book 6

c# 15, f# 11, and .net 11

a complete c# manual with an f# tour

Raeliosol

version 5.0, compiled 10 2026

Contents

1. toolchain and project model 8
   1.1. the sdk 8
   1.2. projects and solutions 8
   1.3. the command surface 9
   1.4. xunit v3 and why this suite is not on it 10
   1.5. top level statements 11
   1.6. how this book verifies claims 11
2. lexical structure 12
   2.1. identifiers 12
   2.2. keywords 12
   2.3. the union keyword 13
   2.4. literals, the full inventory 13
   2.5. strings, five ways 14
   2.6. bool and null 15
   2.7. the preprocessor 15
   2.8. comments and the rest of the surface 15
3. the type system 17
   3.1. value types and reference types 17
   3.2. choosing a kind of type 18
   3.3. tuples 18
   3.4. records 19
   3.5. union types 20
   3.6. closed hierarchies 22
   3.7. enums 23
   3.8. nullable value types 25
   3.9. nullable reference types 25
4. members 27
   4.1. properties and accessors 27
   4.2. the field keyword 27
   4.3. init and required 28
   4.4. indexers 29
   4.5. events 29
   4.6. extension members 30
5. generics 33
   5.1. constraints 33
   5.2. variance 33
   5.3. static abstract members and generic math 35
6. pattern matching and control flow 37
   6.1. the three constructs, the ten patterns 37
   6.2. the switch expression 37
   6.3. switching over unions without a fallback 38
   6.4. property patterns 39
   6.5. positional patterns 39
   6.6. list patterns 40
   6.7. one switch, several patterns 40
   6.8. the switch statement and goto 41
   6.9. loops 41
   6.10. labeled break and continue 43

6.11. the null operators ... 44
7. delegates, lambdas, events ... 46
7.1. declaring and invoking ... 46
7.2. Func, Action, and lambdas ... 46
7.3. closures and captured variables ... 47
7.4. multicast ... 48
7.5. delegates as parameters ... 49
7.6. events, the restricted multicast ... 49
7.7. local functions ... 49
8. exceptions ... 51
8.1. throwing ... 51
8.2. try, catch, finally ... 52
8.3. exception filters ... 52
8.4. rethrowing, the difference one word makes ... 53
8.5. expected failure is not an exception ... 55
9. async ... 56
9.1. async, await, Task ... 56
9.2. concurrency with WhenAll ... 56
9.3. ValueTask ... 57
9.4. cancellation ... 58
9.5. async streams ... 59
9.6. the rules that keep async honest ... 60
10. memory and runtime ... 61
10.1. the garbage collector ... 61
10.2. spans ... 62
10.3. stackalloc ... 62
10.4. passing by reference ... 63
10.5. overflow ... 64
10.6. unsafe in outline ... 64
10.7. the memory safety preview ... 65
10.8. what .NET 10 and 11 changed ... 67
11. allocation: the managed heap, pools, and arenas ... 69
11.1. the managed heap ... 69
11.2. the large object heap ... 70
11.3. the pinned object heap ... 70
11.4. write barriers ... 71
11.5. arraypool, deep ... 72
11.6. memorypool and imemoryowner ... 74
11.7. arenas as a pattern ... 74
11.8. the particle kernel ... 76
12. profilers: benchmarkdotnet, counters, and p99 ... 78
12.1. benchmarkdotnet, the grammar ... 78
12.2. the rc gap and the inprocess toolchain ... 79
12.3. dotnet-counters, live ... 80
12.4. dotnet-trace and eventpipe ... 81
12.5. the gc panel ... 82
12.6. p99 for a managed loop ... 83
12.7. the row kernel ... 85
12.8. which tool when ... 86
12.9. across the six books ... 86
13. reflection, attributes, source generators ... 88

13.1. attributes ... 88
13.2. reflection ... 88
13.3. dynamic ... 89
13.4. source generators ... 90
14. linq ... 93
14.1. method syntax and query syntax ... 93
14.2. deferred execution ... 94
14.3. the operator families ... 95
14.4. grouping and aggregation ... 95
14.5. join and let ... 96
14.6. outer joins ... 97
14.7. what the chapter 2 keyword matrix promised ... 98
15. stdlib tour 1: collections, memory, numerics ... 99
15.1. collections ... 99
15.2. frozen collections ... 100
15.3. memory utilities ... 101
15.4. numerics ... 101
15.5. decimal floating point ... 102
16. stdlib tour 2: io, text, net, channels, diagnostics ... 104
16.1. System.Text.Json ... 104
16.2. unions on the wire ... 105
16.3. files and streams ... 106
16.4. text ... 107
16.5. http ... 107
16.6. channels ... 108
16.7. small utilities that matter ... 109
17. idioms ... 111
17.1. disposal ... 111
17.2. the equality contract ... 112
17.3. string comparison discipline ... 113
17.4. nullable discipline ... 114
17.5. the testing posture ... 114
18. functional programming ... 115
18.1. the five pieces ... 115
18.2. data over behavior ... 115
18.3. exhaustiveness as a type-level test ... 117
18.4. pipelines ... 118
18.5. what c# borrows from f# ... 120
19. f# 11: values, functions, and types ... 121
19.1. let bindings and immutability ... 121
19.2. functions as values ... 121
19.3. type inference ... 122
19.4. records ... 123
19.5. discriminated unions ... 123
19.6. pattern matching and exhaustiveness ... 124
19.7. options ... 125
20. f# 11: collections, pipelines, and async ... 127
20.1. the pipe and compose operators ... 127
20.2. the collection modules against linq ... 127
20.3. the lexer, index arithmetic over a string ... 128
20.4. the parser, precedence by recursion ... 130

20.5. computation expressions ... 131
20.6. async, deliberately absent ... 132
21. f# on .net 11 ... 133
21.1. the toolchain ... 133
21.2. c# interop, both directions ... 133
21.3. two unions, two wire shapes ... 134
21.4. trimming and parallel compilation ... 135
21.5. the versioning story, verified ... 135
21.6. the side quest: the evaluator in f# ... 135
22. api kernel ... 139
22.1. the request delegate contract ... 139
22.2. endpoint routing as the route table ... 139
22.3. the error envelope ... 140
22.4. json in and out ... 141
22.5. the in-process test server ... 142
22.6. the app skeleton and its wiring table ... 143
22.7. what the kernel proves ... 144
23. middleware ... 146
23.1. the layer type ... 146
23.2. exception to envelope ... 146
23.3. request id ... 147
23.4. the access log ... 148
23.5. the handler deadline ... 149
23.6. wiring order and why ... 150
23.7. testing the stack ... 151
24. the user resource ... 153
24.1. the representation ... 153
24.2. decoding with strictness ... 154
24.3. validation from first principles ... 155
24.4. patch merge semantics ... 156
24.5. keyset pagination ... 157
24.6. the status contract ... 158
24.7. golden handler tests ... 158
25. authn ... 160
25.1. password storage with pbkdf2 ... 160
25.2. login and timing flattening ... 161
25.3. server-side sessions ... 161
25.4. cookie mechanics ... 162
25.5. jwt from scratch with hmac ... 163
25.6. the asymmetric token over ecdsa ... 164
25.7. the refresh rotation and reuse detection ... 165
25.8. testing auth without walls ... 165
26. authz and rbac ... 167
26.1. authentication versus authorization ... 167
26.2. roles and the grant table ... 167
26.3. policies as functions ... 168
26.4. 401 versus 403 ... 169
26.5. object-level authorization ... 169
26.6. deny by default with a route audit ... 170
26.7. the rbac matrix test ... 171
27. store ... 173

27.1. ado.net and microsoft.data.sqlite ... 173
27.2. migrations test driven ... 174
27.3. queries with cancellation every time ... 175
27.4. the transaction idiom ... 176
27.5. isolation levels and what sqlite actually gives you ... 176
27.6. the lost update demo and the CAS answer ... 177
27.7. analytics with window functions ... 178
27.8. store tests without docker ... 179
28. concurrency control ... 181
28.1. the lost update prevented ... 181
28.2. If-None-Match and 304 ... 182
28.3. idempotency keys ... 182
28.4. singleflight ... 183
28.5. the primitives revisited ... 184
28.6. the race canary ... 185
29. caching ... 187
29.1. what caching promises ... 187
29.2. http caching semantics for private apis ... 187
29.3. LRU from first principles ... 188
29.4. TTL beside LRU ... 189
29.5. the stampede and its guard ... 190
29.6. invalidation on write ... 191
29.7. what not to cache ... 192
30. rate limiting ... 194
30.1. why limits exist ... 194
30.2. the token bucket from first principles ... 194
30.3. System.Threading.RateLimiting in production ... 195
30.4. the 429 contract ... 197
30.5. what to key on ... 199
30.6. limits under test ... 200
30.7. admission preview ... 201
31. observability ... 203
31.1. ILogger the real setup ... 203
31.2. structured beats interpolated ... 204
31.3. tracing from scratch ... 205
31.4. metrics and the prometheus text format ... 206
31.5. p99 done honestly ... 207
31.6. the diagnostics surface ... 208
31.7. one request fully observed ... 209
32. load and p99 ... 210
32.1. the load harness ... 210
32.2. the report ... 211
32.3. the benchmark discipline ... 212
32.4. graceful shutdown ... 213
32.5. testing shutdown without sleeps ... 213
32.6. admission control ... 214
32.7. the task leak check ... 215
33. ship it ... 217
33.1. the image ... 217
33.2. compose one command ... 218
33.3. the docker lane as a gate ... 219

33.4. ci on GitHub Actions ... 220
33.5. helm the service envelope ... 221
33.6. terraform for this service ... 222
33.7. the shipped checklist ... 222
34. the test suite ... 224
34.1. what the suite already holds ... 224
34.2. properties as seeded theories ... 225
34.3. the cache as a state machine ... 226
34.4. forgery resistance as a target ... 228
34.5. strictness without a framework ... 228
34.6. the xunit surface and the truths it hides ... 229
34.7. coverage without ceremony ... 230
34.8. what the suite proves ... 231
35. capstone: a small language interpreter ... 232
35.1. the pipeline ... 232
35.2. the language ... 232
35.3. lexer, a ref struct over a span ... 233
35.4. the tree, a c# 15 union ... 234
35.5. parser, precedence by method chain ... 238
35.6. checker, pattern matching over the tree ... 238
35.7. evaluator, async end to end ... 240
35.8. builtins and the generator that registers them ... 242
35.9. saves, one sqlite file ... 245
35.10. what the capstone proves ... 246
35.11. profiled, and why ... 246
35.12. the service part ... 248
36. appendices: coverage and sources ... 249
36.1. keyword coverage ... 249
36.2. stdlib coverage ... 253
36.3. pinned sources ... 254

1. toolchain and project model

C# 14 compiles on .NET 10, the long term support release this book teaches from, and the C# 15 chapters fold in .NET 11 builds. This chapter covers what you install, how projects are shaped, and the command line used throughout the book. Every listing in this book is a slice of a real file in this repo's sample solution, and every sample file is compiled and tested before the book is, the Makefile target `verify-csharp` runs the whole suite.

1.1. the sdk

The .NET SDK is one download that includes the compiler (Roslyn), the runtime, and the build tooling. .NET 10 stays the teaching base, and since the C# 15 folds the book builds and tests on the .NET 11 SDK pinned by the book local `global.json`:

```
$ dotnet --version
11.0.100-rc.1
```

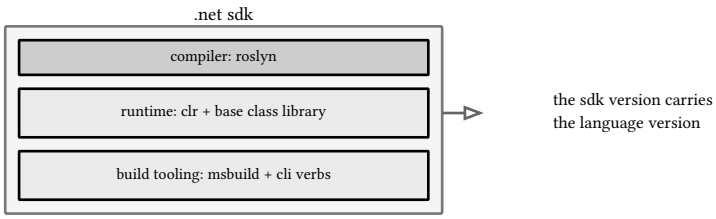


figure 1: the sdk, one download stacking compiler, runtime, and build tooling

The runtime layer is the clr, the common language runtime: it executes il, the intermediate language every .NET compiler emits, and the jit, its just in time compiler, turns that il into machine code while the program runs.

The SDK version carries the language version. Targeting `net10.0` defaults to C# 14 and targeting `net11.0` defaults to C# 15, no `LangVersion` property needed in either case. .NET 10 is LTS, supported to November 2028, so it stays the teaching base, while the C# 15 chapters build under the book local pin at 11.0.100-rc.1; the appendices' rc1 build pin note records what re-pinning at GA will mean.

1.2. projects and solutions

A C# project is an XML file with a target framework and properties. One project compiles to one assembly, the dll or exe the runtime loads. The sample solution's project file is the whole configuration:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net11.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <!-- the generator and the preview project live in nested folders, keep
         them out of this project's glob -->
    <Compile Remove="CsharpBook.Generators\**" />
    <Compile Remove="CsharpBook.Preview\**" />
    <ProjectReference Include="CsharpBook.Generators\CsharpBook.Generators.csproj"
                      OutputItemType="Analyzer" ReferenceOutputAssembly="false" />
  </ItemGroup>

  <ItemGroup>
    <!-- pinned native engine for every SQLite touch in the samples -->
    <PackageReference Include="Microsoft.Data.Sqlite" Version="10.0.12" />
    <PackageReference Include="SQLitePCLRaw.bundle_e_sqlite3" Version="3.0.5" />
  </ItemGroup>

</Project>
```

listing 1: the sample project file, complete (source: csharp-net/samples/src/CsharpBook.Samples.csproj, lines 1-26)

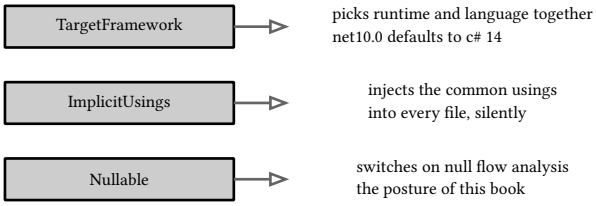


figure 2: three csproj properties, what each one controls

Three properties do real work. `TargetFramework` picks runtime and language version together. `ImplicitUsings` adds the common namespace usings (`System`, `System.Linq`, `System.Collections.Generic`, and friends) to every file silently. `Nullable` turns on nullable reference types, the compiler flow analysis that warns when a reference might be null, which is the default posture of modern C# and the posture of every line in this book.

Solutions group projects. With the .NET 10 SDK, `dotnet new sln` creates the new XML based `.slnx` format rather than the historical `.sln`:

```
$ dotnet new sln -n CsharpBook
CsharpBook.slnx created.
```

The old `.sln` still works, `dotnet sln` reads both, but new solutions default to `.slnx` and that is what this repo uses.

1.3. the command surface

The verbs this book uses:

```
dotnet new console -o src           scaffold a project
dotnet build                        compile
dotnet test                         build and run the test suite
```

```
dotnet run                          build and execute
dotnet publish -c Release           produce deployable output
```

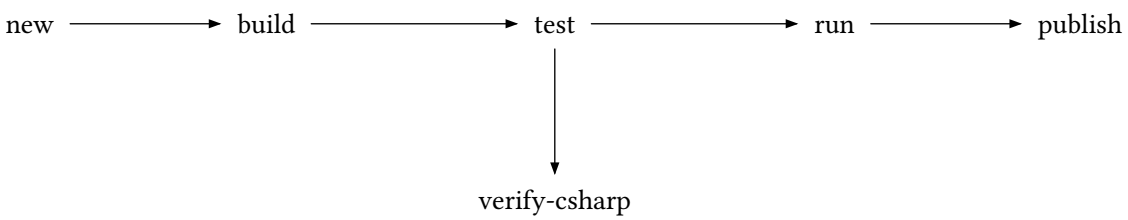


figure 3: the dev loop as a pipeline, test the gate every listing depends on

`dotnet test` on the solution runs the xunit suite over every test project. The sample suite is what gives book listings their guarantee: a listing exists here because its file passed that suite.

1.4. xunit v3 and why this suite is not on it

xUnit.net v3 is the current major version of the test framework this book runs on. It ships its own templates, installed explicitly:

```
$ dotnet new install xunit.v3.templates
xUnit.net v3 Test Project       xunit3            [C#],F#,VB  Test/xUnit
xUnit.net v3 Extension Project  xunit3-extension  [C#],F#,VB  Test/xUnit
```

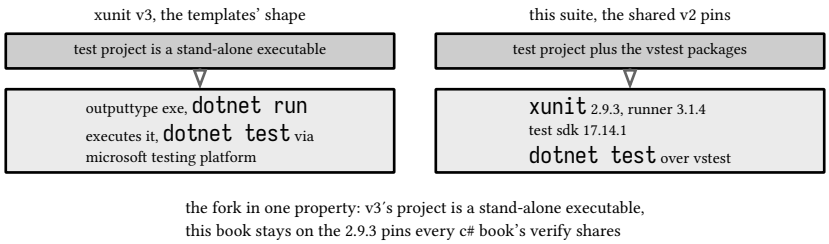


figure 4: the v2/v3 fork in one property, a v3 suite executes itself while this book stays on the shared v2 pins

A v3 project differs from the v2 shape this book uses in one structural way: the test project is a stand-alone executable, `OutputType` set to `Exe`, so `dotnet run` executes the suite directly and `dotnet test` goes through Microsoft Testing Platform by default. Choosing the VSTest runner instead adds back the two packages the v2 world knows, `xunit.runner.visualstudio` and `Microsoft.NET.Test.Sdk`. The SDK side grew a companion feature for single file programs: file-based apps, configured by `#:` directives at the top of the file. The `#:include` directive, available since .NET SDK 10.0.300 and .NET 11 Preview 3, pulls another file into the build, `*.cs` files map to `Compile`, globs and MSBuild properties are allowed, and the included files can add declarations but not top level statements:

```
#:include helpers.cs
#:include models/customer.cs
#:include shared/**/*.cs
```

This suite stays on xunit 2.x, and the csproj is the honest statement: `xunit` 2.9.3, `xunit.runner.visualstudio` 3.1.4, `Microsoft.NET.Test.Sdk` 17.14.1. Two reasons. Every C# suite in this repo, four books, runs through the same `make verify-csharp` `dotnet test` pipeline on those identical pins, and the v3 packages the templates reference were still 4.0.0 pre-release when this was written. Nothing this book teaches needs the v3 additions, `[Fact]` and `[Theory]` mean the same thing in both versions, so the upgrade would buy runner changes and nothing pedagogical. When the 4.0.0 packages go stable the delta to record is mechanical: install the templates, swap the packages, keep the tests.

1.5. top level statements

Since C# 9, a console program needs no `Main` method declaration. A file of top level statements becomes the entry point, and the magic `args` contextual keyword carries the command line arguments:

```
// entry point for the sample project, prints the two ch02 lexical samples
using CsharpBook.Samples.Ch02;

Console.WriteLine(Lexical.RawStringLiteral());
Console.WriteLine(Lexical.InterpolatedForm());
```

listing 2: the whole entry point of the sample project (source: csharp-net/samples/src/Program.cs, lines 1-6)

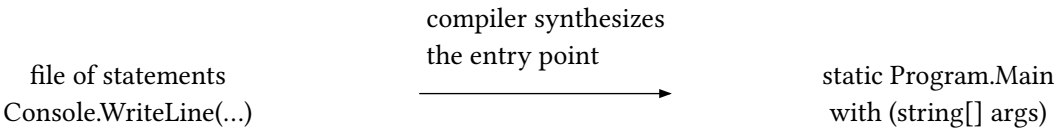


figure 5: top level statements and explicit main, the same program in two shapes

Top level statements are a convenience with rules: only one file per project may have them, any classes must come after the statements, and the compiler synthesizes a `Main` under the hood. For programs that grow, the ordinary shape with an explicit `Program.Main` returns, and that shape gets its full treatment in the structure chapter of book 4's Go material for comparison, and in the idioms chapter here.

1.6. how this book verifies claims

VERIFY: the claim discipline for this book

Every chapter ends with a sources line naming the official documentation sections it was written against, fetched while writing. Code claims are backed by the sample suite, run with `make verify-csharp`. When a behavior surprised the suite, the chapter says so.



figure 6: the evidence chain, from a sample file to a prose claim

sources: learn.microsoft.com, what is new in c# 14 and .net 10, accessed 2026-09-07; learn.microsoft.com, c# language reference keywords page, accessed 2026-09-08; xunit.net, getting started with xunit.net v3 and what's new in v3 pages; learn.microsoft.com, file-based apps page, accessed 2026-09-13; the sample solution in this repo, built under the book local `global.json` pin at 11.0.100-rc.1.

2. lexical structure

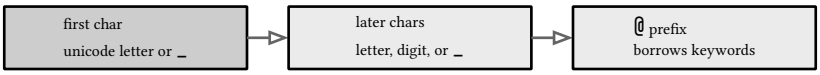
Everything the compiler sees before it understands types: identifiers, keywords, literals, operators, and the preprocessor. C#'s lexical design goal was familiarity for C and Java programmers with the rough edges filed off. The places it files hardest: raw string literals, digit separators, and the `@` and contextual keyword system.

2.1. identifiers

Identifiers start with a letter or underscore and continue with letters, digits, or underscores, where letter means any unicode letter. Vietnamese diacritics are legal identifiers, and this book's author takes advantage:

```
// unicode letters are valid identifier characters
public static int TổngHaiSố(int a, int b) => a + b;
```

listing 3: unicode identifiers are legal (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 11-12)



`TổngHaiSố`: one identifier, every position legal
`@class`: keyword borrowed, `class` alone: syntax error

figure 7: identifier grammar, position one, later positions, and the @ escape

Reserved keywords cannot be identifiers, but the `@` prefix borrows any keyword as a name. This is occasionally necessary for interop with other languages and otherwise a curiosity to avoid:

```
// reserved word borrowed as an identifier
string @class = "not a class declaration here";
int @int = 3;      // legal, discouraged
```

fragment, illustrative, not executed

2.2. keywords

C# has 77 reserved keywords, `abstract` through `while`, listed completely in the appendix coverage matrix. The design that matters more: since C# 2, every new language word has been a **contextual** keyword, meaningful only in position, so old code that used `async`, `yield`, or `from` as variable names keeps compiling. C# 14 continued with `extension` and `field`, C# 15 continues with `union`, the next section's subject. The count of contextual keywords now rivals the reserved set, and the appendix maps every one to the chapter that teaches it.

NOTE: why contextual matters

A language that keeps its reserved set frozen can add features without breaking its own ecosystem. When you meet a new word in C#, check whether it is contextual. If it is, the word is also a legal identifier, which is the tell that it arrived in a later version.

	reserved (77)	contextual (grows)
count	77, frozen since c# 1	one or more per version
as identifier	never	yes, outside its position
examples	`class`, `while`, `return`	`async`, `yield`, `field`

figure 8: reserved vs contextual keywords, what each class allows

2.3. the union keyword

C# 15 adds `union`, and the word arrives the way every new word since C# 2 has arrived: contextual. Chapter 3 owns the union semantics, the cases, the conversions, the exhaustiveness. This chapter owns the word itself. In type declaration position, `union` followed by a name and a parenthesized case list, the parser reads a type declaration. Everywhere else the word is a plain identifier, and the suite proves it with code that compiles on the default language version, no preview flag:

```
// union is contextual: outside its declaration form it is a plain name
public static int UnionAsIdentifier()
{
    int union = 5;
    var total = union + 1;
    return total + Union();
}

private static int Union() => 1;
```

listing 4: a local named union, an addition, a call, all ordinary (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 43-51)

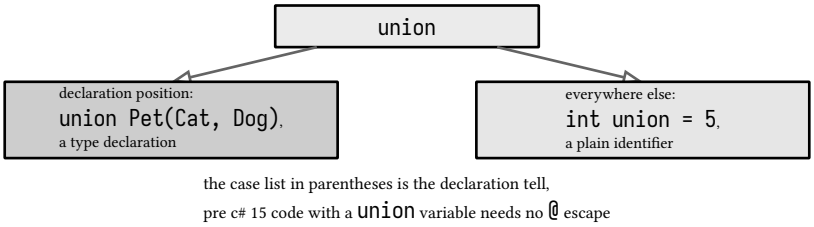


figure 9: union, one word, two parses

The census was recounted from the live keywords reference for this edition rather than carried forward. The reserved table still lists 77 words, `abstract` through `while`, unchanged. The contextual table lists 48 words as of the read date: `partial`, `unmanaged`, `when`, and `where` each appear twice in the table because each word serves two documented contexts, they count once. `union` was not yet in that table, the index page lags the compiler, but the union reference page documents the word and the listing above shows it parsing as an identifier outside its position, which is the contextual signature. The appendix matrix files it under contextual, 49 rows, taught here.

2.4. literals, the full inventory

Integer literals come in decimal, hexadecimal (`0x`), and binary (`0b`) forms, with `_` digit separators allowed anywhere between digits and type suffixes forcing width:

```
// integer literals: decimal, hex, binary, digit separators, suffixes
public static int IntegerForms() => 1_000_000 + 0xFF_FF + 0b1010_1010;
public static long LongSuffix() => 9_007_199_254_740_993L;
public static uint UnsignedSuffix() => 4_000_000_000u;
```

listing 5: integer literal forms (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 14-17)

Real literals come in three precisions: `double` by default, `f` for 32 bit `float`, `m` for `decimal`, the base 10 fixed point type for money where `0.1` must mean exactly 0.1:

```
// real literals: double, float single precision, decimal for money
public static double DoubleForm() => 1.5e3;
public static float FloatSuffix() => 2.5f;
public static decimal DecimalSuffix() => 19.99m;
```

listing 6: real literal forms and suffixes (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 19-22)

Character literals take a single character, a \u unicode escape, or a \x hex escape:

```
    // char literals: plain, hex escape, unicode escape
    public static string CharForms() => $"{'A'}{'\u0042'}{'\x0043'}";
```

listing 7: char literal forms (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 24-25)

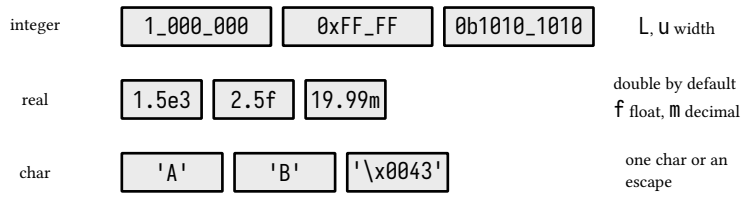


figure 10: literal inventory, bases, separators, suffixes, escapes

2.5. strings, five ways

The string literal system deserves its own section because it has grown five forms, each solving a real problem:

```
    // string literals: regular, verbatim, raw, interpolated
    public static string RegularString() => "tab\t newline\n quote\"";
    public static string VerbatimString() => @"C:\temp\raw\path";
    public static string RawStringLiteral() => """
        quotes " inside, braces { } inside, no escaping needed
        """;
```

listing 8: regular, verbatim, and raw string literals (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 27-32)

Regular strings interpret escapes. Verbatim strings (@"...") take everything literally, doubling "" for a quote, the natural form for Windows paths and regex patterns. Raw string literals (three or more quotes) take everything literally with no escaping at all, and strip indentation to the closing delimiter, which makes embedded SQL, JSON, and multi line text painless.

Interpolation is $"...", with format specifiers after : and alignment after ,:

```
    public static string InterpolatedForm() => $"{Math.PI:F2} and {41,5} and {42:X}";
```

listing 9: interpolation with format and alignment (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 33-33)

Combining interpolation with raw strings doubles the braces, $$""" with {{...}} for holes, which keeps single braces literal for templating:

```
    public static string RawInterpolated() => $$"""
        {Math.PI:F2} stays literal, {{Math.PI:F2}} interpolates
        """;
```

listing 10: raw interpolated strings (source: csharp-net/samples/src/Ch02/Lexical.cs, lines 35-37)

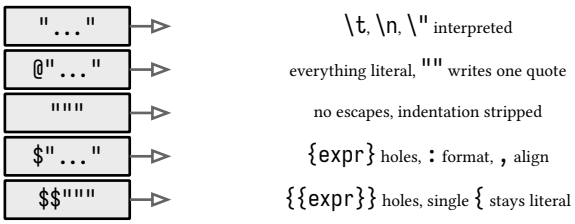


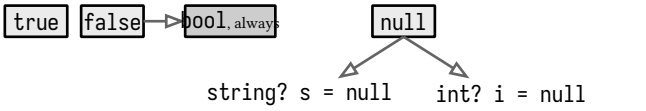
figure 11: the five string forms and their escape policies

PITFALL: the colon in interpolation holes

> {Math.PI:F2} parses because the compiler knows Math.PI is an expression, but a conditional operator inside a hole needs parentheses: {(flag ? "a" : "b")}, because the bare : would be read as the start of a format specifier.

2.6. bool and null

true and false are literals of type bool. null is the null literal, typed by context, and under nullable reference types the compiler tracks which references can hold it, which is the type system chapter's subject.



the context picks the type, and under nullable reference types
the compiler tracks nullness while the runtime type never changes

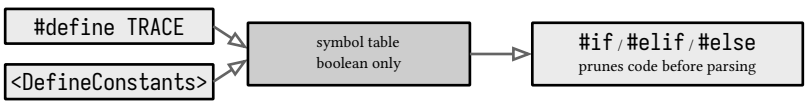
figure 12: true, false, null, two literals with fixed types and one typed by context

2.7. the preprocessor

Lines beginning with # are directives, evaluated before parsing proper. The honest description: C#'s preprocessor is not a text macro processor. There is no #define PI 3.14. The directives that matter:

```
#define TRACE              // boolean symbol only, no value symbols
#if TRACE
  Console.WriteLine("traced");
#elif DEBUG
#else
#endif
#pragma warning disable CS8600   // silence one warning by id
#nullable enable                 // set nullable context per file
#endregion                      // editor folding, nothing more
```

fragment, illustrative, not executed



no value macros: #define PI 3.14 does not exist,
#pragma and #nullable are compiler directives, not macros

figure 13: preprocessor symbols, two feeds, boolean only, pruning before parse

Conditional compilation uses boolean symbols set by #define or the <DefineConstants> property, and the #if family is how cross platform code paths compile selectively. It is a feature to use sparingly, most conditional behavior belongs in runtime if statements where tests can reach it.

2.8. comments and the rest of the surface

Line comments //, block comments /* */ which do not nest, and documentation comments /// that produce XML docs, covered with reflection in chapter 13. Statements end in ;. Whitespace has no meaning beyond separation, and the grammar is free form. That is the entire lexical surface, everything from here on is grammar and types.

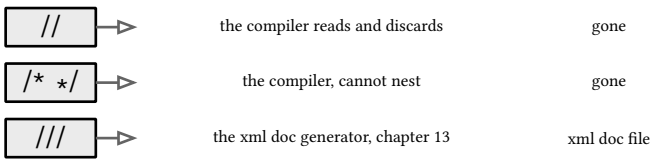


figure 14: three comment forms, their consumers, and what survives

sources: learn.microsoft.com, c# language reference, lexical structure and keywords pages, accessed 2026-09-08, the keywords reference recounted and the union types reference page, accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 10 tests.

3. the type system

Every type in C# is either a value type or a reference type. The split runs through the whole language: it decides what assignment does, what == means, where data lives, and which keywords can declare what. This chapter builds the full model: the two categories and their conversions, tuples, the record family, enums, and the two nullable systems, one for value types and one for references.

3.1. value types and reference types

A value type variable holds its data directly. A reference type variable holds a reference to an object on the heap. Assignment is where the difference shows up, so the samples declare one of each:

```
// a value type: the variable holds the data itself
public struct Money
{
    public decimal Amount;
    public string Currency;
}

// a reference type: the variable holds a reference to heap data
public class Account
{
    public decimal Balance;
}
```

listing 11: a struct and a class, same shape, different semantics (source: csharp-net/samples/src/Ch03/Types.cs, lines 3-14)

Assigning a struct copies the entire value. Assigning a class copies only the reference, so both names end up looking at one object:

```
    public static (decimal First, decimal Second) StructAssignmentCopies()
    {
        var salary = new Money { Amount = 100m, Currency = "usd" };
        var raise = salary;             // copies the whole struct
        raise.Amount = 200m;            // the copy changes, the original does not
        return (salary.Amount, raise.Amount);
    }
```

listing 12: struct assignment copies the data (source: csharp-net/samples/src/Ch03/Types.cs, lines 49-55)

```
    public static (decimal First, decimal Second) ClassAssignmentShares()
    {
        var joint = new Account();
        var alias = joint;              // copies the reference, one object total
        alias.Balance = 200m;           // visible through both names
        return (joint.Balance, alias.Balance);
    }
```

listing 13: class assignment copies the reference (source: csharp-net/samples/src/Ch03/Types.cs, lines 57-63)

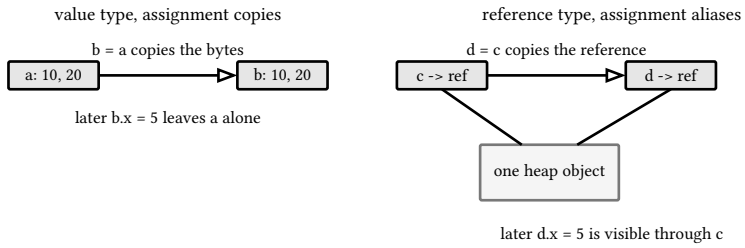


figure 15: what assignment does in each category

The unified hierarchy sits on top: every type derives from `object`, value types through `System.ValueType`. A value type becomes an object by **boxing**, a heap allocation that copies the value in. The box is separate storage, which the sample proves by mutating the original afterwards:

```
public static (int Original, int Boxed) BoxingCopiesTheValue()
{
    int i = 123;
    object boxed = i;              // heap allocation, value copied in
    i = 456;                       // the box is unaffected
    return (i, (int)boxed);        // unboxing is an explicit cast
}
```

listing 14: boxing copies, later mutations do not reach the box (source: csharp-net/samples/src/Ch03/Types.cs, lines 65-71)

PITFALL: boxing hides in hot paths

Boxing happens implicitly whenever a value type is assigned to `object` or to an interface it implements, including calls like `Console.WriteLine($"{x}")` in older code shapes and non generic collections. Each box is an allocation the garbage collector must later track. Chapter 10 covers spans and generics as the standard tools that keep numeric and collection code boxing free.

3.2. choosing a kind of type

The official guidance orders the options by weight. Use a tuple for a temporary grouping that never leaves the method. Use a `struct` or `record struct` for small data, roughly 64 bytes or less, where value semantics fit. Use a `record class` for data centric types that want value equality and nondestructive mutation. Use a `class` when there is real behavior, mutable state, or polymorphism, which is most custom types. Use an `interface` for a contract unrelated types can share. Use an `enum` for a fixed set of named constants.

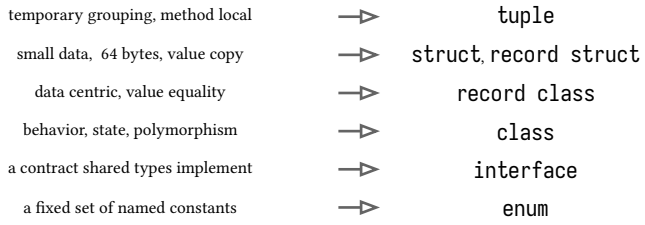


figure 16: the kind of type ladder, lightest to heaviest

3.3. tuples

A tuple groups values into a lightweight value type with no declared type at all. Elements can carry names, tuples deconstruct into variables with discards for the parts you do not want, and == compares element by element, names irrelevant:

```
    public static (int Kept, bool EqualTuples) TupleBehavior()
    {
        var stats = (Min: 5, Max: 15);  // named elements, still a value type
        var (min, _) = stats;           // deconstruct, discard the second
        bool equal = (1, 2) == (1, 2);  // == compares element by element
        return (min, equal);
    }
```

listing 15: named elements, deconstruction with a discard, tuple equality (source: csharp-net/samples/src/Ch03/Types.cs, lines 73-79)

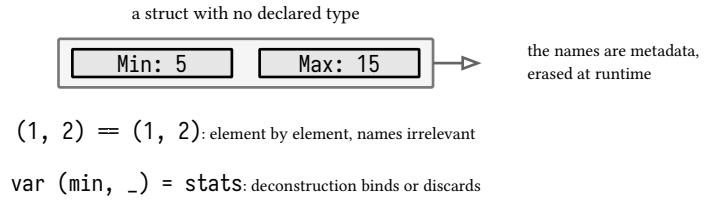


figure 17: a tuple, inline elements, borrowed names, element wise equality

Tuples are the honest return type for a function that computes several things at once. The moment the grouping grows a meaning of its own and travels between methods, promote it to a record.

3.4. records

A record is a type whose members and equality the compiler synthesizes from a positional declaration. One line declares a type with `init`-only positional properties, a primary constructor, value equality, a readable `ToString`, and a `Deconstruct` method:

```
// record class: positional properties, value equality, with-expressions
public record Book(string Title, int Year)
{
    public required string Isbn { get; init; }
}

// record classes support inheritance; equality includes the runtime type
public record Audiobook(string Title, int Year, TimeSpan Runtime)
    : Book(Title, Year);

// record struct: record conveniences on a value type
public readonly record struct Point(int X, int Y);
```

listing 16: record class, derived record, and record struct in three declarations (source: csharp-net/samples/src/Ch03/Types.cs, lines 16-27)

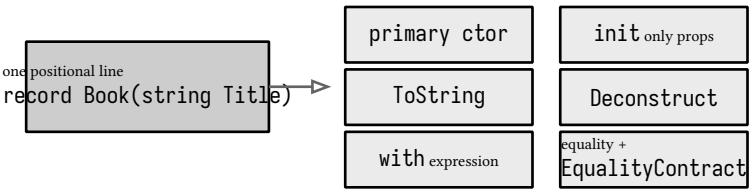


figure 18: one record declaration, the six members the compiler synthesizes

The `with` expression is nondestructive mutation: it copies the instance and applies changes to the copy, leaving the original untouched. Records print their contents, which makes them the natural currency of tests and logs:

```
    public static (bool WithChanged, bool CopyEqual, string Printed) RecordClassBehavior()
    {
        var original = new Book("tolkien", 1954) { Isbn = "978-0618640157" };
        var modified = original with { Title = "the fellowship of the ring" };
        var copy = original with { };   // same values, new instance
        return (original == modified, original == copy, copy.ToString());
    }
```

listing 17: with produces a changed copy, equality and printing are synthesized (source: csharp-net/samples/src/Ch03/Types.cs, lines 81-87)

Record equality is structural but not blind. The synthesized equality compares the runtime type first through an `EqualityContract`, so a base record and a derived record with identical data are not equal:

```
    public static bool RecordEqualityIncludesRuntimeType()
    {
        Book plain = new("dune", 1965) { Isbn = "1" };
        Book audio = new Audiobook("dune", 1965, TimeSpan.FromMinutes(666)) { Isbn = "1" };
        return plain.Equals(audio);     // false: same data, different runtime types
    }
```

listing 18: same data, different runtime type, not equal (source: csharp-net/samples/src/Ch03/Types.cs, lines 89-94)

A `record struct` puts the same conveniences on a value type. Its positional properties are read-write rather than init-only, it cannot inherit, and `default` gives the all-zero value without running any constructor:

```
    public static (bool Equal, int DefaultX) RecordStructBehavior()
    {
        var a = new Point(1, 2);
        var b = new Point(1, 2);
        var origin = default(Point);    // all-zero value, no constructor runs
        return (a == b, origin.X);
    }
```

listing 19: record struct equality and the zero default (source: csharp-net/samples/src/Ch03/Types.cs, lines 96-102)

3.5. union types

Records model data with one fixed shape. A union models a choice: a value that is exactly one of a declared set of case types. Until C# 15 the idiom was a record hierarchy plus a discard arm, correct but trusted only by discipline. C# 15, the default language version when targeting `net11.0` under the book's pinned SDK 11.0.100-rc.1 (RC1, GA November 2026), declares the domain in one line:

```
// the case types: one sealed record per alternative
public sealed record Cat(string Name);
public sealed record Dog(string Name);
public sealed record Bird(string Name);

// the union: one line declares the whole domain
public union Pet(Cat, Dog, Bird);
```

listing 20: three case records and the one line union declaration (source: csharp-net/samples/src/Ch03/Unions.cs, lines 5-11)

The declaration compiles to a sealed struct implementing `System.Runtime.CompilerServices.IUnion`: an `object Value` property holds the active case, one public constructor per case type builds the union, and an implicit conversion from each case type means the wrapper does not appear at the construction site:

```
public static string ImplicitFromCase()
{
    Cat cat = new("milo");
    Pet pet = cat;              // implicit conversion from the case type
    return pet is Cat ? cat.Name : "not a cat";
}
```

listing 21: the case converts into the union implicitly (source: csharp-net/samples/src/Ch03/Unions.cs, lines 33-38)

Patterns are union aware, `is` and switch arms reach the case rather than the wrapper:

```
public static (bool Cat, bool Dog, bool Bird) IsTesting()
{
    Pet cat = new Cat("milo");
    Pet dog = new Dog("rex");
    Pet bird = new Bird("kiwi");
    return (cat is Cat, dog is Dog, bird is Bird);
}
```

listing 22: is testing unwraps to each case (source: csharp-net/samples/src/Ch03/Unions.cs, lines 40-46)

A switch that names every case is exhaustive, no discard arm, which removes the silent default arm chapter 6 examines with its failure modes:

```
// exhaustive over the declared cases, no discard arm
public static string Describe(Pet pet) => pet switch
{
    Cat c => $"cat {c.Name}",
    Dog d => $"dog {d.Name}",
    Bird b => $"bird {b.Name}",
};
```

listing 23: exhaustive over the declared cases, no fallback arm (source: csharp-net/samples/src/Ch03/Unions.cs, lines 48-54)

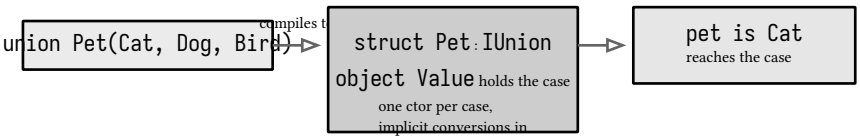


figure 19: a union declaration, the struct the compiler generates

The union is a value type, so the boxing rules from the start of this chapter apply unchanged. Assigning the union to `object` copies the wrapper to the heap, and the box holds the union, not the case inside it: `boxed is Pet` is true while `boxed is Cat` is false, and the direct `pet is Cat` still unwraps:

```
public static (bool BoxedIsPet, bool BoxedIsCat, bool DirectIsCat) Boxing()
{
    Pet pet = new Cat("milo");
    object boxed = pet;         // the union struct itself goes on the heap
    return (boxed is Pet, boxed is Cat, pet is Cat);
}
```

listing 24: the box holds the union, only the direct is unwraps (source: csharp-net/samples/src/Ch03/Unions.cs, lines 68-73)

WARNING: a boxed union hides its case from object tests

> Code that receives `object` and pattern tests it, the `KindOf` classifier of chapter 6, sees the wrapper type and nothing else once a union is boxed. Keep unions statically typed across boundaries or unbox with a cast before matching.

The declaration covers the common case. When the type needs members of its own, `[Union]` on a class implementing `IUnion` makes the constructors that take case types the union creation members. Delete them all and the compiler rejects the type with error CS9385, a union type must have at least one union creation member. This minimal form keeps construction explicit and consumes through `Value`: the compiler did not generate implicit conversions or unwrap patterns for it, and supplying the `TryGetValue` and `HasValue` members the union reference calls the non boxing access pattern is what lets patterns match a custom union's cases directly:

```
// a custom [Union] for cases that need a wrapper you own
public sealed record Ok(int Value);
public sealed record Err(string Message);

[Union]
public sealed class Result<TOk, TErr> : IUnion
{
    public Result(TOk ok) => Value = ok!;   // constructors are union creation members
    public Result(TErr err) => Value = err!;
    public object Value { get; }
}
```

listing 25: a custom union, constructors are the creation members (source: csharp-net/samples/src/Ch03/Unions.cs, lines 13-23)

```
    public static (bool Ok, int Value) ResultOk(int value)
    {
        var result = new Result<Ok, Err>(new Ok(value));
        return (result.Value is Ok, ((Ok)result.Value).Value);
    }

    public static (bool Err, string Message) ResultErr(string message)
    {
        var result = new Result<Ok, Err>(new Err(message));
        return (result.Value is Err, ((Err)result.Value).Message);
    }
```

listing 26: ok and err paths through the custom union (source: csharp-net/samples/src/Ch03/Unions.cs, lines 56-66)

NOTE: two union attributes exist

`System.Runtime.CompilerServices.UnionAttribute` is the compiler's, it is what `[Union]` resolves to in the listings above. `System.Text.Json.Serialization.JsonUnionAttribute` is the serializer's customization point for how cases are discovered and named, part of .NET 11's union serialization support. Same word, different machinery, do not swap them.

3.6. closed hierarchies

Unions declare their cases inline on a new struct. `closed` solves the other half of the problem: an existing class hierarchy whose descendant set should be complete. A `closed` base class can only be derived from inside its declaring assembly, so the compiler knows every direct descendant and a switch that handles them all is exhaustive:

```
// a closed hierarchy: the class form, descendants beside their root
public closed record class PaymentMethod;
public record class Cash : PaymentMethod;
public record class Card(string Last4) : PaymentMethod;
public record class BankTransfer(string Iban) : PaymentMethod;
```

listing 27: a closed record class root with three descendants (source: csharp-net/samples/src/Ch03/Unions.cs, lines 25-29)

```
    // exhaustive over the direct descendants, no discard arm
    public static string DescribePayment(PaymentMethod method) => method switch
    {
        Cash => "cash",
        Card(var last4) => $"card ending {last4}",
        BankTransfer(var iban) => $"transfer to {iban}",
    };
```

listing 28: exhaustive over the direct descendants, no fallback arm (source: csharp-net/samples/src/Ch03/Unions.cs, lines 75-81)

The modifier is honest about what it does. A closed class is implicitly abstract, `new PaymentMethod()` does not compile, and `closed` cannot combine with `sealed`, `static`, or an explicit `abstract`. Derivation is not transitive: a descendant stays open unless it is itself closed or sealed, and marking intermediate descendants `closed` pushes exhaustiveness checking down the tree. One boundary from the language reference: exhaustiveness follows visibility. A direct descendant less accessible than the closed base, an `internal` case under a `public` root, makes switches in other assemblies warn as non exhaustive because the case cannot be named there.

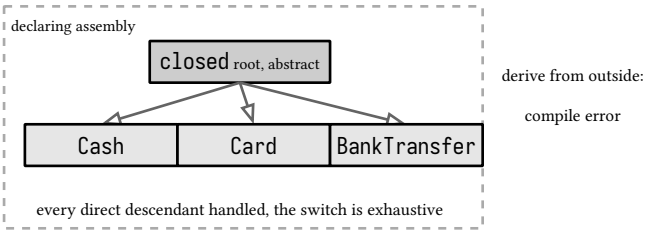


figure 20: closed, the descendant set is fixed inside the assembly

3.7. enums

An enum names a fixed set of integral constants. The underlying type defaults to `int` and can be any integral type except `char`, which matters for interop and for packing enums into structs. A `[Flags]` enum defines members as powers of two so that combinations are themselves values, with named combinations thrown in:

```
// simple enum: one choice out of a fixed set, explicit values, ushort backing
public enum HttpStatus : ushort
{
    Ok = 200,
    NotFound = 404,
}

// flags enum: powers of two so members combine with |
[Flags]
public enum Permissions
{
    None = 0,
    Read = 1,
    Write = 2,
    Execute = 4,
    ReadWrite = Read | Write,
}
```

listing 29: a simple enum and a flags enum (source: csharp-net/samples/src/Ch03/Types.cs, lines 29-45)

The operators `|`, `&`, `^`, and `~` work on every enum type because they operate on the underlying integer. `ToString` on a flags enum prints the combination as names, and `HasFlag` tests membership:

```
    public static (bool Combined, string Printed, bool HasRead) FlagsBehavior()
    {
        var rw = Permissions.Read | Permissions.Write;
        return (rw == Permissions.ReadWrite, rw.ToString(), rw.HasFlag(Permissions.Read));
    }
```

listing 30: combining, printing, and testing flags (source: csharp-net/samples/src/Ch03/Types.cs, lines 104-108)

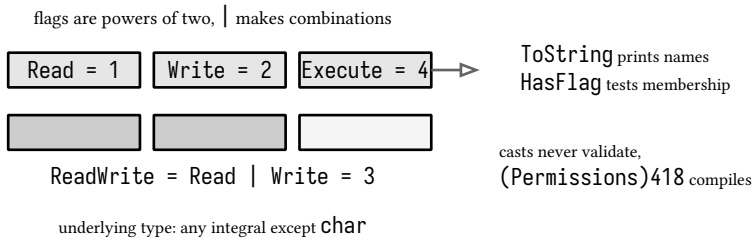


figure 21: enum bits, powers of two combine, names print, casts unchecked

WARNING: numeric to enum casts do not validate

Casting any integer to any enum type compiles and produces a value, defined or not. The literal `0` converts implicitly, which is why every enum should define a zero member. The sample checks the boundary with `Enum.IsDefined`, the standard guard when numbers arrive from outside.

```
    public static (bool Defined, bool NotDefined) NumericCastsDoNotValidate()
    {
        var found = (HttpStatus)404;    // real member
        var invented = (HttpStatus)418; // no member with this value
        return (Enum.IsDefined(found), Enum.IsDefined(invented));
    }
```

listing 31: 418 is not a member, the cast still succeeds (source: csharp-net/samples/src/Ch03/Types.cs, lines 110-115)

3.8. nullable value types

`T?` on a value type is `System.Nullable<T>`, a small wrapper struct that adds "no value" to a type whose domain could not otherwise express it. `HasValue` and `Value` inspect it, `??` supplies a fallback, and `GetValueOrDefault` reads without throwing:

```
public static (bool HasValue, int Fallback, int DefaultValue) NullableValueBehavior()
{
    int? measured = 42;
    int? missing = null;
    return (measured.HasValue, missing ?? -1, missing.GetValueOrDefault());
}
```

listing 32: the three ways to read a nullable value type (source: csharp-net/samples/src/Ch03/Types.cs, lines 117-122)

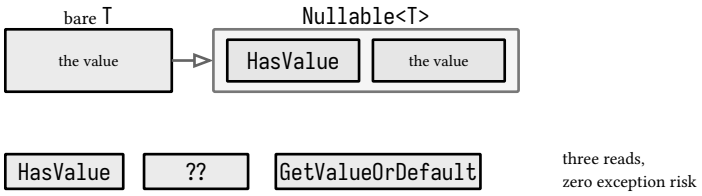


figure 22: nullable value types, a wrapper with a has flag beside the value

The predefined operators are lifted over `T?` as well: arithmetic yields null when either operand is null, the relational operators evaluate to false instead, and equality treats two nulls as equal. `GetValueOrDefault` or `??` before the arithmetic is what keeps a null from cascading through a whole calculation.

3.9. nullable reference types

`T?` on a reference type is not a wrapper and not a different runtime type. It is a compile time annotation, one of three parts that together form nullable reference types: annotations say which variables may hold `null`, flow analysis tracks whether each expression is **not null** or **maybe null** at every point, and attributes on APIs express contracts the syntax cannot, such as "returns null only when the argument was null". The runtime behavior of a `string` and a `string?` is identical. What changes is which code the compiler accepts without a warning:

```
public static int NameLength(string? name)
{
    if (name is null) return -1;    // after this, flow analysis knows more
    return name.Length;             // no warning: name is not null here
}
```

listing 33: a null check narrows the flow state of the parameter (source: csharp-net/samples/src/Ch03/Types.cs, lines 124-128)

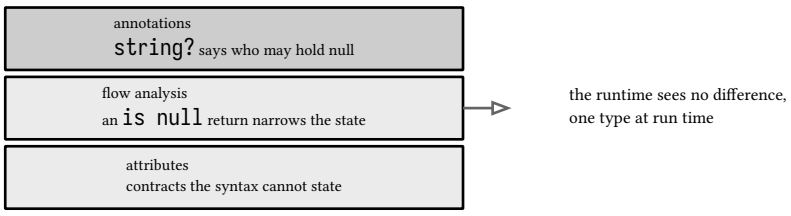


figure 23: nullable reference types, three compile time layers over one runtime type

After the `is null` return, the compiler knows `name` is not null, so `name.Length` needs no forgiveness operator. The `!` operator exists for the cases where you know better than the analysis, and chapter 17 covers when that is ever justified.

sources: learn.microsoft.com, c# type system, value types, reference types, records, record structs, tuple types, enumeration types, nullable value types, nullable reference types pages accessed 2026-09-08, union types, custom

union types, closed modifier, closed hierarchy patterns pages accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 26 tests.

4. members

Classes and structs are containers of members: fields, constants, properties, methods, events, indexers, operators, constructors, and nested types. This chapter covers the data bearing members and the two C# 14 additions that change how they are written. Methods get their own treatment spread over the pattern matching, delegate, and async chapters.

4.1. properties and accessors

A property is a member that reads like a field and runs like a method. The common form is the auto implemented property, where the compiler synthesizes the storage. Accessors control each side: `get` reads, `set` writes, `init` writes only during construction. The sample type uses all three plus `required`, which forces callers to initialize the member explicitly:

```
public class Temperature
{
    public decimal Celsius
    {
        get;
        set ⇒ field = value ≥ -273.15m
            ? value
            : throw new ArgumentOutOfRangeException(nameof(value), "below absolute zero");
    }

    public string Sensor { get; init; } = "default";

    public required string Site { get; set; }
}
```

listing 34: one class showing field, init, and required in C# 14 style (source: csharp-net/samples/src/Ch04/Members.cs, lines 5-18)

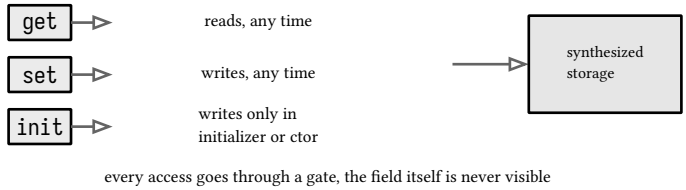


figure 24: a property, one storage behind three accessor gates

4.2. the field keyword

Before C# 14, a property that validated its input needed a hand declared backing field, and the property accessor talked to that field. C# 14 makes the synthesized backing field addressable with the contextual keyword `field` inside an accessor. Validation no longer costs you the auto property, and the backing field cannot drift out of sync with the accessor because there is only one declaration:

```
// pre-14: declare storage yourself
private decimal _celsius;
public decimal Celsius
{
    get ⇒ _celsius;
    set ⇒ _celsius = value ≥ -273.15m ? value
        : throw new ArgumentOutOfRangeException();
```

```
}

// C\# 14: same behavior, one declaration
public decimal Celsius
{
    get;
    set => field = value >= -273.15m ? value
        : throw new ArgumentOutOfRangeException();
}
```

fragment, illustrative, not executed

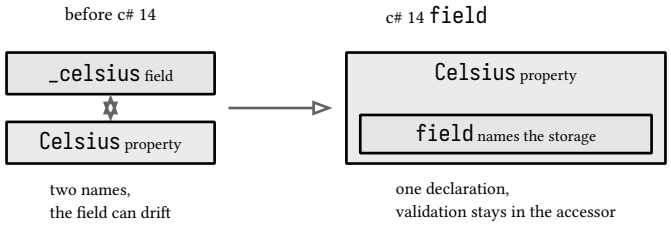


figure 25: backing fields before and after the c# 14 field keyword

The suite drives both sides of the validation through the real property:

```
    public static (decimal Stored, bool Rejected) FieldValidation()
    {
        var t = new Temperature { Site = "north" };
        t.Celsius = 21.5m;
        var rejected = false;
        try
        {
            t.Celsius = -300m;
        }
        catch (ArgumentOutOfRangeException)
        {
            rejected = true;
        }
        return (t.Celsius, rejected);
    }
```

listing 35: valid input stores, invalid input throws (source: csharp-net/samples/src/Ch04/Members.cs, lines 77-91)

4.3. init and required

`init` (C# 9) makes a property settable only in an object initializer or constructor, so a type can be immutable without being awkward to build. `required` (C# 11) closes the other gap: it makes the compiler reject any construction that leaves the member unset. Together they replace most telescoping constructors. Forgetting a required member is a compile error, not a null at runtime:

```
var t = new Temperature();
// error CS9035: required member 'Temperature.Site' must be set
// in the object initializer or attribute constructor
```

fragment, illustrative, not executed

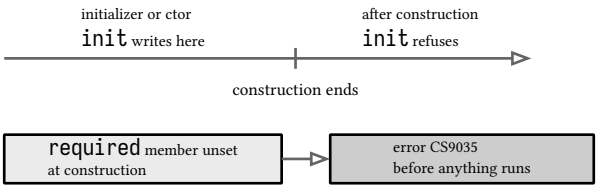


figure 26: the construction timeline, init's window and required's gate

```
    public static string InitDefault() => new Temperature { Site = "north" }.Sensor;

    public static string InitOverridden()
        => new Temperature { Site = "north", Sensor = "north" }.Sensor;
```

listing 36: init defaults survive, initializers override (source: csharp-net/samples/src/Ch04/Members.cs, lines 93-96)

4.4. indexers

An indexer is a property with parameters, invoked with bracket syntax. It turns an object into something indexable without exposing the underlying storage. Indexers can overload on their parameter lists, take any parameter types, and use `get`, `set`, and `init` accessors like properties. There is no auto implemented indexer, storage is always yours:

```
// indexer: array-style access through a member with parameters
public class ReadingLog
{
    private readonly decimal[] _readings = new decimal[7];

    public decimal this[int day]
    {
        get => _readings[day];
        set => _readings[day] = value;
    }
}
```

listing 37: an array backed indexer over days (source: csharp-net/samples/src/Ch04/Members.cs, lines 20-30)

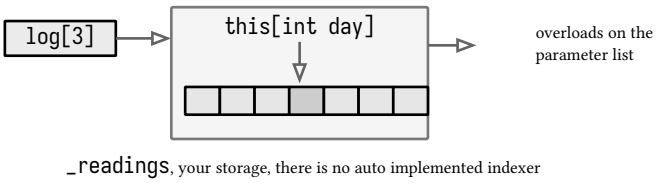


figure 27: an indexer, bracket syntax delegated to private storage

```
    public static decimal IndexerRoundTrip()
    {
        var log = new ReadingLog();
        log[3] = 18.5m;
        return log[3];
    }
```

listing 38: bracket syntax on the calling side (source: csharp-net/samples/src/Ch04/Members.cs, lines 98-103)

4.5. events

An event is a delegate field with restricted access, a delegate being chapter 7's type whose values are methods: external code can only attach and detach handlers with `+=` and `-=`, and only the declaring type can invoke it. That

restriction is the point, it stops subscribers from invoking each other's handlers or clearing the list. Inside the declaring class the event is an ordinary delegate field:

```
// event: a delegate field the outside world can only add and remove from
public class ThresholdMonitor
{
    public event EventHandler<decimal>? Breached;

    public decimal Limit { get; }

    public ThresholdMonitor(decimal limit) => Limit = limit;

    public void Record(decimal value)
    {
        if (value > Limit)
        {
            Breached?.Invoke(this, value); // ?. because no one may have subscribed
        }
    }
}
```

listing 39: an event declared, raised with null conditional invocation (source: csharp-net/samples/src/Ch04/Members.cs, lines 32-48)

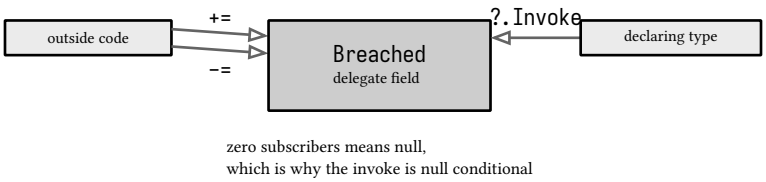


figure 28: an event, one delegate field behind permission walls

The `?.` in `Breached?.Invoke(...)` guards the case where no one subscribed, since an event with no handlers is null. The test subscribes a lambda, records a value under the limit where the handler must stay silent, then records one over it:

```
    public static (bool Fired, decimal Value) EventFiresOverLimit()
    {
        var monitor = new ThresholdMonitor(10m);
        decimal seen = 0m;
        monitor.Breached += (_, value) => seen = value;
        monitor.Record(5m);   // under the limit, handler must not run
        var quiet = seen == 0m;
        monitor.Record(12m);
        return (quiet && seen == 12m, seen);
    }
```

listing 40: subscribing and observing the threshold event (source: csharp-net/samples/src/Ch04/Members.cs, lines 105-114)

Chapter 7 covers delegates, the multicast model, and unsubscription discipline in depth.

4.6. extension members

Extensions add members to a type you do not own, without inheritance and without editing the original. Before C# 14 there was exactly one form, a static method in a static class with `this` on its first parameter. C# 14 adds the `extension` block: a declaration inside a static class that names a receiver once, then declares methods, properties, and operators against it:

```
public static class SequenceExtensions
{
    // the pre-14 form: this modifier on the first parameter
    public static int CountTraditional(this IEnumerable<int> source)
    {
        var count = 0;
        foreach (var _ in source) count++;
        return count;
    }

    // C# 14 extension block: instance members share one receiver
    extension(IEnumerable<int> source)
    {
        public bool IsEmpty => !source.Any();

        public int SecondOrZero() => source.ElementAtOrDefault(1);
    }

    // a second block in the same class extends the type itself
    extension(IEnumerable<int>)
    {
        public static IEnumerable<int> CountingTo(int n) => Enumerable.Range(1, n);
    }
}
```

listing 41: the this form and two C# 14 extension blocks side by side (source: csharp-net/samples/src/Ch04/Members.cs, lines 50-73)

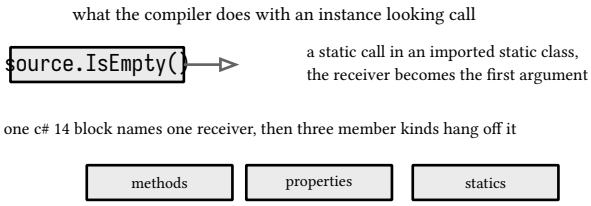


figure 29: extension members, an instance call rewritten to a static one

The three blocks in that listing are worth reading line by line. The `this` form is unchanged since C# 3 and still compiles. The first extension block takes a named receiver, `source`, and everything inside can use it, so `IsEmpty` is a property, something the old form could never express. The second block omits the receiver name and declares a static member, which makes `CountingTo` appear as a static member of `IEnumerable<int>` itself. Calls on the using side look native:

```
    public static (bool Empty, int Second, int Count) ExtensionMembers()
    {
        int[] none = [];
        int[] some = [7, 9];
        return (none.IsEmpty, some.SecondOrZero(), some.CountTraditional());
    }

    public static List<int> StaticExtension() => [.. IEnumerable<int>.CountingTo(3)];
```

listing 42: instance extension property, method, and a static extension in use (source: csharp-net/samples/src/Ch04/Members.cs, lines 116-123)

NOTE: extensions are syntax, not mutation

> Extension members are static calls the compiler resolves at compile time. They cannot see private state, cannot be overridden, and only apply when their containing namespace is imported. A receiver declared `ref` in the block lets an extension mutate a struct receiver in place, which is the one form with by reference semantics.

C# 15 completes the set with extension indexers, indexed access on a type you do not own. Indexers have no name and are always instance members, so the block must name its receiver: `extension(Path path)` gives the indexer body a variable to index through, an unnamed `extension(Path)` does not. Everything else is an ordinary indexer, including overloads on the parameter list:

```
// C# 15: indexers complete the member kinds an extension block can add.
// Indexers are always instance members, so the receiver must be named.
extension(IEnumerable<int> sequence)
{
    public int this[int index] => sequence.ElementAt(index);

    // indexer overloads work like ordinary ones, on the parameter list
    public int this[Index position] => position.IsFromEnd
        ? sequence.ElementAt(sequence.Count() - position.Value)
        : sequence.ElementAt(position.Value);
}
```

listing 43: a C# 15 extension indexer and an overload, one named receiver (source: csharp-net/samples/src/Ch04/ExtensionIndexers.cs, lines 5-15)

Resolution walks the same scopes as extension method calls, and instance indexers win first: arrays and lists keep their own bracket access untouched. The extension indexer binds only where no instance indexer applies, which is why the receiver below is declared `IEnumerable<int>`, a lazy sequence with no indexer of its own:

```
// the receiver is declared IEnumerable<int>, so the extension indexer binds
public static (int Third, int Last) LazyIndexed()
{
    IEnumerable<int> numbers = Enumerable.Range(1, 10); // no indexer of its own
    return (numbers[2], numbers[^1]);
}
```

listing 44: the call site reads as membership, the extension carries it (source: csharp-net/samples/src/Ch04/ExtensionIndexers.cs, lines 20-25)

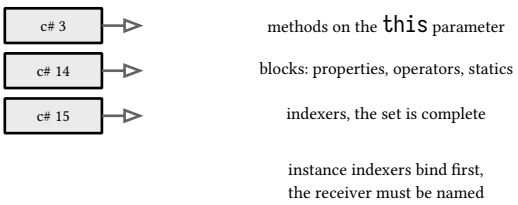


figure 30: extension indexers, the last member kind joins the block

sources: learn.microsoft.com, c# 14 what's new, extension members, extension declaration, field keyword, init, required, indexers, events, and object initializers pages accessed 2026-09-08, c# 15 what's new, extension indexers, and extension declaration pages accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 10 tests.

5. generics

Generics let one declaration work over many types with compile time checking, no casts at use sites, and one copy of the logic. A type parameter like `T` is filled in at each use, producing a **constructed type** such as `List<int>`. Unlike Java's erasure, .NET generics are reified: `List<int>` and `List<string>` are distinct types at runtime, and `typeof(List<int>)` knows its own argument. This chapter covers the three parts that matter: constraints, variance, and static abstract members.

5.1. constraints

A bare `T` can do almost nothing, because the compiler only knows it derives from `object`. A `where` clause narrows T, and every constraint purchases operations. `IComparable<T>` unlocks `CompareTo`. `new()` unlocks construction. `class`, `struct`, `notnull`, `unmanaged`, and base type constraints restrict the argument domain, and a parameterless constructor constraint composes with them:

```
// constraints narrow what T can be; each one unlocks operations on T
public static class Picker
{
    public static T Max<T>(T a, T b) where T : IComparable<T> =>
        a.CompareTo(b) ≥ 0 ? a : b;

    public static List<T> Fresh<T>(int count) where T : new()
    {
        var list = new List<T>(count);
        for (var i = 0; i < count; i++) list.Add(new T());
        return list;
    }
}
```

listing 45: two constraints, each one unlocking an operation (source: csharp-net/samples/src/Ch05/Generics.cs, lines 5-17)

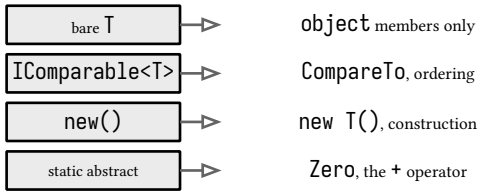


figure 31: each constraint purchases the operations it unlocks

The standard library leans on constraints everywhere, `List<T>` itself enumerates fine as bare `T` but sorts only when the element is `IComparable<T>`.

5.2. variance

Should `IProducer<Cat>` convert to `IProducer<Animal>`? With an invariant type parameter, no: the two constructed types are unrelated even though their arguments are related. Variance is the declaration that makes the conversion legal. Mark a parameter `out` when it only appears in output positions, then everyone producing cats produces animals, which is safe. Mark it `in` when it only appears in input positions, then anything that consumes objects consumes strings:

```
// variance: out parameters produce T, in parameters consume T
public interface IProducer<out T>
{
    T Next();
}

public interface IConsumer<in T>
{
    string Render(T item);
}
```

listing 46: covariant out and contravariant in declared (source: csharp-net/samples/src/Ch05/Generics.cs, lines 19-28)

Both conversions are happening in the sample's callers, through classes that implement the narrow forms:

```
public abstract record Animal(string Name);

public sealed record Cat(string Name) : Animal(Name);

// IProducer<Cat> through IProducer<out T>: covariance needs reference types
public sealed class CatProducer : IProducer<Cat>
{
    private readonly Cat[] _litter = [new("milo"), new("otis")];
    private int _i;

    public Cat Next() => _litter[_i++ % _litter.Length];
}

public sealed class ObjectRenderer : IConsumer<object>
{
    public string Render(object item) => item.ToString() ?? "?";
}
```

listing 47: a cat producer read as an animal producer, an object renderer used as a string consumer (source: csharp-net/samples/src/Ch05/Generics.cs, lines 30-46)

VERIFY: the sample suite hit this rule itself

The first draft of this chapter's sample produced `int` and tried to view it as `IProducer<object>`. The compiler rejected it: variance conversions are reference conversions, and a value type argument makes the constructed type invariant. The fix was the `Cat` to `Animal` hierarchy above. The rule costs nothing in practice, because boxing an `IEnumerable<int>` into an `IEnumerable<object>` would allocate a wrapper and destroy the point.

```
IProducer<object> wide = new IntProducer(7);
// error CS0029: cannot implicitly convert type 'IntProducer'
// to 'IProducer<object>'   value types do not vary
```

fragment, illustrative, not executed

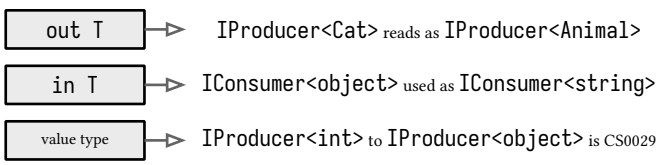


figure 32: variance legality, out and in convert, value types freeze

Variance annotations are only allowed on interfaces and delegates, never on classes or structs. `IEnumerable<out T>`, `Func<in T, out TResult>`, and `Action<in T>` from the standard library are all variant, which is why lambdas and sequences compose across related types without casts.

5.3. static abstract members and generic math

The oldest gap in generics: `T a + T b` did not compile, because operators were static members of concrete types and static members were invisible to type parameters. C# 11 closed it by letting interfaces declare `static abstract` members, including operators. A type parameter constrained to such an interface can then use the operator, and each concrete type supplies its own implementation:

```
// static abstract members (C# 11) put operators on interfaces,
// which lets a type parameter do arithmetic
public interface ICounter<TSelf> where TSelf : ICounter<TSelf>
{
    static abstract TSelf Zero { get; }
    static abstract TSelf operator +(TSelf left, TSelf right);
}

public readonly record struct Tally(int Value) : ICounter<Tally>
{
    public static Tally Zero => new(0);

    public static Tally operator +(Tally left, Tally right) => new(left.Value + right.Value);
}
```

listing 48: a monoid interface: a zero element and a plus operator (source: csharp-net/samples/src/Ch05/Generics.cs, lines 48-61)

With that interface, summation over any complying type is one generic method. `TSelf.Zero` and `+=` both resolve through the constraint:

```
    public static TSelf SumCounters<TSelf>(IEnumerable<TSelf> items)
        where TSelf : ICounter<TSelf>
    {
        var total = TSelf.Zero;         // static abstract member through the parameter
        foreach (var item in items) total += item; // and the operator likewise
        return total;
    }
```

listing 49: generic summation driven entirely by the constraint (source: csharp-net/samples/src/Ch05/Generics.cs, lines 65-71)

The standard library ships this pattern as `System.Numerics`. `INumber<T>` constrains the whole numeric tower, integers, floats, and decimals, so one `Sum` covers them all with no overloads and no boxing:

```
    // generic math: one body over every .NET numeric type
    public static T Sum<T>(IEnumerable<T> values) where T : INumber<T>
    {
        var total = T.Zero;
        foreach (var v in values) total += v;
        return total;
    }
```

listing 50: the same summation over the standard numeric hierarchy (source: csharp-net/samples/src/Ch05/Generics.cs, lines 73-79)

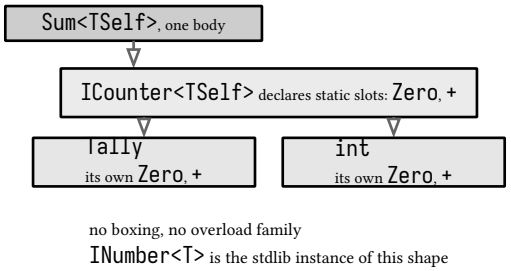


figure 33: generic math, one sum resolved through per-type static slots

NOTE: generic math, not numeric reflection

Before static abstracts, generic numeric code either boxed to `dynamic` or shipped one overload per type. The `INumber<T>` family replaced both. The interpreter capstone uses the same monoid shape to fold values over a visitor, which is the pattern to reach for whenever a generic algorithm needs an identity element and a combine operation.

sources: learn.microsoft.com, generic type parameters, constraints on type parameters, covariance and contravariance in generics, variance in generic interfaces, generic math, and system.numerics pages, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 6 tests.

6. pattern matching and control flow

Control flow in C# is pattern driven. An `if` or `switch` does not merely compare a value against constants, it matches an input against patterns that can test runtime type, deconstruct position, walk properties, and slice sequences, all while binding parts of the input to variables. This chapter covers the full pattern inventory, the loop family, and the null handling operators that remove most guards.

6.1. the three constructs, the ten patterns

Patterns appear in three places: the `is` expression, the `switch` statement, and the `switch` expression. The pattern kinds are: constant (`32`), declaration (`string s`), type (`int[]`), relational (`> 212`), logical (`and`, `or`, `not`), property (`{ Length: 3 }`), positional, `var`, discard (`_`), and list. The property, positional, and list kinds are recursive, any pattern can nest inside them.

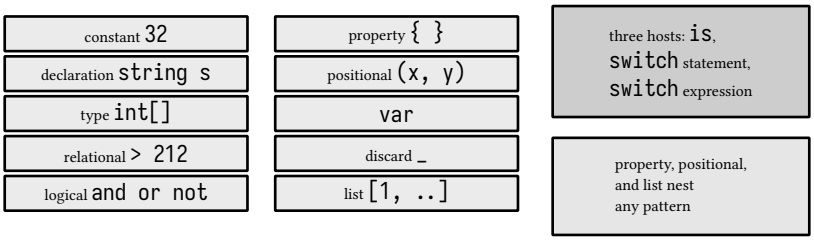


figure 34: the pattern inventory, ten kinds, three hosts, the recursive three

6.2. the switch expression

The switch expression turns selection into an expression that produces a value. Arms are tested top to bottom and the first match wins:

```
// switch expression with relational, constant, and discard patterns
public static string WaterState(int fahrenheit) => fahrenheit switch
{
    < 32 => "solid",
    32 => "melting",
    < 212 => "liquid",
    212 => "boiling",
    _ => "gas",
};
```

listing 51: relational, constant, and discard arms in one switch expression (source: csharp-net/samples/src/Ch06/Flow.cs, lines 16-24)

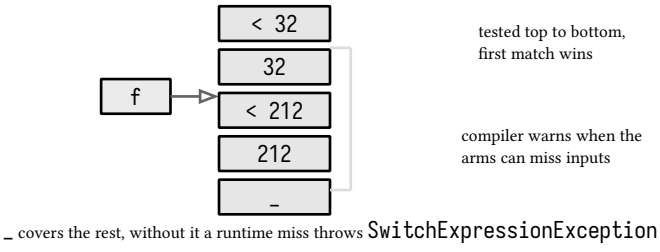


figure 35: switch expression arms, top to bottom, first match wins

The compiler warns when the arms do not cover all possible inputs, which is why explicit boundary arms like `32` and `212` exist even where the relational arms would return the same string. If no arm matches at runtime, the expression throws `SwitchExpressionException`, so an exhaustive arm set or a discard is not cosmetic.

A `when` clause guards an arm with an arbitrary boolean expression when the pattern alone cannot express the condition:

```
var label = reading switch
{
    { Celsius: < 0 } r when r.Sensor == "north" => "frozen north",
    { Celsius: < 0 } => "frozen",
    _ => "ok",
};
```

fragment, illustrative, not executed

6.3. switching over unions without a fallback

The union from chapter 3 hands this machinery a complete domain. A switch over `Pet` that names every case is exhaustive, the compiler accepts it with no discard arm, and the `_ =>` habit that silently swallowed future cases is gone:

```
// C# 15: exhaustive over the chapter 3 Pet union, no discard arm
public static string PetLabel(Ch03.Pet pet) => pet switch
{
    Ch03.Cat c => $"cat {c.Name}",
    Ch03.Dog d => $"dog {d.Name}",
    Ch03.Bird b => $"bird {b.Name}",
};
```

listing 52: every case named, no fallback arm (source: csharp-net/samples/src/Ch06/Flow.cs, lines 124-131)

Leave a case out and the compiler says so. Dropping the `Bird` arm from that switch produces warning CS8509, verbatim: The switch expression does not handle all possible values of its input type (it is not exhaustive). For example, the pattern 'CsharpBook.Samples.Ch03.Bird' is not covered. The message names the missing case by its fully qualified name. It is a warning, so the build still succeeds, and the failure moves to runtime: feed the switch a bird and it throws `System.Runtime.CompilerServices.SwitchExpressionException`. That combination, clean build plus a value that crashes later, is why CS8509 should not stay a warning:

```
// deliberately missing the Bird arm: the runtime miss is the point.
// CS8509 is suppressed here only, the chapter callout explains why.
#pragma warning disable CS8509
public static string PetLabelWithoutBird(Ch03.Pet pet) => pet switch
{
    Ch03.Cat c => $"cat {c.Name}",
    Ch03.Dog d => $"dog {d.Name}",
};
#pragma warning restore CS8509
```

listing 53: the bird arm deliberately missing, CS8509 suppressed at this one spot (source: csharp-net/samples/src/Ch06/Flow.cs, lines 132-140)

WARNING: promote CS8509 to an error

The samples suite compiles with zero warnings because its switches are exhaustive, with one exception: the helper above exists to demonstrate the runtime failure, so it suppresses CS8509 on that one method and the test suite watches the throw. CI should do the opposite of that pragma, `<WarningsAsErrors>CS8509</WarningsAsErrors>` in the project file or `dotnet build -warnaserror:CS8509`, so a missed case can never ride a green build into production.

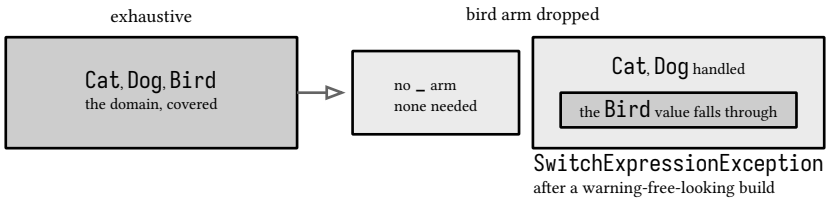


figure 36: exhaustive versus non exhaustive, where the unmatched value lands

6.4. property patterns

A property pattern matches when the input is non-null and every nested pattern matches the corresponding property. Extended property patterns walk with dots, and the `or` combinator joins alternatives:

```
// property pattern with an extended path and a logical or pattern
public static bool IsNightPeak(Reading r) =>
    r is { Sensor: "north", Hour: > 22 or < 4, Celsius: > 30m };
```

listing 54: an extended property pattern with a logical or inside (source: csharp-net/samples/src/Ch06/Flow.cs, lines 26-28)

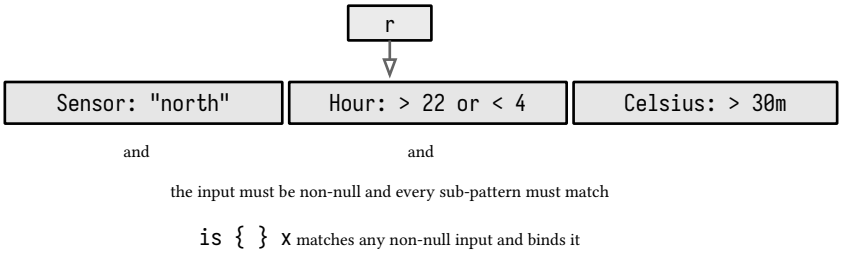


figure 37: a property pattern, every nested sub-pattern must match

The empty property pattern `is { } x` matches anything non-null and binds it, a compact form of the not-null check that also produces a variable.

6.5. positional patterns

A positional pattern calls the type's `Deconstruct` method and matches the results against nested patterns. Records deconstruct for free, so a record becomes a set of coordinates you can branch on:

```
// positional pattern: deconstructs the record, matches nested patterns
public static string Quadrant(Vector v) => v switch
{
    (0, 0) => "origin",
    ( > 0, > 0) => "northeast",
    ( < 0, > 0) => "northwest",
    ( < 0, < 0) => "southwest",
    ( > 0, < 0) => "southeast",
    (_, 0) => "x axis",
    _ => "y axis",
};
```

listing 55: a plane split by nested relational patterns (source: csharp-net/samples/src/Ch06/Flow.cs, lines 30-40)

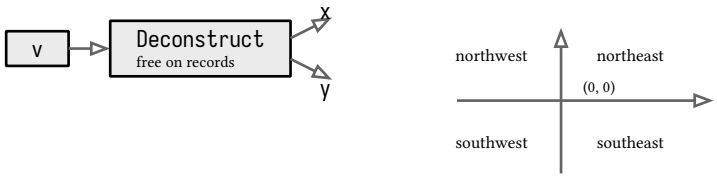


figure 38: positional patterns, deconstructed coordinates split the plane

6.6. list patterns

A list pattern matches an array or list element by element. A slice `..` absorbs zero or more elements and can itself capture with `var`:

```
// list pattern with a slice capturing the middle
public static string DescribeSequence(int[] xs) => xs switch
{
    [] => "empty",
    [var only] => $"one element: {only}",
    [var first, .. var middle, var last] =>
        $"first {first}, middle {middle.Length}, last {last}",
};
```

listing 56: empty, single, and first-middle-last shapes (source: csharp-net/samples/src/Ch06/Flow.cs, lines 42-49)

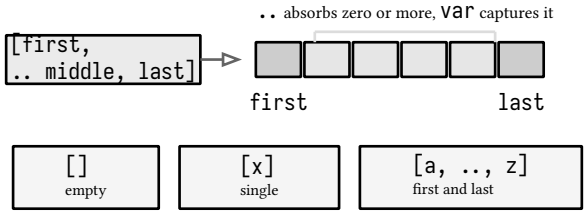


figure 39: list patterns, element wise with a slice in the middle

6.7. one switch, several patterns

The classifier below mixes constant, declaration, type, property, and var patterns in a single expression. Note the `int[] { Length: var n }` arm, a type pattern extended with a property pattern that binds through a var pattern:

```
// declaration, constant, type, and property-with-var patterns in one place
public static int KindOf(object? o) => o switch
{
    null => -1,
    string s => s.Length,
    int[] { Length: var n } => n,
    _ => -2,
};
```

listing 57: constant, declaration, type, and nested var patterns (source: csharp-net/samples/src/Ch06/Flow.cs, lines 51-58)

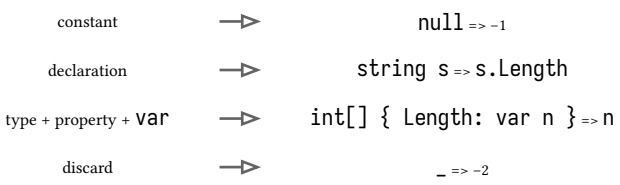


figure 40: one switch expression, four pattern kinds composed

NOTE: pattern combinators have precedence

`not` binds before `and`, which binds before `or`. The pattern `not ≥ 'a' and ≤ 'z'` parses as `(not ≥ 'a') and ≤ 'z'` and never matches what you meant. Parenthesize the range: `not (≥ 'a' and ≤ 'z')`. The compiler cannot help you here, the wrong version is legal code.

6.8. the switch statement and goto

The `switch` statement is the older form, a statement per arm with no fall-through between sections. Multiple labels may share one section, and `goto case` moves control deliberately when a section should continue into another:

```
switch (shape.Kind)
{
    case Kind.Circle:
    case Kind.Ellipse:        // two labels, one section
        return Round(shape);
    case Kind.Path when shape.Points > 100:
        goto case Kind.Polygon;  // explicit fall-through
    case Kind.Polygon:
        return Triangulate(shape);
    default:
        return shape;
}
```

fragment, illustrative, not executed

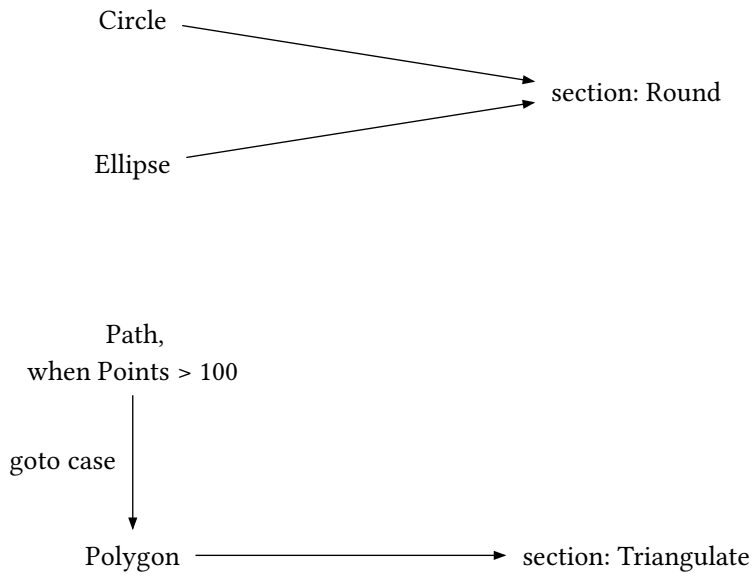


figure 41: switch statement sections, shared labels and the one explicit goto

New code prefers the switch expression, which cannot fall through at all. Reach `if` and the ternary `?:` before reaching for a switch when there are one or two cases.

6.9. loops

The four loops. `for` when an index is needed, `foreach` over anything enumerable, `while` tests before running, `do` tests after and so runs at least once:

```
// for with continue, foreach with break
public static (int EvenSum, int FirstOver) LoopKinds(int[] xs)
{
    var evenSum = 0;
    for (var i = 0; i < xs.Length; i++)
    {
        if (xs[i] % 2 == 1) continue; // skip the odds
        evenSum += xs[i];
    }

    var firstOver = -1;
    foreach (var x in xs)
    {
        if (x > 50)
        {
            firstOver = x;
            break; // stop scanning at the first hit
        }
    }
    return (evenSum, firstOver);
}
```

listing 58: for with continue, foreach with break (source: csharp-net/samples/src/Ch06/Flow.cs, lines 60-80)

```
public static int Factorial(int n)
{
    var product = 1;
    var i = n;
    while (i > 1)
    {
        product *= i;
        i--;
    }
    return product;
}
```

listing 59: while counts down (source: csharp-net/samples/src/Ch06/Flow.cs, lines 82-92)

```
// the body runs before the condition is ever tested
public static int DoWhileAlwaysRunsOnce(int threshold)
{
    var runs = 0;
    do
    {
        runs++;
    } while (runs < threshold);
    return runs;
}
```

listing 60: do while runs before it checks (source: csharp-net/samples/src/Ch06/Flow.cs, lines 94-103)

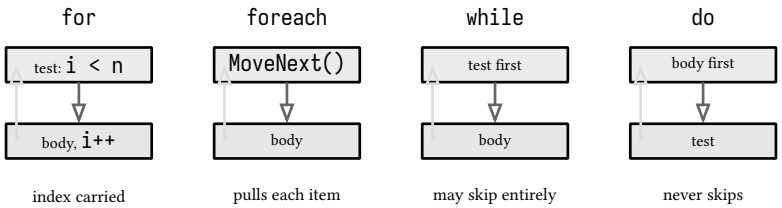


figure 42: four loop shapes, where the test sits in each

goto also jumps to a label and cannot jump into a block, only out of one. For escaping nested loops, the labeled jumps below have retired it, goto case in a switch statement is its remaining everyday use.

6.10. labeled break and continue

C# 15 lets break and continue name their target. The label sits directly on an enclosing loop, rows: foreach (...), and the jump names it: continue rows; starts the next iteration of that loop, break rows; leaves it and everything nested inside it. The two workarounds this replaces are the Boolean flag copied out level by level and the goto aimed past the loops, and the IDE0410 style rule flags both:

```
// C# 15 labeled jumps: continue rows abandons a row, break rows ends the scan
public static (int Sum, int RowsSeen) LabeledJumps(int[][] grid)
{
    var sum = 0;
    var rowsSeen = 0;
    rows: foreach (var row in grid)
    {
        rowsSeen++;
        foreach (var cell in row)
        {
            if (cell < 0) continue rows;
            if (cell > 100) break rows;
            sum += cell;
        }
    }
    return (sum, rowsSeen);
}
```

listing 61: continue rows abandons a row, break rows ends the whole scan (source: csharp-net/samples/src/Ch06/Flow.cs, lines 142-158)

The rules are small. break can name an enclosing loop or switch statement, continue names loops only, because starting the next iteration of a switch is meaningless. Only the statement immediately under a label carries it, a: b: while (...) labels the while as b, so break a; inside finds no target. Naming a label with no enclosing loop or switch to break is error CS9393, no enclosing loop to continue is CS9394. The label can sit on any of the four loop kinds, the second sample names a foreach and jumps from a while nested two levels down:

```
    // the label names a foreach, the jump comes from a nested while
    public static (int Count, int Rows) QuietCount(int[][] samples, int limit)
    {
        var count = 0;
        var rows = 0;
        scan: foreach (var row in samples)
        {
            rows++;
            var i = 0;
            while (i < row.Length)
            {
                if (row[i] > limit) break scan;
                count++;
                i++;
            }
        }
        return (count, rows);
    }
}
```

listing 62: the label names a foreach, the break comes from a nested while (source: csharp-net/samples/src/Ch06/Flow.cs, lines 160-178)

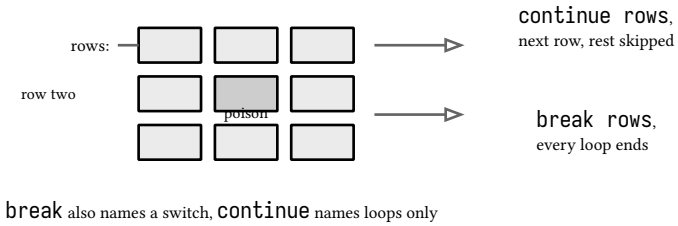


figure 43: labeled exits, break rows leaves, continue rows advances

6.11. the null operators

Four operators carry most null handling. ?. accesses a member only when the receiver is non-null, short-circuiting the rest of the chain. ?[] is the same for element access. ?? yields the right operand when the left is null. ??= assigns only when the left is null:

```
    // null conditional indexer: null array in, null int? out
    public static int? ElementOrNull(int[]? xs, int index) => xs?[index];
```

listing 63: null conditional indexer access (source: csharp-net/samples/src/Ch06/Flow.cs, lines 105-106)

```
    // ??= assigns only when the left side is null
    public static List<int> LazilyFilled(List<int>? cache)
    {
        cache ??= [];
        cache.Add(1);
        return cache;
    }
```

listing 64: coalescing assignment as lazy init (source: csharp-net/samples/src/Ch06/Flow.cs, lines 108-114)

C# 14 extends the family with null conditional assignment, making ?. and ?[] valid assignment targets. The right side evaluates only when the receiver is non-null, so the guard and the write are one statement:

```
    // ?. chains, ?? fallback, and C# 14 null conditional assignment
    public static (string Theme, int Length) NullChains(Chain? link)
    {
        var theme = link?.Name ?? "anon";
        link?.Name = "set"; // no-op when link is null, C# 14
        return (theme, link?.Name?.Length ?? -1);
    }
```

listing 65: chains, fallbacks, and C# 14 conditional assignment (source: csharp-net/samples/src/Ch06/Flow.cs, lines 116-122)

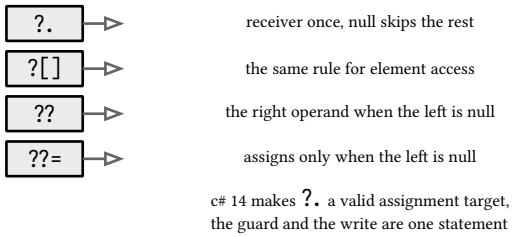


figure 44: the null operators, what each short circuits

NOTE: why ?. exists

`a?.M()` evaluates `a` once and cannot be rewritten as a plain check when `a` is a field another thread may write to between the test and the call. That single evaluation rule is also what makes `handler?.Invoke(...)` the thread safe way to raise an event, chapter 7 covers that use.

sources: learn.microsoft.com, patterns, switch expression, selection statements, iteration statements, member access operators, null coalescing operator, and c# null operators pages accessed 2026-09-08, labeled break and continue, jump statements, and jump statement errors pages accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 29 tests.

7. delegates, lambdas, events

A delegate is a type whose values are methods. Given a delegate value, you can call it, store it, pass it as an argument, combine several into a chain, and remove them again. Delegates are the mechanism under events, LINQ, callbacks, and most async APIs, which makes this the chapter that explains how the rest of the library composes.

7.1. declaring and invoking

A delegate declaration introduces a signature. Instances hold methods matching that signature, and invoking the delegate calls whatever it holds. Assignment from a bare method name is a **method group conversion**, and it works for static and instance methods alike:

```
// a custom delegate type names a method shape
public delegate int Transformer(int input);
```

listing 66: a custom delegate type is one line (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 3-4)

```
    public static (int First, int Second) MethodGroupAndCall()
    {
        Transformer t = Square;          // method group conversion
        var first = t(5);                // invoked like a method
        t = Increment;                   // delegates are immutable, t is rebound
        return (first, t(5));
    }
```

listing 67: binding, invoking, and rebinding a delegate (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 12-18)

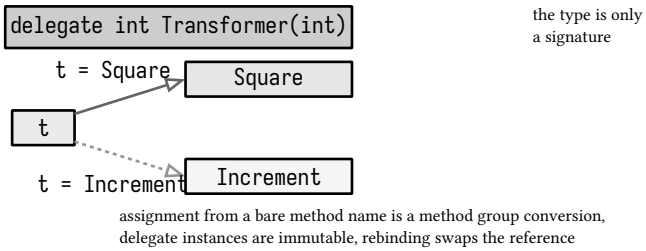


figure 45: a delegate type, a bound method group, and a rebinding

Delegate instances are immutable. Reassigning `t` swaps which method the variable refers to, it does not modify any delegate object.

7.2. Func, Action, and lambdas

Custom delegate types are rarely needed. The library ships `Func<...>` for methods that return a value, with the last type argument as the return type, and `Action<...>` for methods that return void, each family covering 0 to 16 parameters. A lambda expression creates the delegate inline, and the compiler checks the lambda against the target type:

```
public static (int Bumped, List<string> Log) FuncAndAction()
{
    var log = new List<string>();
    Func<int, int, int> add = (left, right) => left + right;
    Action<string> record = message => log.Add(message); // captures log
    record($"{add(2, 3)}");
    Func<int, int> bump = static x => x + 1; // static lambda, no captures
    return (bump(41), log);
}
```

listing 68: Func, Action, a capturing lambda, and a static lambda (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 20-28)

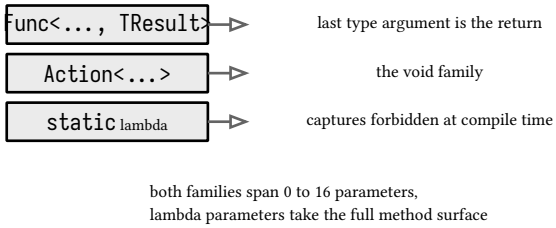


figure 46: the two delegate families and the lambda surface

The `static` modifier on a lambda forbids captures entirely. When a helper needs no outside state, declaring it static documents that and catches accidental captures at compile time. Lambda parameters can also be declared `ref`, `out`, or `in`, take default values, and use `params` collections, the full method parameter surface.

7.3. closures and captured variables

A lambda that references an outer variable captures the variable itself, not its current value. The pair of lambda and captured variables is a closure. Three lambdas sharing one counter see each other's increments:

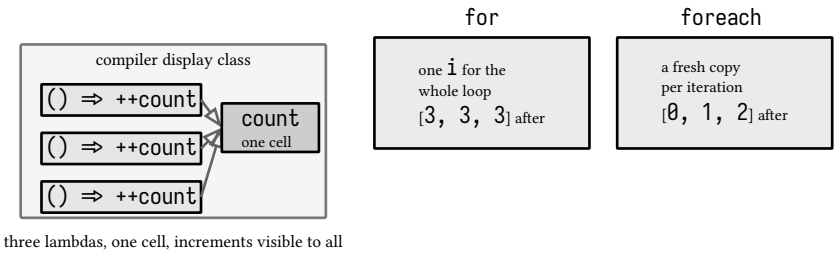
```
public static List<int> CapturedVariableIsShared()
{
    var count = 0;
    var clicks = new List<Func<int>> { () => ++count, () => ++count, () => ++count };
    return [clicks[0](), clicks[1](), clicks[2]()]; // 1, 2, 3: one variable
}
```

listing 69: three closures, one shared variable (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 30-35)

The classic trap is loop capture. A `for` loop declares a single variable reused across iterations, so closures created inside the loop all observe the final value when invoked after the loop. A `foreach` loop by contrast introduces a fresh variable per iteration, exactly because of this trap:

```
public static List<int> ForLoopSharesItsVariable()
{
    var fs = new List<Func<int>>();
    for (var i = 0; i < 3; i++)
    {
        fs.Add(() => i);            // every closure captures the same i
    }
    return [fs[0](), fs[1](), fs[2]()]; // 3, 3, 3: the loop has finished
}
```

listing 70: the for loop variable is one variable (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 37-45)



three lambdas, one cell, increments visible to all

figure 47: closures, one hoisted cell shared, the loop capture trap

PITFALL: capture rules that bite

A lambda cannot capture `ref`, `out`, or `in` parameters of the enclosing method, and captured variables stay alive as long as any delegate referencing them is alive, which is how event subscriptions cause memory leaks. When a closure outlives its scope, you have a leak, and unsubscription is the fix.

7.4. multicast

Delegates are multicast: `+` chains two delegates of the same type into one whose invocation list is both, invoked in order. `-` removes by instance, which is why handlers stored for later removal must be kept in variables, an inline lambda can never be removed because no reference to it survives. The return value of a multicast invocation is the last entry's return, earlier returns are discarded:

```
public static (string Calls, int LastReturn) MulticastCombineAndRemove()
{
    var calls = "";
    Transformer a = x => { calls += "a"; return x + 1; };
    Transformer b = x => { calls += "b"; return x * 10; };
    Transformer both = a + b;        // one delegate, two entries
    var last = both(4);              // runs a then b, result is b's return
    Transformer? rest = both - a;    // removal needs the same instance
    rest?.Invoke(4);                 // subtraction can yield null, guard it
    return (calls, last);
}
```

listing 71: combine, invoke in order, remove by instance (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 47-57)

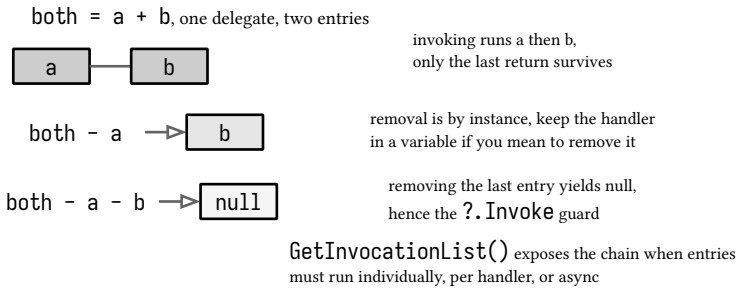


figure 48: the multicast chain, combine, invoke in order, remove by instance

Subtraction deserves the `?.Invoke` guard in that listing: removing the last entry produces a null delegate, and invoking null throws. The `GetInvocationList()` method exposes the chain as an array when you need to invoke entries individually, catch per-handler exceptions, or run them asynchronously, which is the standard eventing pattern for libraries.

7.5. delegates as parameters

A delegate parameter is how a method accepts behavior. The `Map` helper below takes a transformation, callers pass a lambda or a method group interchangeably:

```
public static int[] Map(int[] xs, Func<int, int> f)
{
    var result = new int[xs.Length];
    for (var i = 0; i < xs.Length; i++) result[i] = f(xs[i]);
    return result;
}
```

listing 72: a transformation passed as a delegate (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 59-64)

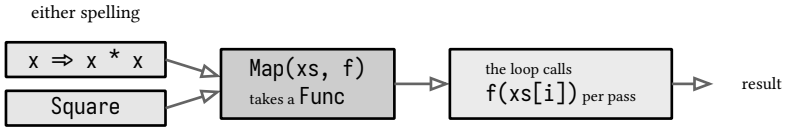


figure 49: behavior as data, one map over any transformation

When a signature carries parameters the body ignores, discards name that choice at the use site:

```
public static List<string> DiscardParameters()
{
    var seen = new List<string>();
    Action<int, int, string> onStatus = (_, _, message) => seen.Add(message);
    onStatus(200, 3, "ok");
    return seen;
}
```

listing 73: discard parameters in a lambda (source: csharp-net/samples/src/Ch07/Delegates.cs, lines 66-72)

7.6. events, the restricted multicast

Chapter 4 introduced events as members. The machinery underneath is this chapter: `event` wraps a delegate field so that outside code can only `+=` and `-=`, which are `Delegate.Combine` and `Delegate.Remove`. Subscribing with `+=` adds your handler to the invocation list, unsubscribing with `-=` removes it by reference, which is why instance methods must be unsubscribed with the same method group and lambdas must be stored to be removable. Raising an event is invoking the underlying multicast delegate, guarded with `?.` because an event with no subscribers is null.

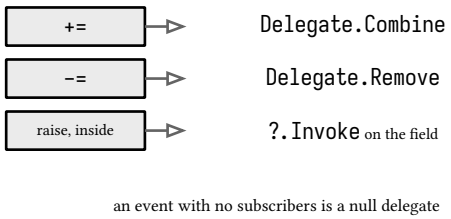


figure 50: event operations, the two outside verbs and the guarded raise

7.7. local functions

A local function is a method declared inside another method. It can do everything a lambda can and recurses naturally, since it can refer to itself before assignment. Lambdas are converted to delegates at declaration, local functions only when used as delegates, so a local function that is only called directly can avoid the closure allocation a lambda always pays:

```
int Fact(int n) => n <= 1 ? 1 : n * Fact(n - 1); // local function

// lambda equivalent needs the variable first
Func<int, int> fact = null!;
fact = n => n <= 1 ? 1 : n * fact(n - 1);
```

fragment, illustrative, not executed

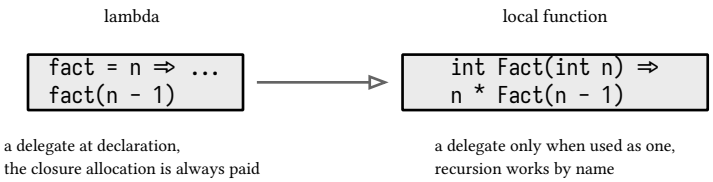


figure 51: lambda vs local function, when the delegate is created

sources: learn.microsoft.com, delegates, lambda expressions, delegates with named versus anonymous methods, how to combine delegates, system.delegate, and local functions pages, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 7 tests.

8. exceptions

Exceptions are the language's mechanism for failures a method cannot handle locally: bad arguments, broken invariants, io errors, cancelled operations. The design rule that keeps them useful is narrow: throw when a caller further up can make a better decision, catch only what you can actually handle, and never let an expected outcome become an exception.

8.1. throwing

`throw` takes any expression convertible to `System.Exception`. The standard library covers most needs, argument problems throw `ArgumentException` and its subclasses, state problems throw `InvalidOperationException`, and .NET 6 added one line guards for the common argument checks:

```
    // the modern argument guards, one line each
    public static decimal RateFor(string? currency)
    {
        ArgumentNullException.ThrowIfNull(currency);
        ArgumentException.ThrowIfNullOrEmpty(currency);
        return currency.Length;
    }
```

listing 74: ThrowIfNull and ThrowIfNullOrEmpty replace four line guard clauses (source: csharp-net/samples/src/Ch08/Errors.cs, lines 105-111)

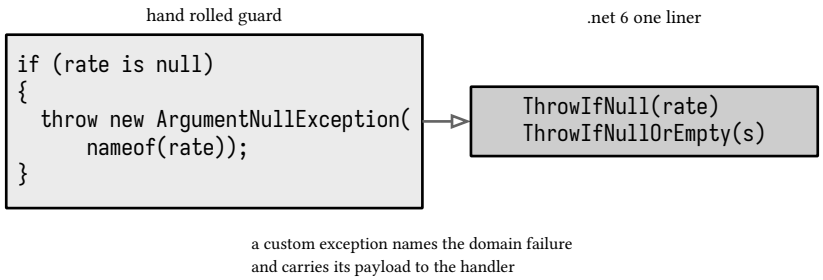


figure 52: the guard clause, four lines to one

When a failure is specific to your domain, name it. A custom exception is a small class carrying the data the handler needs:

```
// a custom exception: name the domain failure, carry the payload
public sealed class TransferException(string message, decimal amount)
    : Exception(message)
{
    public decimal Amount { get; } = amount;
}
```

listing 75: a custom exception carrying its payload (source: csharp-net/samples/src/Ch08/Errors.cs, lines 7-12)

```
    public static void Reject(decimal amount)
        => throw new TransferException("transfer rejected", amount);
```

listing 76: throwing it (source: csharp-net/samples/src/Ch08/Errors.cs, lines 113-114)

`throw` is also an expression, valid on the right side of `?:`, `??`, and in lambda bodies, which is how the guard clauses above are implemented.

8.2. try, catch, finally

`try` wraps the code that might fail. Catch clauses are examined top to bottom and at most one runs, most derived type first. `finally` runs on every exit path, normal completion, a `return` or `break` out of the block, or an exception propagating. A `try` block must have at least one catch or a finally:

```
public static string FinallyOrder(List<string> trace, bool throwIt)
{
    try
    {
        trace.Add("try");
        if (throwIt) throw new InvalidOperationException("bang");
    }
    finally
    {
        trace.Add("finally");
    }
    trace.Add("after");
    return string.Join(",", trace);
}
```

listing 77: finally runs before the exception escapes (source: csharp-net/samples/src/Ch08/Errors.cs, lines 90-103)

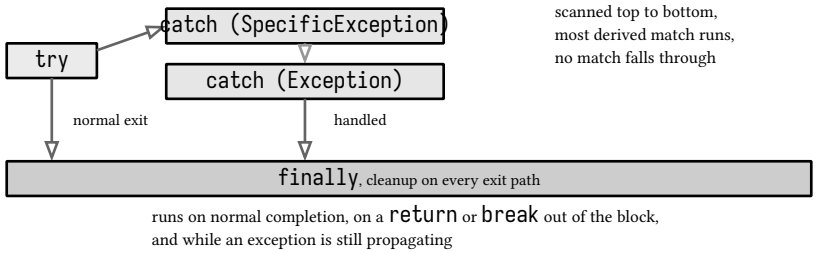


figure 53: try catch finally, the scan and the guaranteed bar

The test suite checks both paths: the happy path appends `after` after `finally`, the throwing path never reaches `after` but the `finally` entries land before the exception surfaces to the caller. `finally` is for cleanup, with the `using` statement covered in chapter 17 as the usual better tool. Only process termination skips it.

8.3. exception filters

A `when` clause between the catch type and the block adds an arbitrary boolean condition. The filter runs **before the stack unwinds**, so locals at the throw site are still alive in the debugger, and a filter that returns false leaves the exception propagating untouched, same type, same stack, as if the clause did not exist:

```
// two catch clauses for one type, split by a when filter
public static string Filtered(int status)
{
    try
    {
        ThrowFor(status);
        return "none";
    }
    catch (HttpRequestException e) when ((int?)e.StatusCode ≥ 500)
    {
        return "server";
    }
    catch (HttpRequestException e) when ((int?)e.StatusCode ≥ 400)
    {
        return "client";
    }
}

private static void ThrowFor(int status)
{
    if (status < 400) return; // 2xx is not an error
    throw new HttpRequestException($"http {status}", null, (HttpStatusCode)status);
}
```

listing 78: two filters on one exception type, plus the thrower (source: csharp-net/samples/src/Ch08/Errors.cs, lines 16-38)

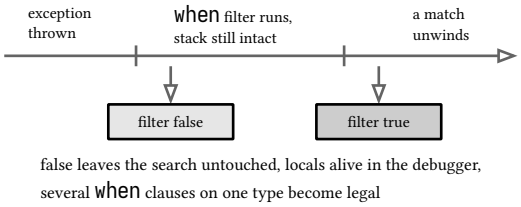


figure 54: the when filter, evaluated before the stack unwinds

Filters make several catch clauses for one type legal, and they are the honest way to write “catch this except when it is cancellation”, which appears in almost every async pipeline:

```
catch (Exception e) when (e is not OperationCanceledException)
{
    Log(e);
    throw; // cancellation is not an error, let it fly
}
```

fragment, illustrative, not executed

8.4. rethrowing, the difference one word makes

Inside a catch block, `throw;` rethrows the caught exception with its original stack trace. `throw e;` rethrows the same instance but resets the stack trace to the current frame, erasing the origin. The suite asserts the difference on both forms, with `NoInlining` so the frame survives optimized builds:

```
    [MethodImpl(MethodImplOptions.NoInlining)]
    private static void OriginOfTrouble() => throw new InvalidOperationException("origin");

    // bare throw: same instance, original stack trace preserved
    public static (bool SameInstance, bool StackNamesOrigin) BareRethrow()
    {
        try
        {
            OriginOfTrouble();
        }
        catch (Exception caught)
        {
            try
            {
                throw;
            }
            catch (Exception again)
            {
                return (ReferenceEquals(caught, again),
                    again.StackTrace?.Contains(nameof(OriginOfTrouble)) ?? false);
            }
        }
        throw new UnreachableException();
    }
```

listing 79: bare throw keeps the origin in the stack trace (source: csharp-net/samples/src/Ch08/Errors.cs, lines 40-63)

```
    // throw e: same instance, but the stack trace restarts here.
    // CA2200 is suppressed because demonstrating this anti-pattern is the point.
#pragma warning disable CA2200
    public static (bool SameInstance, bool StackNamesOrigin) RethrowByValue()
    {
        try
        {
            OriginOfTrouble();
        }
        catch (Exception caught)
        {
            try
            {
                throw caught;
            }
            catch (Exception again)
            {
                return (ReferenceEquals(caught, again),
                    again.StackTrace?.Contains(nameof(OriginOfTrouble)) ?? false);
            }
        }
        throw new UnreachableException();
    }
#pragma warning restore CA2200
```

listing 80: throw e keeps the instance but loses the origin (source: csharp-net/samples/src/Ch08/Errors.cs, lines 65-88)

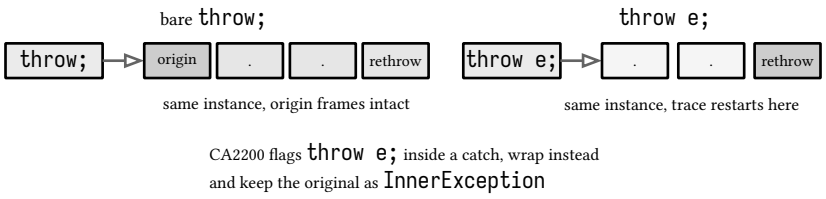


figure 55: rethrowing, one word decides whether the origin survives

PITFALL: the analyzer watches this too

CA2200 flags `throw e;` in a catch block for exactly this reason, and the second listing carries its suppression as documentation. When you must wrap rather than rethrow, keep the original as `InnerException`: `throw new TransferException("context", amount, caught);` so the chain stays walkable.

8.5. expected failure is not an exception

Exceptions are for the exceptional. Parsing untrusted input, probing a cache, reading an optional setting, these fail routinely, and the library reflects that with `TryParse`, `TryGetValue`, and `TryAdd` pairs that return a bool and push the value through an `out` parameter, `out` being the by-reference parameter the callee must assign, chapter 10 has the family:

```
// expected failure is a bool out, not an exception
public static int ParseCount(string? s) => int.TryParse(s, out var n) && n ≥ 0 ? n : 0;
```

listing 81: TryParse turns a throwing parse into a branch (source: csharp-net/samples/src/Ch08/Errors.cs, lines 116-117)

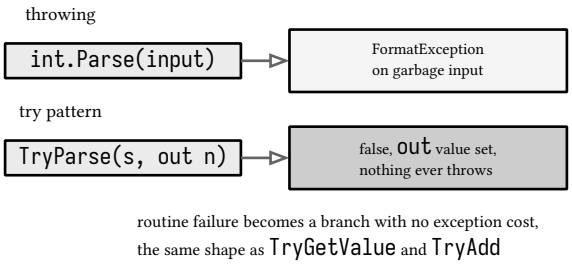


figure 56: two error models, throwing parse vs try pattern

The suite drives that line with valid input, garbage, a negative number, and null. None of them throw. Chapter 15 collects this into the wider discipline of choosing between error models.

sources: learn.microsoft.com, exception handling statements, creating and throwing exceptions, exception handling programming guide, and ca2200 pages, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 14 tests.

9. async

The async model in C# is cooperative single thread handoff. An `await` marks a point where the method may suspend: it registers the rest of the method as a continuation, returns control to the caller immediately, and resumes on completion, by default back on the same synchronization context or thread pool work item. A synchronization context is a queue-and-post abstraction binding continuations to an environment, the UI thread being the classic one, and console apps have none, so their continuations ride the thread pool. The compiler builds this from your code as a state machine, so async methods read sequentially and run concurrently where it matters.

9.1. async, await, Task

`async` on a method enables `await` inside it. The return type describes the future: `Task` for no value, `Task<T>` for one, `ValueTask<T>` as an allocation avoiding refinement, and `IAsyncEnumerable<T>` for a stream of values over time:

```
public static async Task<int> SumAfterAsync(int a, int b)
{
    await Task.Delay(1);            // suspension point, control returns to caller
    return a + b;
}
```

listing 82: the smallest useful async method (source: csharp-net/samples/src/Ch09/AsyncWork.cs, lines 26-30)

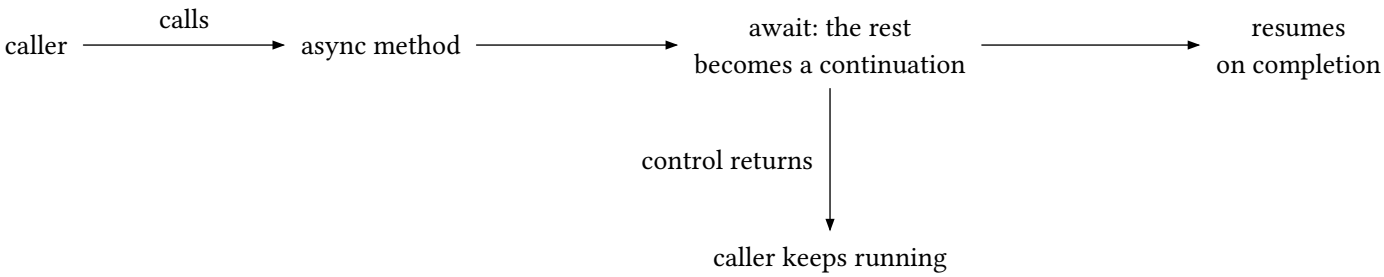


figure 57: await as suspension, one method lowered to a state machine

Awaiting a `Task<T>` yields the `T`. Awaiting a faulted task rethrows the original exception at the await site, which means async code needs no special error plumbing:

```
public static async Task<int> FailAsync()
{
    await Task.Delay(1);
    throw new InvalidOperationException("no result");
}
```

listing 83: the throw site and the await site (source: csharp-net/samples/src/Ch09/AsyncWork.cs, lines 84-88)

9.2. concurrency with WhenAll

Independent awaits run one at a time if you await them in sequence. `Task.WhenAll` starts them together and completes when every task does, returning the results in input order no matter which finished first. The sample schedules the digits so the last one completes first, then asserts the order anyway:

```
    private static async Task<int> DigitAfterDelayAsync(int digit, int delayMs)
    {
        await Task.Delay(delayMs);
        return digit;
    }

    // the digits are scheduled so 3 finishes first; results stay in input order
    public static async Task<int[]> DigitsAsync()
    {
        var tasks = new List<Task<int>>
        {
            DigitAfterDelayAsync(1, 30),
            DigitAfterDelayAsync(2, 20),
            DigitAfterDelayAsync(3, 1),
        };
        return await Task.WhenAll(tasks);
    }
```

listing 84: three tasks, reverse completion order, input order results (source: csharp-net/samples/src/Ch09/AsyncWork.cs, lines 32-48)

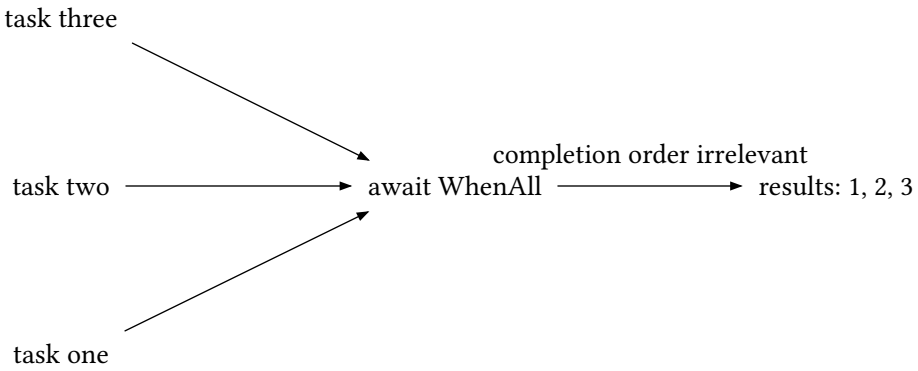


figure 58: WhenAll fans in, one await point, results in input order

`Task.WhenAny` is the sibling for races: it completes when the first task does and is the base for timeout patterns. One caution that follows from WhenAll's error model: when several tasks fail, await throws only the first exception. `task.Exception.InnerExceptions` holds the rest.

9.3. ValueTask

Every `Task<T>` you return that is not already running is a heap allocation. When a method often completes synchronously, a cache hit, a validation failure, `ValueTask<T>` lets the synchronous path return a value with no allocation and only pay for a Task on the slow path:

```
// ValueTask demo: a read-through cache that only pays for a Task on a miss
public sealed class SlowCounter
{
    private readonly Dictionary<int, int> _hits = [];

    public int Misses { get; private set; }

    public async ValueTask<int> ReadAsync(int key)
    {
        if (_hits.TryGetValue(key, out var hit))
        {
            return hit;                 // completes synchronously, no allocation
        }
        Misses++;
        await Task.Delay(1);            // the slow path stands in for real io
        return _hits[key] = key * 10;
    }
```

listing 85: a read through cache: hits return a value, misses return a task (source: csharp-net/samples/src/Ch09/AsyncWork.cs, lines 5-21)

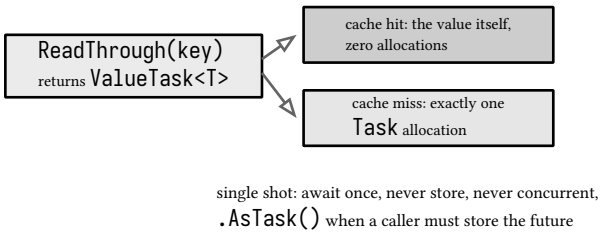


figure 59: valuetask, one allocation free fast path and one slow path

WARNING: ValueTask is single shot

Unlike Task, a ValueTask may only be awaited once, never stored and never concurrently awaited, because the fast path result lives on the stack. If a caller needs to await twice or store the future, convert with `.AsTask()`. The book's rule: default to Task, switch to ValueTask where a profile shows allocation pressure on a hot synchronous path.

9.4. cancellation

Cooperative cancellation is a `CancellationToken` threaded through the call chain. Code that respects it checks `ThrowIfCancellationRequested` at loop boundaries and passes the token into every awaitable call, including `Task.Delay`. Calling `ThrowIfCancellationRequested` raises `OperationCanceledException`, which callers are expected to treat as control flow, not error:

```
public static async Task<int> CountToAsync(int target, CancellationToken token)
{
    var sum = 0;
    for (var i = 1; i ≤ target; i++)
    {
        token.ThrowIfCancellationRequested(); // fail fast before the delay
        await Task.Delay(10, token);          // and inside it
        sum += i;
    }
    return sum;
}
```

listing 86: a loop that checks before and inside its delay (source: csharp-net/samples/src/Ch09/AsyncWork.cs, lines 50-60)

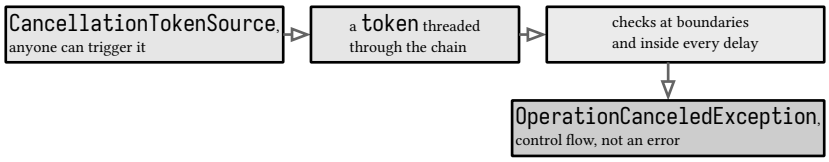


figure 60: cancellation, one token threaded to every await

The token itself comes from a `CancellationTokenSource`, which a timeout, another task, or caller code can trigger. Cancellation is also the one exception type async code should let propagate, the filter in chapter 8's snippet exists for exactly that.

9.5. async streams

An async method can also be a producer. `yield return` inside an `async IAsyncEnumerable<T>` method makes an iterator whose advancement is asynchronous, consumed with `await foreach`:

```
public static async IAsyncEnumerable<int> TicksAsync(
    [EnumeratorCancellation] CancellationToken token = default)
{
    for (var i = 1; i ≤ 3; i++)
    {
        await Task.Delay(1, token);
        yield return i;
    }
}
```

listing 87: an async iterator producing three ticks (source: csharp-net/samples/src/Ch09/AsyncWork.cs, lines 62-70)

The `[EnumeratorCancellation]` attribute marks which parameter the consumer's token should flow into when the caller uses `WithCancellation`, the composition that makes iterators cancellable without a second overload:

```
// WithCancellation feeds the consumer token into the iterator
// through the EnumeratorCancellation parameter
public static async Task<List<int>> CollectTicksAsync(CancellationToken token)
{
    var got = new List<int>();
    await foreach (var tick in TicksAsync().WithCancellation(token))
    {
        got.Add(tick);
    }
    return got;
}
```

listing 88: WithCancellation feeding the iterator's token (source: csharp-net/samples/src/Ch09/AsyncWork.cs, lines 72-82)

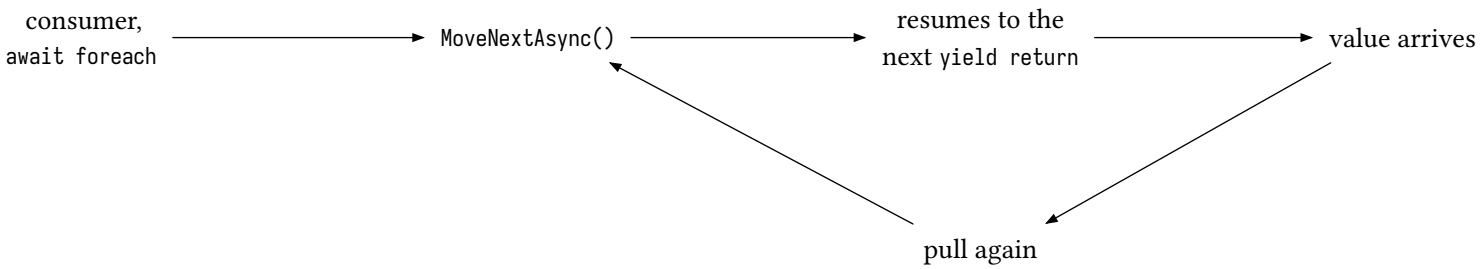


figure 61: async streams, one value per movenextasync, pulled by the consumer

9.6. the rules that keep async honest

Async methods flow one direction. The compounding list:

WARNING: never block on async code

`.Result`, `.Wait()`, and `.GetAwaiter().GetResult()` on a task that is not complete can deadlock when a synchronization context exists, and they burn a thread either way. Await instead, all the way up. `await` cannot appear inside `lock` or a `catch`/`finally` in older code shapes, cannot preserve a `ref struct` across suspension (chapter 10 owns the stack-only type), and `async void` is reserved for event handlers, where exceptions crash the process because nothing can await them.

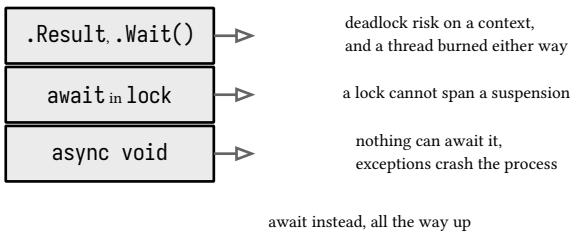


figure 62: the banned list, each rule with its cause

sources: learn.microsoft.com, async programming with async and await, task asynchronous programming model, async return types, await operator, cancellation in managed threads, and iasyncenumerable pages, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 8 tests.

10. memory and runtime

Chapter 3 established the value versus reference split. This chapter is about what the runtime does with those two worlds: how the garbage collector organizes the heap, and how the language hands you controlled ways to step around allocation when a hot path needs it, spans, `stackalloc`, `ref` passing, and `ref struct`. None of it requires `unsafe`.

10.1. the garbage collector

The GC is generational and compacting. New objects allocate in generation 0, cheaply, by bumping a pointer in a reserved segment. Collections happen most often in generation 0, because most objects die young, survivors are promoted to generation 1 and then 2, and each collection marks live objects, relocates them, and compacts the freed space. Generation 2 collections are the expensive full collections. Objects of 85,000 bytes or more allocate directly on the large object heap, which by default is not compacted because copying that much is its own cost problem. The suite watches a promotion happen:

```
    // survivors get promoted to older generations
    public static (int Fresh, int AfterCollect) Generations()
    {
        var fresh = new object();
        var gen = GC.GetGeneration(fresh);
        GC.Collect(); // force promotion, never do this in production code
        return (gen, GC.GetGeneration(fresh));
    }
```

listing 89: a fresh object, a forced collection, a promotion (source: csharp-net/samples/src/Ch10/Memory.cs, lines 56-63)



figure 63: the generational heap, survivors climb, big objects stay put

Two flavors exist. Workstation GC, the default, collects on the triggering thread and suits client and high density workloads. Server GC creates a heap and a dedicated high priority collector thread per logical CPU for throughput. Background GC collects generation 2 on dedicated threads while generation 0 and 1 collections happen as needed, and it is on by default. All of this is configuration (`System.GC.Server`, `ServerGarbageCollection` in the project file), not code.

WARNING: GC.Collect is a diagnostic

Calling `GC.Collect()` in production code is almost always wrong, it forces blocking collections at times the collector did not choose and can measurably hurt throughput. It has one legitimate use, measuring the reachable heap while investigating a leak. The listing calls it for exactly that reason and says so in its comment.

What the GC cannot free is anything still referenced. The managed leak is a rooted object nobody uses, a static cache that only grows, an event handler never detached, which is the chapter 7 closure leak wearing a different hat.

10.2. spans

A span is a view over contiguous memory: an array, a string's chars, a stack allocation, or native memory. Slicing updates a pointer and a length, no copy happens, and writes through the span reach the underlying storage:

```
// a span is a view: writes through it reach the array
public static int DoubleMiddle(int[] xs)
{
    var slice = xs.AsSpan(2, 3); // elements 2, 3, 4
    for (var i = 0; i < slice.Length; i++) slice[i] *= 2;
    return xs[3];
}
```

listing 90: a slice of the middle, doubled through the view (source: csharp-net/samples/src/Ch10/Memory.cs, lines 8-14)

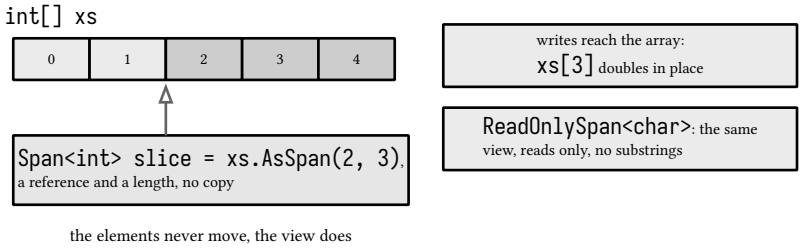


figure 64: a span is a reference and a length into storage, no copy

`ReadOnlySpan<T>` is the immutable form, and it is the standard answer to substring churn. Parsing a header with `Substring` allocates a new string for the digits. Parsing it with a span reads the same characters in place:

```
// ReadOnlySpan<char> parses a header without allocating a substring
public static int ParseContentLength(ReadOnlySpan<char> header)
{
    var colon = header.IndexOf(':');
    var digits = header[(colon + 1)..].Trim();
    return int.Parse(digits);
}
```

listing 91: slice, trim, parse, zero allocations (source: csharp-net/samples/src/Ch10/Memory.cs, lines 16-22)

`Span<T>` is a `ref struct`, and the compiler polices its lifetime: it cannot be boxed, cannot be a field of a class or a normal struct, cannot cross an `await` or `yield` boundary, and cannot escape the method whose stack it points into. When a buffer must live on the heap or cross an async boundary, `Memory<T>` is the wrapper without those restrictions, and you take the span for the synchronous work inside.

```
Span<int> M()
{
    Span<int> numbers = stackalloc int[3];
    return numbers; // error CS8352: cannot escape the method
}
```

fragment, illustrative, not executed

10.3. stackalloc

`stackalloc` reserves memory on the stack frame, which costs nothing at collection time because the frame just goes away. Inside a `ref struct` safe context it combines with spans into the standard scratch buffer pattern, small buffers on the stack, big ones on the heap, one type for both:

```
    // stackalloc under a size limit, heap above it, one span either way
    public static int SumScratch(int[] xs)
    {
        const int maxStackLimit = 1024;
        Span<int> scratch = xs.Length ≤ maxStackLimit
            ? stackalloc int[xs.Length]
            : new int[xs.Length];
        scratch.Fill(1);
        var sum = 0;
        foreach (var v in scratch) sum += v;
        return sum;
    }
```

listing 92: the ternary scratch buffer, stack under the limit (source: csharp-net/samples/src/Ch10/Memory.cs, lines 24-35)

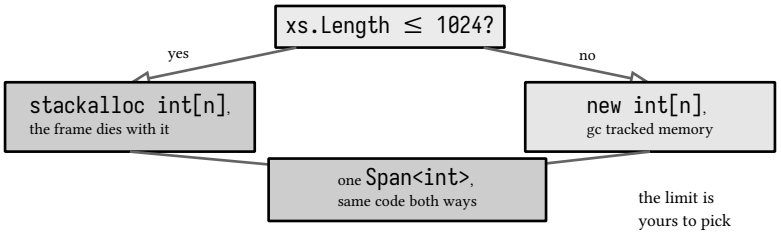


figure 65: stackalloc, stack under the limit, heap above, one span either way

The limit is a judgement call you own, stack space is finite and environment dependent. Stack allocated memory is uninitialized, so write before you read, and never put `stackalloc` inside a loop, hoist the buffer out and reuse it.

10.4. passing by reference

`ref`, `out`, `in`, and `ref readonly` parameters pass a reference to the caller's storage instead of a copy. `ref` requires the caller to initialize, `out` makes the callee assign, `in` is a read only reference that avoids copying large structs. Combined with `readonly`
`struct`, which guarantees no member can mutate, `in` parameters are free of defensive copy surprises:

```
// a readonly struct: in parameters copy nothing and stay safe
public readonly record struct Matrix(long M00, long M01, long M10, long M11);
```

listing 93: a readonly struct with no mutable state to defend (source: csharp-net/samples/src/Ch10/Memory.cs, lines 3-4)

```
    // in: a big struct passed by reference, readable only
    public static long Trace(in Matrix m) => m.M00 + m.M11;
```

listing 94: in takes the big struct without copying it (source: csharp-net/samples/src/Ch10/Memory.cs, lines 48-49)

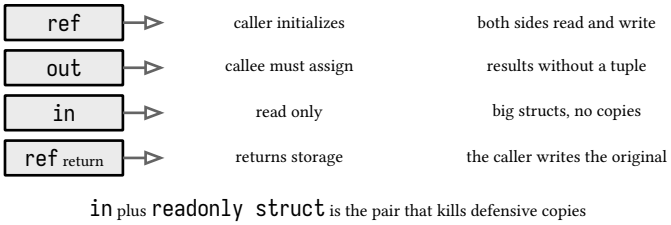


figure 66: the by-reference modifiers, who initializes and who writes

References can also flow back out. A `ref` return hands the caller a reference to storage the method can see, an array element, a field of a class. Ref locals rebind with `ref =` and write through to the original:

```
// ref return: the caller receives a reference into the array
public static ref int RefToMax(int[] xs)
{
    ref var best = ref xs[0];
    for (var i = 1; i < xs.Length; i++)
    {
        if (xs[i] > best) best = ref xs[i]; // rebind the reference
    }
    return ref best;
}
```

listing 95: ref return and ref local rebinding (source: csharp-net/samples/src/Ch10/Memory.cs, lines 37-46)

The test writes `100` through the returned reference and asserts the array changed. The compiler enforces the safety here: a reference to a local cannot leave its method, and ref safe contexts are checked at compile time, so a dangling reference is a compile error, not a crash.

10.5. overflow

Integer overflow wraps silently by default. `checked` and `unchecked` make the choice per expression or per block, with `checked` throwing `OverflowException` instead:

```
// overflow is a choice, made per expression
public static int WrapAdd(int a, int b) => unchecked(a + b);

public static int CheckedAdd(int a, int b) => checked(a + b);
```

listing 96: the same addition, two overflow policies (source: csharp-net/samples/src/Ch10/Memory.cs, lines 51-54)

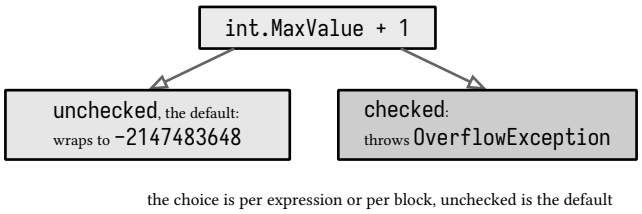


figure 67: overflow policy, unchecked wraps silently, checked throws

10.6. unsafe in outline

`unsafe` enables pointer types, pointer arithmetic, and `fixed`, which pins a managed object so the GC cannot move it while native code holds its address. `sizeof` measures unmanaged types at compile time. This book treats the pointer surface as interop territory, reached for when calling native libraries or parsing wire formats byte by byte, and always behind a safe managed facade. The modern interop path uses source generated marshalling rather than hand written `fixed` blocks, covered in chapter 13.

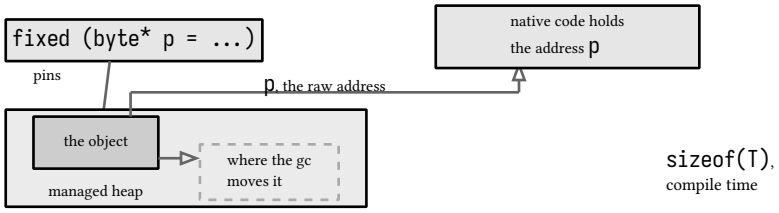


figure 68: fixed pins the object while native code holds its address

10.7. the memory safety preview

C# 15 opens a multi release effort to redefine what `unsafe` means. The original model ties the `unsafe` context to the existence of pointers. The updated model ties it to the operations that touch memory the runtime does not manage, so holding, passing, and returning a pointer becomes safe code while reading through it stays unsafe. The first slice ships in this SDK and every claim below was probed on 11.0.100-rc.1, none of it is blog inheritance. With `LangVersion` set to `preview`: declaring a pointer type, address-of `&`, the `fixed` statement, a `stackalloc` expression converted to a pointer, and `sizeof` on an unmanaged type all compile in safe code. Dereference `*p`, member access `p→m`, element access `p[i]`, and function pointer invocation still require an `unsafe` context, the probe measured error CS9360 on a bare `*pointer` in safe code. The preview lives in its own project because a preview language version must not leak into the main samples assembly:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net11.0</TargetFramework>
    <!-- the memory safety preview needs LangVersion=preview, which must not
         leak into the main samples assembly pinned to the default language -->
    <LangVersion>preview</LangVersion>
    <!-- the unsafe keyword (block, modifier, expression) still gates on this -->
    <AllowUnsafeBlocks>true</AllowUnsafeBlocks>
    <!-- runtime async toggle demo, the .net 11 rc1 claim probed in chapter 10 -->
    <Features>runtime-async=on</Features>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

listing 97: the isolated preview project, preview language plus the runtime async toggle (source: csharp-net/samples/src/CsharpBook.Preview/CsharpBook.Preview.csproj, lines 1-17)

The relaxations in one slice of the real sample:

```
// pointer declaration and address-of, no unsafe context anywhere
public static nint AddressOfLocal()
{
    int number = 42;
    int* pointer = &number;
    return (nint)pointer;
}

// sizeof on an unmanaged struct, previously an unsafe-only operation
public static int PointSize() => sizeof(Point);

// the fixed statement pins the array while the pointer exists
public static nint PinnedFirst(int[] xs)
{
    fixed (int* first = xs)
    {
        return (nint)first; // no dereference inside
    }
}

// stackalloc converting straight to a pointer, safe under the preview
public static nint Scratch()
{
    int* scratch = stackalloc int[4];
    return (nint)scratch;
}
```

listing 98: pointer existence, address-of, fixed, sizeof, stackalloc to pointer, all in safe code (source: csharp-net/samples/src/CsharpBook.Preview/MemorySafety.cs, lines 10-35)

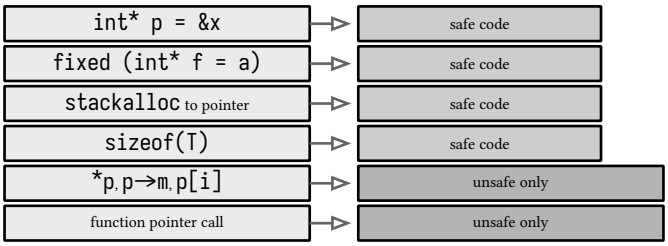


figure 69: the boundary moved, pointer existence is safe, access is not

Two more pieces of the preview round it out. The `unsafe(expr)` expression form scopes an unsafe context to a single expression, which matters where an `unsafe` block cannot appear syntactically: the probe compiled it in a field initializer and in a `catch` filter. Both usages still gate on `AllowUnsafeBlocks`, the probe measured error CS0227 with it off, while the pure pointer relaxations above compile even with it off. And the `safe` contextual keyword is recognized as a modifier on `extern` members and explicit layout struct fields, where the compiler cannot classify safety itself. The bigger half of the model, caller unsafe obligations and the assembly opt in, is not enforced at RC1: marking a member `unsafe` currently has no effect on its callers:

```
// dereference still needs unsafe, the expression form scopes it to one
// expression so the member itself stays safe for every caller
public static int ReadLocal()
{
    int number = 7;
    int* pointer = &number;
    return unsafe(*pointer);
}

// a field initializer cannot hold an unsafe block, the expression form
// is legal exactly there
static readonly int Signature = unsafe(ReadSignature());

static unsafe int ReadSignature()
{
    int raw = 0x1234;
    int* pointer = &raw;
    return *pointer;
}

public static int FieldSignature() => Signature;
```

listing 99: the unsafe expression form, in a return and in a field initializer (source: csharp-net/samples/src/CsharpBook.Preview/MemorySafety.cs, lines 37-57)

10.8. what .NET 10 and 11 changed

The JIT got more aggressive about escaping the heap. Small arrays of value types and of reference types that provably do not outlive their method are stack allocated, and escape analysis covers local struct fields and delegates. The practical effect: code written in the plain allocation style above gets faster without changing shape, which is the runtime trend to expect, measure, and let happen rather than pre-optimizing around.

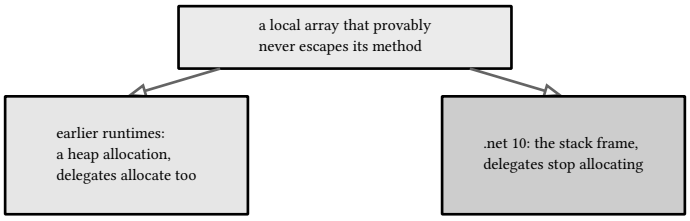


figure 70: escape analysis, provably local allocations move to the frame

.NET 11 adds two more runtime moves worth naming. Runtime Async shifts async suspension from compiler generated state machines into the runtime itself. A project opts in with `<Features>runtime-async=on</Features>`, which no longer needs the preview feature switch on `net11.0`, and the runtime libraries ship compiled with it on. The cheap part is demonstrable and demonstrated: the preview project carries the toggle and its plain `await` code compiles and runs under it on this SDK. The interesting part, cleaner live stack traces and lower allocation overhead, stays a release note claim here, this suite does not measure it. Chapter 9 covers the await model the feature runs underneath. The JIT side continues the story above: bounds check elimination for the `i + cns < len` shape and index from end access, and bounds checks dropped after an empty span guard. Those are release note claims too, the pattern to apply is the one this chapter keeps repeating, write the plain shape and let the JIT close the gap.

```
    // compiled with runtime-async=on from the project file, the chapter 10
    // toggle demo: plain await shape, runs unchanged under the feature
    public static async Task<int> Twice(int x)
    {
        await Task.Yield();
        return x * 2;
    }
```

listing 100: the runtime async toggle demo, plain await shape under runtime-async=on (source: csharp-net/samples/src/CsharpBook.Preview/MemorySafety.cs, lines 59-65)

sources: learn.microsoft.com, fundamentals of garbage collection, workstation and server gc, background gc, the large object heap, memory and span types, span t and readonlyspan t api pages, method parameters, stackalloc expression, performance with ref safety, and what's new in the .net 10 runtime pages, accessed 2026-09-08, what's new in c# 15 memory safety, unsafe code two models and preview pages, the safe keyword page, and what's new in the .net 11 runtime page, accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 14 tests.

11. allocation: the managed heap, pools, and arenas

Chapter 10 described the collector from the language side, spans, `stackalloc`, and the rules that keep references honest. This chapter walks the heap itself: how the allocator is organized, where big objects land, what pinning costs and how .NET 5 gave it a home, how the write barrier is observable without a debugger, and then the two reuse disciplines the base class library ships, `ArrayPool` and `MemoryPool`, plus the arena pattern you build yourself when a run of work should allocate once and reset. The chapter closes on the particle kernel the whole corpus measures: one workload in four representations, checksums asserted equal in the suite, nanoseconds per particle per step measured on this machine.

11.1. the managed heap

The heap is one virtual address space the collector carves into segments, reserved from the OS and committed as needed. Logically it is three generations plus two special heaps. Allocation is a bump of a generation 0 pointer, which is why small allocation is cheap: no free list, no coalescing, just a pointer and a bounds check. Each generation carries a **budget**, and when an allocation would exceed it, that generation is collected. Budgets are tuned by the runtime as the program runs, which is why the same code can collect at different times across runs. The suite watches the two probes that anchor the rest of the chapter:

```
    // this many bytes or more allocates on the loh, which is swept, not
    // compacted, and collected with generation 2
    public const int LohThresholdBytes = 85_000;

    // a double[] crosses at 10_622 elements: 24 header bytes + 8 * 10_622
    // = 85_000 exactly, measured. GC.GetGeneration reports 2 for loh
    // allocations straight out of the allocation, before any collection.
    public const int LohBoundaryDoubles = 10_622;

    // a quiet heap first: force a full collection so generation 0 is empty,
    // then a fresh small array reports generation 0
    public static int FreshSmallArrayGeneration()
    {
        GC.Collect();
        return GC.GetGeneration(new double[10_000]);
    }

    // the same allocation one field bigger than the boundary lands on the
    // loh and reports generation 2 immediately
    public static int LargeArrayGeneration()
    {
        GC.Collect();
        return GC.GetGeneration(new double[LohBoundaryDoubles + 1]);
    }
```

listing 101: threshold constants and the two generation probes (source: csharp-net/samples/src/Ch11/Heap.cs, lines 10-34)

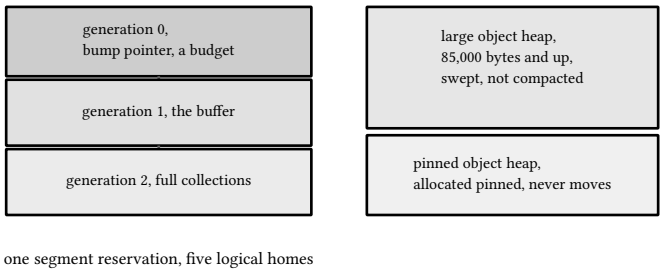


figure 71: the managed heap: three generations with budgets, two special heaps beside them

Only two of those homes accept user allocations: generation 0 for small objects, the large object heap for big ones. Generation 1 and 2 are filled by the collector promoting survivors, and the pinned object heap is reached only through the pinned allocation APIs later in this chapter.

11.2. the large object heap

The documented default threshold is 85,000 bytes, configurable through `System.GC.LOHThreshold` and `DOTNET_GCLOHThreshold`, and the number came from performance tuning, not from an address boundary. An object at or above the threshold allocates directly on the LOH, is collected with generation 2, and is swept rather than compacted, because copying megabytes to close a gap costs more than the gap. The boundary is measurable from safe code, and it lands at exactly 85,000 object bytes: a `double` array crosses at 10,622 elements, 24 bytes of array header plus 8 bytes per element, measured on this machine under the pinned SDK:

```
// walk the boundary: the smallest double[] length that reports
// generation 2 on allocation. deterministic after a full collect
public static int MeasuredLohBoundary()
{
    GC.Collect();
    for (var n = LohBoundaryDoubles - 64; n ≤ LohBoundaryDoubles + 64; n++)
    {
        if (GC.GetGeneration(new double[n]) == 2) return n;
    }
    return -1;
}
```

listing 102: walking the boundary: the first length that reports generation 2 (source: csharp-net/samples/src/Ch11/Heap.cs, lines 35-45)

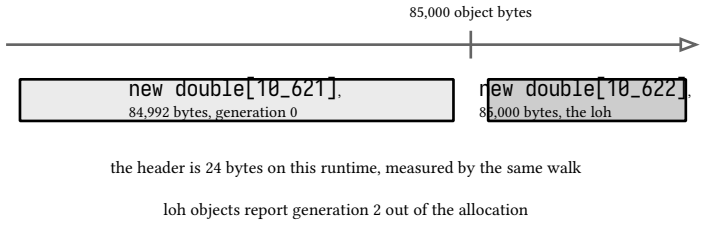


figure 72: the threshold, measured: 24 header bytes plus 8 per element crosses at 10,622

The LOH's failure mode is fragmentation: swept free space becomes a free list, long-lived and short-lived large objects interleave, and the gaps cannot close without a compaction. When one is genuinely needed, `GCSettings.LargeObjectHeapCompactionMode = CompactOnce` followed by `GC.Collect()` compacts it once, on demand, as an explicit decision rather than a policy. The cheaper cure is not to allocate large temporary arrays at all, which is the pool's job below.

11.3. the pinned object heap

`fixed` pins an object wherever it lives, and every pinned object blocks compaction around it. .NET 5 added the honest alternative: allocate the object where it never moves in the first place. `GC.AllocateArray(length, pinned: true)` and the uninitialized variant put the array on the pinned object heap, a heap with no compaction ever, so the address you take stays valid without pinning anything. The restriction is the same as for `fixed` buffers: the element type must not contain references, because a never-moving heap cannot have its references rewritten. The suite watches the heap grow:

```
    // generation info entries, in order: gen 0, gen 1, gen 2, loh, poh.
    // five entries on this runtime, measured
    public const int PohGenerationInfoIndex = 4;

    // the pinned object heap: GC.AllocateUninitializedArray with pinned true
    // (or GC.AllocateArray) puts the array on the poh, where the collector
    // never moves it. returns how much the poh grew for one pinned
    // allocation, measured across a full collection
    public static long PohGrowthForPinnedAllocation(int bytes)
    {
        GC.Collect();
        var before = GC.GetGCMemoryInfo().GenerationInfo[PohGenerationInfoIndex].SizeAfterBytes;
        var pinned = GC.AllocateUninitializedArray<byte>(bytes, pinned: true);
        GC.Collect();
        var after = GC.GetGCMemoryInfo().GenerationInfo[PohGenerationInfoIndex].SizeAfterBytes;
        GC.KeepAlive(pinned);
        return after - before;
    }

    // pinned arrays still report generation 2 through GetGeneration: the poh
    // is collected with generation 2 like the loh, but never compacted
    public static int PinnedArrayGeneration()
    {
        GC.Collect();
        return GC.GetGeneration(GC.AllocateArray<byte>(1 << 16, pinned: true));
    }
```

listing 103: pinned allocation grows the pinned object heap, measured across a collection (source: csharp-net/samples/src/Ch11/Heap.cs, lines 47-72)

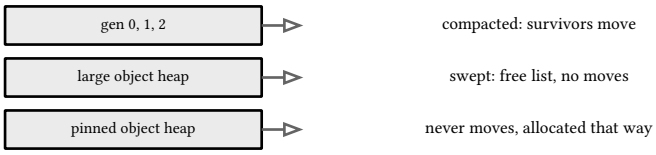


figure 73: three heaps, three movement policies

The `GenerationInfo` array the listing reads has five entries on this runtime, gen 0, gen 1, gen 2, LOH, POH, measured, and the pinned heap is the fifth. Pinned arrays still report generation 2 through `GC.GetGeneration`, both special heaps are collected with generation 2, the difference is only whether their objects can move.

11.4. write barriers

A generational collector needs to find references from old objects into young ones without scanning the whole old heap. The write barrier is the trick: every store of a reference into an object older than generation 0 marks a card, a coarse region of the old heap, so a generation 0 collection only walks the dirty cards. You cannot call the barrier, but you can watch it work: store a million young references into a generation 2 array and they all survive the next full collection, found through the cards the stores dirtied:

```
    // the write barrier, observed: after a full collect the array is
    // generation 2, every store of a young reference into it dirties a card
    // so the next generation 0 collection can find the young object through
    // the old one. returns how many stored references survive a full
    // collection and where the array lives
    public static (int Live, int ArrayGeneration, int StoredGeneration) YoungRefsIntoOldArray(
        int stores, int slots)
    {
        GC.Collect();
        var old = new object?[slots]; // small enough to stay off the loh
        GC.Collect(); // the array is now generation 2
        for (var i = 0; i < stores; i++)
        {
            old[i & (slots - 1)] = new object();
        }
        var live = old.Count(o => o ≠ null);
        GC.Collect(); // survivors promote through the dirty cards
        return (live, GC.GetGeneration(old), GC.GetGeneration(old[0]!));
    }
```

listing 104: young references stored into an old array survive through the dirty cards (source: csharp-net/samples/src/Ch11/Heap.cs, lines 74-92)

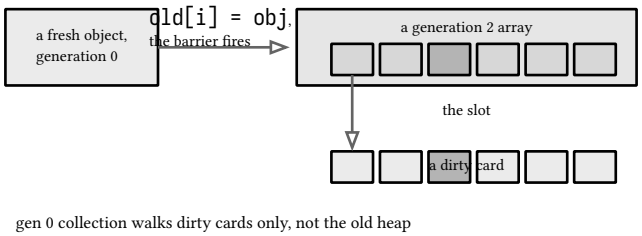


figure 74: the card table: a store into an old object dirties the card that names it

The barrier has a cost the chapter measures rather than asserts: storing a million young references into the old array ran 2 generation 0 collections where dropping them ran 1, measured on this machine, 2026-09-13, because every survivor keeps a little more of generation 0 alive per pass. The barrier is also why `fixed` and the POH matter to throughput: pinned objects dirty no cards, but they block the compaction that keeps generation 0's survivors dense.

11.5. arraypool, deep

Chapter 15 sketched the pool as a line in the collections tour. The deep version is the contract: one shared pool per process, buckets by power of two from 16 up, `Rent` returns at least the requested length and may return more, `Return` is voluntary and returns the array to the bucket, and nothing is zeroed. The rounding is the first trap: the caller must carry its own length, because `Rent(1000)` hands back a 1024-slot array:

```
    // bucket sizes are powers of two from 16: Rent rounds up, Rent(1000)
    // yields a 1024 slot array, so callers must carry their own length
    public static int[] RentLengths(params int[] requests)
    {
        var pool = ArrayPool<double>.Shared;
        var lengths = new int[requests.Length];
        var rented = new double[requests.Length][];
        for (var i = 0; i < requests.Length; i++)
        {
            rented[i] = pool.Rent(requests[i]);
            lengths[i] = rented[i].Length;
        }
        foreach (var buffer in rented)
        {
            pool.Return(buffer);
        }
        return lengths;
    }

    // the rent-return-rent cycle on one thread hands back the same array
    // instance, and with clearArray false the previous contents survive.
    // this is why Return counts as giving up ownership: the next renter
    // sees your bytes unless you pay the clear
    public static (bool SameInstance, double SeenValue) ReturnThenRentKeepsContents()
    {
        var pool = ArrayPool<double>.Shared;
        var first = pool.Rent(64);
        first[0] = 42;
        pool.Return(first);
        var second = pool.Rent(64);
        var same = ReferenceEquals(first, second);
        var seen = second[0];
        pool.Return(second);
        return (same, seen);
    }
```

listing 105: bucket rounding, the rent-return-rent cycle, and the surviving bytes (source: csharp-net/samples/src/Ch11/Pools.cs, lines 13-47)

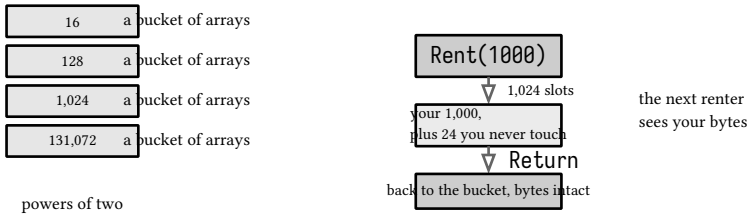


figure 75: the bucket ladder: rent rounds up, return is voluntary, nothing clears

The second trap is ownership. `Return` is a give-up: the same instance can be handed to the next renter on the same thread immediately, the suite asserts the reference equality and the surviving 42, and a returned-then-used buffer is a use-after-free the compiler cannot see. The rule the platform docs state and the samples follow: keep `Rent` and `Return` in one method, return in `finally` only when no escape path can still hold the array, and pay `clearArray: true` when the next renter must not read what you left.

11.6. memorypool and imemoryowner

`ArrayPool` rents raw arrays, which is wrong the moment the buffer must cross an async boundary or live in a class, because the responsibility to return it has no type. `MemoryPool<T>` is the owning form: `Rent` returns an `IMemoryOwner<T>`, the handle whose `Dispose` returns the buffer, and the `Memory<T>` it hands out is valid exactly as long as the owner is alive. It rounds to powers of two like the array pool, measured, `Rent(1000)` yields a `Memory` of length 1024:

```
// MemoryPool: the default pool also rounds to powers of two, Rent(1000)
// yields a Memory of length 1024, Rent(64) yields 64, measured. the
// owner is the lifetime: Dispose returns the buffer, and handing the
// Memory out is safe only while the owner is alive
public static int[] MemoryPoolLengths(params int[] requests)
{
    var lengths = new int[requests.Length];
    for (var i = 0; i < requests.Length; i++)
    {
        using var owner = MemoryPool<double>.Shared.Rent(requests[i]);
        lengths[i] = owner.Memory.Length;
    }
    return lengths;
}

// writes through the owner's Memory reach the buffer, reads after the
// write see them, one handle owns the buffer
public static double OwnerMemoryIsTheBuffer()
{
    using var owner = MemoryPool<double>.Shared.Rent(128);
    owner.Memory.Span[7] = 3.5;
    return owner.Memory.Span[7];
}
```

listing 106: the owner handle: rent, use through the memory, dispose returns (source: csharp-net/samples/src/Ch11/Pools.cs, lines 63-85)

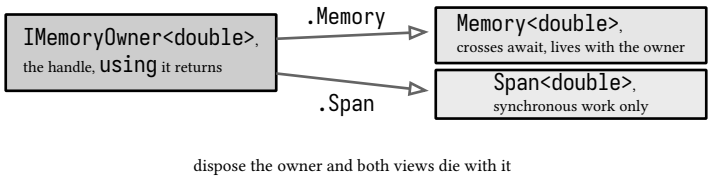


figure 76: ownership as a type: the owner outlives the memory it hands out

The pool boundary is a design decision, not a performance one. A `Memory<T>` handed to an async consumer outliving its owner is the same use-after-free with better syntax, which is why the owner is the thing you pass, never the bare `Memory`.

11.7. arenas as a pattern

A pool amortizes individual buffers. An arena changes the shape entirely: one slab rented up front, allocations are bumps of a cursor, nothing is garbage during the run because everything is reachable from the arena, and the run ends with one `Reset` that rewinds the cursor instead of freeing anything. Per-frame work, per-request parsing, any phase whose allocations die together at a known point, fits an arena. The sample arena is 60 lines over rented slabs:

```
    // bump-allocate: the span is valid while the arena is alive and until a
    // Reset hands the same region to the next caller. a rental bigger than
    // one slab is refused: pick a slab size that fits the workload
    public Span<double> Rent(int doubles)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        var slabLength = _slab.Length;
        if (doubles > slabLength)
        {
            throw new ArgumentOutOfRangeException(
                nameof(doubles), "over-slab rentals need a bigger slab, not an arena slice");
        }
        if (_used + doubles > slabLength)
        {
            // current slab exhausted, take another; slabs are kept so older
            // spans stay valid until Reset or Dispose
            _slabIndex++;
            if (_slabIndex == _slabs.Count)
            {
                _slab = _pool.Rent(slabLength);
                _slabs.Add(_slab);
            }
            else
            {
                _slab = _slabs[_slabIndex];
            }
            _used = 0;
        }
        var span = _slab.AsSpan(_used, doubles);
        _used += doubles;
        return span;
    }

    // rewind to the first slab and hand the same regions out again. the
    // caller promises no span rented before the Reset is used after it
    public void Reset()
    {
        _slabIndex = 0;
        _slab = _slabs[0];
        _used = 0;
    }
```

listing 107: bump allocation with a rewind: rent, exhaust, reset (source: csharp-net/samples/src/Ch11/Arena.cs, lines 33-73)

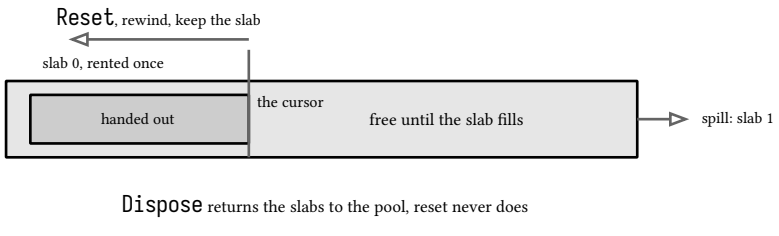


figure 77: the arena slab: bump, spill, rewind, return

Two contracts make it honest. A rental bigger than a slab is refused, the caller sizes the slab to the run, and `Reset` rewinds without clearing, so a span from the previous pass aliases a span from the next one, which the suite asserts on purpose: 5 written before the reset is 5 read after it. That aliasing is the arena's power and its whole hazard, and it is why arenas live behind a phase boundary, not in general-purpose APIs.

11.8. the particle kernel

The corpus-wide workload: N particles, eight float64 fields each (x, y, z, vx, vy, vz, m, q), sixteen steps of pure arithmetic under a constant analytic field, one stated seed for the PRNG so every representation fills identically. Four representations of the same state: owned `double[8][]` structure-of-arrays, `Particle[]` array-of-structs, pool rentals sliced to N, and arena slices off one slab. The step runs over spans, so the same code serves the last three:

```
    // one step over spans: pos += vel * dt + 0.5 * a * dt * dt, then
    // vel += a * dt, with a = q * e / m elementwise. no allocation, the
    // spans are the whole contract
    public static void Step(
        Span<double> x, Span<double> y, Span<double> z,
        Span<double> vx, Span<double> vy, Span<double> vz,
        ReadOnlySpan<double> m, ReadOnlySpan<double> q)
    {
        var halfDt2 = 0.5 * Dt * Dt;
        for (var i = 0; i < x.Length; i++)
        {
            var ax = q[i] * Ex / m[i];
            var ay = q[i] * Ey / m[i];
            var az = q[i] * Ez / m[i];
            x[i] += vx[i] * Dt + ax * halfDt2;
            y[i] += vy[i] * Dt + ay * halfDt2;
            z[i] += vz[i] * Dt + az * halfDt2;
            vx[i] += ax * Dt;
            vy[i] += ay * Dt;
            vz[i] += az * Dt;
        }
    }
```

listing 108: the span step: pos += vel times dt plus half a dt squared, vel += a dt (source: csharp-net/samples/src/Ch11/Particles.cs, lines 85-106)

The suite asserts the four representations agree bitwise, and the agreement is not free: the fold and the update had to associate identically, one `+=` per field in checksum order and the update parenthesized to match the span statement, or the last bits drift. The bench, opt-in under `samples/bench/`, measured the step loop only, warm, median of five reps after a warmup rep, 1,000,000 particles, .NET 11.0.0, this machine, 2026-09-13:

representation	ns per particle per step	setup ms	allocated per rep
`double[8][]` soa	13.1	30	64.0 MB
`Particle[]` aos	5.7	39	128.0 MB
pooled soa	6.4	20	432 B warm
arena soa	3.9	27	488 B warm

The folklore answer, SoA always wins, is not what this loop measures. The step touches all eight fields of every particle, so the AoS walk reads one 64-byte cache line per particle, perfectly sequential, while the SoA loop touches eight separate streams a stride apart, eight lines in flight for the same particle, and loses despite zero per-element bookkeeping. SoA pays off when work is field-wise or vectorizable across a lane, which is exactly the split to test before choosing, not assume. The arena's edge over the pooled variant, 3.9 against 6.4 ns, is the same locality story one level up: eight slices of one contiguous slab against eight separately allocated segments. The allocation column is the chapter's thesis in one number: the owned arrays pay 64 to 128 MB of garbage per run at this scale, the pooled and arena variants pay a few hundred bytes warm, and the checksum, 6995752.357206226, is identical in every representation and every rep.

The scale ladder above 1m is a one-off and arithmetic. 100 million particles, SoA only, run alone and sequentially on this machine, 2026-09-13: fill 4,372 ms, the 16 steps 34,903 ms, which is 21.815 ns per particle per step against

13.1 at 1m, the step loop is memory bandwidth bound once the eight arrays are 800 MB each, and the managed heap held 6,400,250,392 bytes with two collections in each generation across the whole run, the arrays are all large objects, the ephemeral machinery barely engages. One billion is arithmetic, not a measurement on this machine: 64 GB of SoA state, ten times this run, past this machine's 32 GB.

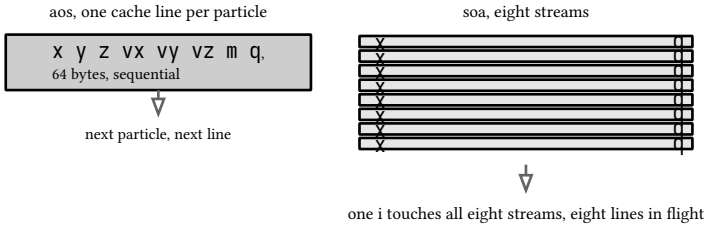


figure 78: four representations, one loop: lines in flight per particle

sources: learn.microsoft.com, fundamentals of garbage collection, the large object heap on windows systems, runtime configuration options for garbage collection, gc.allocatearray and gc.allocateuninitializedarray, arraypool, arraypool shared rent and return, and memorypool pages, accessed 2026-09-13. Heap, pool, and arena behavior verified by `make verify-csharp`, 20 tests. The 1m particle measurements above are from `samples/bench/` on this machine, 2026-09-13, .NET 11.0.0 under SDK 11.0.100-rc.1.

12. profilers: benchmarkdotnet, counters, and p99

Chapter 11 made allocation measurable from inside the process. This chapter is the tooling around it: how to A/B two implementations honestly, how to watch a live process without touching it, how to read a trace, and how to report latency as percentiles instead of an average. Every command and every number below ran on this machine under the pinned SDK 11.0.100-rc.1, and the two slow workloads, the particle kernel and the ten-million-row SQLite kernel, are the same contract the other language books in this corpus measure.

12.1. benchmarkdotnet, the grammar

BenchmarkDotNet is the standard harness, and its grammar is small. Mark methods with `[Benchmark]`, one of them `Baseline = true` so the table reports ratios against it, do setup once in `[GlobalSetup]`, return the result so the JIT cannot dead-code the loop, and let the runner own the iterations: warmup runs, measured runs, outlier removal, and a Mean, Error, StdDev, Ratio block at the end. The opt-in bench project under `samples/bench/` carries the book's copy:

```
// InProcess is the toolchain that works on this rc sdk, Classic is the
// default toolchain whose failure the chapter quotes verbatim.
[MemoryDiagnoser]
public class SoaVsAos
{
    private double[][] _fields = [];
    private Particle[] _particles = [];

    [GlobalSetup]
    public void Setup()
    {
        _fields = Particles.CreateSoA(100_000);
        _particles = Particles.ToAoS(Particles.CreateSoA(100_000));
    }

    [Benchmark(Baseline = true)]
    public double Soa()
    {
        Particles.StepSoA(_fields, 16);
        return Particles.ChecksumSoA(_fields);
    }

    [Benchmark]
    public double Aos()
    {
        Particles.StepAoS(_particles, 16);
        return Particles.ChecksumAoS(_particles);
    }
```

listing 109: the chapter 11 step as an a slash b, baseline, setup, checksum returned (source: csharp-net/samples/bench/Bdn.cs, lines 14-41)

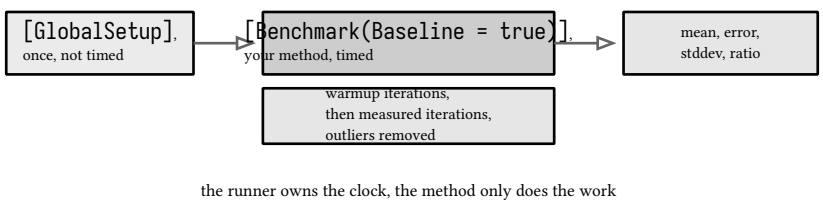


figure 79: what the harness adds around your method

The number one rule is isolation: the harness exists so a quick `Stopwatch` around two candidates, with a cold JIT and one iteration each, cannot masquerade as a measurement. The table it prints carries its own honesty, Error is half the confidence interval and Ratio is the distribution of per-iteration ratios, not a division of two means.

12.2. the rc gap and the inprocess toolchain

On a stable SDK the default toolchain generates a project per benchmark, builds it, and runs it out of process. On this RC SDK that path dies before the first benchmark, reproduced by the `bdnclassic` mode of the bench, exit code 82, 2026-09-13, this machine:

```
// Validating benchmarks:
Unhandled exception. System.NotImplementedException: GetRuntimeVersion not implemented for NotRecognized
   at BenchmarkDotNet.Extensions.RuntimeMonikerExtensions.GetRuntimeVersion(RuntimeMoniker runtimeMoniker)
   at BenchmarkDotNet.Validators.DotNetSdkValidator.ValidateCoreSdks(...)
```

BenchmarkDotNet 0.15.8 does not recognize the RC runtime moniker, so its SDK validation throws instead of validating. The working toolchain on this SDK is `[InProcess]` territory: `InProcessEmitToolchain` runs every benchmark inside the host process, no generated project, no toolchain resolution. That is the toolchain this book runs:

```
public static class Bdn
{
    // the working toolchain on 11.0.100-rc.1: the benchmark runs inside
    // this process, no generated project, no toolchain resolution
    public static void RunInProcess()
    {
        var config = ManualConfig.Create(DefaultConfig.Instance)
            .AddJob(Job.Default
                .WithToolchain(InProcessEmitToolchain.Instance)
                .WithWarmupCount(3)
                .WithIterationCount(15))
            .AddDiagnoser(MemoryDiagnoser.Default);
        _ = BenchmarkRunner.Run<SoaVsAos>(config);
    }

    // the default toolchain: emits a generated project and builds it. on
    // this rc sdk it dies before the first benchmark with the exception the
    // chapter quotes
    public static void RunClassic()
        => _ = BenchmarkRunner.Run<SoaVsAos>();
}
```

listing 110: inprocess for the runs, the classic entry kept as the documented failure (source: csharp-net/samples/bench/Bdn.cs, lines 44-64)

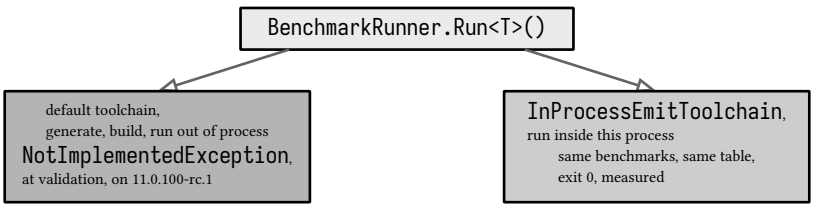


figure 80: two toolchains, one dead on the rc sdk, one working

The measured run, 100,000 particles, 16 steps plus the checksum fold, warmup 3, 15 iterations, this machine, 2026-09-13:

```
| Method | Mean     | Error     | StdDev    | Ratio | Allocated | Alloc Ratio |
|------- |---------:|----------:|----------:|------:|----------:|------------:|
| Soa    | 4.935 ms | 0.0335 ms | 0.0313 ms |  1.00 |         - |          NA |
| Aos    | 4.909 ms | 0.0258 ms | 0.0229 ms |  0.99 |         - |          NA |
```

One outlier was removed from `Aos` automatically, and the ratio at this scale is parity, which is itself a result: at 100k particles both representations are 6.4 MB and L3 resident, where chapter 11′s 1m run, 64 MB each, measured the AoS loop 2.3 times faster. Scale moved the answer, which is the argument for keeping both numbers rather than a favorite. The GA SDK is expected to fix the classic toolchain; when it does, re-run `bdnclassic` and record the delta here instead of silently deleting the gap.

12.3. dotnet-counters, live

`dotnet-counters` watches a process through the EventPipe, no profiler attach, no code change. The bench installs it as a local tool from its own manifest, so nothing pollutes the machine:

```
dotnet tool restore
dotnet dotnet-counters collect --counters System.Runtime --format csv \
    --output counters.csv -p <pid> --refresh-interval 1 \
    --duration 00:00:00:04
```

fragment, illustrative, not executed

The capture committed with this book was taken that way against the bench's spinner, a busy loop churning short-lived arrays, 4 seconds at a one second refresh, this machine, 2026-09-13. The names are the new meter names, `dotnet.gc.*`, not the legacy counter names:

```
Timestamp,Provider,Counter Name,Counter Type,Mean/Increment
09/13/2026 18:11:15,System.Runtime,dotnet.thread_pool.thread.count ({thread}),Metric,0
09/13/2026 18:11:15,System.Runtime,dotnet.gc.last_collection.heap.size (By)
[gc.heap.generation=gen1],Metric,119792
09/13/2026 18:11:15,System.Runtime,dotnet.gc.last_collection.heap.size (By)
[gc.heap.generation=gen0],Metric,560
09/13/2026 18:11:15,System.Runtime,dotnet.gc.last_collection.heap.size (By)
[gc.heap.generation=poh],Metric,8184
09/13/2026 18:11:15,System.Runtime,dotnet.gc.last_collection.heap.size (By)
[gc.heap.generation=gen2],Metric,241992
```

listing 111: the committed capture, five of its 104 rows (source: csharp-net/samples/captures/counters-spinner-2026-09-13.csv, lines 1-6)

```
    public static List<CounterSample> Parse(IEnumerable<string> lines)
    {
        var samples = new List<CounterSample>();
        foreach (var line in lines)
        {
            if (string.IsNullOrWhiteSpace(line) || line == Header)
            {
                continue;
            }
            var parts = line.Split(',');
            if (parts.Length != 5)
            {
                throw new FormatException($"expected 5 csv fields, got {parts.Length}: {line}");
            }
            samples.Add(new CounterSample(
                DateTime.Parse(parts[0], CultureInfo.InvariantCulture),
                parts[1],
                parts[2],
                parts[3],
                double.Parse(parts[4], CultureInfo.InvariantCulture)));
        }
        return samples;
    }
```

listing 112: the capture parses into records, header skipped, five fields per row (source: csharp-net/samples/src/Ch12/CountersParse.cs, lines 24-46)

PITFALL: never pipe these tools through select-object -first

The capture commands are long, and trimming their output with `Select-Object -First` in PowerShell kills the capture: the early pipeline stop tears down the session and leaves a 1 KB stub file. Measured twice on this machine. Redirect to a file and read the file, the way `run-bench.ps1` does.

12.4. dotnet-trace and eventpipe

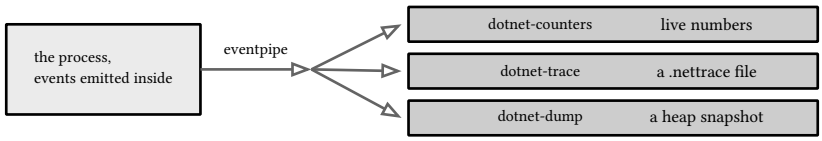
`dotnet-trace` writes the same EventPipe stream to a `.nettrace` file for after-the-fact analysis. The capture that quoted this section, 3 seconds of the same spinner:

```
$ dotnet dotnet-trace collect -p <pid> --duration 00:00:00:03 -o spinner.nettrace
...
Trace completed.
```

1,061,498 bytes for 3 seconds of a busy loop, measured 2026-09-13. The file is a stream of events: CPU samples, GC events, JIT events, thread switches. `--format speedscope` rewrites it for the speedscope flame graph viewer, and `--profile gc-verbose` narrows the stream to allocation and collection events when only the collector is in question. The spinner being observed:

```
// a busy loop for dotnet-counters and dotnet-trace to observe: churns gen 0
// with short-lived allocations and burns cycles, run for N seconds
public static class Spinner
{
    public static void Run(int seconds)
    {
        Console.WriteLine($"spinner pid={Environment.ProcessId} for {seconds}s");
        var clock = Stopwatch.StartNew();
        var sink = 0.0;
        while (clock.Elapsed.TotalSeconds < seconds)
        {
            for (var i = 0; i < 100_000; i++)
            {
                var buffer = new double[64];
                buffer[0] = i;
                sink += buffer[0];
            }
            Thread.Sleep(50);
        }
        Console.WriteLine($"sink {sink:R}");
    }
}
```

listing 113: the workload the captures watched: allocation churn plus sleep (source: csharp-net/samples/bench/ParticleBench.cs, lines 162-184)



no code change, no attach, the same stream read three ways

figure 81: one eventpipe stream, three consumers

12.5. the gc panel

Percent time in GC, generation counts, and pause time together tell you which of chapter 11's levers to pull, and all three are meters in the committed capture. Reading it:

```
    // % time in gc over the window: dotnet.gc.pause.time is seconds of
    // pause per second, so the mean of the rate is the fraction directly
    public static double TimeInGcPercent(IEnumerable<CounterSample> samples)
    {
        var rates = samples.Where(s => s.Meter == "dotnet.gc.pause.time").Select(s => s.Value).ToList();
        return rates.Count == 0 ? 0.0 : rates.Average() * 100.0;
    }
```

listing 114: pause time is a rate, seconds of pause per second, so the mean is the fraction (source: csharp-net/samples/src/Ch12/CountersParse.cs, lines 66-74)

meter	measured	reading
dotnet.gc.pause.time	about 0.0037 s per s	0.37% of wall time paused, healthy for this churn

meter	measured	reading
dotnet.gc.collections gen 0	65 to 66 per s	the ephemeral budget recycling the spinner's arrays
dotnet.gc.collections gen 1, 2	0	nothing survives the loop, no promotion pressure
dotnet.gc.heap.total_allocated	about 858 MB per s	the allocation rate the gen 0 cadence is paying for
dotnet.gc.last_collection.heap.size gen 0	560 B	the post-collection ephemeral remainder
dotnet.process.memory.working_set	47.7 to 49.1 MB	the process cost of the churn

The panel is a diagnosis loop: allocation rate explains the gen 0 cadence, the cadence times the pause rate gives the wall-clock tax, and if the tax is too high, chapter 11's pools and arenas are the lever, verified by capturing again and watching the gen 0 count fall.

12.6. p99 for a managed loop

The mean of a latency distribution is a marketing number. The honest report is nearest-rank percentiles over a count of samples: sort, take the value at `ceil(p / 100 * n)`. The suite pins the math on known inputs, p50 of 1..100 is 50, p99 of 1..1000 is 990, and one floating point trap is handled in code, the rank product rounded to ten decimals before the ceiling, because `99.9 / 100 * 1000` computes to 999.00000000000006 and a bare ceiling silently asks for rank 1000:

```
// ceil(p / 100 * n) - 1. p50 of 1..100 is 50, p99 of 1..1000 is 990
public static double Percentile(double[] samples, double p)
{
    ArgumentOutOfRangeException.ThrowIfZero(samples.Length);
    var sorted = (double[])samples.Clone();
    Array.Sort(sorted);
    return PercentileOfSorted(sorted, p);
}

private static double PercentileOfSorted(double[] sorted, double p)
{
    // round the product to ten decimals before the ceiling: p99.9 of
    // 1000 samples is exactly 999 ranks, but 99.9 / 100 * 1000 computes
    // to 999.00000000000006 and a bare ceiling overshoots to 1000
    var rank = (int)Math.Ceiling(Math.Round(p / 100.0 * sorted.Length, 10));
    if (rank < 1) rank = 1;
    if (rank > sorted.Length) rank = sorted.Length;
    return sorted[rank - 1];
}

// the four numbers a latency report owes its reader, in one call
public static (double P50, double P95, double P99, double P999) Summary(double[] samples)
{
    var sorted = (double[])samples.Clone();
    Array.Sort(sorted);
    return (
        PercentileOfSorted(sorted, 50),
        PercentileOfSorted(sorted, 95),
        PercentileOfSorted(sorted, 99),
        PercentileOfSorted(sorted, 99.9));
```

listing 115: nearest rank percentiles and the four-number summary (source: csharp-net/samples/src/Ch12/P99.cs, lines 14-43)

The harness loop runs deterministic chunks with a shaped load, every 16th chunk does 8 times the work, so the tail is designed rather than accidental:

```
    // a managed chunk loop with a shaped load: chunk i does base work, and
    // every stride-th chunk does multiplier times more, so the tail is real
    // rather than scheduler noise. returns the per-chunk latencies in
    // milliseconds and a checksum of everything computed, which is what the
    // suite asserts (deterministic) while the latencies feed percentiles
    // (measured)
    public static (double[] ChunkMs, double Checksum) ChunkLoop(
        int chunks, int chunkSize, int seed = 7,
        int heavyStride = 16, int heavyMultiplier = 8)
    {
        var rng = new Random(seed);
        var buffer = new double[chunkSize];
        var chunkMs = new double[chunks];
        var checksum = 0.0;
        var clock = Stopwatch.StartNew();
        for (var c = 0; c < chunks; c++)
        {
            var work = (c % heavyStride == heavyStride - 1) ? chunkSize * heavyMultiplier : chunkSize;
            for (var i = 0; i < work; i++)
            {
                buffer[i % chunkSize] = rng.NextDouble() - 0.5;
            }
            for (var i = 0; i < chunkSize; i++)
            {
                checksum += buffer[i];
            }
            chunkMs[c] = clock.Elapsed.TotalMilliseconds;
            clock.Restart();
        }
        return (chunkMs, checksum);
    }
}
```

listing 116: a chunk loop with a real tail, one latency per chunk (source: csharp-net/samples/src/Ch12/P99.cs, lines 45-77)

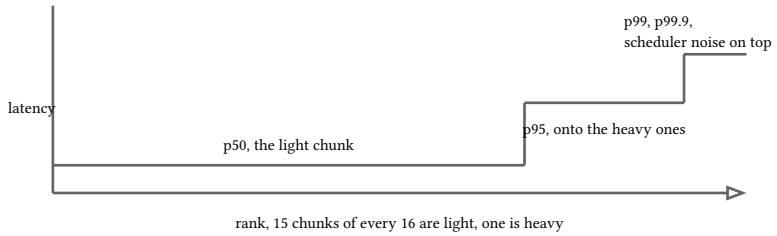


figure 82: the distribution a shaped tail produces: the median floor, the p95 step, the p99 cliff

Measured through the bench, 2,000 chunks of 1,000 doubles, run twice back to back, this machine, 2026-09-13: run 1 measured p50 0.0035 ms, p95 0.0244, p99 0.0245, p99.9 0.0454, max 1.1676; run 2 measured p50 0.0035, p95 0.0244, p99 0.0381, p99.9 0.0558, max 1.2187. The median and p95 repeat, the p99 and beyond move, and the checksum, −383.24379846230437, is identical in both runs. That split is the lesson: the deterministic half of the tail belongs to your workload, the moving half belongs to the machine, and a latency claim that does not say which is which is not a claim.

12.7. the row kernel

The second corpus workload: ten million rows into SQLite through `Microsoft.Data.Sqlite` 10.0.12 over the pinned `bundle_e_sqlite3` 3.0.5 build, both reporting engine version 3.53.4, the same pins the capstone's save store uses. The schema is fixed across every book in the corpus, 412 stations, 8 REAL columns, and the values are a stated integer hash of station, row, and field, so any run resumable anywhere produces the identical stream:

```
    public static long InsertBatched(
        SqliteConnection connection, long rows, List<double>? chunkMs = null)
    {
        long inserted = 0;
        using var insert = connection.CreateCommand();
        insert.CommandText =
            "INSERT INTO readings (station, v0, v1, v2, v3, v4, v5, v6, v7) " +
            "VALUES ($s, $v0, $v1, $v2, $v3, $v4, $v5, $v6, $v7)";
        var pStation = insert.CreateParameter();
        pStation.ParameterName = "$s";
        _ = insert.Parameters.Add(pStation);
        var pValue = new SqliteParameter[8];
        for (var f = 0; f < 8; f++)
        {
            pValue[f] = insert.CreateParameter();
            pValue[f].ParameterName = $"$v{f}";
            _ = insert.Parameters.Add(pValue[f]);
        }
        var clock = Stopwatch.StartNew();
        while (inserted < rows)
        {
            var inBatch = (int)Math.Min(BatchSize, rows - inserted);
            using (var transaction = connection.BeginTransaction())
            {
                insert.Transaction = transaction;
                for (var r = 0; r < inBatch; r++)
                {
                    var row = inserted + r;
                    var station = (int)(row % Stations);
                    pStation.Value = Station(station);
                    for (var f = 0; f < 8; f++)
                    {
                        pValue[f].Value = Value(station, row, f);
                    }
                    _ = insert.ExecuteNonQuery();
                }
                transaction.Commit();
            }
            chunkMs?.Add(clock.Elapsed.TotalMilliseconds);
            clock.Restart();
            inserted += inBatch;
        }
        return inserted;
    }
```

listing 117: 10k-row batched transactions, one prepared statement, per-batch commit latency (source: csharp-net/samples/src/Ch12/RowKernel.cs, lines 50-93)

The suite runs the 1,000-row smoke against a temp file and asserts the two aggregation passes agree, SQL `GROUP BY` min, max, and avg times 8 against a manual cursor walk of the same data. At measurement scale, 10,000,000 rows through the bench, 1,000 batches, db deleted after, this machine, 2026-09-13:

stage	measured	rate
insert, 10k-row transactions	29.5 s for 10,000,000 rows	339,091 rows/s
batch commit p50 / p95	27.1 ms / 37.9 ms	over 1,000 batches
batch commit p99 / p99.9	48.5 ms / 120.8 ms	the tail the inserts buy
aggregate, sql group by	13.8 s	723,248 rows/s
aggregate, manual cursor walk	19.9 s	501,820 rows/s
agreement fold, sql against cursor	3296.2817847056854 both	delta 0.00
database size	870,072,320 bytes	about 830 MB, deleted after

The engine's aggregate beats the cursor walk by 1.4 times at this scale, the GROUP BY does the same scan in C with the page cache hot. The 10m rate extrapolates sanely, so the 100 million row one-off ran, alone and sequentially, database deleted after, this machine, 2026-09-13: insert 371.4 s at 269,216 rows/s across 10,000 batches with p50 36.5 ms, p99 50.2 ms, p99.9 135.0 ms, aggregate 314,663 rows/s by SQL against 267,968 by cursor walk, folds equal at 3296.81315815615, database 8,735,449,088 bytes. The rates drop a fifth from the 10m run and the SQL-to-cursor gap narrows from 1.4 to 1.2, both the page cache giving way to disk, and both one-off databases are gone. One billion rows is arithmetic, not a measurement on this machine, roughly 87 GB at this row shape and half an hour of insert at the measured rate.

12.7.1. the same kernel on duckdb

The corpus ran this kernel on duckdb too, from c and from go, and those chapters hold the engine-side numbers. The c twin, same 412 stations and rows bit-identical to its sqlite lane, measured the appender at 1.05 to 1.12 million rows a second against sqlite's batched 464,047, the group by at 79.6 to 83.3 million against 555,402, 143x to 150x, while the go lane appended 774,437 rows a second with its group by at 124,918,491 over the engine's own thread pool, all 2026-09-20 on duckdb 1.5.5. `Microsoft.Data.Sqlite` stays the engine here anyway: the tail quoted above is a batch commit's, p99 over 1,000 transactions, and the capstone's save store rides the same pins, while the appender opens no transaction and offers no per-batch commit to time. Which engine a workload wants is measured in book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine) and chosen in book 13, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine).

12.8. which tool when

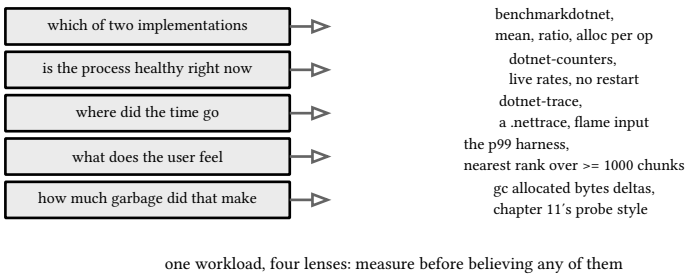


figure 83: which tool answers which question

The tools compose: the counters panel flags a GC tax, the trace says which phase pays it, the benchmark A/Bs the fix, and the percentile harness proves the tail moved. Nothing in this chapter replaces the discipline chapter 11 closed on, the representation you pick has to be re-measured at your scale, because both of this chapter's measured workloads changed their answer between 100k and 1m particles and between a cache-hot engine scan and a manual walk.

12.9. across the six books

The same two kernels ran in all six language books under one contract, N particles of 8 float64 fields stepped 16 times and 10 million readings rows of one station key plus 8 real columns, every number below quoted from the

chapter that measured it on this machine, 2026-09-13. The first two columns are the particle step at 1m in each language's row-of-objects repr and its flat column repr, and they disagree on purpose: the flat repr wins 1.4x in c, 2.6x in javascript, and 21x in python, the object repr wins 2.3x in c# and 3.8x in go because the step touches all eight fields of every particle and one cache line per particle beats eight streams in flight, and lua's ffi cdata wins 11.5x over boxed tables. The last four columns are the shared sqlite contract at 10m rows, insert through 10k-row batched transactions, the sql group by, the manual cursor walk, and the p99 of one 10k-row commit. Lua's p99 cell sits empty on purpose: its os.clock ticks at 1 ms and the chapter refuses to quote a percentile it cannot read, the boundary every book states in its own clock.

book	objects ns	flat ns	insert/s	group/s	walk/s	p99 ms
c, clang -O2	6.15 to 6.75	4.53 to 4.91	468,803	584,696	2,242,885	28.3
c#, .net 11	5.7	13.1	339,091	723,248	501,820	48.5
go 1.27	8.2	30.9	267,942	324,614	610,720	67.957
javascript, node 26	12.4	4.72	530.42k	851k	562.3k	17.914
python 3.14	345.7	16.18	327,703	536,378	307,199	39.60
lua 5.5 + luajit	114.1	9.9	504,765	573,915	580,353	

Table 1: the two kernels across the six language books: particle step ns at 1m in each book's object and flat repr, then the 10m row kernel with insert, group by, and walk in rows per second and p99 in milliseconds per 10k-row commit, every cell quoted from the chapter that measured it

The ladder above the table stays in its own book: the 100m particle one-off measured 5.768 ns flat and 7.229 ns object in c, 21.815 ns flat in c# (memory bound, soa only), 8.898 ns flat in go, 4.63 ns flat in javascript, 16.88 ns flat in python, and 8.2 ns ffi in lua, with the boxed 100m priced at 13.9 GB and refused. The 100m row one-off held its 10m insert rate within 5 percent in c, javascript, and lua, gained 4 percent in python, and paid 8 percent in go and 21 percent in c#. One billion is arithmetic in every book, 64 GB of particle state and roughly 81 to 87 GB of rows, not measured on this machine.

sources: benchmarkdotnet.org, benchmarkdotnet docs overview and the inprocess toolchain page, learn.microsoft.com, dotnet-counters, dotnet-trace, well-known event counters in .net, and the .net runtime metrics pages, accessed 2026-09-13. Counter and trace behavior verified against the committed capture in `samples/captures/` and the bench under `samples/bench/`, both made on this machine, 2026-09-13. Sample behavior verified by `make verify-csharp`, 12 tests.

13. reflection, attributes, source generators

Three mechanisms let code talk about code. Reflection reads types, members, and metadata at runtime. Attributes attach metadata to those types. Source generators read the compilation at compile time and add code to it, which moves work that reflection or hand written boilerplate used to do into the build. `dynamic` sits at the edge of the group, late bound dispatch without the reflection ceremony.

13.1. attributes

An attribute is a class deriving from `Attribute`, applied in brackets. `AttributeUsage` constrains where it can appear, and constructor parameters plus properties become the stored data:

```
// an attribute is a class; metadata is data
[AttributeUsage(AttributeTargets.Class)]
public sealed class AuditedAttribute(string tag) : Attribute
{
    public string Tag { get; } = tag;
}
```

listing 118: a custom attribute with one payload (source: csharp-net/samples/src/Ch13/Meta.cs, lines 5-10)

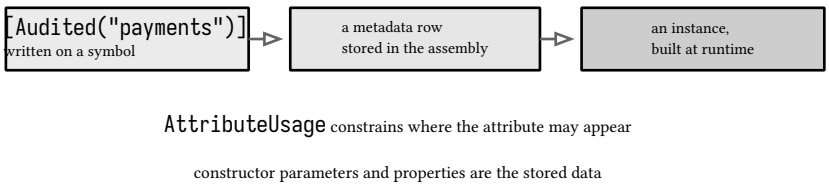


figure 84: an attribute is a metadata row that materializes on demand

Convention ends attribute class names in `Attribute` and lets the suffix disappear at the use site, `[Audited("payments")]` in the listing below is `AuditedAttribute`.

13.2. reflection

`typeof(T)` and `obj.GetType()` produce `System.Type`, the entry point to everything the runtime knows: base types, interfaces, properties, methods, fields, and applied attributes. `MethodInfo.Invoke` calls a method found by name:

```
[Audited("payments")]
public sealed class Payment
{
    public decimal Amount { get; init; }

    public string Owner { get; init; } = "";
}
```

listing 119: the attributed type the samples reflect over (source: csharp-net/samples/src/Ch13/Meta.cs, lines 12-18)

```
public static List<string> PublicPropertyNames()
    => [.. typeof(Payment).GetProperties().Select(p => p.Name)];

public static string AuditTag(Type type)
    => type.GetCustomAttribute<AuditedAttribute>()?.Tag ?? "none";

public static string InvokeToString(object target)
    => (string)target.GetType()
        .GetMethod("ToString", Type.EmptyTypes)!
        .Invoke(target, null)!;
```

listing 120: listing properties, reading an attribute, invoking by name (source: csharp-net/samples/src/Ch13/Meta.cs, lines 22-31)

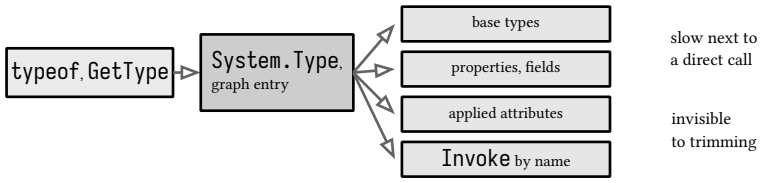


figure 85: system.type, the entry point to the metadata graph

Reflection is how test frameworks find your tests, how serializers map records to JSON, and how dependency containers build object graphs. It is also slow compared to a direct call and invisible to trimming, which is why every hot use of it in the modern library has been replaced by source generation. Trimming, the publish step that strips unreferenced code to shrink the output, cannot see what reflection will look up at runtime, the second reason the library moved to generators.

`nameof` is the compile time cousin: it produces the name of any symbol as a string constant, so renames keep logs and argument checks honest:

```
public static (string Type, string Prop) NamesAtCompileTime()
    => (nameof(Payment), nameof(Payment.Amount));
```

listing 121: names without string literals (source: csharp-net/samples/src/Ch13/Meta.cs, lines 39-40)

13.3. dynamic

`dynamic` defers binding from compile time to runtime. The compiler skips checking member access on `dynamic` values, and the runtime binds using the actual type:

```
public static string DynamicToString(object target)
{
    dynamic d = target;
    return d.ToString(); // bound at runtime, checked nowhere
}
```

listing 122: a call the compiler never checks (source: csharp-net/samples/src/Ch13/Meta.cs, lines 33-37)

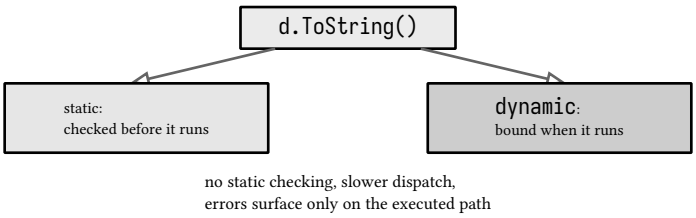


figure 86: dynamic defers binding from compile time to run time

The cost is exactly what the listing says: no static checking, slower dispatch, and errors that surface only on the executed path. It earns its keep talking to COM, dynamic languages, and JSON payloads whose shape is known

but never modeled. Everywhere else, a record plus a pattern match over `object` gives the same flexibility with the compiler still in the room.

13.4. source generators

A source generator is a small program the compiler runs during the build. It reads the compilation's syntax trees, picks out what interests it, and emits new source files that compile as if you wrote them. Generated code is checked, debuggable, and free of the runtime cost reflection pays. The modern Roslyn shape is the **incremental** generator, a pipeline of cached stages so an edit re-runs only the stages whose inputs changed.

The sample solution carries a real one. The consumer side is one class with a marker attribute:

```
// the marker the source generator looks for, matched by name
[AttributeUsage(AttributeTargets.Class)]
public sealed class GenerateSummaryAttribute : Attribute;

// Describe() is not in this file. The generator adds it at compile time.
[GenerateSummary]
public sealed class Widget
{
    public int Part { get; init; }
}
```

listing 123: the marker attribute and a class with no Describe method in sight (source: csharp-net/samples/src/Ch13/Generated.cs, lines 3-12)

The generator finds every class carrying `[GenerateSummary]` and emits an extension method for it:

```
// an incremental source generator: syntax in, source out, cached between edits
[Generator]
public sealed class SummaryGenerator : IIncrementalGenerator
{
    public void Initialize(IncrementalGeneratorInitializationContext context)
    {
        var marked = context.SyntaxProvider.CreateSyntaxProvider(
            static (node, _) => node is ClassDeclarationSyntax { AttributeLists.Count: > 0 },
            static (ctx, _) => (ClassDeclarationSyntax)ctx.Node)
            .Where(static cls => cls.AttributeLists
                .SelectMany(static list => list.Attributes)
                .Any(static attr => attr.Name.ToString()
                    is "GenerateSummary" or "GenerateSummaryAttribute"));

        context.RegisterSourceOutput(marked, static (spc, cls) =>
        {
            var ns = NamespaceOf(cls);
            var name = cls.Identifier.ValueText;
            var source = new StringBuilder()
                .AppendLine("// <auto-generated by CsharpBook.Generators.SummaryGenerator />")
                .AppendLine("#nullable enable")
                .AppendLine($"namespace {ns};")
                .AppendLine()
                .AppendLine($"public static class {name}Summaries")
                .AppendLine("{")
                .AppendLine($"    public static string Describe(this {name} self)")
                .AppendLine($"        => \"generated summary for {name}\";")
                .AppendLine("}")
                .ToString();
            spc.AddSource($"{name}Summary.g.cs", source);
        });
    }
```

listing 124: the initialize pipeline: filter syntax, emit source (source: csharp-net/samples/src/CsharpBook.Generators/SummaryGenerator.cs, lines 9-40)

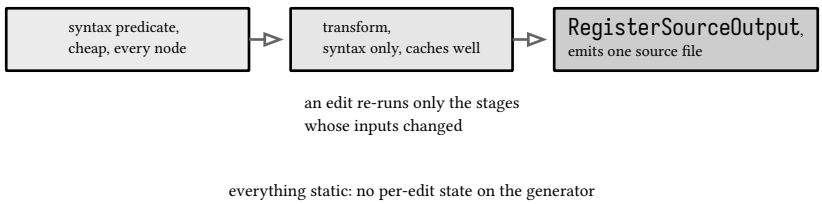


figure 87: the incremental generator, cached stages, static end to end

The pipeline is the whole design. `CreateSyntaxProvider` runs a cheap predicate over every syntax node, then a transform on the survivors. `Where` filters by attribute name, still syntax only, no semantic model, so the stage caches well. `RegisterSourceOutput` finally emits one source file per match. Everything is `static` so no per-edit state accumulates on the generator instance.

Wiring it in is one project reference with `OutputItemType="Analyzer"`, a `netstandard2.0` target for the generator itself:

```
  <ItemGroup>
    <!-- the generator and the preview project live in nested folders, keep
         them out of this project's glob -->
    <Compile Remove="CsharpBook.Generators\**" />
    <Compile Remove="CsharpBook.Preview\**" />
    <ProjectReference Include="CsharpBook.Generators\CsharpBook.Generators.csproj"
                      OutputItemType="Analyzer" ReferenceOutputAssembly="false" />
  </ItemGroup>
```

listing 125: an analyzer reference, no runtime assembly (source: csharp-net/samples/src/CsharpBook.Samples.csproj, lines 10-17)

The test is the proof. It was written first, failed to compile with CS1061 because `Widget.Describe` did not exist, and passed only after the generator ran:

```
    public void SourceGenerator_AddedDescribeAtCompileTime()
        => Assert.Equal("generated summary for Widget", new Widget().Describe());
```

listing 126: the red test that the generator turned green (source: csharp-net/samples/tests/Ch13/MetaTests.cs, lines 31-32)

NOTE: what generators replaced

`[GeneratedRegex]` compiles a regex at build time instead of the first call. `System.Text.Json` source generation writes the serializer metadata a profile used to build with reflection. System interop uses generated marshallers instead of hand written `fixed` blocks, which is why chapter 10 called the pointer surface interop territory. The capstone interpreter uses the same shape to register its builtin functions.

sources: learn.microsoft.com, reflection overview, attributes, nameof, dynamic type, the roslyn sdk overview, and the incremental generators cookbook at github.com/dotnet/roslyn, accessed 2026-09-08. Generator behavior verified by `make verify-csharp`, 6 tests.

14. linq

LINQ is a query language over sequences. Anything implementing `IEnumerable<T>` can be filtered, projected, ordered, grouped, and joined with a uniform vocabulary, and because the operators are just extension methods over `IEnumerable<T>`, the whole system is open: LINQ to Objects runs in memory, the same syntax targets databases through `IQueryable<T>`, where operators become expression trees, the lambdas captured as data that the provider translates. LINQ is also where this book's chapter 5 variance and chapter 7 delegates pay off, every operator takes a `Func<...>` and `IEnumerable<out T>` is what lets one query method serve every element type.

The demonstrations below all run over rows in a real SQLite database: `OpenShop` seeds the `product`, `shopper`, and `line` tables with integer-cent prices in an in-memory file, and typed readers turn each `SELECT` into the `IEnumerable<T>` the operators consume, so the test suite checks every LINQ answer against the equivalent plain SQL.

```
// an in-memory SQLite shop: schema plus seeded demo rows
public static SqliteConnection OpenShop()
{
    var connection = new SqliteConnection("Data Source=:memory:");
    connection.Open();
    using var seed = connection.CreateCommand();
    seed.CommandText = """
        CREATE TABLE product (
            name     TEXT PRIMARY KEY,
            category TEXT NOT NULL,
            cents    INTEGER NOT NULL
        );
        CREATE TABLE shopper (name TEXT PRIMARY KEY, category TEXT NOT NULL);
        CREATE TABLE line (text TEXT NOT NULL);
        INSERT INTO product (name, category, cents) VALUES
            ('masa', 'kitchen', 12000),
            ('pot',   'kitchen', 4500),
            ('desk',  'office',  30000),
            ('lamp',  'office',  8000),
            ('pen',   'office',  200);
        INSERT INTO shopper (name, category) VALUES
            ('ana',  'office'),
            ('binh', 'kitchen');
        INSERT INTO line (text) VALUES ('alpha beta'), ('beta gamma');
        """;
    seed.ExecuteNonQuery();
    return connection;
}
```

listing 127: the shop as schema and seed rows, one in-memory SQLite file (source: csharp-net/samples/src/Ch14/Queries.cs, lines 14-41)

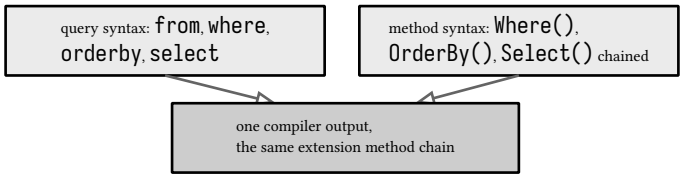
14.1. method syntax and query syntax

Two spellings, one compiler output. Method syntax chains extension methods with lambda arguments. Query syntax is the `from ... where ... select` form the compiler translates into exactly those calls:

```
// method syntax: a chain of operators
public static List<string> CheapSorted(SqliteConnection connection)
    => Products(connection).Where(p => p.Price < 100m)
        .OrderBy(p => p.Price)
        .Select(p => p.Name)
        .ToList();

// query syntax: the same plan, compiler-identical
public static List<string> CheapSortedQuery(SqliteConnection connection)
    => (from p in Products(connection)
        where p.Price < 100m
        orderby p.Price
        select p.Name).ToList();
```

listing 128: one plan written twice, producing equal results (source: csharp-net/samples/src/Ch14/Queries.cs, lines 89-101)



query syntax covers the common relational shape, method syntax covers every operator

figure 88: two spellings, one compiler output

Method syntax covers every operator, query syntax covers the common relational shape and reads better once `join` and `group` appear. Use either consistently within a file.

14.2. deferred execution

A query expression builds a plan, it does not run. Nothing executes until enumeration, and every enumeration runs the plan again against the current data:

```
// a query is a plan, each enumeration runs it against the current table
public static (int Before, int After) DeferredExecution(SqliteConnection connection)
{
    var query = Products(connection).Where(p => p.Price > 0m); // nothing has run yet
    var before = query.Count();                                // one SELECT: 5
    Insert(connection, new("crate", "office", 60m));
    var after = query.Count();                                 // a fresh SELECT: 6
    return (before, after);
}
```

listing 129: two counts, one query, different answers (source: csharp-net/samples/src/Ch14/Queries.cs, lines 103-111)

The operators that return something other than `IEnumerable<T>` (`ToList`, `ToArray`, `ToDictionary`, `ToHashSet`, `First`, `Count`, `Any`, `Sum`) execute immediately and materialize. That is the lever: keep the query deferred while composing, materialize once at the end, and never hand a live query to code that expects a snapshot.

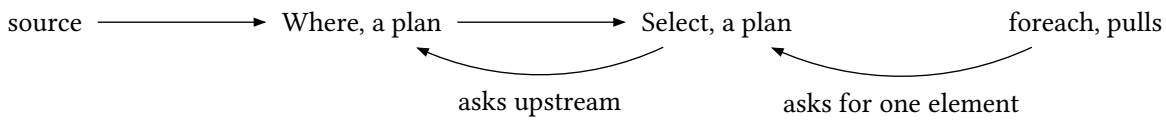


figure 89: deferred execution, enumeration pulls, nothing runs until asked

14.3. the operator families

Filtering and projection are `Where` and `Select`. Flattening is `SelectMany`. Ordering is `OrderBy` and `ThenBy`, both stable. The element operators are `First`, `FirstOrDefault`, `Single`, `SingleOrDefault`, each with a reason to be chosen: `Single` throws on duplicates by design, `First` takes the earliest match. Quantifiers `Any`, `All`, `Contains` answer boolean questions and short circuit. Set operators `Distinct`, `Union`, `Intersect`, `Except` use default or supplied equality. Partitioning is `Take`, `Skip`, `TakeWhile`, `SkipWhile`:

```
public static List<string> DistinctWords(SqliteConnection connection)
    => Lines(connection).SelectMany(line => line.Split(' ')).Distinct().Order().ToList();
```

listing 130: SelectMany flattens, Distinct dedupes, Order sorts (source: csharp-net/samples/src/Ch14/Queries.cs, lines 146-147)

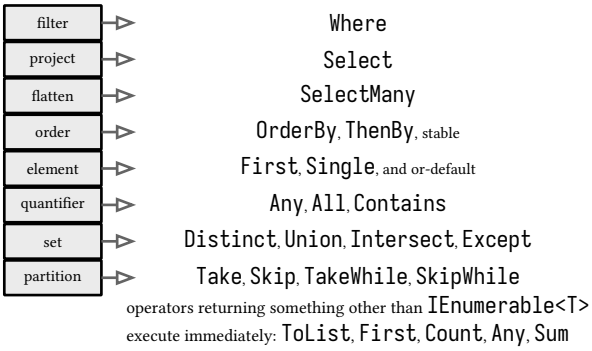


figure 90: the operator families, one job each

14.4. grouping and aggregation

`GroupBy` clusters a sequence by a key and returns a sequence of groups, each carrying its key and its members:

```
public static Dictionary<string, int> CountByCategory(SqliteConnection connection)
    => Products(connection).GroupBy(p => p.Category)
        .ToDictionary(g => g.Key, g => g.Count());
```

listing 131: GroupBy into a dictionary of counts (source: csharp-net/samples/src/Ch14/Queries.cs, lines 113-115)

In query syntax, `group ... by ... into` introduces the group as a new range variable and the query continues per group:

```
// group ... by ... into continues the query per group
public static Dictionary<string, List<Product>> GroupByQuery(SqliteConnection connection)
    => (from p in Products(connection)
        group p by p.Category into g
        select (g.Key, Items: g.ToList()))
        .ToDictionary(x => x.Key, x => x.Items);
```

listing 132: group by with an into continuation (source: csharp-net/samples/src/Ch14/Queries.cs, lines 117-122)

Aggregation folds a sequence to a value. `Sum`, `Min`, `Max`, `Average`, `Count` are specialized folds, and the general one is `Aggregate`, which takes a function from accumulator and element to accumulator:

```
public static (decimal Total, decimal Max) AggregatePrices(SqliteConnection connection)
{
    var products = Products(connection).ToList();
    var total = products.Sum(p => p.Price);
    var max = products.Aggregate((a, b) => a.Price > b.Price ? a : b).Price;
    return (total, max);
}
```

listing 133: a specialized fold and a general one (source: csharp-net/samples/src/Ch14/Queries.cs, lines 138-144)

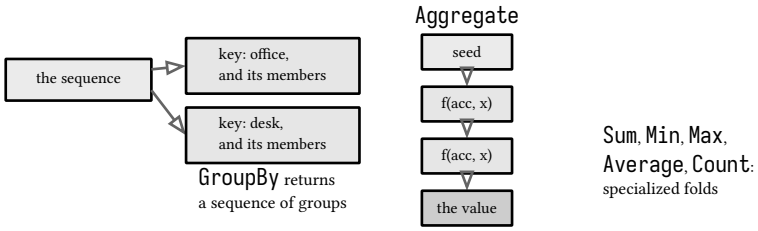


figure 91: groupby clusters by key, aggregate folds to a value

14.5. join and let

`join ... on ... equals ...` is an equi-join by key, inner by default, `join ... into` for grouping joins. Note the shape of the `on` clause, it names both sides around the `equals` keyword rather than a boolean expression:

```
// join ... on ... equals pairs two sources by key
public static List<string> WhoBuysWhat(SqliteConnection connection)
    => (from s in Shoppers(connection)
        join p in Products(connection) on s.Category equals p.Category
        where p.Price > 100m
        select $"{s.Name} wants {p.Name}").ToList();
```

listing 134: an equi-join across shoppers and catalog (source: csharp-net/samples/src/Ch14/Queries.cs, lines 131-136)

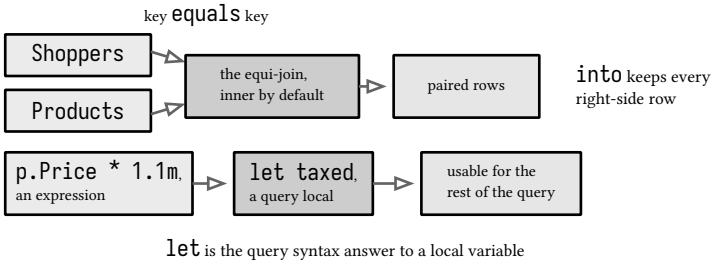


figure 92: join pairs sources by key, let binds a name mid query

`let` binds a name to an intermediate value usable for the rest of the query, the query syntax answer to a local variable:

```
// let names an intermediate inside the query
public static List<string> WithLet(SqliteConnection connection)
    => (from p in Products(connection)
        let taxed = p.Price * 1.1m
        where taxed > 100m
        select $"{p.Name}: {taxed:F0}").ToList();
```

listing 135: let computing a taxed price mid query (source: csharp-net/samples/src/Ch14/Queries.cs, lines 124-129)

14.6. outer joins

`join` is inner-only: a row without a counterpart on the other side disappears. .NET 11 completes the relational vocabulary with three operators, `LeftJoin` keeps every left row, `RightJoin` keeps every right row, `FullJoin` keeps both, and each returns `(outer, inner)` tuples with `default` on whichever side found no match. The seed tables match on both sides, so the demo extends the live SELECTs with the two rows that make a gap visible, a shopper in a category with no products and a product in a category with no shoppers:

```
// the seed tables match on both sides, so the outer join demos below
// extend the live SELECTs with the rows that make a gap visible:
// chi browses a category with no products, rose sits in one with no shoppers
private static IEnumerable<Shopper> ShoppersWithGap(SqliteConnection connection)
    => Shoppers(connection).Append(new("chi", "garage"));

private static IEnumerable<Product> ProductsWithGap(SqliteConnection connection)
    => Products(connection).Append(new("rose", "garden", 12m));

// .NET 11: LeftJoin keeps every left row, default (null) fills the right
public static List<string> WhoMightBuyWhat(SqliteConnection connection)
    => ShoppersWithGap(connection)
        .LeftJoin(Products(connection), s => s.Category, p => p.Category)
        .OrderBy(pair => pair.Item1.Name)
        .ThenBy(pair => pair.Item2?.Name ?? "")
        .Select(pair => $"{pair.Item1.Name} wants {pair.Item2?.Name ?? "nothing"}")
        .ToList();
```

listing 136: LeftJoin keeps chi, default fills the missing right (source: csharp-net/samples/src/Ch14/Queries.cs, lines 155-171)

```
// RightJoin is the mirror: every right row, default (null) fills the left
public static List<string> WhatNobodyWants(SqliteConnection connection)
    => Shoppers(connection)
        .RightJoin(ProductsWithGap(connection), s => s.Category, p => p.Category)
        .OrderBy(pair => pair.Item2.Name)
        .Select(pair => $"{pair.Item2.Name} suits {pair.Item1?.Name ?? "nobody"}")
        .ToList();
```

listing 137: RightJoin is the mirror, rose survives without a shopper (source: csharp-net/samples/src/Ch14/Queries.cs, lines 173-179)

```
// FullJoin keeps both sides, default on whichever side found no match
public static List<string> CategoryCoverage(SqliteConnection connection)
    => ShoppersWithGap(connection)
        .FullJoin(ProductsWithGap(connection), s => s.Category, p => p.Category)
        .OrderBy(pair => pair.Item1?.Name ?? pair.Item2?.Name)
        .ThenBy(pair => pair.Item2?.Name ?? "")
        .Select(pair => $"{pair.Item1?.Name ?? "-"} pairs {pair.Item2?.Name ?? "-"}")
        .ToList();
```

listing 138: FullJoin keeps both sides of the gap (source: csharp-net/samples/src/Ch14/Queries.cs, lines 181-188)

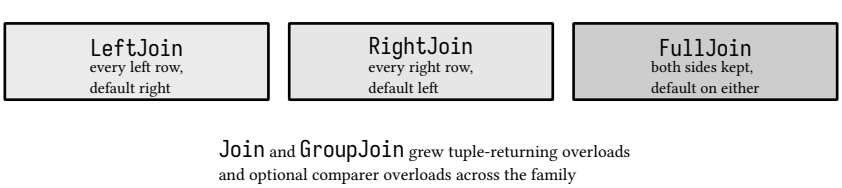


figure 93: which rows survive each join

The tuple shape feeds deconstruction and patterns directly, which is the ergonomic win over the old `join ... into` plus `Any()` dance for left joins. The family also grew: `Join` and `GroupJoin` have tuple-returning overloads that drop the result selector, every join operator takes an optional `IEqualityComparer<TKey>`, and the same operators exist on `Queryable` and `AsyncEnumerable`. EF Core 11 translates `FullJoin` to SQL `FULL JOIN`, so the vocabulary crosses the `IQueryable` boundary.

PITFALL: default is not the same as no row

A missing right side surfaces as `default(TInner)`, which is `null` for the record types above but a real zero for a value-typed element. Test the gap with a pattern, `pair.Item2 is Product p`, which is false for both null and a zeroed struct, and only reach for `?.` once the pattern says a value is there. A `NullReferenceException` inside a left join projection is almost always a missing-match case handled as if it were present.

14.7. what the chapter 2 keyword matrix promised

The two ordering directions close the set. `ascending` is the default and is usually left implicit, `descending` flips a key, and one `orderby` can mix both:

```
    // explicit ordering directions, one orderby can mix both
    public static List<string> CategoryUpPriceDown(SqliteConnection connection)
        => (from p in Products(connection)
            orderby p.Category ascending, p.Price descending
            select p.Name).ToList();
```

listing 139: explicit directions, category ascending then price descending (source: csharp-net/samples/src/Ch14/Queries.cs, lines 149-153)

Query syntax is where the contextual keywords from the lexical chapter all land: `from`, `where`, `select`, `group`, `into`, `join`, `on`, `equals`, `by`, `orderby`, `ascending`, `descending`, and `let`. Every one of them is demonstrated in the listings above, none is reserved, and all of them are method calls after compilation.

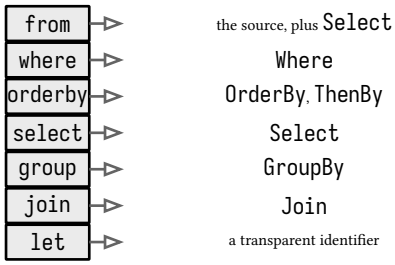


figure 94: every query keyword lands as a method call

PITFALL: captured variables and multiple enumeration

Deferred execution composes with the closure rules of chapter 7: a lambda in a query captures the variable, not the value. Changing a captured variable between building and enumerating a query changes the result. And enumerating a deferred query twice does the work twice, which is fine for a list and wrong for a database round trip, where `ToList` first is the rule.

sources: learn.microsoft.com, linq overview, query syntax and method syntax, standard query operators overview, deferred execution and lazy evaluation in linq pages accessed 2026-09-08, what's new in .NET 11 libraries, enumerable.leftjoin, rightjoin, and fulljoin pages accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 14 tests.

15. stdlib tour 1: collections, memory, numerics

Two chapters walk the parts of the base class library a working C# programmer touches weekly. This one covers collections, the memory utilities that pair with chapter 10, and the numerics stack. Chapter 16 covers io, text, networking, channels, and diagnostics.

15.1. collections

The workhorse types and their contracts. `List<T>` is the dynamic array, amortized growth, indexed access, the default sequence holder. `Dictionary<TKey, TValue>` is the hash map, and its idioms matter: `GetValueOrDefault` reads a possibly missing key without a throw, `TryGetValue` reads and tests in one step, and the indexer writes or replaces in one:

```
// List: the dynamic array
public static string ListOps()
{
    var xs = new List<int> { 1, 2 };
    xs.AddRange([3, 4]);
    xs.Remove(2);
    xs.Insert(0, 0);
    return string.Join(",", xs); // 0,1,3,4
}
```

listing 140: List as the dynamic array (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 8-16)

```
// Dictionary: the hash map, GetValueOrDefault reads without throwing
public static string DictOps()
{
    var counts = new Dictionary<string, int>();
    foreach (var word in "a b a c a".Split(' '))
    {
        counts[word] = counts.GetValueOrDefault(word) + 1;
    }
    var hit = counts.TryGetValue("a", out var a);
    return $"{hit}/{a}/{counts.GetValueOrDefault("z")}"; // True/3/0
}
```

listing 141: counting words the dictionary way (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 18-28)

.NET 11 removes the wrapper record for one small hashing problem: `EqualityComparer<T>.Create` builds a comparer from a key selector, so a dictionary can hash by the trimmed, cased, or otherwise projected form of its keys:

```
// .NET 11: EqualityComparer.Create keys equality through a selector,
// the dictionary then hashes "a", " a ", and "a " to one bucket
public static string ComparerCreate()
{
    var byTrimmed = EqualityComparer<string>.Create(s => s.Trim());
    var counts = new Dictionary<string, int>(byTrimmed);
    foreach (var word in "a, a ,a ".Split(','))
    {
        counts[word] = counts.GetValueOrDefault(word) + 1;
    }
    return $"{counts["a"]}/{byTrimmed.Equals("x", " x ")}"; // 3/True
}
```

listing 142: equality keyed through a projection, no wrapper record (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 74-85)

`HashSet<T>` is the membership structure with set algebra built in. `Queue<T>` and `Stack<T>` are fifo and lifo. `PriorityQueue<TElement, TPriority>` pops by priority, and `SortedList` and `SortedDictionary` keep order at the cost of operations becoming logarithmic:

```
// HashSet: membership and set algebra
public static string SetOps()
{
    var left = new HashSet<int> { 1, 2, 3 };
    left.UnionWith([3, 4]);
    var overlaps = left.Overlaps([4]);
    return $"{string.Join(",", left.Order())}/{left.Count}/{overlaps}"; // 1,2,3,4/4/True
}
```

listing 143: set union and membership (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 30-37)

```
// Queue is fifo, Stack is lifo, PriorityQueue pops by priority
public static string QueueAndStack()
{
    var queue = new Queue<string>();
    queue.Enqueue("first");
    queue.Enqueue("second");

    var stack = new Stack<int>();
    stack.Push(1);
    stack.Push(2);

    var tickets = new PriorityQueue<string, int>();
    tickets.Enqueue("low", 2);
    tickets.Enqueue("high", 1);
    return $"{queue.Dequeue()}/{stack.Pop()}/{tickets.Dequeue()}"; // first/2/high
}
```

listing 144: fifo, lifo, and priority order in one method (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 39-54)

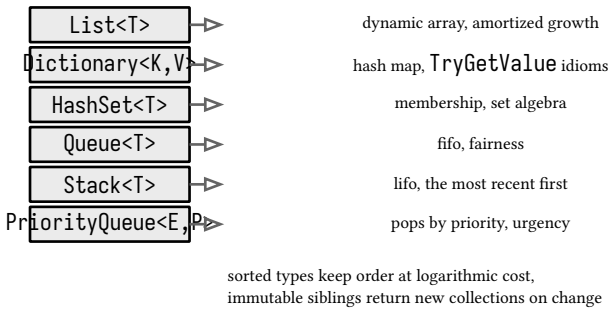


figure 95: the workhorse collections, one contract each

The immutable siblings (`ImmutableList<T>`, `ImmutableDictionary<TKey, TValue>`, and friends) share interfaces with these but return new collections on every change, the functional option for state you want to share without defensive copies.

15.2. frozen collections

`FrozenDictionary<TKey, TValue>` and `FrozenList<T>` (.NET 8) are the read heavy end of the spectrum. Building one costs more than a dictionary, reads after that are measurably faster because the structure optimizes for the actual keys at freeze time. The rule: build once at startup from a dictionary, read for the process lifetime:

```
// frozen collections: pay once at build, read cheaply forever after
public static string FrozenOps()
{
    var frozen = new Dictionary<string, int> { ["a"] = 1, ["b"] = 2 }
        .ToFrozenDictionary();
    return $"{frozen["a"]}/{frozen.ContainsKey("c")}"; // 1/False
}
```

listing 145: freeze a dictionary once, look up forever (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 56-62)

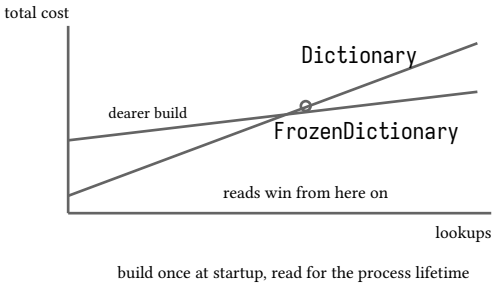


figure 96: frozen collections, pay at freeze, read cheaper forever

15.3. memory utilities

Chapter 10 covered `Span<T>`, `Memory<T>`, and `stackalloc`. The collection side of that story is `ArrayPool<T>`, the shared pool for large temporary arrays: rent, use, return, and let the next caller reuse the same allocation instead of feeding the large object heap. `ReadOnlySequence<T>` chains buffer segments for parsers. Chapter 11 goes deep on the pool's contract and its ownership rules, and both matter in parsing and networking code, which chapter 16 revisits.

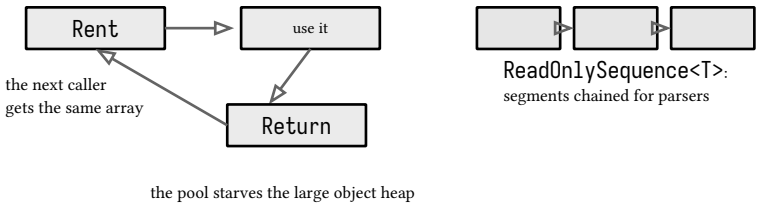


figure 97: arraypool cycles arrays, the sequence chains segments

15.4. numerics

`Math` carries the scalar functions, `Math.Clamp`, `Math.Sqrt`, and friends, with no surprises. `BigInteger` is arbitrary precision with no operator limits. The `System.Numerics` vector types, `Vector<T>` and the fixed size `Vector128<T>` through `Vector512<T>`, expose SIMD (single instruction, multiple data) without intrinsics, one expression that the JIT compiles to the widest register the machine has:

```
// numerics: Math, BigInteger, hardware vectors
public static string Numerics()
{
    var digits = BigInteger.Pow(2, 100).ToString().Length;
    var rooted = Math.Sqrt(144);
    var clamped = Math.Clamp(180, 0, 100);
    var simd = Vector.IsHardwareAccelerated ? "hw" : "soft";
    return $"{digits}/{rooted}/{clamped}/{simd}"; // 31/12/100/hw or soft
}
```

listing 146: big integers, scalar math, and a hardware check (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 64-72)

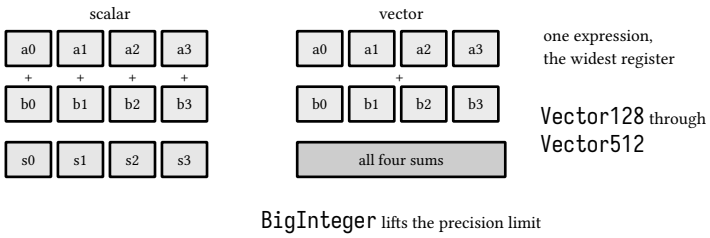


figure 98: simd lanes, one wide add against four narrow ones

The `INumber<T>` family from chapter 5 is the bridge between these: generic algorithms written once run over `int`, `double`, `BigInteger`, and the vectors' element types.

15.5. decimal floating point

`double` cannot store one tenth: 0.1 is an infinitely repeating binary fraction, and ten additions land just under one. .NET 11 adds the IEEE 754-2019 decimal encodings, `Decimal32` with 7 significant digits, `Decimal64` with 16, and `Decimal128` with 34, where a decimal digit is stored directly and one tenth is exact. They carry infinities and NaN like the binary floats, they implement the generic math interfaces so the chapter 5 algorithms run over them, and conversion from `decimal` keeps the scale:

```
// .NET 11: IEEE 754-2019 decimal floats, one tenth is exact,
// so ten additions land on exactly one where the binary sum misses
public static string DecimalExactness()
{
    var decimalSum = Decimal128.Zero;
    var doubleSum = 0.0;
    for (var i = 0; i < 10; i++)
    {
        decimalSum += (Decimal128)0.1m;
        doubleSum += 0.1;
    }
    return $"{decimalSum == Decimal128.One}/{doubleSum == 1.0}"; // True/False
}
```

listing 147: tenths sum to exactly one, the binary sum misses (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 87-99)

Decimal encodings also make trailing zeros meaningful. `1.00` and `1` are equal values at different quantum, the unit of the last stored digit, which is what `decimal`'s scale was approximating all along. Two values can be equal and still fail `HaveSameQuantum`, money that claims cent precision against money that does not:

```
// trailing zeros are data: 1.00 and 1 are equal values at different
// quantum, the unit of the last stored digit
public static string QuantumScale()
{
    var priced = (Decimal128)1.00m;
    var whole = (Decimal128)1m;
    return $"{priced == whole}/" +
        $"{Decimal128.HaveSameQuantum(priced, whole)}"; // True/False
}
```

listing 148: equal values, different quantum, trailing zeros are data (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 101-109)

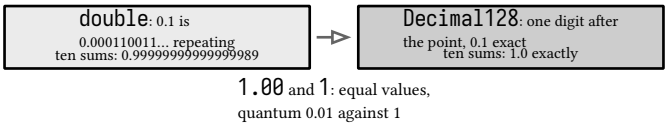


figure 99: binary cannot name one tenth, decimal can

The same release adds `BFloat16`, the 16-bit brain float from the ML world: `float`'s exponent range with 8 mantissa bits, roughly three decimal digits, for storing tensors that are computed elsewhere at full width. `BitConverter` converts it to and from bits, and `Complex<T>` now runs over it, the decimal types, `float`, and `Half` alike:

```
    // BFloat16: the 16-bit brain float, float's exponent range with
    // 8 mantissa bits, built for ML tensor storage
    public static string TruncatedFloat()
    {
        var exact = (BFloat16)1.5f;
        var rounded = (BFloat16)1.23456789f;
        return $"{exact}/{rounded}"; // 1.5/1.234
    }
```

listing 149: range kept, precision spent: 1.23456789 lands on 1.234 (source: csharp-net/samples/src/Ch15/Stdlib.cs, lines 111-118)

NOTE: decimal the type against decimal the encodings

`decimal` stays the money type for in-process arithmetic: base-10 scaled integers, exact for addition and subtraction, no NaN, and every existing API takes it. Reach for the IEEE decimal types when something outside the process speaks decimal, SQL `DECIMAL`, JSON numbers past 28 digits, or another implementation of the standard, or when the huge exponent range, infinities, and NaN are the shape of the data rather than a bug.

NOTE: choosing a collection

Default to `List<T>` for order and `Dictionary<TKey, TValue>` for lookup. Switch to `HashSet<T>` when membership questions dominate, `Queue<T>` when fairness matters, `PriorityQueue` when urgency does, sorted types when iteration order is the product, immutable types when sharing beats mutation, and frozen types when reads outnumber writes by thousands to one. Book 10 implements most of these from first principles, which is where the performance intuition comes from.

sources: learn.microsoft.com, system.collections.generic overview, frozen collections, arraypool, system.numerics, and bigint pages accessed 2026-09-08, what's new in .NET 11 libraries, decimal32, decimal64, decimal128, bfloat16, and equalitycomparer.create pages accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 10 tests.

16. stdlib tour 2: io, text, net, channels, diagnostics

The second half of the weekly toolbox: serialization, files, text, http, channels, and the small utilities every program ends up needing.

16.1. System.Text.Json

JSON support is built around records and reflection-free speed. A record serializes property by property and deserializes through its constructor, and the source generated mode from chapter 13 removes the reflection cost for trim-safe publishing. Options live in `JsonSerializerOptions`, and reusing one instance matters, each new options object rebuilds its metadata cache:

```
// System.Text.Json: records serialize by property name
public static string JsonRoundTrip()
{
    var invoice = new Invoice("acme", 99.5m, ["desk", "lamp"]);
    var json = JsonSerializer.Serialize(invoice, Options);
    var back = JsonSerializer.Deserialize<Invoice>(json);
    return $"{back!.Customer}/{back.Total}/{back.Lines[1]}"; // acme/99.5/lamp
}
```

listing 150: a record through serialize and deserialize (source: csharp-net/samples/src/Ch16/Streams.cs, lines 29-36)

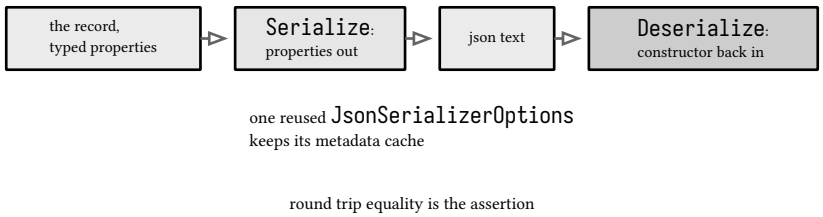


figure 100: the json round trip, properties out, constructor back in

The naming policy set gains `JsonNamingPolicy.PascalCase` in .NET 11, so a .NET type can meet a Pascal-cased wire without renaming anything on either side. The per-type `[JsonNamingPolicy]` attribute covers the other policies but rejects `PascalCase` under the book's pinned RC1 SDK, the options-level lever is the one that works:

```
// .NET 11: JsonNamingPolicy.PascalCase, the options-level lever
public static string PascalPolicy()
{
    var options = new JsonSerializerOptions
    {
        PropertyNamingPolicy = JsonNamingPolicy.PascalCase,
    };
    return JsonSerializer.Serialize(new Invoice("acme", 99.5m, []), options);
    // {"Customer":"acme","Total":99.5,"Lines":[]}
}
```

listing 151: the wire in Pascal, the record untouched (source: csharp-net/samples/src/Ch16/Streams.cs, lines 138-147)

The same namespace carries `Utf8JsonReader` and `Utf8JsonWriter` for span level, allocation free work, and `JsonDocument` for read only DOM access. The chapter 35 capstone uses none of them: its serialization work is `JsonSerializer` round trips over the c# 15 union AST, and its save store is `Microsoft.Data.Sqlite`.

16.2. unions on the wire

.NET 11 teaches the serializer the chapter 3 unions, reflection and source generator both. Serialization writes the active case and nothing else, no wrapper, no envelope: a union of `int`, `string`, and a record puts `7`, `"ping"`, or the record's object on the wire. Reading runs one rule: the JSON value kind must name the case. Number, string, and object each own a kind, so a union whose cases are distinct kinds round trips with no configuration at all:

```
// .NET 11: a union serializes as its active case, nothing else. When the
// cases are value kinds (number, string, object) each kind names its
// case, so the round trip needs no configuration
public static string UnionByValueKind()
{
    var number = JsonSerializer.Serialize<Telemetry>(7);              // 7
    var text = JsonSerializer.Serialize<Telemetry>("ping");           // "ping"
    var reading = JsonSerializer.Serialize<Telemetry>(new Sample(2.5, "ps"));
    var back = JsonSerializer.Deserialize<Telemetry>(reading);
    return $"{number}/{text}/{back is Sample s && s.Level == 2.5}"; // 7/"ping"/True
}
```

listing 152: the case is the payload, the value kind names the case on the way back (source: csharp-net/samples/src/Ch16/Streams.cs, lines 105-115)

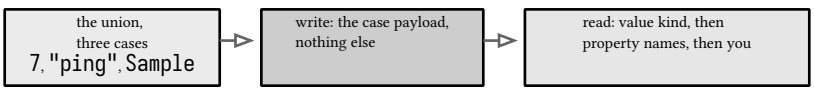
Object cases collide, every record is a JSON object, and the deserializer refuses to guess. The built-in `JsonUnionTypeStructuralClassifier` resolves what the property names can: registered once on a reused options instance, it picks the case whose required properties the payload carries:

```
// object cases need a classifier: the structural one reads the property
// names to pick the case, added once to a reused options instance
private static readonly JsonSerializerOptions Classified = BuildClassified();

private static JsonSerializerOptions BuildClassified()
{
    var options = new JsonSerializerOptions();
    options.TypeClassifiers.Add(
        new System.Text.Json.Serialization.JsonUnionTypeStructuralClassifier());
    return options;
}

public static string UnionByPropertyShape()
{
    var json = JsonSerializer.Serialize<Adjustment>(new Discount(10m));
    var back = JsonSerializer.Deserialize<Adjustment>(json, Classified);
    var charge = JsonSerializer.Deserialize<Adjustment>(
        """{"Amount":4.5,"Reason":"rush"}""", Classified);
    return $"{json}/{back is Discount}/{charge is Charge}"; // {"Percent":10}/True/True
}
```

listing 153: the structural classifier reads property names to pick the case (source: csharp-net/samples/src/Ch16/Streams.cs, lines 117-136)



distinct value kinds need no config, object cases need the classifier

same-shaped cases: the classifier rejects them at configure,
a custom `JsonTypeClassifier` or a discriminant property is the escape hatch

figure 101: serialize writes the case, deserialize classifies it

PITFALL: same-shaped cases are the wall

The classifier only works while property names can tell the cases apart. Chapter 3's `Pet(Cat, Dog, Bird)`, three cases that each carry one `Name`, serializes fine, but the structural classifier refuses the union the first time the options touch it: a case that can never be selected uniquely, because `Cat` recognizes every property `Dog` does, is a design the classifier rejects rather than misreads. Same-shaped cases need a custom `JsonTypeClassifier` through `[JsonUnion]` or `TypeClassifiers`, or a discriminant property on the wire.

16.3. files and streams

`File` and `Path` cover the common cases in one call each. Anything owning a handle is `IDisposable`, and `using` scopes the lifetime to the block, the compiler writes the `try/finally`. `Stream` is the base abstraction, `MemoryStream` for buffers, `FileStream` for disk, and readers and writers layer encoding on top:

```
// file io: the using statement owns the handle
public static string WriteAndRead()
{
    var path = Path.Combine(Path.GetTempPath(), $"csharpbook-{Guid.NewGuid():N}.txt");
    try
    {
        File.WriteAllText(path, "handle with care");
        return File.ReadAllText(path).Trim();
    }
    finally
    {
        File.Delete(path);
    }
}
```

listing 154: write, read, and clean up a temp file (source: csharp-net/samples/src/Ch16/Streams.cs, lines 38-51)

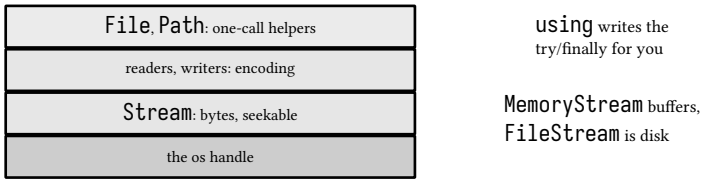


figure 102: the io stack, one call helpers over streams over handles

.NET 11 closes the gap between text and streams in one direction: `StringStream` is a string as a read-only, forward-only UTF-8 stream, so an API that wants a `Stream` gets one without the `GetBytes` array in the middle. Seeking is off, `CanSeek` is false and `Length` throws, which is the honest contract for text being consumed once. The span companions `ReadOnlyMemoryStream` and `WritableMemoryStream` cover the seekable cases over buffers you already own:

```
// .NET 11: StringStream is a string as a read-only, forward-only
// UTF-8 stream, no intermediate byte array
public static string StringToStream()
{
    using var stream = new StringStream("handle with care", Encoding.UTF8);
    var flags = $"{stream.CanRead}/{stream.CanWrite}/{stream.CanSeek}";
    using var reader = new StreamReader(stream);
    return $"{reader.ReadToEnd()}/{flags}"; // handle with care/True/False/False
}
```

listing 155: text handed to a stream api, no byte array in between (source: csharp-net/samples/src/Ch16/Streams.cs, lines 149-157)

16.4. text

`string` is immutable UTF-16, so repeated concatenation in a loop allocates a new string each pass. `StringBuilder` is the mutable scratch buffer, and `Encoding.UTF8` converts between text and bytes, with `GetByteCount` measuring before you allocate:

```
// StringBuilder composes, encoding measures the wire cost
public static string TextWork()
{
    var sb = new StringBuilder("field1,field2");
    sb.Append(',').Append("field3");
    var bytes = Encoding.UTF8.GetByteCount(sb.ToString());
    return $"{sb}/{bytes}"; // field1,field2,field3/20
}
```

listing 156: composing and measuring without churn (source: csharp-net/samples/src/Ch16/Streams.cs, lines 96-103)

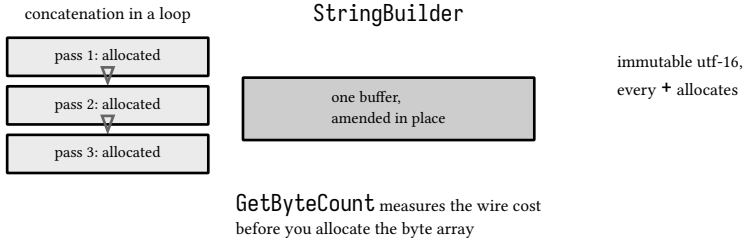


figure 103: string churn against one builder buffer

Compare strings with `string.Equals(a, b, StringComparison.Ordinal)` or its case-insensitive sibling, never with `==` when the inputs come from outside the process. The culture-aware comparisons belong to display, not logic, and chapter 17 returns to this rule.

.NET 11 finishes the rune story on `string` itself. `IndexOf`, `Contains`, `StartsWith`, `Trim`, and `Split` take a `Rune`, `StringBuilder` enumerates, appends, and replaces by rune, and `ToUpperOrdinal` and `ToLowerOrdinal` case text with no culture in sight. One rune is not one `char`: the rocket below is one code point, two UTF-16 units, and `IndexOf` reports the unit where it starts:

```
// .NET 11: rune-aware string APIs, one rune is not always one char
public static string RuneWork()
{
    var mixed = "ab🚀cd";
    var rocket = new Rune(0x1F680);
    return $"{mixed.Length}/{mixed.EnumerateRunes().Count()}/" +
        $"{mixed.IndexOf(rocket)}/{"école café".ToUpperOrdinal()}";
    // 6/5/2/ÉCOLE CAFÉ
}
```

listing 157: six chars, five runes, and the unit index where the rocket starts (source: csharp-net/samples/src/Ch16/Streams.cs, lines 159-167)

16.5. http

`HttpClient` is built for reuse, one client per process or per endpoint, not one per request, sockets exhaust otherwise. Its handler pipeline is injectable, which is what makes http code testable, the sample ships a stub handler answering every request offline:

```
// a stub handler keeps the http test offline and deterministic
public sealed class StubHandler : HttpMessageHandler
{
    protected override Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request, CancellationToken cancellationToken)
        => Task.FromResult(new HttpResponseMessage(HttpStatusCode.OK)
        {
            Content = new StringContent("pong", Encoding.UTF8),
        });
}
```

listing 158: a stub handler, the seam for offline tests (source: csharp-net/samples/src/Ch16/Streams.cs, lines 11-20)

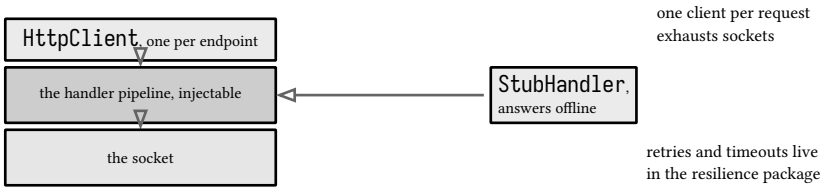


figure 104: the handler chain is the http test seam

```
    // HttpClient through an injected handler, no real network
    public static async Task<string> FetchAsync()
    {
        using var client = new HttpClient(new StubHandler());
        var response = await client.GetAsync("https://unit.test/ping");
        return await response.Content.ReadAsStringAsync();
    }
```

listing 159: the client through the stub, no network touched (source: csharp-net/samples/src/Ch16/Streams.cs, lines 81-87)

For real work, `IHttpClientFactory` manages handler lifetimes, and resilience policies, retries, timeouts, circuit breakers, live in `Microsoft.Extensions.Http.Resilience`, the same patterns book 12 implements by hand.

16.6. channels

`System.Threading.Channels` is the bounded queue the thread pool era wanted. A channel has a writer side and a reader side, unbounded or bounded with a policy, and the bounded form gives you backpressure: when the buffer is full, `WriteAsync` parks until the consumer takes a value. The sample's buffer holds one element, so the producer and consumer genuinely interleave, and `TryComplete` closes the stream so `ReadAllAsync` can finish:

```
// channels: bounded handoff with backpressure, one buffer slot here,
// so the producer waits until the consumer takes each value
public static async Task<List<int>> ChannelPipelineAsync()
{
    var channel = Channel.CreateBounded<int>(new BoundedChannelOptions(1)
    {
        SingleReader = true,
        SingleWriter = true,
    });

    var producer = Task.Run(async () =>
    {
        foreach (var value in new[] { 1, 2 })
        {
            await channel.Writer.WriteAsync(value); // parks when the slot is full
        }
        channel.Writer.TryComplete();
    });

    var read = new List<int>();
    await foreach (var value in channel.Reader.ReadAllAsync())
    {
        read.Add(value);
    }
    await producer;
    return read;
}
```

listing 160: capacity one, backpressure real, completion observed (source: csharp-net/samples/src/Ch16/Streams.cs, lines 53-79)

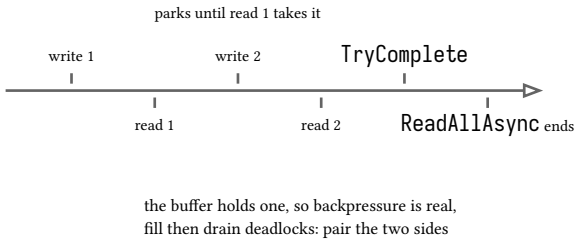


figure 105: capacity one, the writer and reader interleave

PITFALL: this sample deadlocked on its first draft

The first version awaited both writes before it began reading. With capacity one, the second write waits for a reader that never comes, and the test hangs. Bounded channels pair a producer and a consumer, they are not a buffer you fill then drain. The fix, run the producer concurrently, is the shape to copy.

Channels are the in-process messaging primitive that book 12 builds pipelines on, and `await foreach` from chapter 9 is how the reading side consumes them.

16.7. small utilities that matter

`Random` with an explicit seed is reproducible, which is how the game systems book gets deterministic replays. `Guid` names things. `Stopwatch` measures elapsed time without wall clock drift concerns. `DateTimeOffset` over `DateTime` when the moment matters across machines. `Interlocked` and `lock` belong to book 12's concurrency chapters:

```
    // Random is deterministic under a fixed seed
    public static List<int> SeededRange()
    {
        var first = new Random(42);
        return [first.Next(0, 100), first.Next(0, 100)];
    }
```

listing 161: seeded random, the determinism lever (source: csharp-net/samples/src/Ch16/Streams.cs, lines 89-94)

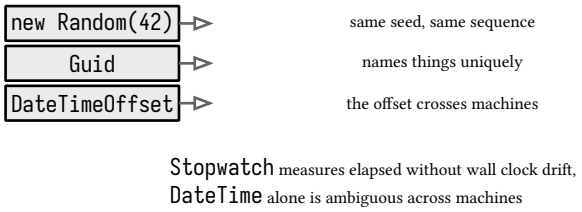


figure 106: the determinism levers, seed, name, and offset

Diagnostics on this machine, `Stopwatch`, `dotnet-counters`, `dotnet-trace`, and the benchmarking and percentile discipline around them, get their full treatment in chapter 12. Authoring observability, `EventSource`, `DiagnosticListener`, and the metrics APIs you emit from a service, stays with book 12′s distributed observability material.

sources: learn.microsoft.com, system.text.json overview, system.io, stream class, stringbuilder, system.net.http, system.threading.channels, and random pages accessed 2026-09-08, what's new in .NET 11 libraries, serialize C# union types, jsonuniontypestructuralclassifier, jsonnamingpolicy.pascalcase, stringstream, and rune pages accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 11 tests.

17. idioms

Everything so far was mechanics. This chapter is the set of habits that separates fluent C# from correct C#: deterministic cleanup, the equality contract, string comparison discipline, nullable discipline, and the testing posture this book's own suite demonstrates.

17.1. disposal

`IDisposable` says "I hold something the garbage collector cannot be trusted to reclaim promptly", a file handle, a socket, a native buffer. The `using` statement scopes the lifetime and calls `Dispose` at the closing brace, on every exit path, including an exception in flight:

```
// classic disposable: deterministic cleanup at scope exit
public sealed class TempScope : IDisposable
{
    public bool Disposed { get; private set; }

    public void Dispose() => Disposed = true;
}
```

listing 162: the smallest honest disposable (source: csharp-net/samples/src/Ch17/Craft.cs, lines 5-11)

```
    public static bool UsingDisposesAtScopeExit()
    {
        TempScope scope;
        using (scope = new TempScope())
        {
            _ = scope.Disposed; // still false inside
        }
        return scope.Disposed; // true after the closing brace
    }
```

listing 163: alive inside, disposed after (source: csharp-net/samples/src/Ch17/Craft.cs, lines 39-47)

```
    public static bool UsingDisposesOnThrow()
    {
        var scope = new TempScope();
        try
        {
            using (scope)
            {
                throw new InvalidOperationException();
            }
        }
        catch (InvalidOperationException)
        {
            // swallowed on purpose: the test observes disposal, not the throw
        }
        return scope.Disposed;
    }
```

listing 164: the exception unwinds through the using, dispose still runs (source: csharp-net/samples/src/Ch17/Craft.cs, lines 49-64)



figure 107: using is a state machine over exit paths, dispose always runs

When teardown itself awaits, implement `IAsyncDisposable` and consume with `await using`, the chapter 9 rules apply to `DisposeAsync` like any await:

```
// async disposable for resources whose teardown awaits
public sealed class AsyncScope : IAsyncDisposable
{
    public bool Disposed { get; private set; }

    public async ValueTask DisposeAsync()
    {
        await Task.Delay(1);
        Disposed = true;
    }
}
```

listing 165: an async disposable whose cleanup awaits (source: csharp-net/samples/src/Ch17/Craft.cs, lines 13-23)

```
    public static async Task<bool> AwaitUsingDisposesAsync()
    {
        AsyncScope scope;
        await using (scope = new AsyncScope())
        {
        }
        return scope.Disposed;
    }
```

listing 166: await using closes the async scope (source: csharp-net/samples/src/Ch17/Craft.cs, lines 66-73)

The `using` declaration form (`using var x = ...`) disposes at the end of the enclosing block and is the shape to prefer when the scope is the whole method.

17.2. the equality contract

If two instances of your type can be meaningfully equal, the contract is three parts: implement `IEquatable<T>` so collections compare typed, override `object.Equals` to delegate to it, and override `GetHashCode` consistently, equal values must produce equal hashes, or dictionaries lose them. Classes do none of this automatically. Records do all of it, plus the == operator:

```
// manual equality: IEquatable first, then the object override pair
public sealed class Tag(string key) : IEquatable<Tag>
{
    public string Key { get; } = key;

    public bool Equals(Tag? other) => other is not null && Key == other.Key;

    public override bool Equals(object? obj) => obj is Tag tag && Equals(tag);

    public override int GetHashCode() => StringComparer.Ordinal.GetHashCode(Key);
}
```

listing 167: the manual contract, three members (source: csharp-net/samples/src/Ch17/Craft.cs, lines 25-35)

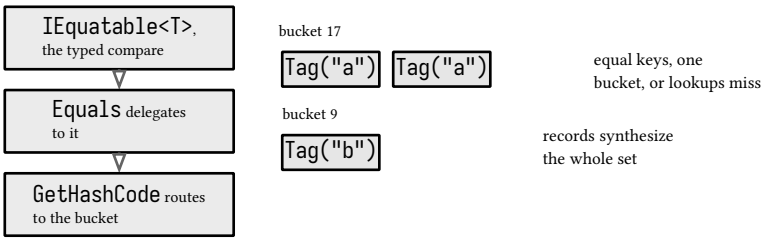


figure 108: three members, one equality contract

```
    public static (bool Manual, bool Record) EqualityDiscipline()
    {
        var manualEqual = new Tag("a").Equals(new Tag("a"));
        var recordEqual = new Note("x", 1) == new Note("x", 1);
        return (manualEqual, recordEqual);
    }

    public static bool HashMatchesEquality()
        => new Tag("a").GetHashCode() == new Tag("a").GetHashCode();
```

listing 168: manual equality beside record equality (source: csharp-net/samples/src/Ch17/Craft.cs, lines 75-83)

WARNING: equal objects, equal hashes, always

Overriding `Equals` without `GetHashCode` compiles and corrupts hash based collections quietly: two equal keys can land in different buckets and lookups miss. The pair is one contract. Prefer records for any type whose equality is value shaped, they synthesize the whole set including the runtime type check from chapter 3.

17.3. string comparison discipline

== on strings is ordinal, culture blind. That is right for program logic and wrong to assume about comparisons you did not write. Explicit comparisons should say what they mean: ordinal for machine keys, ordinal ignore case for user facing keys, culture aware only when producing text for humans to read:

```
    public static bool OrdinalComparisonsOnly()
        => string.Equals("HELLO", "hello", StringComparison.OrdinalIgnoreCase)
            && !string.Equals("hello", "hellö", StringComparison.Ordinal);
```

listing 169: explicit rules, no defaults inherited (source: csharp-net/samples/src/Ch17/Craft.cs, lines 85-87)

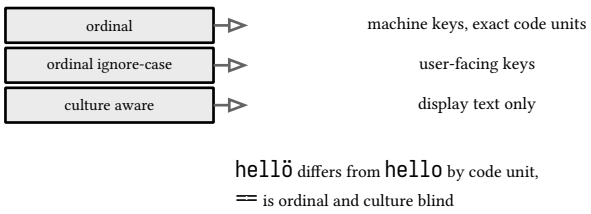


figure 109: the comparer grid, which rule for which job

The test asserts both directions: case-insensitive matches across casing, and ordinal treats `hellö` as different from `hello` because the code units differ.

17.4. nullable discipline

Chapter 3 built the model. The working rules: annotations tell the truth, `string?` only where null is a real answer. Null checks narrow the flow state, so check at the boundary and let the compiler carry the knowledge inward. `!` is a debt, each occurrence is a claim the compiler could not verify, and a candidate for a test. `??` and `?.` remove most of the branches the checks would otherwise create.

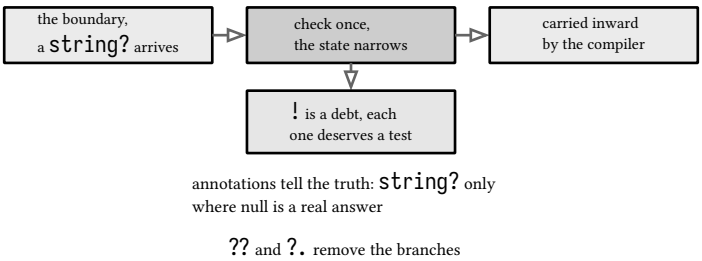


figure 110: nullable discipline, check once at the boundary

17.5. the testing posture

This book's sample suite is the reference implementation of the habit. Facts for one behavior, theories with `InlineData` when the behavior is parameter shaped, and assertions that name the behavior in the test method's own name, so a failure reads as a sentence. The red-green rhythm from chapter 13's generator applies everywhere: write the failing test, run it, watch it fail for the predicted reason, then make it pass and refactor.

The suite's own conventions are worth copying: static pure sample functions returning tuples, so tests read as input to output pairs, deterministic seeds over wall clock, `Assert.Throws` family over try-catch rethrow in tests, and one behavioral assertion per test so a failure points at one fact.

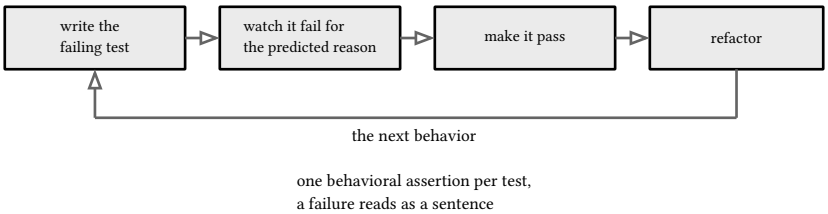


figure 111: red green refactor, the loop the suite runs

sources: learn.microsoft.com, implementing dispose, iasyncdisposable, equals and gethashcode, string comparison best practices, and xunit documentation at xunit.net, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 6 tests.

18. functional programming

The pieces are already on the table. Chapter 3 built records and, with C# 15, unions and closed hierarchies. Chapter 4 added extension members. Chapter 6 made switches exhaustive over a closed domain. Chapter 14 built LINQ. This chapter assembles them into one working style: model the world as data, transform it with functions, and let the compiler prove the transforms total. Everything runs on one sample, a document processor small enough to read in a sitting and complete enough to exercise all five pieces.

18.1. the five pieces

Functional C# is five features that compose. Records carry plain data with value equality, chapter 3's records section. A union closes a domain in one line, chapter 3's union types section, or `closed` fixes an existing hierarchy, its closed hierarchies section. Pattern matching branches on the shape of a value rather than on flags, chapter 6's switching over unions without a fallback. Collection expressions build and spread the sequences those branches produce. LINQ chains the transformations, chapter 14. The document model is all five at once:

```
// the case records: data only, no methods, no visitors
public sealed record Heading(int Level, string Text);
public sealed record Paragraph(string Text);
public sealed record Items(string[] Entries);
public sealed record Quote(string Attribution, Doc[] Nested);

// the whole document domain in one line, four cases, closed forever
public union Doc(Heading, Paragraph, Items, Quote);
```

listing 170: four case records and the one line domain (source: csharp-net/samples/src/Ch18/Functional.cs, lines 7-14)

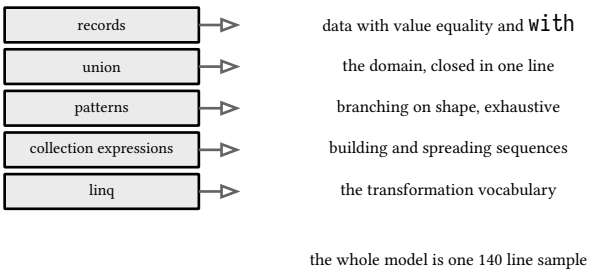


figure 112: five pieces, one job each, composed into one style

The four case records hold data and nothing else, no methods, no visitor interfaces, no behavior. A heading is a level and a text, an item list is an array of entries, and the quote case is the one that nests: its `Nested` field is more document, which is what makes the model a tree rather than a list. The union declaration is the entire contract. Every function that follows is written against those four cases, and the compiler holds each one to all of them.

The `is` operator is union aware, chapter 3 measured that, and so is every switch arm below. The boxing rule from that chapter applies unchanged: keep values statically typed as `Doc` across boundaries, because a union boxed to `object` hides its case from a type test.

18.2. data over behavior

The object oriented reflex is to put behavior inside the model, one class per case with a virtual method. The functional shape inverts it: the model stays dumb, transforms are functions from document to document, and behavior that needs to look at every block becomes a fold. The sample document is built with collection expressions, the `[a, b]` literal and `..` spread that build sequences inline (C# 12), and the spread in the flatten below is the same syntax at work:

```
// a tiny spec built with collection expressions, the spread unrolls the nested quote
public static List<Doc> Sample() =>
[
    new Heading(1, "pipeline notes"),
    new Paragraph("sequences flow through where select and aggregate"),
    new Heading(2, "steps"),
    new Items(["flatten", "renumber", "count"]),
    new Quote("seneca",
    [
        new Paragraph("every transform is a function from doc to doc"),
        new Items(["no mutation", "no visitor"]),
    ]),
    new Paragraph("counts close the loop"),
];
```

listing 171: a document built with collection expressions, one quote nested (source: csharp-net/samples/src/Ch18/Functional.cs, lines 18-31)

A tree to tree transform walks the structure and returns a new one. Demoting every heading copies each heading with a `with` expression, the nondestructive mutation from chapter 3's records section, and recurses into the quote's children. The original document is untouched, which the tests assert rather than trust:

```
// tree to tree: with copies the heading, the quote recurses into its children
private static Doc DemoteOne(Doc block) => block switch
{
    Heading h => h with { Level = h.Level + 1 },
    Paragraph p => p,
    Items i => i,
    Quote q => new Quote(q.Attribution, [.. q.Nested.Select(DemoteOne)]),
};

public static List<Doc> DemoteHeadings(IEnumerable<Doc> blocks)
    => [.. blocks.Select(DemoteOne)];
```

listing 172: with copies the heading, the recursion rebuilds the quote (source: csharp-net/samples/src/Ch18/Functional.cs, lines 96-106)

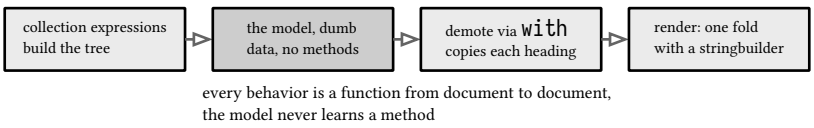


figure 113: data over behavior, transforms walk a model that stays dumb

Behavior is a fold over the flattened model. Rendering to text is one `Aggregate`, the general fold chapter 14's grouping and aggregation section introduced, carrying a `StringBuilder` as its accumulator and a switch as its step function:

```
// behavior as a fold: rendering is one Aggregate over the flattened model
public static string PlainText(IEnumerable<Doc> blocks)
    => Flatten(blocks).Aggregate(
        new StringBuilder(),
        (text, block) => text.AppendLine(block switch
        {
            Heading h => new string('#', h.Level) + " " + h.Text,
            Paragraph p => p.Text,
            Items i => "- " + string.Join(", ", i.Entries),
            Quote q => "> " + q.Attribution,
        }),
        text => text.ToString().TrimEnd());
```

listing 173: rendering as one fold, a switch per block kind (source: csharp-net/samples/src/Ch18/Functional.cs, lines 108-119)

NOTE: data over behavior is a default, not a law

The dumb model earns its keep when many behaviors read the same structure, outlines, word counts, renderers, all in this sample. When exactly one behavior owns the type, or when a hot path needs to mutate in place, a class with methods is still the honest tool, the same trade chapter 3's choosing a kind of type ladder walked.

18.3. exhaustiveness as a type-level test

Every switch in the sample names all four cases and carries no default arm. That discipline turns the compiler into a test suite for the domain. Declare a fifth case, say `Table`, and every transform in the file stops compiling until it says what a table means: CS8509, the warning chapter 6's switching over unions without a fallback section quoted verbatim, fires at each switch that misses it. A rename or a removed case fails the same way. The chapter 6 callout stands for this chapter too, promote CS8509 to an error so the proof cannot ride a green build:

```
// recursive walk: only the quote case nests, and the switch says so with no default arm
private static IEnumerable<Doc> FlattenOne(Doc block) => block switch
{
    Heading h => [h],
    Paragraph p => [p],
    Items i => [i],
    Quote q => [q, .. q.Nested.SelectMany(FlattenOne)],
};

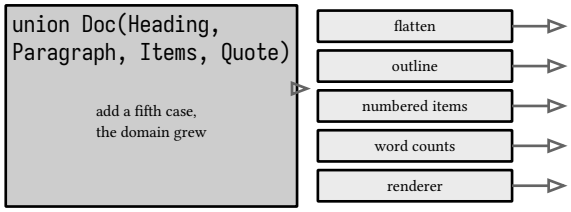
public static List<Doc> Flatten(IEnumerable<Doc> blocks)
    => [.. blocks.SelectMany(FlattenOne)];
```

listing 174: the recursive walk, four arms, no fallback (source: csharp-net/samples/src/Ch18/Functional.cs, lines 33-43)

The empty arms are the point. `HeadingsIn` returns nothing for 3 of the 4 cases, and those arms exist precisely so the compiler can count them. Writing `_ => []` instead would compile today and silently swallow the fifth case tomorrow, the failure mode chapter 6 diagrammed:

```
// only the heading case carries a heading, the other three arms exist for the compiler
private static IEnumerable<Heading> HeadingsIn(Doc block) => block switch
{
    Heading h => [h],
    Paragraph => [],
    Items => [],
    Quote => [],
};
```

listing 175: three empty arms, each one named for the compiler (source: csharp-net/samples/src/Ch18/Functional.cs, lines 45-52)



every arrow ends at the same error, CS8509, `Table` not covered

figure 114: a new case turns every transform red until handled

This is the difference from the pre C# 15 idiom. A record hierarchy with a discard arm trusted the writer to update every switch, and the compiler had nothing to say when they forgot. With the domain in the union declaration, an unhandled case is a compile time fact, not a runtime `SwitchExpressionException` waiting for the one input that carries it. Chapter 6 measured both halves of that failure, the warning text and the throw.

18.4. pipelines

LINQ is the backbone, and chapter 4's extension members section is the reason it reads left to right: every operator is a static extension method over `IEnumerable<T>`, resolved at compile time, which is also why the vocabulary is open to the same plumbing. The pattern in this sample is flatten once, then query. `Flatten` walks the tree into a block sequence, and every query below composes over it:

```
public static List<string> Outline(IEnumerable<Doc> blocks)
    => [.. Flatten(blocks)
        .SelectMany(HeadingsIn)
        .Select(h => $"{new string(' ', 2 * (h.Level - 1))}{h.Text}")];
```

listing 176: outline, flatten then project the headings (source: csharp-net/samples/src/Ch18/Functional.cs, lines 54-57)

```
// only the items case carries entries
private static string[] EntriesIn(Doc block) => block switch
{
    Items i => i.Entries,
    Heading => [],
    Paragraph => [],
    Quote => [],
};

// the indexed Select threads one counter through both lists
public static List<string> NumberedItems(IEnumerable<Doc> blocks)
    => [.. Flatten(blocks)
        .SelectMany(EntriesIn)
        .Select((entry, index) => $"{index + 1}. {entry}")];
```

listing 177: renumber, the indexed select threads one counter through both lists (source: csharp-net/samples/src/Ch18/Functional.cs, lines 59-72)

Word counts are chapter 14's grouping and aggregation shape exactly: `SelectMany` to words, `GroupBy` to cluster, `OrderByDescending` and `ThenBy` to rank, with the ordinal rule from chapter 17's string comparison discipline keeping the ties deterministic:

```
    // every case contributes its words, headings and attributions included
    private static string[] WordsIn(Doc block) => block switch
    {
        Heading h => Words(h.Text),
        Paragraph p => Words(p.Text),
        Items i => [.. i.Entries.SelectMany(Words)],
        Quote q => Words(q.Attribution),
    };

    private static string[] Words(string text)
        => text.Split(' ', StringSplitOptions.TrimEntries | StringSplitOptions.RemoveEmptyEntries);

    // grouping and aggregation from chapter 12, ordered by the ordinal rule from chapter 15
    public static List<(string Word, int Count)> TopWords(IEnumerable<Doc> blocks, int take)
        => [.. Flatten(blocks)
            .SelectMany(WordsIn)
            .GroupBy(word => word, StringComparer.Ordinal)
            .Select(group => (Word: group.Key, Count: group.Count()))
            .OrderByDescending(pair => pair.Count)
            .ThenBy(pair => pair.Word, StringComparer.Ordinal)
            .Take(take)];
```

listing 178: words to groups to ranked counts (source: csharp-net/samples/src/Ch18/Functional.cs, lines 74-94)

The join earns its place at the roster: which authors never got quoted. That is the outer joins section of chapter 14 verbatim, `LeftJoin` keeps every roster name and fills the missing right side with `default`, so `null` is the answer, and the pattern filters the gap. The pitfall from that chapter is why the filter is a pattern rather than a null check on a property:

```
    // only the quote case carries an attribution to join on
    private static IEnumerable<string> AttributionsIn(Doc block) => block switch
    {
        Quote q => [q.Attribution],
        Heading => [],
        Paragraph => [],
        Items => [],
    };

    // the chapter 12 outer join shape: every roster name kept, default where no quote matched
    public static List<string> UnquotedAuthors(IEnumerable<Doc> blocks, IEnumerable<string> roster)
        => [.. roster
            .LeftJoin(
                Flatten(blocks).SelectMany(AttributionsIn),
                name => name,
                attribution => attribution,
                StringComparer.Ordinal)
            .Where(pair => pair.Item2 is null)
            .Select(pair => pair.Item1)];
```

listing 179: left join over attributions, unmatched names are the result (source: csharp-net/samples/src/Ch18/Functional.cs, lines 121-139)

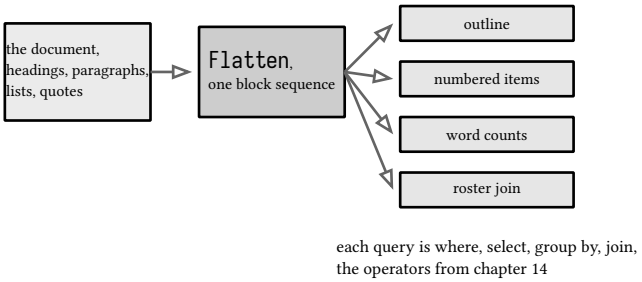


figure 115: the sample as pipelines, flatten once then query

Deferred execution from chapter 14 applies to these pipelines the same as any other: the queries are plans until enumerated, and `[.. ]` materializes at the end. `Flatten` itself materializes once inside each query, which is the honest shape for a tree walk that a query cannot express.

18.5. what c# borrows from f#

None of the five pieces is a C# invention. Records, `with`, and pattern matching came through the functional lineage into C# 7 through 10. Collection expressions and the pipeline reading of LINQ echo list literals and the |> operator. The `union` keyword in C# 15 is the newest loan, and it is the largest: a discriminated union is the native data type of F#, where the exhaustiveness this chapter treats as a test suite has been the default posture of every `match` since the language began.

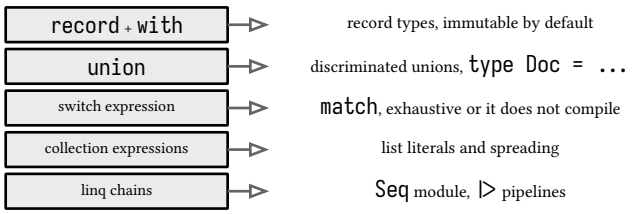


figure 116: each c# piece, its f# ancestry

C# keeps its own posture. The union is a struct wrapper rather than a heap allocated tree, equality and conversions are synthesized to feel like the rest of the type system, and LINQ remains method calls all the way down. The f# part of this corpus takes the same idea further: the document model in this chapter reappears there as a discriminated union with a `match` per transform, written in the language where the pattern is native and the borrowings were never borrowed. Read it after this chapter, the five pieces will all be familiar, only the syntax moved.

sources: learn.microsoft.com, union types, custom union types, closed hierarchy patterns, switch expressions, and standard query operators pages accessed 2026-09-13. Sample behavior verified by `make verify-csharp`, 8 tests.

19. f# 11: values, functions, and types

Chapter 18 ended with a promise: the five pieces of functional C# are loans, and the lender is a real language. These three chapters pay the visit. The same repo carries a second implementation of the interpreter, `books/csharp-net/fsharp/`, four F# files and a test project, built with the same `dotnet test` verbs on the same pinned SDK 11.0.100-rc.1. This chapter reads the language core on that code, let bindings through options, chapter 20 reads collections, pipelines, and async, and chapter 21 reads the .NET side. The tree is real and small, 497 lines across four files, so every claim below has a file and a line.

19.1. let bindings and immutability

`let` is the only binding form. It binds a name to a value or a function, and the binding is immutable by default: reassigning a `let` bound name is a compile error, so the value a name refers to cannot change under a reader. Mutation exists but must ask for it, `let mutable`, and assignment uses `<-` rather than `=`, so the two intentions never look alike. The lexer is the one place in the port that needs mutable state, a cursor over the source string, and its declaration is honest about it:

```
type Lexer(src: string) =
    let mutable pos = 0
    let mutable line = 1

    member _.Peek(ahead: int) =
        if pos + ahead < src.Length then src.[pos + ahead] else System.Char.MinValue
```

listing 180: the lexer class, two mutable fields, everything else immutable (source: csharp-net/fsharp/src/FSharpBook/Lexer.fs, lines 12-17)

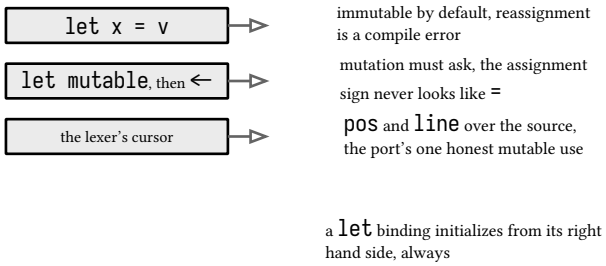


figure 117: let as the only binding form, immutable by default, mutation asking

The `src` parameter, the `pos` and `line` cursors, and every helper bound below them are values with fixed identity. The C# reflex of declaring first and assigning later does not exist: a `let` binding initializes from its right hand side, always. Chapter 3's nullable reference types exist because C# locals and fields can hold `null`. In F# a name either has a value or has type `option`, the last section of this chapter.

19.2. functions as values

Functions are let bindings too, with argument names before the colon and no ceremony around passing them around. A function value can sit in a list, travel as an argument, come back as a return value, and the test suite opens with two:

```
let private lex (source: string) : Token list = Lexer(source).Scan()

let private kinds (source: string) : TokenKind list =
    lex source |> List.map (fun t -> t.Kind)
```

listing 181: two helpers, one wrapping a method, one transforming a list (source: csharp-net/fsharp/tests/FSharpBook.Tests/LexerTests.fs, lines 7-10)

`lex` wraps constructor and method calls into a plain function of `string` to `Token list`. `kinds` composes `lex` with a lambda over `List.map`, and the lambda is just another value passed as the argument. There is no delegate type, no `Func<_,_>`, no declaration tax on a function the way chapter 7's `delegate` family taxes C#: every function is curried by default, `f a b` means apply `f` to `a` and then to `b`, and partial application, fixing the first argument and keeping the rest, falls out of the syntax with no new constructs.

The xunit facts make the same point from another side. A test is a let bound function marked with an attribute:

```
[<Fact>]
let LetStatement_ProducesItsTokens() =
    let tokens = lex "let x = 1.5;"
    Assert.Equal<TokenKind list>(
        [Let; Ident; Eq; Number; Semicolon; Eof],
        kinds "let x = 1.5;")
    Assert.Equal("x", tokens.[1].Text)
    Assert.Equal(1.5, tokens.[3].Number)
    Assert.Equal(1, tokens.[3].Line)
```

listing 182: a fact is a let bound function, the attribute names the unit (source: csharp-net/fsharp/tests/FSharpBook.Tests/LexerTests.fs, lines 12-20)

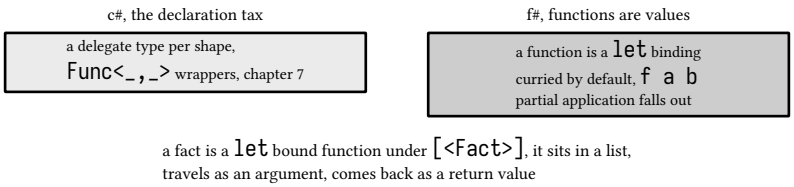


figure 118: functions as values, the delegate tax c# pays against the f# default

NOTE: an attribute on a non function is now a warning

F# 10 enforces attribute targets across let bound values and functions. Before it, `[<Fact>]` on a value rather than a function compiled silently and the test runner ignored it, a test that could never fail. The compiler now warns FS0842 that the attribute's valid target is a method. The fact above takes `()` and is a real function, so it runs.

19.3. type inference

Annotations are rare. `Token list` on `Scan`, `TokenKind` and `string` on the record fields, and almost nothing else: the lexer infers `ResizeArray<Token>` from its constructor, the parser infers the `Stmt list` it accumulates, and `kinds` above infers `string -> TokenKind list` from use alone. The inference is two phase and left to right, values are generalized where the language allows, and the places F# still wants a type are the places a reader would: method arguments at object boundaries and recursive functions, which name themselves with `rec` so the compiler knows the cycle. Compared with `var` in C#, the inference runs in the other direction: `var` infers a left hand side from a known right hand side, F# infers the right hand side from how the value is used.

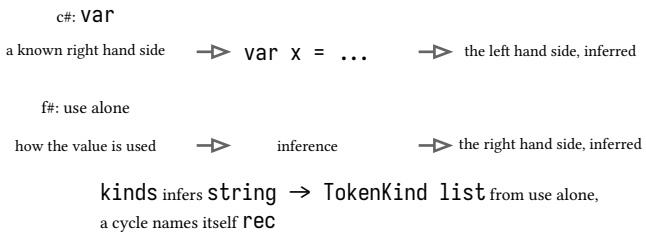


figure 119: the two inference directions, `var` from a known right, f# from use

19.4. records

The `Token` type is a record, five labeled fields in braces:

```
type Token =
    { Kind: TokenKind
      Text: string
      Number: float
      Str: string
      Line: int }
```

listing 183: the token record, five fields, immutable, structurally compared (source: csharp-net/fsharp/src/FSharpBook/Ast.fs, lines 13-18)

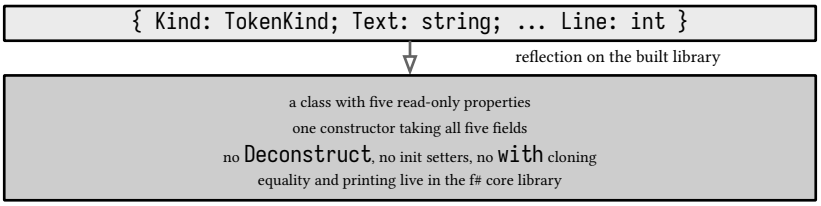


figure 120: one brace declaration, the compiled shape reflection measures

One declaration buys what C# spends a `record` on, chapter 3's records section: structural equality, a readable printing, and immutability without opting in. Copy and update uses `with` in braces, `{ token with Line = 3 }`, the same nondestructive mutation the C# keyword offers. The compiled shape differs from a C# record in one way a consumer feels: measured by reflection on the built library, F# compiles `Token` to a class with five read-only properties, one constructor taking all five fields, and no `Deconstruct`, `with`- style cloning, or init setters. Equality and printing live in the F# core library and generated code, not in the type's own members.

19.5. discriminated unions

The expression tree is one declaration:

```
type Expr =
    | NumberExpr of value: float * line: int
    | TextExpr of value: string * line: int
    | BoolExpr of value: bool * line: int
    | IdentExpr of name: string * line: int
    | UnaryExpr of op: TokenKind * operand: Expr * line: int
    | BinaryExpr of op: TokenKind * left: Expr * right: Expr * line: int
    | CallExpr of name: string * args: Expr list * line: int

// statement nodes
type Stmt =
    | LetStmt of name: string * value: Expr * line: int
    | AssignStmt of name: string * value: Expr * line: int
    | PrintStmt of value: Expr * line: int
    | IfStmt of condition: Expr * thenBody: Stmt list * otherwise: Stmt list * line: int
    | WhileStmt of condition: Expr * body: Stmt list * line: int
    | BlockStmt of body: Stmt list * line: int

type ProgramAst = { Body: Stmt list }
```

listing 184: expressions, statements, and the program, three unions over labeled fields (source: csharp-net/fsharp/src/FSharpBook/Ast.fs, lines 21-39)

`Expr` is a choice of seven cases, each carrying labeled fields inline, and one case nests the type itself, `UnaryExpr` holds an `Expr`, which is the whole recursion with no extra declaration. The file also carries `TokenKind`, a union whose cases carry nothing, F#'s spelling of C#'s `enum`, chapter 3's enums section.

This is the same idea as the C# 15 union of chapter 3's union types section, and the difference is entirely in the runtime representation. The measured shapes, reflection over both builds under the pinned SDK:

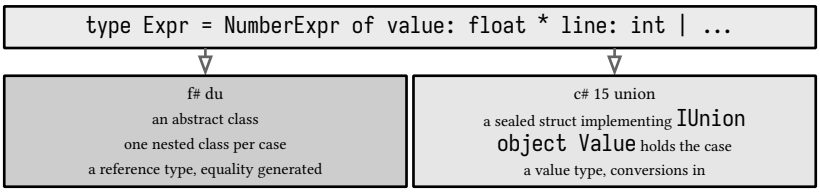


figure 121: same idea, two runtimes, the union in c# and the du in f#

F# compiles the union to an abstract class `Expr` with a nested class per case, each deriving from it, heap allocated and polymorphic. C# 15 compiles its union to a sealed struct wrapping an `object` that holds the case, stack friendly and boxing under the rules chapter 3 measured. Neither carries a discriminant field: the runtime type is the tag in F#, the case type's identity is the tag in C#. The C# record cases of the capstone tree, `Kind` tag and all, chapter 35's tree section, correspond to F#'s labeled fields, `of value: float * line: int`, which ride inside the case class as read-only properties.

19.6. pattern matching and exhaustiveness

`match` is the branch form and it deconstructs as it branches. The lexer's keyword table is the smallest honest example:

```
        let kind =
            match text with
            | "let" -> Let
            | "print" -> Print
            | "if" -> If
            | "else" -> Else
            | "while" -> While
            | "true" -> True
            | "false" -> False
            | _ -> Ident
        { Kind = kind; Text = text; Number = 0.0; Str = ""; Line = line }
```

listing 185: keywords as a match over the scanned word, the wildcard closes it (source: csharp-net/fsharp/src/FSharpBook/Lexer.fs, lines 99-109)

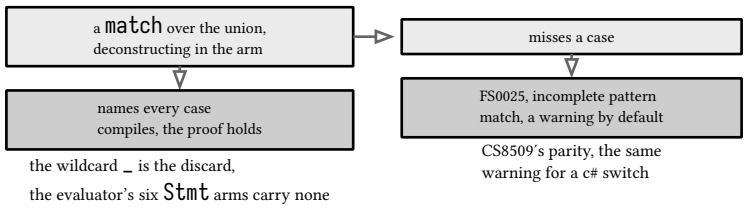


figure 122: match deconstructing as it branches, the incomplete case warned

The wildcard _ is the discard chapter 6 measured in C#. What F# adds is deconstruction in the arm: `match x.MatchAny [EqEq; BangEq]` `with | Some op -> ... | None -> ...` tests and unwraps in one line, the same reach C# 15 patterns get through union aware is, chapter

3. A `match` that misses a case is FS0025, an incomplete pattern

match, a warning by default, exactly as CS8509 is for a C# switch. The evaluator's statement walk names all six `Stmt` cases and carries no wildcard, so adding a case turns every match over the tree amber until handled, the compiler as a test suite, the posture chapter 18 called exhaustiveness as a type level test.

PITFALL: fs0025 is a warning, promote it

The F# compiler does not refuse an incomplete match, it warns. The parity move with chapter 18's advice on CS8509 is to treat FS0025 as an error in the project file, so an added case stops the build rather than shipping a match that throws at runtime.

19.7. options

`option<'T>` is a two case union, `Some 'T` or `None`, and it is the language's answer to "may not be there". A lookup returns it, the caller must match, and the type checker holds the caller to both arms. The parser's optional consume is the pattern in miniature:

```
    member private x.MatchAny(kinds: TokenKind list) : Token option =
        match kinds |> List.tryFind (fun k -> x.Check k) with
        | Some _ ->
            pos <- pos + 1
            Some tokens.[pos - 1]
        | None -> None
```

listing 186: tryfind returns an option, the match unwraps or falls through (source: csharp-net/fsharp/src/FSharpBook/Parser.fs, lines 149-154)

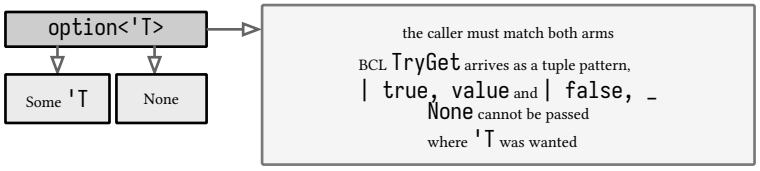


figure 123: option, absence as a value the type system counts

`List.tryFind` is the option returning sibling of `List.find`, and `Token option` in the signature is the whole contract. The BCL's `TryGet` idiom, `bool` plus `out`, arrives as a tuple pattern in F#: the evaluator's name lookup matches `names.TryGetValue name` against `| true, value` and `| false, _`, chapter 21 shows the site. Against C#'s nullable reference types, chapter 3's two nullable systems, the difference is placement: absence is a value the type system counts, and `None` cannot be passed where `'T` was wanted.

sources: learn.microsoft.com, f# language reference let bindings, records, unions, pattern matching, and options pages, the f# 9 and f# 10 what's new pages, accessed 2026-09-13. Compiled shapes measured by reflection on the built library. Verified by `dotnet test fsharp/FSharpBook.slnx`, 20 tests.

20. f# 11: collections, pipelines, and async

Chapter 19 read the port's types. This chapter reads its motion: how F# code moves values from one function to the next. The pipe and compose operators make function application read in data order, the List, Array, and Seq modules carry the vocabulary LINQ gives C#, chapter 14, and computation expressions wrap both patterns around effects like asynchrony. The same four files are the evidence, and the parser and lexer carry most of the chapter: their slices show the pipelines running in the working interpreter.

20.1. the pipe and compose operators

`|>` applies left to right, `x |> f` is `f x`, so the data flows in reading order instead of inside out. `>>` composes forward, `f >>`
`g` is a function that runs `f` then `g`. Both are ordinary functions in the core library, `(|>)` and `(>>)`, not syntax, which is the difference from C#: chapter 4's extension members made LINQ read left to right by putting every operator on the receiver, F# reaches the same reading with one infix operator over plain functions. The test helper for the whole evaluator is the idiom in six lines:

```
let private run (source: string) : string list =
    let tokens = Lexer(source).Scan() |> Array.ofList
    let program = Parser(tokens).ParseProgram()
    let lines = ResizeArray<string>()
    Evaluator(fun line -> lines.Add line).Run(program)
    List.ofSeq lines
```

listing 187: the run helper, one pipe per stage, list out at the end (source: csharp-net/fsharp/tests/FSharpBook.Tests/EvalTests.fs, lines 8-13)

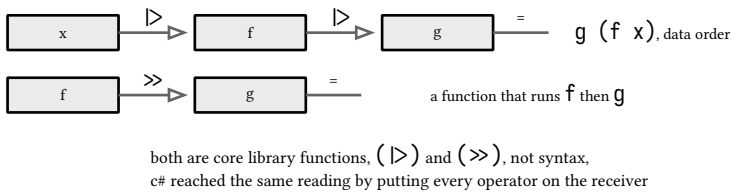


figure 124: pipe and compose, library functions that read in data order

`Scan()` returns a `Token list`, `|> Array.ofList` rehouses it as an array, and the parser wants the array because its cursor indexes by position, constant time on arrays and linear on the linked list. The last line converts the BCL `ResizeArray` back into an F# list, the boundary traffic this chapter keeps returning to. The `|> ignore` idiom in the parser, applying the discard function to a value that must be consumed, is the same operator doing ceremony removal.

20.2. the collection modules against linq

F#'s vocabulary lives in three modules, `List`, `Array`, and `Seq`, one per collection kind: `List` over the immutable linked list, `Array` over the CLR array, `Seq` over `IEnumerable<'T>`. LINQ's operators from chapter 14's the operator families section all have their counterparts, `List.filter` is `Where`, `List.map` is `Select`, `List.collect` is `SelectMany`, `List.sortBy` is `OrderBy`, and chapter 14's grouping and aggregation section maps `GroupBy` to `List.groupBy` and `Aggregate` to `List.fold`. The lookup chapter 19 showed in the options section, `List.tryFind` feeding a `match`, is the module's option returning flavor of LINQ's `FirstOrDefault`.

Two differences from LINQ are structural rather than notational. Eagerness: `List` and `Array` functions materialize their result when called, while `Seq` is lazy, the same split as chapter 14's deferred execution section, where `Where` builds a plan and `ToList` runs it. In F# the choice is by module: `Seq.filter` returns a lazily computed sequence, `List.filter` returns a finished list, and no operator returns to change the decision. Joins: chapter 14's join and let

and outer joins sections have no `List` counterpart, relational joining is LINQ's home ground, and the F# posture is to fold or recurs instead. The argument list in a call expression shows the accumulate and convert pattern the modules are built on:

```
            if x.Match LParen then
                let args = ResizeArray<Expr>()
                if not (x.Check RParen) then
                    args.Add(x.Expression())
                    while x.Match Comma do
                        args.Add(x.Expression())
                x.Expect(RParen, "')' after the arguments") |> ignore
                CallExpr(name.Text, List.ofSeq args, name.Line)
            else
                IdentExpr(name.Text, name.Line)
```

listing 188: resizearray accumulates, list.ofseq converts, the boundary is explicit (source: csharp-net/fsharp/src/FSharpBook/Parser.fs, lines 132-141)

`ResizeArray<'T>` is `List<'T>` under an F# alias, chosen when mutation is the honest tool, and converted back the moment it crosses a boundary, which keeps each side of the code base in its own vocabulary.

linq	f# module		what to know
`Where`	`List.filter`	→	eager list, lazy seq
`Select`	`List.map`	→	same two flavors
`SelectMany`	`List.collect`	→	flatten, both modules
`OrderBy`	`List.sortBy`	→	no thenby, sort by tuple
`GroupBy`	`List.groupBy`	→	pairs of key and list
`Aggregate`	`List.fold`	→	explicit seed and state
`join`, `group join`	none	→	linq's home ground

figure 125: the linq operator, the module function, and what changes

20.3. the lexer, index arithmetic over a string

The port's lexer walks the source string by index, the same design as the C# capstone's lexer, chapter 35′s lexer section, with one honest substitution: C# walked a `ReadOnlySpan<char>` from a `ref struct`, chapter 10′s allocation free shape, and F# indexes the immutable `string` directly, `src.[pos]`, which is constant time but not span based. The dispatch on the current character is four branches:

```
    member private x.Next() : Token =
        x.SkipWhitespaceAndComments()
        if pos >= src.Length then
            { Kind = Eof; Text = "<eof>"; Number = 0.0; Str = ""; Line = line }
        else
            let c = src.[pos]
            if System.Char.IsDigit c then x.ScanNumber()
            elif c = '"' then x.ScanString()
            elif System.Char.IsLetter c || c = '_' then x.ScanWord()
            else x.ScanSymbol()
```

listing 189: digit, quote, letter, everything else, four dispatch branches (source: csharp-net/fsharp/src/FSharpBook/Lexer.fs, lines 28-37)

Number scanning enforces the same malformed-double rule against the character stream, one decimal point, digits after it, no second one:

```
    member private _.ScanNumber() : Token =
        let start = pos
        let digits() =
            while pos < src.Length && System.Char.IsDigit src.[pos] do pos <- pos + 1
        digits()
        if pos < src.Length && src.[pos] = '.' then
            pos <- pos + 1
            if pos >= src.Length || not (System.Char.IsDigit src.[pos]) then
                raise (LexException(line, "a digit must follow the decimal point"))
            digits()
            if pos < src.Length && src.[pos] = '.' then
                raise (LexException(line, "a digit must follow the decimal point"))
        let text = src.Substring(start, pos - start)
        let value = System.Double.Parse(text, CultureInfo.InvariantCulture)
        { Kind = Number; Text = text; Number = value; Str = ""; Line = line }
```

listing 190: digits, one decimal point, invariant parse at the end (source: csharp-net/fsharp/src/FSharpBook/Lexer.fs, lines 53-67)

The symbol scanner is the chapter's payoff listing, a `match` over a pair of characters with a tuple pattern per two character operator:

```
    member private x.ScanSymbol() : Token =
        let c = src.[pos]
        let kind, width =
            match c, x.Peek 1 with
            | '=', '=' -> (EqEq, 2)
            | '!', '=' -> (BangEq, 2)
            | '<', '=' -> (Le, 2)
            | '>', '=' -> (Ge, 2)
            | '(', _ -> (LParen, 1)
            | ')', _ -> (RParen, 1)
            | '{', _ -> (LBrace, 1)
            | '}', _ -> (RBrace, 1)
            | ';', _ -> (Semicolon, 1)
            | ',', _ -> (Comma, 1)
            | '=', _ -> (Eq, 1)
            | '!', _ -> (Bang, 1)
            | '<', _ -> (Lt, 1)
            | '>', _ -> (Gt, 1)
            | '+', _ -> (Plus, 1)
            | '-', _ -> (Minus, 1)
            | '*', _ -> (Star, 1)
            | '/', _ -> (Slash, 1)
            | _ -> raise (LexException(line, $"unexpected character '{c}'"))
        let text = src.Substring(pos, width)
        pos <- pos + width
        { Kind = kind; Text = text; Number = 0.0; Str = ""; Line = line }
```

listing 191: the two character operators first, then singles, else an error (source: csharp-net/fsharp/src/FSharpBook/Lexer.fs, lines 111-136)

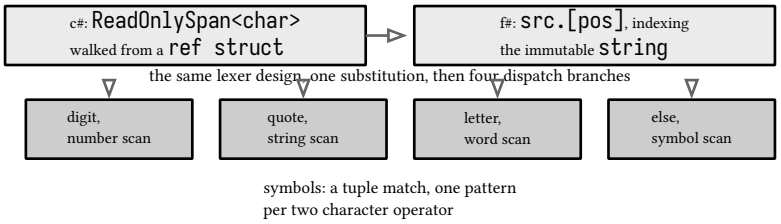


figure 126: the lexer, index arithmetic over the immutable string

20.4. the parser, precedence by recursion

The C# capstone encodes precedence as method depth, chapter 35's parser section. The port encodes it as recursive local functions, each level a `let rec loop` that folds its own operator set over the next level down:

```
    member private x.Expression() : Expr = x.Equality()

    member private x.Equality() : Expr =
        let rec loop left =
            match x.MatchAny [EqEq; BangEq] with
            | Some op -> loop (BinaryExpr(op.Kind, left, x.Comparison(), op.Line))
            | None -> left
        loop (x.Comparison())

    member private x.Comparison() : Expr =
        let rec loop left =
            match x.MatchAny [Lt; Le; Gt; Ge] with
            | Some op -> loop (BinaryExpr(op.Kind, left, x.Additive(), op.Line))
            | None -> left
        loop (x.Additive())

    member private x.Additive() : Expr =
        let rec loop left =
            match x.MatchAny [Plus; Minus] with
            | Some op -> loop (BinaryExpr(op.Kind, left, x.Multiplicative(), op.Line))
            | None -> left
        loop (x.Multiplicative())

    member private x.Multiplicative() : Expr =
        let rec loop left =
            match x.MatchAny [Star; Slash] with
            | Some op -> loop (BinaryExpr(op.Kind, left, x.Unary(), op.Line))
            | None -> left
        loop (x.Unary())

    member private x.Unary() : Expr =
        match x.MatchAny [Bang; Minus] with
        | Some op -> UnaryExpr(op.Kind, x.Unary(), op.Line)
        | None -> x.Primary()
```

listing 192: the ladder, equality to unary, one loop per level (source: csharp-net/fsharp/src/FSharpBook/Parser.fs, lines 86-119)

The shape is the same claim as C#'s: tighter binding operators sit deeper and finish first, and each level looping over its own set is what makes subtraction left associative. Statement dispatch is a flat match over the leading token, `Check Let`, `Check Ident`, and company, replacing the C# if chain:

```
    member private x.Statement() : Stmt =
        if x.Check Let then x.LetStatement()
        elif x.Check Ident && x.CheckAhead Eq then x.AssignStatement()
        elif x.Check Print then x.PrintStatement()
        elif x.Check If then
            let start = x.Advance()
            x.IfStatement(start.Line)
        elif x.Check While then x.WhileStatement()
        elif x.Check LBrace then
            let openBrace = tokens.[pos]
            BlockStmt(x.Block(), openBrace.Line)
        else
            raise (x.Error(tokens.[pos], "a statement"))
```

listing 193: statement dispatch, one branch per leading token (source: csharp-net/fsharp/src/FSharpBook/Parser.fs, lines 21-33)

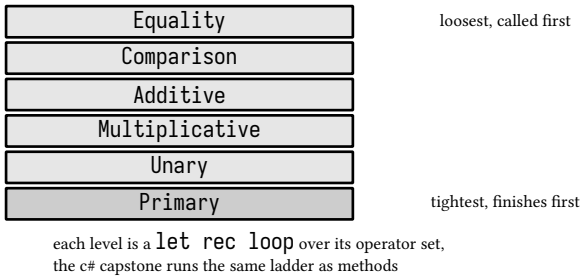


figure 127: precedence as call depth, one recursive loop per level

20.5. computation expressions

Sequences, asynchronous workflows, and tasks are not special cased syntax in F#, they are instances of one pattern, the computation expression: braces after a builder name, `let!` binds a value in the computation's world, `return` wraps, `use!` scopes a resource, and the builder class supplies the plumbing. `seq { 1..10 }` builds a lazy sequence because the `seq` builder's `Bind` yields, `async { ... }` builds a workflow because the `async` builder schedules. F# 9 added empty bodied expressions, `seq { }`, by giving builders a `Zero` default.

The one computation expression this book's C# side cares about is `task`, which maps directly onto `System.Threading.Tasks.Task`, the same type chapter 9's async and await consume:

```
task {
    let! a = fetchA()
    and! b = fetchB()
    return a + b
}
```

fragment, illustrative, not executed

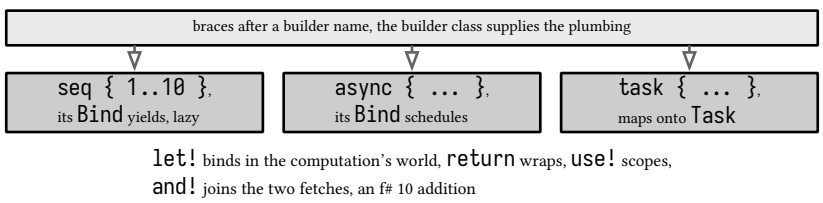


figure 128: computation expressions, one pattern behind seq, async, and task

`and!` binds two computations together, so the two fetches run concurrently rather than in sequence, the semantics of `Task.WhenAll` without the combinator. The honest label: `and!` in task expressions is an F# 10 addition, the F# 10 release notes name it as such, it is not new in 11.

NOTE: the port contains no computation expression

The four files behind chapters 19 through 21 use none of this. The lexer and parser are loops and recursion, the evaluator is synchronous end to end, and the code comment on the builtin table says why: `clock_async` left with async. The constructs matter here as the native form of what chapter 9 does with `await` and what chapter 35′s evaluator section does with an awaited `RunAsync`, the F# port deliberately took the smaller scope.

20.6. async, deliberately absent

The C# capstone's evaluator is async end to end, a cancellation token through every await, chapter 35. The port is not, and the omission is a scope decision the code states in a comment: async came out, and the one async builtin, `clock_async`, came out with it. What remains is the synchronous core, a dictionary of names, a match per statement, and the remaining builtins computing values with no awaits anywhere. The two designs are the two languages' resting positions: C# reaches for `async` methods that return `Task<T>`, F# for `task { … }` blocks that build one. The port's choice keeps the comparison honest, the same tree, the same tests, one async dimension removed, and chapter 21 returns to what that buys on the .NET side.

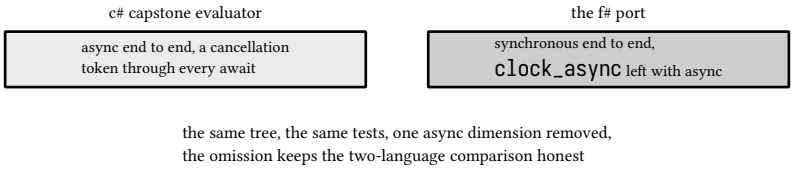


figure 129: async, deliberately absent, the port's smaller scope

sources: learn.microsoft.com, f# language reference, lists, arrays, sequences, computation expressions, and task expressions pages, the f# 10 what's new page for `and!` in task expressions, accessed 2026-09-13. Sample behavior verified by `dotnet test`
`fsharp/FSharpBook.slnx`, 20 tests, zero skipped.

21. f# on .net 11

F# is not a separate runtime. It compiles to the same IL, runs on the same CLR, ships in the same SDK download, and calls the same base class library, which is why the port behind chapters 19 and 20 builds with the verbs chapter 1 already covered. This chapter reads the .NET side of the language: the toolchain as the port uses it, interop in both directions, what .NET 11′s serializer does with a discriminated union, the trimming and compilation notes from the F# release documentation, the versioning story verified against the docs and the SDK, and the side quest's own accounting.

21.1. the toolchain

The pin is the book's pin. `global.json` in `books/csharp-net` selects SDK 11.0.100-rc.1, and everything the part claims was built under it:

```
$ dotnet --version
11.0.100-rc.1.26425.128

$ dotnet test fsharp/FSharpBook.slnx
Passed! - Failed: 0, Passed: 20, Skipped: 0, Total: 20
```

The project file is a plain fsproj: `TargetFramework` `net11.0`, `IsPackable` false, and four `Compile` items, one per file. The order of those items is load bearing, the one structural difference from a csproj: F# compiles files top to bottom and a file may only use what earlier files define, so the list reads `Ast.fs`, `Lexer.fs`, `Parser.fs`, `Evaluator.fs`, dependency order made explicit. C# resolves symbols across the whole project regardless of file order, chapter 1′s project model, which is why the C# capstone's file order is a convention and the port's is a constraint.

F# Interactive ships inside the SDK, no separate install, `dotnet fsi` runs a session or a script. A session on this SDK reports itself:

```
$ dotnet fsi --quiet version.fsx
runtime: 11.0.0
FSharp.Core 11.0.0.0
FSharp.Compiler.Service 43.13.101.0
```

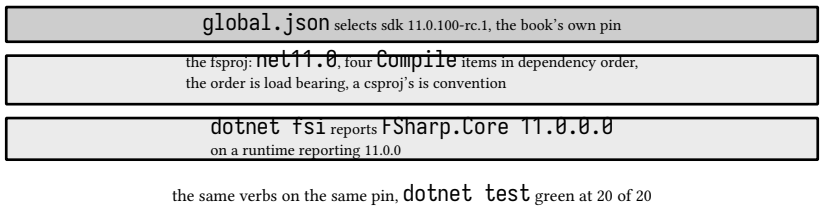


figure 130: the toolchain, one pin from global.json down to fsi

The script is three lines that enumerate the loaded assemblies and print their versions. `FSharp.Core 11.0.0.0` is the number this chapter's versioning section leans on: the core library the RC SDK carries is versioned 11. The wire shapes and the compiled shapes quoted below were measured the same way, a script that `#r` loads the built FSharpBook.dll and prints what reflection sees.

21.2. c# interop, both directions

F# toward .NET needs no ceremony, and the port leans on the BCL throughout: `System.Char.IsDigit` classifies characters in the lexer, `System.Text.StringBuilder` accumulates string bodies, `Double.Parse` takes a `CultureInfo.InvariantCulture` so decimal points never localize, and the evaluator's name table is a BCL

`Dictionary<string, Value>`, created and indexed from F# with no adapter. `ResizeArray<'T>` in the parser is the same BCL generic list under an F# name, chapter 20 showed the boundary crossing.

.NET toward F# is a question about compiled shape, and it has a measured answer. Reflection over the built library shows `Expr` compiles to an abstract class with one nested class per case, each deriving from it, and `Token` compiles to a class with five read only properties, one constructor over all five fields, and a `CompilationMapping` attribute marking it as an F# record. A C# consumer therefore sees ordinary classes: instantiate the case class directly, read its properties, branch with `is` and `switch` over the hierarchy. What the C# consumer does not get is the F# side of the deal, structural equality and comparison live in the F# core library's generated code, and C# 15's union patterns reach unions built with the C# `union` keyword through `IUnion`, chapter 3, not an F# discriminated union compiled as a class hierarchy.

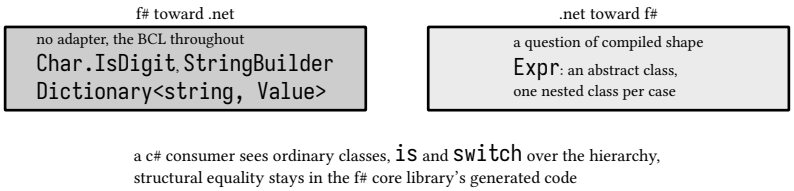


figure 131: interop both directions, plain bcl calls out, ordinary classes in

NOTE: one direction is measured from the outside

The BCL consumption direction runs inside a green test suite, the port's 20 tests exercise every BCL call listed above. The C# consumption direction has no such suite, no C# project in this repo references FSharpBook.dll, so its claims rest on the measured compiled shape and the language reference, not on a test.

21.3. two unions, two wire shapes

.NET 11's serializer learned both union families in the same release. The what's new in .NET 11 libraries page lists "F# discriminated union support" in System.Text.Json alongside the C# union serialization chapter 16 measured. The two do not write the same bytes, and both shapes below were measured on the pinned SDK with default options.

The F# discriminated union writes an envelope: a `$type` discriminator naming the case, then the case's fields by their declared names:

```
{"$type":"NumberExpr","value":1.5,"line":1}
{"$type":"BinaryExpr","op":"Plus","left":{...},"right":{...},"line":3}
```

Reading runs one rule: the discriminator must be present. A payload without `$type` is refused with a `JsonException`, "required type discriminator property not found", measured. Unnamed case fields serialize as `Item`, a two case evaluator value measured `{"$type":"Number","Item":2.5}`, and web defaults camel case both the case name and the field names.

The C# 15 union of chapter 16's unions on the wire section writes the bare case payload, no envelope, the JSON value kind names the case, and object cases that share a kind need the `JsonUnionTypeStructuralClassifier`. Both round trip with zero configuration under their own conventions, and the trade sits in the envelope:

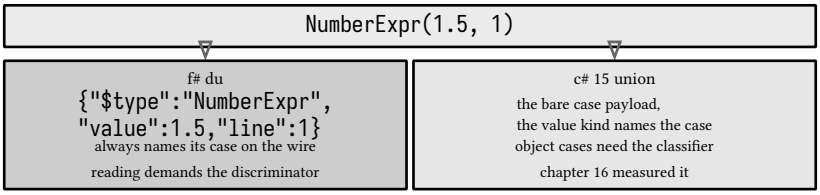


figure 132: one case, two wires, the envelope is the difference

WARNING: the two shapes are not interchangeable

A C# reader handed F# union JSON must cope with the `$type` envelope, and an F# reader handed C# union JSON will refuse it for lacking the discriminator. One serializer, one release, two conventions: agree on the writer before the wire exists.

21.4. trimming and parallel compilation

Two F# release notes claims carry into .NET 11 and both are worth the source. Trimming: F# assemblies carry metadata resources the compiler wants and the runtime does not, and F# 10 removed the manual `ILLink.Substitutions.xml` maintenance by generating the substitutions file automatically when publishing with `PublishTrimmed`, so trimmed F# output is smaller by default with the opt out named `DisableILLinkSubstitutions`. Compilation: the `ParallelCompilation` project property groups graph based type checking, parallel IL generation, and parallel optimization behind one switch. F# 10 turned it on by default only for `LangVersion=Preview` projects, the notes state the plan to enable it for all projects in .NET 11, and they state the cost, it does not combine with deterministic builds.

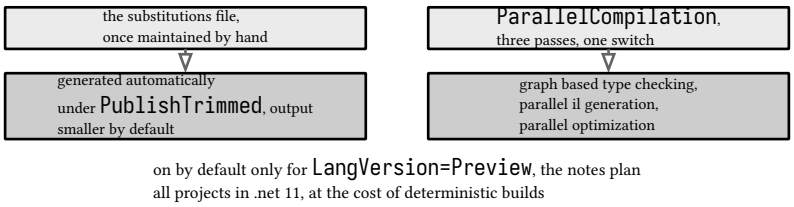


figure 133: two f# 10 release claims carried into .net 11

What the .NET 11 docs at RC1 do not carry is an F# section, the overview's language coverage names C# 15 and nothing for F#. The plan is in writing, the confirmation is not yet, and this chapter records the gap rather than assuming the flip.

21.5. the versioning story, verified

The F# release numbering has tracked the .NET release numbering since F# 9, and the docs confirm the pairing link by link. The .NET 9 overview: "F# 9 ships with the .NET 9 SDK." The F# 10 page: "F# 10 ships with .NET 10 and Visual Studio 2026." For F# 11 the docs are not yet written, there is no what's new in F# 11 page at RC1 and the .NET 11 overview names no F# language version, so the claim rests on the SDK itself: `dotnet fsi` under 11.0.100-rc.1 loads `FSharp.Core 11.0.0.0` on a runtime reporting 11.0.0, the same alignment the last two releases documented.

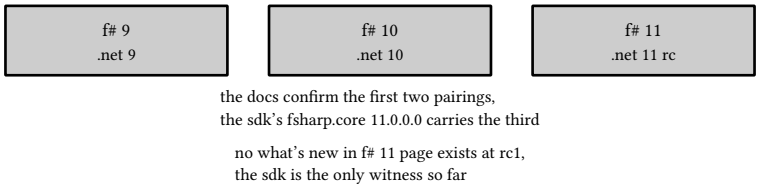


figure 134: three releases, the pairing in the docs and the sdk

21.6. the side quest: the evaluator in f#

The port is four files, all read across this part: `Ast.fs`, 39 lines, the token kinds, the token record, and the two unions, chapter 19, `Lexer.fs`, 136 lines, chapter 20, `Parser.fs`, 183 lines, chapter 20, and `Evaluator.fs`, 139 lines, this section. The evaluator's value domain is one more discriminated union with a printing rule per case, integer valued numbers printed without a decimal point, invariant culture throughout:

```
type Value =
    | Nil
    | Number of float
    | Text of string
    | Flag of bool

    override this.ToString() =
        match this with
        | Nil -> "nil"
        | Number n when n = floor n -> (int64 n).ToString(CultureInfo.InvariantCulture)
        | Number n -> n.ToString(CultureInfo.InvariantCulture)
        | Text t -> t
        | Flag b -> if b then "true" else "false"
```

listing 194: the value union, one tostring for the whole domain (source: csharp-net/fsharp/src/FSharpBook/Evaluator.fs, lines 12-24)

The builtin table is where the port's scope is stated in the code itself. The C# capstone registers builtins with a source generator, chapter 35's builtins section, the port is a two entry map, and the comment on it says what left: the C# registry is source generated, here it is a plain map, and `clock_async` left with async:

```
module Builtins =

    let private require name count (args: Value[]) =
        if args.Length <> count then
            raise (EvalException(0, $"{name} expects {count} arguments, got {args.Length}"))

    let private numberAt name index (args: Value[]) =
        match args.[index] with
        | Number n -> n
        | _ -> raise (EvalException(0, $"{name} argument {index + 1} must be a number"))

    let private math1 name (f: float -> float) (args: Value[]) : Value =
        require name 1 args
        Number(f (numberAt name 0 args))

    // the C# registry is source generated, here it is a plain map, and
    // clock_async left with async
    let All: Map<string, Value[] -> Value> =
        Map.ofList [ "sqrt", math1 "sqrt" sqrt
                     "abs", math1 "abs" abs ]
```

listing 195: sqrt and abs kept, clock async dropped, a plain map (source: csharp-net/fsharp/src/FSharpBook/Evaluator.fs, lines 67-86)

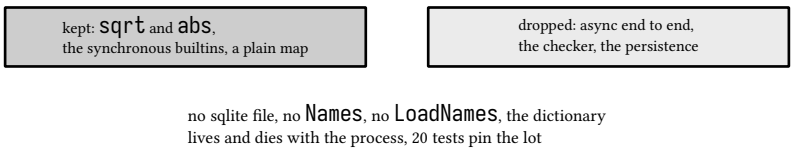


figure 135: the side quest's deliberate scope, kept and dropped

The class comment above the evaluator says the rest of it: "the C# capstone's evaluator minus async, sqlite persistence, and the source generated builtin registry, synchronous end to end". The optional constructor parameters, `?write` and `?builtins`, default through `defaultArg`, and the name table is the BCL dictionary:

```
// deliberate scope: the C# capstone's evaluator minus async, sqlite
// persistence, and the source generated builtin registry, synchronous end
// to end
type Evaluator(?write: (string -> unit), ?builtins: Map<string, Value[] -> Value>) =
    let sink = defaultArg write ignore
    let table = defaultArg builtins Builtins.All
    let names = Dictionary<string, Value>()

    member x.Run(program: ProgramAst) = x.RunBlock program.Body
```

listing 196: deliberate scope in a comment, defaults in two lines, a dictionary for names (source: csharp-net/fsharp/src/FSharpBook/Evaluator.fs, lines 88-96)

Statement execution is a match over the six `Stmt` cases, no wildcard, chapter 19's exhaustiveness posture. Undefined names raise here, at evaluation, because the port has no checker:

```
    member private x.RunStmt(stmt: Stmt) =
        match stmt with
        | LetStmt(name, value, _) -> names.[name] <- x.Eval value
        | AssignStmt(name, value, _) -> names.[name] <- x.Eval value
        | PrintStmt(value, _) -> sink ((x.Eval value).ToString())
        | IfStmt(condition, thenBody, otherwise, _) ->
            if truth (x.Eval condition) then x.RunBlock thenBody
            else x.RunBlock otherwise
        | WhileStmt(condition, body, _) ->
            while truth (x.Eval condition) do
                x.RunBlock body
        | BlockStmt(body, _) -> x.RunBlock body
```

listing 197: six named arms, no discard, assignment writes the dictionary (source: csharp-net/fsharp/src/FSharpBook/Evaluator.fs, lines 102-113)

Expression evaluation is the same discipline over `Expr`, and it is where the evaluator owns what the C# capstone's checker pass owned, undefined names and operand kinds, the test comment states it outright: "no check pass in this port, the evaluator owns kind errors":

```
    member private x.Eval(expr: Expr) : Value =
        match expr with
        | NumberExpr(v, _) -> Number v
        | TextExpr(v, _) -> Text v
        | BoolExpr(v, _) -> Flag v
        | IdentExpr(name, line) ->
            match names.TryGetValue name with
            | true, value -> value
            | false, _ -> raise (EvalException(line, $"undefined name '{name}'"))
        | UnaryExpr(op, operand, line) ->
            let v = x.Eval operand
            if op = Bang then
                match v with
                | Flag b -> Flag(not b)
                | _ -> raise (EvalException(line, $"'!' needs a flag, got {kindOf v}"))
            else
                match v with
                | Number n -> Number(-n)
                | _ -> raise (EvalException(line, $"unary minus needs a number, got {kindOf v}"))
        | BinaryExpr(op, l, r, line) -> apply op (x.Eval l) (x.Eval r) line
        | CallExpr(name, args, line) ->
            match table.TryGetValue name with
            | true, fn ->
                args |> List.map (fun a -> x.Eval a) |> Array.ofList |> fn
            | false, _ -> raise (EvalException(line, $"unknown builtin '{name}'"))
```

listing 198: the eval match, kind errors and unknown builtins raised here (source: csharp-net/fsharp/src/FSharpBook/Evaluator.fs, lines 115-139)

That is the whole deliberate scope. Kept: `sqrt` and `abs` as the synchronous builtins. Dropped: async end to end with `clock_async`, the checker pass, and the persistence that chapter 35's saves section describes, no SQLite file, no `Names`, no `LoadNames`, the evaluator's dictionary lives and dies with the process. The suite that pins all of it: 20 tests, 5 lexer, 4 parser, 11 evaluator, zero skipped, green under the pinned SDK.

sources: learn.microsoft.com, what's new in .net 11 libraries and overview pages, what's new in f# 9 and f# 10 pages, accessed 2026-09-13. fsi, compiler, runtime, and serializer behavior measured in `dotnet fsi` sessions under SDK 11.0.100-rc.1.26425.128. Sample behavior verified by `dotnet test fsharp/FSharpBook.slnx`, 20 tests, zero skipped.

22. api kernel

Everything else in this service part sits on one small folder: the api kernel. This chapter builds it in `CsharpBook.Api/Kernel`: the request delegate and result contract, the route table with the fallback that keeps the platform's two miss answers inside the contract, the constant error envelope every non-2xx response answers with, the System.Text.Json codec whose bytes match the go lane's exactly, the in-process test server, and the composition root the part appends to.

22.1. the request delegate contract

An aspnet handler is one delegate, `RequestDelegate`, a function from `HttpContext` to `Task`, and that shape is the whole server: the pipeline `WebApplication` builds is a chain of these delegates folded over each other, and an endpoint registered with `MapGet` is a terminal one. The kernel routes do not return plain values, they return `IResult`, the deferred response, an object with one `ExecuteAsync` method that writes status, headers, and body into the context when the framework executes it, with `TypedResults` as the typed factory over the built-ins. Deferring instead of writing inline is what keeps the failure path uniform: one result type owns every failure body.

The kernel keeps the shape and adds one convention on top, the exception channel. A handler that fails throws `ApiError` instead of writing a failure, and the exception middleware of the next chapter turns the throw into the contract envelope, so no route formats a failure body by hand and no route can answer a 500 with the wrong shape. Go returns errors, c# throws, and the two idioms land on the same contract. Until the middleware exists, the kernel's own `EnvelopeResult` is the one result that renders an `ApiError`:

```
public sealed class EnvelopeResult(ApiError error) : IResult
{
    public async Task ExecuteAsync(HttpContext context)
    {
        var envelope = new Envelope(ErrorBody.From(error, RequestId.Resolve(context)));
        await context.Response.WriteJsonAsync(error.Status, envelope, context.RequestAborted);
    }

    public static EnvelopeResult From(ApiError error) => new(error);
}
```

listing 199: the kernel's one IResult: execute resolves the request id and writes the envelope (source: csharp-net/api/src/CsharpBook.Api/Kernel/EnvelopeResult.cs, lines 10-19)

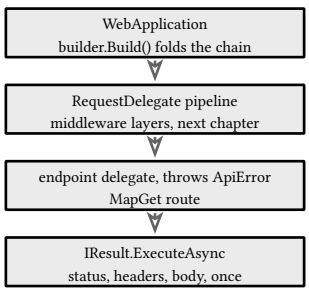


figure 136: the request climbs the pipeline, the result executes into the response

22.2. endpoint routing as the route table

Endpoint routing is a real router. A route registers with `MapGet`, `MapPost`, and siblings, one method per pattern, wildcards like `{id}` come from the template, and matching runs in phases over the whole candidate set, the documentation's wording, so the more specific pattern beats the looser one the way go's ServeMux precedence does. Two of the platform's answers violate the contract. An unknown path gets a bare 404 from the routing

middleware, a known path with the wrong method gets a bare 405 plus an `Allow` header from a sentinel endpoint the matcher selects, and the contract demands the envelope on both.

The fallback registers above `UseRouting`, which the composition root pins explicitly, because the platform's documented behavior is that routes added directly to `WebApplication` execute at the end of the pipeline, so middleware registered before the explicit `UseRouting` wraps the router and every miss unwinds into the fallback before anything is flushed. A 404 with no endpoint selected is a route miss, a bare 405 with no content length is a method miss, and anything a handler answered with a body of its own is left untouched. The 405 keeps the `Allow` header the router computed, and both statuses carry the `not_found` code, the closed code set having no `method_not_allowed` member, with the message carrying the distinction:

```
public async Task Invoke(HttpContext context)
{
    await next(context);
    if (context.Response.HasStarted)
    {
        return;
    }
    var status = context.Response.StatusCode;
    if (status == StatusCodes.Status404NotFound && context.GetEndpoint() is null)
    {
        await Write(context, ApiError.Fail(ErrorCode.NotFound, "route not found"));
        return;
    }
    if (status == StatusCodes.Status405MethodNotAllowed && context.Response.ContentLength is null)
    {
        await Write(
            context,
            ApiError.Fail(ErrorCode.NotFound, "method not allowed")
                .At(StatusCodes.Status405MethodNotAllowed)
        );
    }
}
```

listing 200: the fallback owns the two answers the router would emit bare (source: csharp-net/api/src/CsharpBook.Api/Kernel/EnvelopeFallback.cs, lines 21-42)

NOTE: head does not ride a get pattern

Go's ServeMux serves HEAD on a `GET` pattern by design. Endpoint routing does not: HEAD against a get-only route is a method miss, 405 with `Allow`, the same envelope, pinned by a test, a platform ruling this lane inherits.

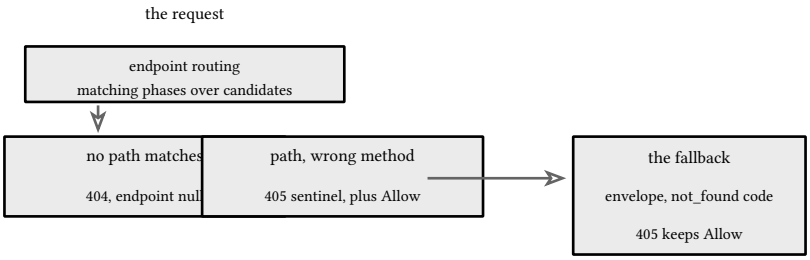


figure 137: one router, two misses, both rewritten into the envelope

22.3. the error envelope

The contract freezes one failure shape for the whole api, smaller than most: an error object carrying a code from a closed set of eleven, one human sentence, optional `details` on `validation_failed` only, and the request id. Every

non-2xx rides it, including the 404 and 405 the fallback owns, the 422 the user chapter adds, and the 429 and 503 the rate and load chapters add. A constant envelope beats bare status codes because a client writes one error path instead of thirteen, the statuses the wire actually serves, a grep for the code string works across every endpoint, and the day a status needs to move the code does not.

The codes are declared once, an `ErrorCode` enum with eleven members, and one table maps each code to the status it carries everywhere, the grid below. The two split codes each have exactly one sanctioned override site, expressed as `At(status)` on the error: `invalid_json` rides 415 on a wrong media type and `not_found` rides 405 on a method miss, while the map supplies every other status. The record is two levels deep so the wire shape stays nested the way the vectors freeze it, property order is declaration order, and `details` carries a `JsonIgnore` condition that keeps the member absent everywhere except validation, the c# spelling of go's omitzero:

```
public sealed record Envelope(ErrorBody Error);

public sealed record ErrorBody
{
    public required ErrorCode Code { get; init; }

    public required string Message { get; init; }

    /// <summary>Rides along on validation_failed only; null keeps the
    /// member off the wire, mirroring go's omitzero.</summary>
    [JsonIgnore(Condition = JsonIgnoreCondition.WhenWritingNull)]
    public IReadOnlyList<object>? Details { get; init; }

    public required string RequestId { get; init; }

    public static ErrorBody From(ApiError error, string requestId) =>
        new()
        {
            Code = error.Code,
            Message = error.Message,
            Details = error.Details,
            RequestId = requestId,
```

listing 201: the envelope: one shape, closed codes, details off the wire unless present (source: csharp-net/api/src/CsharpBook.Api/Kernel/Envelope.cs, lines 11-32)

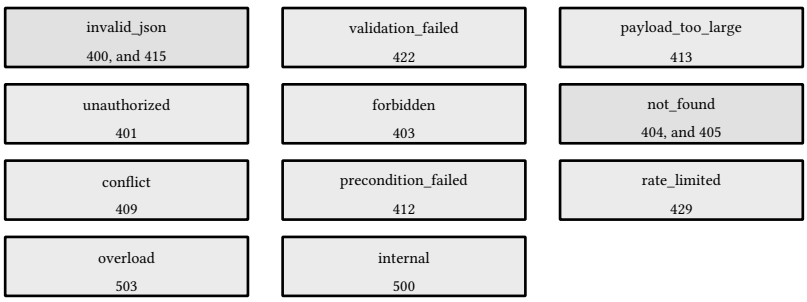


figure 138: eleven codes, eleven statuses in the table, thirteen on the wire, two sanctioned splits

22.4. json in and out

Bodies go out through System.Text.Json behind one options instance, `KernelJson.Wire`, and every property the api writes rides it. Three settings do the contract's work. `SnakeCaseLower`, the built-in naming policy shipped in .NET 8, maps each c# property to the snake case name the wire freezes, `RequestId` to `request_id`. The enum converter writes codes as their snake case strings rather than numbers. The relaxed encoder leaves non-ascii text raw instead of escaping it to \u sequences, which is what go's encoder does, so accented text serializes identically in both lanes.

Declaration order is serialization order, so the dto property order mirrors the go struct order, and the vector bytes match lane for lane.

The write helper commits the status and the content type before the body serializes, the same ordering tradeoff the go kernel makes, safe for the kernel's marshal-by-construction records and wrong for anything that can fail to marshal, and the content type is `application/json` exactly, no charset parameter, because the vectors freeze that header too. Reading is stricter, because request bodies are input. `BindAsync<T>` is the one decode path, generic over the target dto, and it enforces three rungs in a fixed order: the media type must parse as `application/json`, so a charset parameter still passes and anything else is 415 `invalid_json`, the body runs through a stream capped at 1 MiB whose overflow surfaces as a distinct exception type and answers 413 `payload_too_large`, and what survives must parse, a malformed body answering 400 `invalid_json`. A body of the literal `null` is ruled malformed rather than zero-valued, this lane's ruling:

```
    public static readonly JsonSerializerOptions Wire = new()
    {
        PropertyNamingPolicy = JsonNamingPolicy.SnakeCaseLower,
        Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping,
        Converters = { new JsonStringEnumConverter(JsonNamingPolicy.SnakeCaseLower) },
    };

    /// <summary>The content type every json response carries, exactly,
    /// with no charset parameter, because the vectors freeze
    /// it.</summary>
    public const string ContentType = "application/json";

    /// <summary>
    /// The contract's request body cap, 1 MiB, enforced before any
    /// decode work happens.
    /// </summary>
    public const int MaxBodyBytes = 1 << 20;
```

listing 202: the wire options and the caps: one settings object behind every byte (source: csharp-net/api/src/CsharpBook.Api/Kernel/KernelJson.cs, lines 16-32)

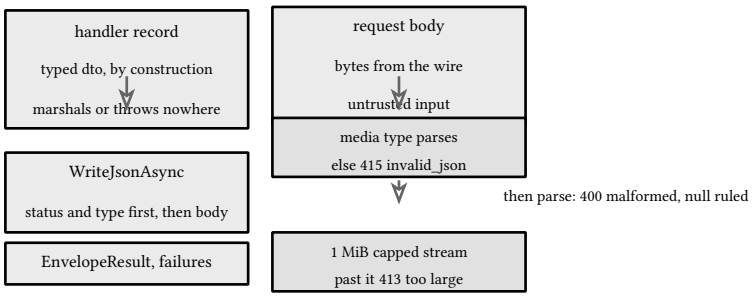


figure 139: two write sites, one decode ladder, four contract statuses

22.5. the in-process test server

`WebApplicationFactory<Program>` gives the kernel its whole test story without a socket. The factory boots the real composition root on an in-memory TestServer, hands back an `HttpClient`, and the round trip is a function call. Nothing listens, so there is no port to collide, no firewall prompt, and no timing, and `public partial class Program` is the one accessibility line the factory needs to see the entry point, the migration documentation's own recipe.

The pattern the whole suite copies is three steps, build a factory, make one client, assert on exact values. Exact bytes are asserted where the contract freezes them, the health body as a whole string, and structural assertions elsewhere, the envelope's code and message and a parseable `request_id` guid. Unit level, `DefaultHttpContext` is the recorder, a memory stream standing in for the body while the same delegates the server would run are invoked

directly, which is how the bind ladder, the id resolution, and the fallback run without a host. One platform gap needs a double: no server sits behind `DefaultHttpContext`, nothing ever starts the response, so a strict feature double pins `HasStarted` to true and the started branches run under test too:

```
    public async Task HealthzAnswersTheExactFrozenBody()
    {
        using var factory = Factory();
        var client = factory.CreateClient();

        var response = await client.GetAsync("/healthz");
        var body = await response.Content.ReadAsStringAsync();

        Assert.Equal(System.Net.HttpStatusCode.OK, response.StatusCode);
        Assert.Equal("application/json", response.Content.Headers.ContentType?.MediaType);
        Assert.Equal("""{"status":"ok","version":"dev"}""", body);
    }
```

listing 203: the factory round trip: exact frozen bytes on the deployment contract (source: csharp-net/api/tests/CsharpBook.Api.Tests/Kernel/KernelEndpointTests.cs, lines 32-43)

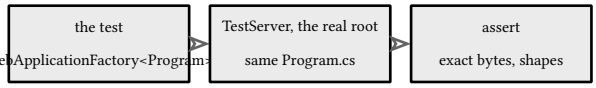


figure 140: the factory boots the real root on an in-memory server

22.6. the app skeleton and its wiring table

The kernel also ships the program skeleton, and the skeleton is a file, `Program.cs`, with a discipline attached: a wiring table with one writer. This chapter pins the root at its smallest, the kernel's own registrations behind one call, and the ship chapter later lands `Deploy/Wire.cs` as the single table every family composes through, the c# mirror of go's `wire.go`, so the root never grows a line per family and every family's wiring lives in one reviewable place. Each family registers services through one extension beside its code, and the table reads top to bottom as the request flows.

The kernel's registrations are the clock and the config hub. `TimeProvider.System` registers once and every family resolves it, the seam the tests swap for the stepped fake. `ApiOptions` carries the build version from `GOAPI_VERSION` and the request budget from `GOAPI_REQUEST_TIMEOUT`, the `GOAPI_` names kept for compose parity with the go lane. The two kernel routes are the deployment contract. `GET /healthz` answers liveness, 200 with the status and version, so a deployment proves what it is running. `GET /readyz` answers readiness, 200 while the probe passes and the envelope's `overload` code at 503 the moment it does not, the hook the drain wiring flips:

```
if (await ShipProbe.RunAsync(args)) return;

var builder = WebApplication.CreateSlimBuilder(args);

// ch33: services and listeners, the table's first half.
Wire.Services(builder);

var app = builder.Build();

// ch33: the stack and every route, the table's second half.
Wire.Stack(app);

app.Run();
```

listing 204: the root at its smallest: the probe line, two table calls, run (source: csharp-net/api/src/CsharpBook.Api/Program.cs, lines 11-23)

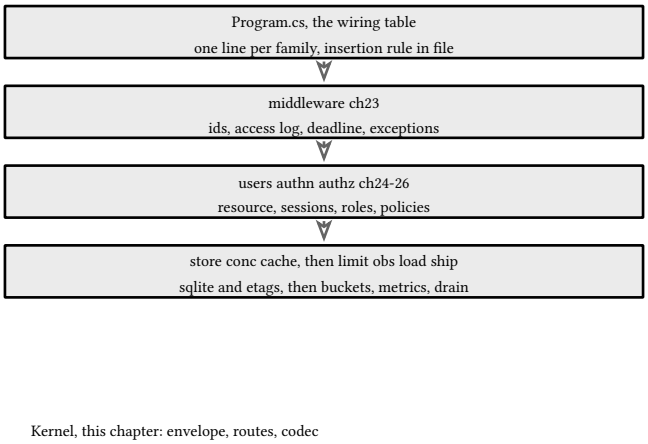


figure 141: the part's families stack on the kernel, the wiring table owns the order

22.7. what the kernel proves

The kernel is small on purpose, 10 files and a test folder, and it proves the part's shape is already fixed. The envelope exists, so every later failure is a `Fail` call and never a handbuilt body, and a test builds the vector 05 body from the frozen file and compares the serialization byte for byte. The decode ladder exists, so every write route inherits the 415, the 413, and the 400 without repeating them, and the route table exists with the fallback that keeps the contract on the two answers the router gets wrong. The composition root exists with its insertion rule, so the roadmap from here is additive, every chapter adding routes or layers, none rewriting the kernel:

```
    public static IEndpointRouteBuilder MapKernel(this IEndpointRouteBuilder endpoints)
    {
        endpoints.MapGet(
            "/healthz",
            (ApiOptions options) =>
                TypedResults.Json(new HealthzBody("ok", options.Version), KernelJson.Wire,
KernelJson.ContentType)
        );
        endpoints.MapGet(
            "/readyz",
            (ApiOptions options) =>
                options.Ready is { } ready && !ready()
                    ? (IResult)
                        EnvelopeResult.From(ApiError.Fail(ErrorCode.Overload, "not ready"))
                    : TypedResults.Json(new ReadyzBody("ready"), KernelJson.Wire, KernelJson.ContentType)
        );
        return endpoints;
    }
```

listing 205: the kernel's two routes, the deployment contract in full (source: csharp-net/api/src/CsharpBook.Api/Kernel/KernelRoutes.cs, lines 13-29)

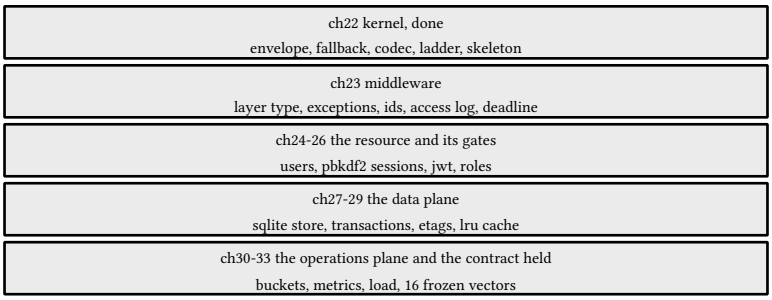


figure 142: the part as six rows: every chapter adds routes or layers, none rewrites the kernel

sources: learn.microsoft.com/aspnet/core/fundamentals/routing for the matching phases and the end-of-pipeline execution of routes added to WebApplication, learn.microsoft.com/aspnet/core/test/integration-tests for WebApplicationFactory and the TestServer hosting model, and learn.microsoft.com/dotnet/api/system.text.json.jsonnamingpolicy.snakecaselower for the naming policy, all accessed 2026-09-26. Verified by `CsharpBook.Api.Tests.Kernel` under `dotnet test Api.slnx`, 47 tests, plus dotnet build and dotnet format over the touched projects.

23. middleware

Cross-cutting behavior does not belong inside the handlers, and aspnet does not make anyone put it there. This chapter composes the service's middleware stack over the previous chapter's kernel, in `CsharpBook.Api/Middleware`: the layer type that folds delegates into one pipeline, exceptions into the contract envelope, request ids born once and quoted everywhere, the access log with its status-capture problem, the handler deadline on a linked token, and the wiring order that makes the stack correct rather than merely present.

23.1. the layer type

One layer is one function: given the next delegate, return the delegate that wraps it. That is the whole `Layer` type, a delegate from `RequestDelegate` to `RequestDelegate`, and it is the same shape the framework's own `IApplicationBuilder` stores, so anything written against it composes with anything else. `Chain.Compose` folds a list of layers into one by wrapping right to left, so the first layer ends up outermost: it sees the request first and the response last, the onion picture every middleware chapter draws because it is the true one. A composed chain is itself invocable, so chains compose into bigger chains.

The loop runs backwards for a reason worth saying out loud: the framework's own pipeline build is this fold, `builder.Build()` walks the registered components in reverse and folds each over the next, which is why middleware registered first runs first, and why the composition root's list reads top to bottom as the request flows. The stack test pins the order `id, log, timeout, exceptions, handler` and a reorder anywhere fails it:

```
public static RequestDelegate Compose(RequestDelegate terminal, params Layer[] layers)
{
    for (var i = layers.Length - 1; i ≥ 0; i--)
    {
        terminal = layers[i](terminal);
    }
    return terminal;
}
```

listing 206: one layer is a function over the next delegate, the fold wraps the first outermost (source: csharp-net/api/src/CsharpBook.Api/Middleware/Chain.cs, lines 20-27)

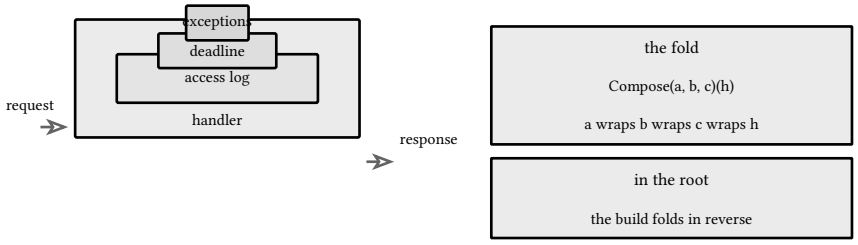


figure 143: the onion: first layer outermost, request in, response out

23.2. exception to envelope

A throwing handler must not take the process down, and the client must still get the contract envelope, and a bare delegate pipeline promises neither on its own. The exception layer wraps `next` in one `try`: an `ApiError` renders as its own code, message, and status through the kernel's one failure shape, and anything else is answered as `internal` with the detail kept for the logs, because a leaked store exception tells a client more than it should know. This layer is the adapter the kernel chapter deferred, go's error-returning handler contract transposed into the exception channel.

Two rethrows are deliberate. `OperationCanceledException` while the request's abort token is canceled means the client is gone, the server will drop the connection, and writing a 500 nobody reads would log a lie, so it propagates

untouched, the aspnet counterpart of go's rethrown `ErrAbortHandler`. The same filter covers a cancellation raised by the deadline layer's linked token, so an abandoned pipeline never answers over the deadline's envelope. And a failure after the response started cannot be answered at all: half a body is already on the wire, the platform's own guidance says headers and status are immutable once the body has begun, so that exception is logged and rethrown, the one case where the envelope stays unwritten:

```
        catch (Exception ex) when (context.Response.HasStarted)
        {
            logger.LogError(ex, "failure after response started {RequestId}", RequestId.Resolve(context));
            throw;
        }
        catch (ApiError error)
        {
            var envelope = new Envelope(ErrorBody.From(error, RequestId.Resolve(context)));
            await context.Response.WriteJsonAsync(error.Status, envelope, context.RequestAborted);
        }
        catch (OperationCanceledException) when (context.RequestAborted.IsCancellationRequested)
        {
            throw;
        }
```

listing 207: ApiError renders, cancellation rethrows, a started response is left alone (source: csharp-net/api/src/CsharpBook.Api/Middleware/ExceptionMiddleware.cs, lines 29-42)

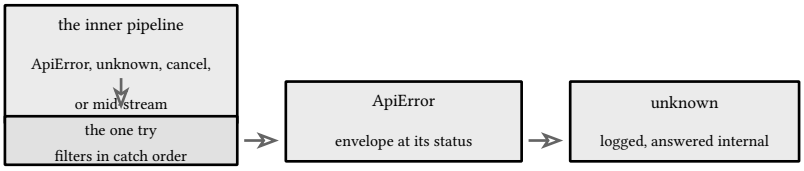


figure 144: one throw, four paths: envelope, rethrow, rethrow, log and rethrow

23.3. request id

Every request needs one id that every artifact agrees on: the access log line, the error envelope, the exception log, and later the trace. The layer resolves it exactly once, at the edge, using the kernel's `RequestId.Resolve` order: the stored value when some layer already installed one, else the client's `X-Request-ID` when it parses as a guid, else a fresh version 7 guid. Parsing is not politeness, it is contract: the envelope promises `request_id` is a guid, and a client-supplied free-text header echoed into logs is log injection. Garbage is replaced, never echoed, and the version 7 choice keeps ids time-ordered the way the store chapter's ids are.

The layer then does two small things with the value: it sets the response header, so a client correlates its own logs with the server's, and it installs the id in `HttpContext.Items` through the kernel's `RequestId.Set`, so everything downstream resolves one value without touching headers again. The tests check all three births: a client guid echoes, garbage becomes a fresh version 7, and a handler that wipes the incoming header still resolves the stored id:

```
    public Task Invoke(HttpContext context)
    {
        var id = RequestId.Resolve(context);
        context.Response.Headers[RequestId.HeaderName] = id;
        RequestId.Set(context, id);
        return next(context);
    }
```

listing 208: resolve once at the edge, echo it, store it for downstream (source: csharp-net/api/src/CsharpBook.Api/Middleware/RequestIdMiddleware.cs, lines 15-21)

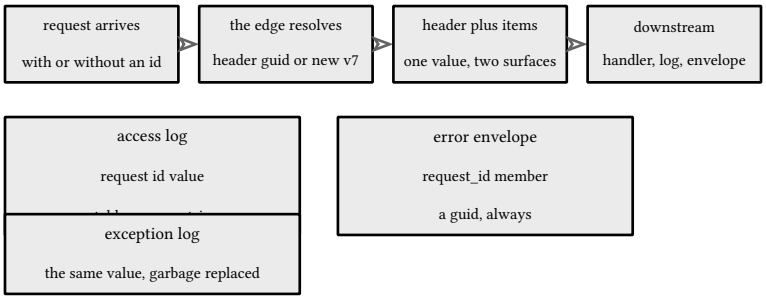


figure 145: one id, born at the edge, quoted by everything downstream

23.4. the access log

Handlers set a status and never return it, and a layer that wants to log what actually went out has to watch it go. AspNet answers the go chapter's status-capturing writer differently: `HttpResponse` exposes `StatusCode` as a readable property, so no wrapper is needed to learn the status, and the load-bearing question is when to read it. The log layer registers a witness through `OnStarting`, the platform's hook for work that must run just before headers flush, and the callback snapshots the status at the exact moment of commit. The documentation calls that callback the last chance to modify headers, status, and reason phrase, which also makes it the only truthful witness of the status the client saw, and registering it after the response started throws, so the layer guards on `HasStarted` first. When no commit ever happens, a unit context with no server, the log falls back to the final property, and both paths are pinned by tests.

Bytes are counted by wrapping the body stream in a `CountingStream` installed before the inner pipeline runs and removed after it returns, the one place every byte of the body flows through in this stack. The line written afterwards carries method, path, status, bytes, duration, and request id, with the duration measured off the injected `TimeProvider`, never a wall clock. The write happens in a `finally`, so a handler that throws is still logged, the fact the wiring order section depends on:

```
        var start = clock.GetUtcNow();
        var witness = new CommitWitness();
        if (!context.Response.HasStarted)
        {
            context.Response.OnStarting(
                static state =>
                {
                    var w = (CommitWitness)state!;
                    w.Status = w.Context!.Response.StatusCode;
                    return Task.CompletedTask;
                },
                witness.With(context)
            );
        }
        var original = context.Response.Body;
        var counting = new CountingStream(original);
        context.Response.Body = counting;
```

listing 209: the witness at commit, the counting wrap, before the inner pipeline runs (source: csharp-net/api/src/CsharpBook.Api/Middleware/AccessLogMiddleware.cs, lines 31-47)

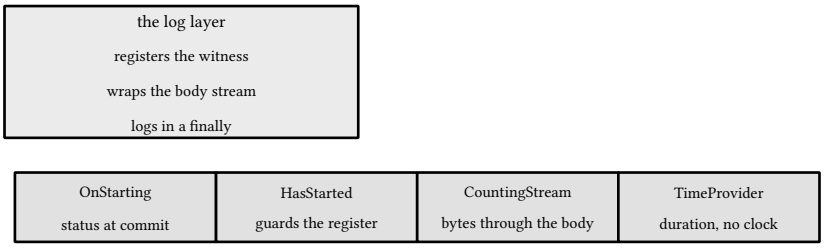


figure 146: status read at commit through OnStarting, bytes counted at the body

23.5. the handler deadline

A service needs a deadline at two layers because the two layers fail differently. The transport layer is the server's own configuration, kestrel's keep-alive and header timeouts, coarse knobs that protect the process from slow clients and drop the connection when they fire. The handler layer is finer, one request's pipeline time, after which the client deserves the contract's `overload` envelope rather than a dropped connection, and that is a middleware.

The deadline layer builds a `CancellationTokenSource` linked to the request's abort token and replaces `context.RequestAborted` with the linked token, so a well-behaved handler observes the deadline the same way it observes a client leaving. A timer from `TimeProvider.CreateTimer`, one callback, one due time, signals a completion source, and the layer awaits the race and then resolves it on the facts, not the winner: a pipeline that ran to completion passes its response through untouched, a fault with the deadline unfired propagates to the layers outside, and everything else is expiry, because the linked token's cancellation can tear the pipeline down before the signal task is observed and the deadline firing is the truth either way. The expiry path answers the envelope only when the response never started and the original client token is still alive, because a client that left has nobody to read a body. The abandoned pipeline keeps running against the canceled token, and its eventual fault is observed by a continuation that logs it with the request id when it is not the expected cancellation, never left unobserved. The clock comes from `TimeProvider` throughout, which is why the tests drive expiry by stepping a fake clock, no sleeps anywhere:

```
                signal.TrySetResult();
                try
                {
                    cts.Cancel();
                }
                catch (ObjectDisposedException)
                {
                    // the success path disposes the source the moment
                    // the pipeline completes, and a real timer's
                    // callback is not fenced against that: firing into
                    // the window means nobody is left to cancel, and an
                    // exception escaping on the timer thread would take
                    // the process down
                }
            },
            (timeout, expired),
            options.RequestTimeout,
            Timeout.InfiniteTimeSpan
        );

        var inner = next(context);
        _ = await Task.WhenAny(inner, expired.Task).ConfigureAwait(false);
        if (inner.IsCompletedSuccessfully)
        {
            timeout.Dispose();
            return;
        }
        // the deadline's fact is the linked source's own state, set
        // synchronously before any cancellation continuations run. A
        // pipeline torn down by the timer can complete its await chain
        // inside Cancel() and reach this check before the wake-up task
        // is marked, so the wake-up artifact is never read as the fact.
        // The client's own token leaving is the other way the linked
        // source cancels, and that fault propagates to the layers
        // outside instead.
        if (!(timeout.IsCancellationRequested && !clientToken.IsCancellationRequested))
```

listing 210: the race resolved: success passes through, a fault without the deadline propagates, the fact read from the linked source, the cancel fenced against a source the success path already disposed (source: csharp-net/api/src/CsharpBook.Api/Middleware/TimeoutMiddleware.cs, lines 43-78)

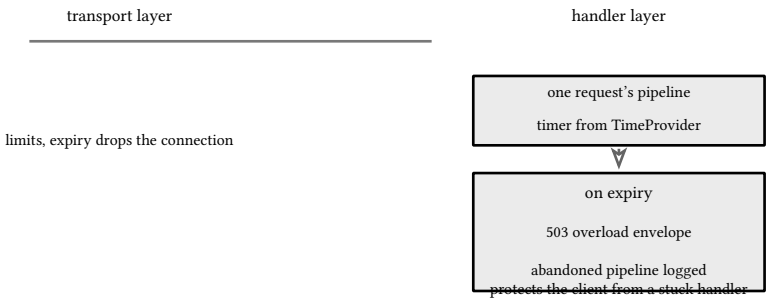


figure 147: two clocks: transport deadlines drop connections, the handler deadline answers

23.6. wiring order and why

The order of the four layers is contract, and each position is a sentence. Request id outermost, because every other artifact quotes it: the access log value, the exception log, the envelope, all of it. Access log second, outside the exception layer, so a converted 500 is logged as the 500 the client saw rather than a status a wrapper never

observed, and because the log line lives in a finally, the request that throws is still logged. Deadline third. Exceptions innermost, and this one is worth the argument: aspnet's conventional placement for an exception handler is outermost, first registered, so it catches failures from every layer including the middleware itself, but this stack's deadline does not move the pipeline onto another thread the way go's timeout moved its handler onto another goroutine, so an inner placement catches everything the handlers throw, and the log sees the answer it wrote. The go chapter's mechanical goroutine rule and this lane's log-visibility rule land on the same order:

```
app.UseRequestId(); // ch23: one id, born at the edge
app.UseAccessLog(); // ch23: one structured line per request
app.UseBookDrain(); // ch32: count in-flight, refuse once draining
app.UseBookTracing(); // ch31: the server span wraps the family layers
app.UseBookLogScope(); // ch31: the request id scope under the log
app.UseBookMetrics(); // ch31: both series per route
app.UseBookAdmission(); // ch32: overload refused at the cap
app.UseBookRateLimiting(); // ch30: the 429 gate, headers on draws
app.UseRequestTimeout(); // ch23: the handler deadline
app.UseApiExceptions(); // ch23: exceptions become the envelope
```

listing 211: the four ch23 lines in the shipped stack, outermost first, later families interleaved between them (source: csharp-net/api/src/CsharpBook.Api/Deploy/Wire.cs, lines 84-93)

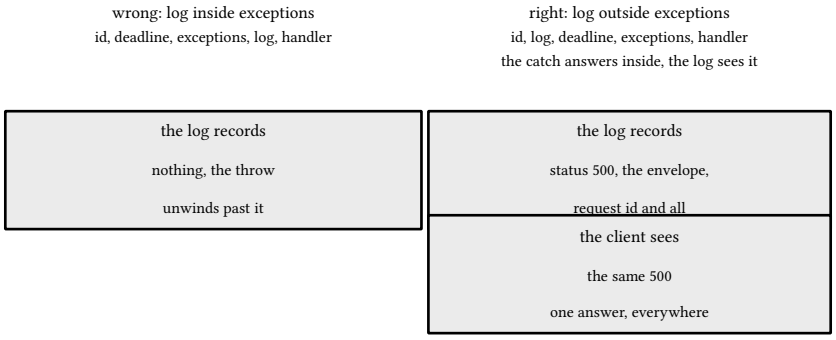


figure 148: wrong order versus right: where the log sits decides what it sees

23.7. testing the stack

Middleware is tested like handlers, mostly. The fast paths run through `DefaultHttpContext`, the unit recorder from the kernel chapter, with each layer constructed directly over a tiny handler and every dependency injected: a recording logger asserts on the structured values that were logged, never on console output, and a stepped `TimeProvider` moves time only when the test says so, its timers firing synchronously inside `Advance`. The blocking path is the addition: a handler that never returns until its token cancels, an infinite delay with no wall clock, and the test steps the fake clock past the budget, awaits the middleware, and joins the abandoned pipeline back through a completion source the way the go chapter joins its abandoned goroutine through a channel. The client-gone path needs no clock at all: a pre-canceled request token distinguishes a client leaving from a deadline expiring with zero timing dependence.

The stepped provider is hand-rolled on purpose. The platform ships `Microsoft.Extensions.TimeProvider.Testing` with its own `FakeTimeProvider`, a Microsoft-official package that is this same fake, and the vehicle declines it for the same reason it declines assertion libraries: the fake is a hundred and forty lines, the learning is in those lines, and a test project that stays dependency-free is one whose doubles can be read in a sitting. The stack test at the end composes all four layers over a throwing handler and asserts the property the whole chapter exists to keep: the log and the envelope agree on the 500, and both quote the id the edge resolved:

```
        var context = TestHttpContext.New();

        await stack(context);

        Assert.Equal(500, context.Response.StatusCode);
        var logged = Assert.Single(accessLogger.Entries);
        Assert.Equal(500, logged.Values["Status"]);
        var requestId = Assert.IsType<string>(logged.Values["RequestId"]);
        var error = JsonDocument.Parse(await TestHttpContext.BodyTextAsync(context))
            .RootElement.GetProperty("error");
        // one answer, everywhere: the envelope and the log quote the id
        // the edge resolved
        Assert.Equal(requestId, error.GetProperty("request_id").GetString());
        Assert.Single(exceptionLogger.Entries);
```

listing 212: the stack test: log and envelope agree on the 500 and the id (source: csharp-net/api/tests/CsharpBook.Api.Tests/Middleware/StackTests.cs, lines 128-141)

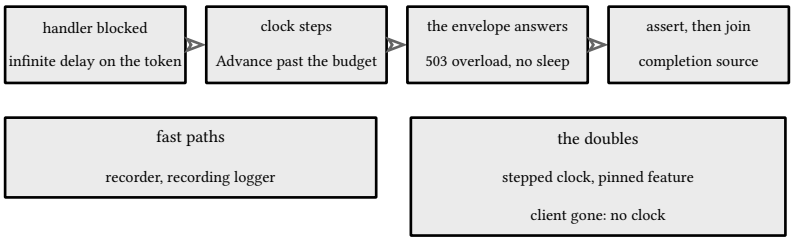


figure 149: the harness: gates hold handlers, clocks step, channels join

sources: learn.microsoft.com/aspnet/core/fundamentals/middleware for the pipeline and order rules, learn.microsoft.com/aspnet/core/ fundamentals/routing for the UseRouting placement, the best-practices page and learn.microsoft.com/dotnet/api/microsoft.aspnetcore.http. httpresponse.onstarting for the HasStarted guard and the commit callback, and learn.microsoft.com/dotnet/api/system.timeprovider. createtimer for the timer surface, all accessed 2026-09-26. Verified by `CsharpBook.Api.Tests.Middleware` under `dotnet test Api.slnx`, 22 tests beside the kernel suite, plus dotnet build and dotnet format.

24. the user resource

The service's first real resource is the user: a record with an id, an email, a display name, roles, a creation time, and a version. This chapter builds the resource layer on the chapter 22 kernel: the representation and its two projections, strict decoding, validation from first principles, merge-patch semantics, keyset pagination, the status contract that binds failures to codes, and the golden handler tests that pin all of it against the frozen contract vectors.

24.1. the representation

A resource is three things at once: the bytes on the wire, the fields each reader may see, and the strong etag that names one exact version of the body. One c# record expresses all three cheaply: the stored record and the full wire representation stay one type, because System.Text.Json maps each property to a member through the snake case naming policy and a JsonIgnore keeps the password hash off the wire without a second type drifting beside the first:

```
    public required string Id { get; init; }
    public required string Email { get; init; }
    public required string DisplayName { get; init; }
    public required IReadOnlyList<string> Roles { get; init; }
    public required DateTimeOffset CreatedAt { get; init; }
    public required int Version { get; init; }

    /// <summary>
    /// The password hash rides along for the authn family but never
    /// serializes, so the stored record and the wire shape stay one
    /// type. Not required: json metadata treats a required member as a
    /// deserialization demand even when ignored.
    /// </summary>
    [JsonIgnore]
    public string PasswordHash { get; init; } = "";
```

listing 213: the record, its wire members, and the hash that never rides (source: csharp-net/api/src/CsharpBook.Api/Users/User.cs, lines 14-28)

Declaration order is wire order, which is why the frozen vectors' bytes come out of the serializer with no configuration beyond the policy. The projection is a second record, and one method chooses between them: Project answers the full record for an admin and a four-member public shape for everyone else, so email and roles stay admin-only data, and a rule enforced in one method cannot be forgotten in a second handler. The version member is the quiet third actor: it bumps exactly once per successful write, and the concurrency chapter promotes it into the compare-and-swap substrate.

The etag is not stored anywhere, it is computed: serialize the full representation once, sha256 those exact bytes with SHA256.HashData, quote the hex with Convert.ToHexStringLower. The etag is a function of the body, so the two can never disagree, and the one UserWire ETagBody call serves every handler.

c# property	json member
Id string	id
Email string	email
DisplayName string	display_name
Roles list	roles
CreatedAt offset	created_at
Version int	version
PasswordHash string	JsonIgnore, never sent

figure 150: the record properties against the wire members, one property kept off the wire

One converter carries the timestamps: whole seconds render as RFC 3339 utc with the Z suffix, because the vectors freeze those exact bytes and the BCL default would render a numeric offset instead. That converter is the one addition this layer makes to the kernel's shared options instance, and every DTO in the vehicle serializes through the combined result.

24.2. decoding with strictness

Input is hostile until proven otherwise, and the kernel's bind helper answers three questions in a fixed order: is the media type json, is the body parseable inside the 1 MiB cap, and does it carry only members the contract knows. The register route runs the whole ladder in one call, with the strictness riding as one attribute on the dto:

```
    [JsonUnmappedMemberHandling(JsonUnmappedMemberHandling.Disallow)]
    private sealed record RegisterRequest(string? Email, string? Password, string? DisplayName);

    private static async Task RegisterAsync(UserFamily family, HttpRequest request, CancellationToken ct)
    {
        var body = await request.BindAsync<RegisterRequest>(ct);
        var details = UserValidation.ValidateRegister(body.Email ?? "", body.Password ?? "",
body.DisplayName ?? "");
        if (details.Count > 0)
        {
            throw ApiError.Fail(ErrorCode.ValidationFailed, "validation failed", [.. details]);
        }
```

listing 214: strict bind, rule validation, then the store, in that order (source: csharp-net/api/src/CsharpBook.Api/Users/UserEndpoints.cs, lines 43-53)

JsonUnmappedMemberHandling.Disallow is the strictness that pays rent, and it is an attribute rather than a decode flag because System.Text.Json puts the decision on the contract type itself: a client that sends a pet member against a dto with no pet gets a 400, not a silently ignored field, so a typo'd member name surfaces at the border instead of corrupting data six layers in. The kernel's bind helper maps the thrown JsonException to the invalid_json envelope.

The order matters for the client's sake: the 415 fires before a body byte is read, the 400 at parse time, and only then does validation answer 422 with one detail per broken rule. A client fixing its request learns everything wrong with it in one round trip.

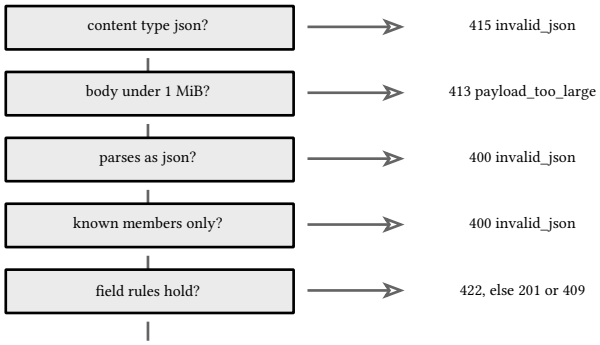


figure 151: the decode ladder: each rung rejects a class of bad input

24.3. validation from first principles

Validation rules come from the contract, not from a library: passwords at least 12 characters and at most 128, one email shape, display names 1 to 64 characters. Stating them as plain code keeps every number greppable and every message owned. The method returns one detail per violated rule rather than failing on the first, so the 422 body's details array tells the whole story:

```
    public static List<string> ValidateRegister(string email, string password, string displayName)
    {
        var details = new List<string>();
        if (email.Length == 0)
        {
            details.Add("email is required");
        }
        else if (!IsValidEmail(email))
        {
            details.Add("email is not a valid address");
        }

        if (password.Length == 0)
        {
            details.Add("password is required");
        }
        else if (password.Length < MinPasswordLength)
        {
            details.Add("password must be at least 12 characters");
        }
        else if (password.Length > MaxPasswordLength)
        {
            details.Add("password must be at most 128 characters");
        }

        if (displayName.Length == 0)
        {
```

listing 215: one detail per broken rule, in field order (source: csharp-net/api/src/CsharpBook.Api/Users/UserValidation.cs, lines 18-44)

Three classes of rule run here. Presence rules catch missing members. Shape rules catch an email that parses as nothing, where MailAddress plus an exact-address check and a dotted-domain test approximates the real grammar without rejecting the addresses people actually hold. Range rules catch lengths. The bounds live in named constants next to the rules, so handler, tests, and prose quote one set of numbers.

The 422 and the 409 are different animals. A rule violation is about the request alone, checkable without touching state, so it is 422 validation. The email conflict is about the store's current contents, so it is 409 conflict, and it

surfaces only after validation passed and the create call answered its conflict outcome. Same request shape, two different truths.

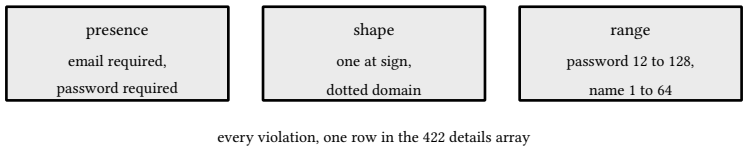


figure 152: three rule classes, one details array

24.4. patch merge semantics

RFC 7386 merge-patch gives updates three-way semantics a typed record cannot express: an absent member leaves the field alone, null deletes it, a value replaces it. So the patch decodes into a member map of JsonElement values, not a dto, and the handler walks the members in sorted order so details are deterministic:

```
        var details = new List<string>();
        var name = current.DisplayName;
        foreach (var key in patch.Keys.Order())
        {
            var value = patch[key];
            switch (key)
            {
                case "display_name":
                    if (value.ValueKind == JsonValueKind.Null)
                    {
                        details.Add("display_name cannot be removed");
                    }
                    else if (value.ValueKind != JsonValueKind.String)
                    {
                        details.Add("display_name must be a string");
                    }
                    else
                    {
                        var candidate = value.GetString()!;
                        details.AddRange(UserValidation.ValidateDisplayName(candidate));
                        name = candidate;
                    }

                    break;
                case "id" or "email" or "created_at" or "version" or "roles":
```

listing 216: the member walk: immutable members refuse, null on the display name too, the rest validate (source: csharp-net/api/src/CsharpBook.Api/Users/UserEndpoints.cs, lines 149-173)

The contract makes one member writable, the display name, and the switch shows the full policy in one expression: null answers 422, because deleting the display name would leave the record invalid, a non-string answers 422, and a string runs the same range rules registration runs. The immutable members, id, email, created_at, version, roles, refuse the moment they appear, null or not: patching them is not a validation typo, it is an attempt to write a field the contract closed. Unknown members refuse too, the same strictness the typed register decode enforced with an attribute, enforced here by the member walk because a dictionary cannot carry that attribute.

After the walk, the precondition and the write run in order. The client's If-Match must equal the etag of the current body, else 412, and the update carries the expected version so a write that lost a race answers 412 as well rather than silently clobbering. The no-op patch, an empty object, is legal and answers 200 with the current representation, nothing bumped.

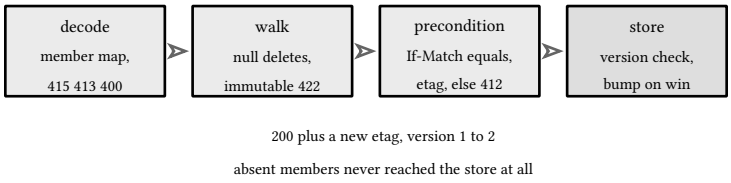


figure 153: a patch request walks the members, then the precondition, then the store

24.5. keyset pagination

Offset pagination asks for row 86041 onward, and it fails twice. The database still walks and discards every skipped row, so page 4000 costs what a table scan costs, and offsets are positions in a moving file: one registration between requests and a row appears twice or vanishes. Keyset pagination fixes both by remembering where it stopped rather than counting from the front. The order is the pair (created_at, id), the cursor encodes the last row's position, and the next page is everything strictly after it:

```
    public static string EncodeCursor(CursorKey key)
    {
        var payload = JsonSerializer.SerializeToUtf8Bytes(new CursorJson(key.T, key.I));
        return Base64Url.EncodeToString(payload);
    }

    public static bool TryDecodeCursor(string raw, out CursorKey key, [NotNullWhen(false)] out string?
error)
    {
        key = default;
        Span<byte> buffer = stackalloc byte[MaxCursorBytes];
        if (raw.Length > MaxCursorChars ||
            Base64Url.DecodeFromChars(raw, buffer, out _, out var written) != OperationStatus.Done)
        {
            error = "cursor is not base64url";
```

listing 217: the cursor: created_at milliseconds plus the id tiebreaker, base64url unpadded (source: csharp-net/api/src/CsharpBook.Api/Users/Pagination.cs, lines 52-65)

The payload is deliberately tiny, two members, because the cursor is a position, not a page number. Time is milliseconds since the epoch, the id breaks ties when two users share a creation millisecond, and uuid v7 ids sort by their own embedded time, so the composite order is total. The encoding rides the BCL's Base64Url type, unpadded by construction: opaque to clients, trivially decodable in a test, and the frozen vectors carry its exact bytes. A cursor the service never issued is garbage, not a missing page, so decode failures answer 422, and the limit must be an integer in 1..100, 422 otherwise, default 20. The store query asks for one row over the limit, so the has-more signal comes from data the page returned.

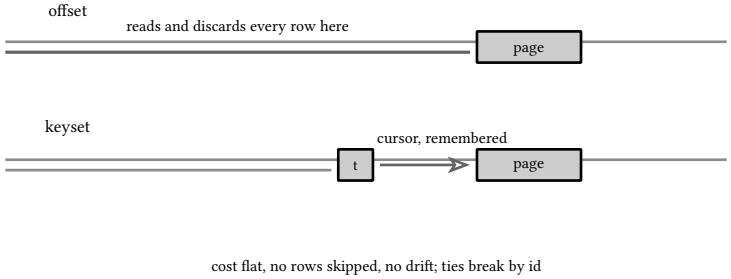


figure 154: offset walks the whole prefix, keyset jumps to the remembered position

24.6. the status contract

Every failure this layer can produce already has a code in the chapter 22 table, so the resource layer states outcomes and the table states statuses. The get handler is the whole ladder in one screen: 401 without an actor, 404 for an unknown id, 403 for a stranger on someone else's record, then the etag work:

```
    private static async Task GetAsync(UserFamily family, HttpContext http, string id, CancellationToken
ct)
    {
        var actor = ActorOr401(http);
        var user = await GetUserOr404Async(family, http, id, ct);
        RequireOwner(actor, user.Id);

        var bytes = user.ToWireBytes();
        var etag = UserWire.ETagBody(bytes);
        if (http.Request.Headers.IfNoneMatch.ToString() == etag)
        {
            http.Response.StatusCode = StatusCodes.Status304NotModified;
            http.Response.Headers.ETag = etag;
            return;
        }

        await WriteBodyAsync(http.Response, StatusCodes.Status200OK, bytes, ct, etag: etag);
    }
```

listing 218: the failure ladder before any body is written (source: csharp-net/api/src/CsharpBook.Api/Users/UserEndpoints.cs, lines 82-98)

After the ladder comes the one status that is not a failure, 304. If the client's If-None-Match equals the computed etag, the body it already holds is current, and the service answers the header and nothing else. Each failure is one thrown ApiError the kernel's exception middleware renders as the one envelope, so no handler in this chapter writes a failure body itself.

The order inside the ladder is a policy choice this chapter owns: the 404 answers before the 403, so an unknown id and a stranger's id are distinguishable. APIs that prefer to hide existence answer 404 for both, and the authz chapter walks that tradeoff. No route here invents a status or an envelope shape.

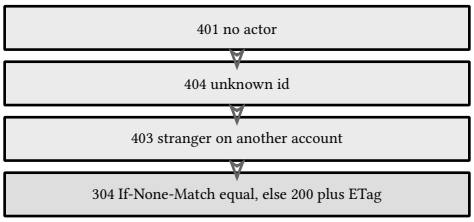


figure 155: one request climbs the ladder, each rung one contract code

24.7. golden handler tests

The frozen vectors under `contract/testdata` are the fixture source of truth, and the tests read them as data: ids, offsets, expected etag strings, expected cursor bytes all come from the json files, so a test never re-derives a constant the contract already froze. The walk test shows the shape: seed three users at the vector's offsets, ask for a page as a non-admin, and demand both pages against the vector bodies:

```
        await SeedListWalkAsync();

        var first = await SendAsync(_harness.Client, v.Steps[0].Request);
        Assert.Equal(200, (int)first.StatusCode);
        AssertJsonEquals(await BodyTextAsync(first), v.Steps[0].Response.Body!);

        var second = await SendAsync(_harness.Client, v.Steps[1].Request);
        Assert.Equal(200, (int)second.StatusCode);
        AssertJsonEquals(await BodyTextAsync(second), v.Steps[1].Response.Body!);
```

listing 219: the vector walk: both pages judged against the frozen bodies (source: csharp-net/api/tests/CsharpBook.Api.Tests/Users/UsersApiVectorTests.cs, lines 185-193)

Exactness is the point. The etag assertion proves the body bytes byte for byte, because the etag is the sha256 of those bytes, and the cursor assertion proves the encoding down to the missing padding. The harness around it injects everything time-dependent: a stepped TimeProvider, a scripted id source handing out the vector's uuids, a scripted request-id source the middleware stamps into every envelope, and a fake hasher standing in for the pbkdf2 the authn chapter brings. The whole service runs over TestServer, the framework's in-memory host, so nothing sleeps, nothing dials a socket, and every response comes out of the same pipeline production traffic takes.

The same discipline covers the rest of the slice: register 201 and 409, the 422 details array, unknown and immutable patch members, the stale If-Match 412, the 25-user default-limit walk, and the ownership grid. The register replay vector, 02, rides the idempotency guard the concurrency chapter ships: the register route buffers the keyed body, the guard stores the 201 snapshot, and the retry answers those exact bytes again, Location and etag derived back from the body because the etag is a function of the body. The same key under a different body answers 422, contract misuse. When a vector changes the code is wrong, and when the code changes an output byte a vector fails.

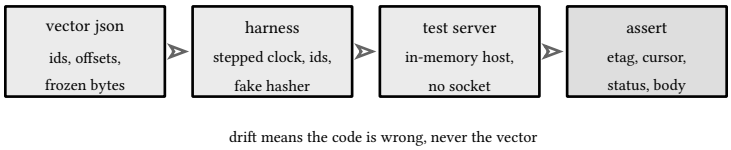


figure 156: a golden test: the vector file drives the fixture and judges the harness

sources: RFC 7386, JSON Merge Patch, and RFC 9110, HTTP Semantics sections 8.8.3 and 13.1.1 on etags and conditional requests, both at rfc-editor.org. Microsoft Learn, verified 2026-09-26: learn.microsoft.com/dotnet/api/system.text.json.serialization.jsonunmappedmemberhandling for Disallow semantics, learn.microsoft.com/dotnet/standard/serialization/system-text-json/missing-members for the attribute placement, learn.microsoft.com/dotnet/api/system.buffers.text.base64url for the base64url codec, and learn.microsoft.com/dotnet/api/system.security.cryptography.sha256.hashdata plus learn.microsoft.com/dotnet/api/system.convert.tohexstringlower for the etag digest rendering. The keyset-versus-offset argument follows use-the-index-luke.com on offset paging. Verified by `books/csharp-net/api` tests under `dotnet test`, the users and authn suites green.

25. authn

Authentication answers one question, who is calling, and this chapter builds every piece of the answer: pbkdf2 password storage with parameters carried in the hash string, a login path flattened so unknown emails and wrong passwords cost the same, server-side sessions behind a store port, the cookie that carries the refresh token, HS256 and ES256 json web tokens built from cryptographic primitives rather than a library, rotation with reuse detection on the refresh family, and the injected-clock test discipline that keeps all of it fast.

25.1. password storage with pbkdf2

Passwords are never stored, only proofs that cost an attacker real time to reproduce. The BCL's answer is Rfc2898DeriveBytes.Pbkdf2 over HMAC-SHA256, and the parameter set travels inside the encoded string, so verification replays the exact parameters the hash was written with, never today's defaults:

```
    public string Hash(string password)
    {
        var salt = RandomNumberGenerator.GetBytes(SaltSize);
        var key = Rfc2898DeriveBytes.Pbkdf2(
            password, salt, _iterations, HashAlgorithmName.SHA256, KeySize);
        return "$pbkdf2-sha256"
            + "$i=" + _iterations.ToString(System.Globalization.CultureInfo.InvariantCulture)
            + ",l=" + KeySize.ToString(System.Globalization.CultureInfo.InvariantCulture)
            + "$" + Convert.ToBase64String(salt)
            + "$" + Convert.ToBase64String(key);
    }
```

listing 220: a fresh salt, one derive call, one PHC-style string (source: csharp-net/api/src/CsharpBook.Api/Authn/Pbkdf2Hasher.cs, lines 48-59)

The issue tier is 210000 iterations, 16 salt bytes, a 32 byte key, the values the frozen contract pins for this lane. The OWASP cheat sheet read the same day recommends 600000 iterations for pbkdf2-hmac-sha256, and the lane sits below it on purpose: the tier is frozen cross-lane vector bytes, and the honesty lives in the PHC string carrying its own parameters, not in chasing a guidance table. The test tier is a separate named constant, 2000 iterations, a confession that keeps every hash in the suite instant while exercising the identical code path, and a structural test pins the production floor without ever running it, so the tier cannot silently decay. Verification parses the string, re-derives under the carried parameters, and compares in constant time, and malformed strings answer false without throwing, so a stolen store row cannot force absurd work through crafted parameters: the parser caps both iteration count and key length.

The memory-hard alternative deserves its name even though the BCL does not ship it. Argon2id is a schedule over a primitive, not a new primitive: each lane fills its strip of a memory array by compressing the previous block and a referenced block from an earlier lane, and each pass re-reads and re-compresses the whole array in a different order. The memory hardness is the schedule's property, an attacker who shrinks the array cannot answer the compression chain, and the parameter string is the same PHC shape, dollar argon2id v 19 m t p salt hash. The platform gap is real, the BCL ships no argon2, so the c# community reaches for a third-party package while this vehicle stays on the platform's pbkdf2 and states the maintenance tradeoff the suite chapter returns to.

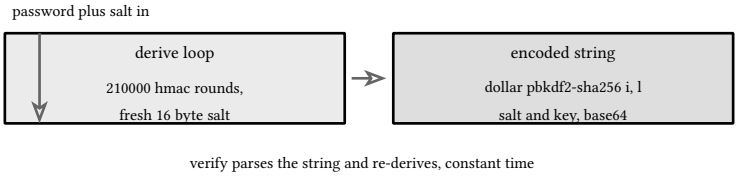


figure 157: pbkdf2: one hmac chain, parameters riding in the string

25.2. login and timing flattening

A login endpoint that answers fast for unknown emails and slowly for wrong passwords hands out an account census to anyone with a stopwatch. The flattening fix sits in the unknown-email branch: run the same pbkdf2 verification the wrong-password path runs, against a dummy hash computed once at service construction, discard the result, and answer the identical bytes:

```
    private static async Task LoginAsync(AuthnService service, HttpRequest request, CancellationToken ct)
    {
        var body = await request.BindAsync<LoginRequest>(ct);
        if (string.IsNullOrEmpty(body.Email) || string.IsNullOrEmpty(body.Password))
        {
            throw ApiError.Fail(
                ErrorCode.ValidationFailed, "validation failed", "email is required", "password is
required");
        }

        var family = service.Family;
        var found = await family.Users.GetByEmailAsync(UserValidation.NormalizeEmail(body.Email), ct);
        if (found.Outcome == UserStoreOutcome.NotFound)
        {
            // Unknown email: verify against the dummy hash so the work,
            // and therefore the latency profile, matches the wrong
            // password path, then answer the identical bytes.
            service.VerifyDummy(body.Password);
            throw ApiError.Fail(ErrorCode.Unauthorized, "invalid credentials");
        }

        if (!service.VerifyPassword(found.Value!.PasswordHash, body.Password))
        {
            throw ApiError.Fail(ErrorCode.Unauthorized, "invalid credentials");
        }
```

listing 221: both failure branches answer the same 401 after the same work (source: csharp-net/api/src/CsharpBook.Api/Authn/AuthnEndpoints.cs, lines 47-70)

Both branches return the same envelope, code, message, and no distinguishing header, and the counting hasher in the tests proves the dummy verify ran on the unknown-email path: two verifies for two failed logins, one real, one dummy, no matter which branch each request took. The success path runs on through Issue, which mints the session row, the refresh token, the cookie, and the access token in one place, so login stays readable as two facts: prove, then issue.

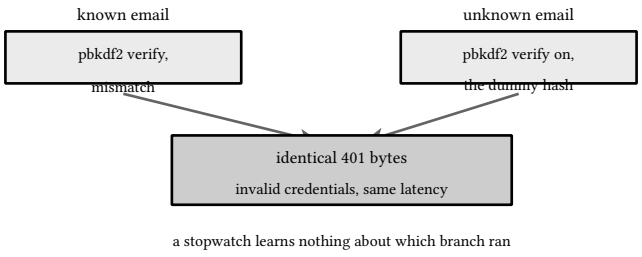


figure 158: the timing-equalized login: unknown email pays what a wrong password pays

25.3. server-side sessions

A session is the server's own record of one login: an id, the user, the expiry, a revoked flag, and the sha256 of the current refresh token. The token itself is never stored, only its hash, so a stolen store dump cannot be

replayed against the live service. The store port mirrors the user store's: an interface frozen here, the in-memory implementation in this chapter, the sqlite implementation the store chapter already ships against the same seam:

```
public interface ISessionStore
{
    Task CreateAsync(Session session, CancellationToken ct = default);

    Task<SessionStoreResult> GetAsync(string id, CancellationToken ct = default);

    Task<SessionStoreResult> GetByRefreshAsync(byte[] refreshHash, CancellationToken ct = default);

    Task<SessionStoreResult> RotateAsync(
        byte[] current, byte[] next, DateTimeOffset expiresAt, CancellationToken ct = default);

    /// <summary>Revoke kills a session and its whole refresh family.
    /// Revoking an unknown session is a no-op.</summary>
    Task RevokeAsync(string id, CancellationToken ct = default);
}
```

listing 222: the session store port: five verbs, four outcomes (source: csharp-net/api/src/CsharpBook.Api/Authn/ISessionStore.cs, lines 37-51)

The lifetime rules stay with the caller, on purpose. The store knows structural state, which token is current, which session is revoked; the service reads the clock and applies the policy, a session counts as alive while not revoked and not expired. That split keeps the engine implementation free of clock reads and makes every expiry test pure arithmetic on an injected clock.

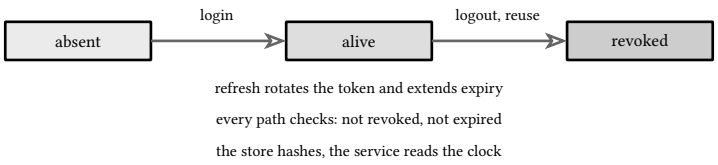


figure 159: the session lifecycle: login creates, refresh extends, logout or reuse revokes

25.4. cookie mechanics

The browser carries the refresh token in a cookie, and the cookie's attributes are its entire protection. Path scopes it to the api tree so it never rides on requests to other origins hosted beside the service. HttpOnly keeps scripts from reading it, so an injected snippet cannot exfiltrate what it cannot see. Secure keeps it off plain http. SameSite Lax blocks the cookie on cross-site posts while allowing ordinary top-level navigation. The line is rendered by hand, not through the framework's cookie writer, because the attributes are the contract and the frozen vector freezes their order and spacing:

```
    public static void Set(HttpResponse response, string token) =>
        response.Headers.Append(HeaderNames.SetCookie, $"{Name}={token}; Path=/api; HttpOnly; Secure;
SameSite=Lax");

    /// <summary>
    /// Clear expires the cookie: Max-Age=0 rendered exactly, which
    /// every browser treats as delete. The attributes stay set so the
    /// deletion overwrites the exact cookie it replaces.
    /// </summary>
    public static void Clear(HttpResponse response) =>
        response.Headers.Append(
            HeaderNames.SetCookie,
            $"{Name}=; Path=/api; Max-Age=0; HttpOnly; Secure; SameSite=Lax");
}
```

listing 223: set and clear: four attributes, one exact line each (source: csharp-net/api/src/CsharpBook.Api/Authn/SessionCookie.cs, lines 19-31)

Logout clears it with Max-Age 0 rendered exactly, and the attributes stay set so the deletion overwrites the exact cookie it replaces. The tests assert the whole Set-Cookie line byte for byte against the frozen vectors, attributes, order, and semicolons included.

attribute	stops	scope
Path /api	leak on sibling paths	one tree
HttpOnly	script exfiltration	read denied
Secure	plaintext transit	https only
SameSite Lax	cross-site post	top nav only

four attributes, four attacks, zero javascript visible

figure 160: the cookie attribute matrix: what each one stops

25.5. jwt from scratch with hmac

A json web token is three base64url parts, header, claims, signature, and the whole format fits in two functions of cryptographic primitives. HMAC-SHA256 signs the first two segments with a shared secret, verification recomputes and compares in constant time, and the claim rules run after the signature, never before:

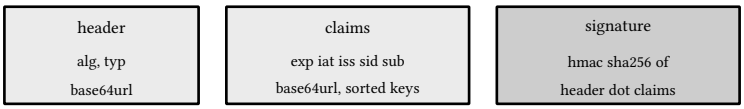
```
    public static string SignHS256(byte[] secret, AccessClaims claims)
    {
        var (header, payload) = Segments(AlgHS256, claims);
        var signature = HMACSHA256.HashData(secret, Encoding.ASCII.GetBytes(header + "." + payload));
        return header + "." + payload + "." + Base64Url.EncodeToString(signature);
    }

    public static JwtResult ReadHS256(string token, byte[] secret, DateTimeOffset now) =>
        Read(token, AlgHS256, (message, signature) =>
            CryptographicOperations.FixedTimeEquals(
                HMACSHA256.HashData(secret, message), signature), now);
```

listing 224: sign and verify HS256, no library, two primitives (source: csharp-net/api/src/CsharpBook.Api/Authn/Jwt.cs, lines 28-38)

The claims record is small and alphabetical by declaration, exp, iat, iss, sid, sub, which makes the serialized form deterministic, the same bytes for the same facts, so the golden test reproduces the frozen vector's token exactly, signature included: the login test at plus ten seconds emits the vector's access token byte for byte under the fixed test secret. There is no roles claim on purpose: authorization reads the store, and the sid claim makes every outstanding token revocable by killing one session row. The parser refuses what the classics try: an alg none

header never reaches the payload, the alg pin runs before any payload byte is trusted, a tampered claim fails the constant-time compare, and an expired or foreign-iss token fails the claim checks with its own failure codes.



dots included, padding stripped, alg pinned before payload reads

figure 161: three segments, the signature covers exactly the first two

25.6. the asymmetric token over ecdsa

HMAC has one secret and both sides hold it, so whoever can verify tokens can also mint them. In one process that is fine, and it is why the issued lane stays HS256: one signer, one verifier, one binary. The moment verification spreads, a public-key scheme earns its complexity, and the platform's asymmetric lane here is ECDsa: the private key signs, the public key verifies, and the verification side provably cannot forge:

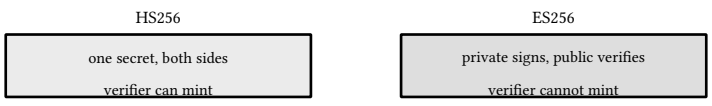
```
    /// <summary>
    /// SignEs256 signs with the ECDsa private key: verifiers hold only
    /// the public key, so the party checking tokens can never mint
    /// them.
    /// </summary>
    public static string SignEs256(ECDsa key, AccessClaims claims)
    {
        var (header, payload) = Segments(AlgES256, claims);
        var signature = key.SignData(Encoding.ASCII.GetBytes(header + "." + payload),
HashAlgorithmName.SHA256);
        return header + "." + payload + "." + Base64Url.EncodeToString(signature);
    }

    public static JwtResult ReadEs256(string token, ECDsa key, DateTimeOffset now) =>
        Read(token, AlgES256, (message, signature) =>
            key.VerifyData(message, signature, HashAlgorithmName.SHA256), now);
```

listing 225: the same envelope, an ES256 signature over a split key (source: csharp-net/api/src/CsharpBook.Api/Authn/Jwt.cs, lines 40-54)

The two readers cross-check each other in the tests: an HS256 reader refuses the ES256 token on the alg pin, the ES256 reader refuses the HS256 token the same way, so the algorithm confusion attack, present a token signed with the algorithm the attacker prefers, dies at the header. The claim checks are shared, so issuer, expiry, and the constant-time habits do not fork between algorithms.

EdDSA is the construction to reach for where the platform provides it, and it decomposes the same way: Ed25519 is a Schnorr signature over the twisted Edwards curve25519 with SHA-512, and the nonce is derived deterministically from the key's hash prefix and the message, so signing never reads a random generator. The gap that rules this lane: Windows cryptography has no EdDSA at all, verified against the platform docs, so the BCL cannot offer it here, and ES256 over a NIST curve is the platform's answer. The lesson stands either way: the envelope, the pin, and the claim rules are the design, the curve is a parameter.



one binary, one signer: the issued lane stays HS256

split the process, split the key: verification scales alone

figure 162: shared secret versus split key: who can mint, who can check

25.7. the refresh rotation and reuse detection

Access tokens live fifteen minutes, the refresh token is the long-lived credential, and it rotates on every use. Rotation means the store remembers the token it just retired, marked superseded, because presenting a superseded token is the strongest theft signal the design gets: the legitimate holder moved on, so someone else is replaying. The answer is family revocation, the replayed token and its successor die together:

```
    public async Task RefreshAsync(HttpContext http, string token, CancellationToken ct)
    {
        var now = family.Clock.GetUtcNow();
        var current = Session.HashToken(token);
        var found = await family.Sessions.GetByRefreshAsync(current, ct);
        if (found.Value is { Revoked: false, ExpiresAt: var expires } && now ≥ expires)
        {
            throw ApiError.Fail(ErrorCode.Unauthorized, "invalid credentials");
        }

        var next = family.NewRefreshToken();
        var rotated = await family.Sessions.RotateAsync(current, Session.HashToken(next), now + SessionTtl,
ct);
        if (rotated.Outcome == SessionStoreOutcome.UnknownRefresh)
        {
            throw ApiError.Fail(ErrorCode.Unauthorized, "invalid credentials");
        }

        if (rotated.Outcome == SessionStoreOutcome.SessionRevoked)
        {
            throw ApiError.Fail(ErrorCode.Unauthorized, "session revoked");
        }
```

listing 226: refresh: expired sessions stay dead, rotation revokes on reuse (source: csharp-net/api/src/CsharpBook.Api/Authn/AuthnService.cs, lines 106-126)

The vector pair pins the whole story. Refresh at plus twenty returns a new cookie and a new access token. Replay the old cookie at plus twenty-one and both die: the replay answers 401 session revoked, and the successor, still perfectly formed, answers 401 session revoked one second later. An attacker who stole the token before the holder refreshed gets one use, then loses the family along with the victim. An expired but never-rotated session does not come back to life either: the expiry check runs before the rotation attempt and answers invalid credentials.

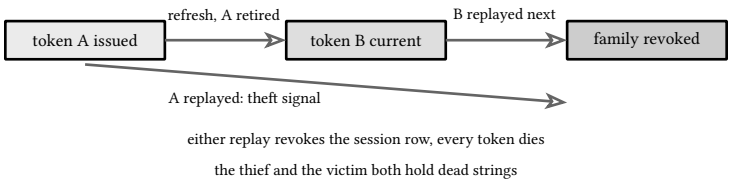


figure 163: the refresh family: rotation forks, reuse revokes everything

25.8. testing auth without walls

Every time-dependent behavior here is tested as math. The clock is a stepped TimeProvider, so token expiry is one addition. The id source hands out the vector's uuids, so the sid claim lands exact. The refresh token source is scripted, so the cookie line and the access token reproduce the frozen vector byte for byte, and a counting hasher proves the dummy verify ran on the unknown-email path:

```
    public async Task Login200ReplaysTheVectorDownToTheSignatureBytes()
    {
        var v = Vectors.Load("04-login-200");
        _harness = await ApiHarness.StartAsync(
            T0.AddSeconds(10), ids: [AdaSid], refreshTokens: [TokenA], withAuthn: true);
        await SeedAdaAsync(_harness);

        var response = await SendAsync(_harness.Client, v.First.Request);
        Assert.Equal(200, (int)response.StatusCode);
        Assert.Equal(v.First.Response.Headers["Set-Cookie"], response.Headers.GetValues("Set-
Cookie").Single());
        Assert.Equal(v.First.Response.Headers["X-RateLimit-Limit"], response.Headers.GetValues("X-
RateLimit-Limit").Single());
        Assert.Equal(v.First.Response.Headers["X-RateLimit-Remaining"], response.Headers.GetValues("X-
RateLimit-Remaining").Single());
        Assert.Equal(v.First.Response.Headers["X-RateLimit-Reset"], response.Headers.GetValues("X-
RateLimit-Reset").Single());
        AssertJsonEquals(await response.Content.ReadAsStringAsync(), v.First.Response.Body!);
```

listing 227: the vector login: cookie line, rate headers, and token body all exact (source: csharp-net/api/tests/CsharpBook.Api.Tests/Authn/AuthnApiVectorTests.cs, lines 99-112)

No test sleeps, no test opens a socket, and the whole service runs over TestServer, the framework's in-memory host, the same pipeline production traffic takes. The session store tests rotate one token from eight concurrent callers and assert exactly one winner, the token reader is probed with garbage, alg none, tampered payloads, and foreign issuers, and the two algorithm readers refuse each other's tokens. When the vectors change the code is wrong, and nothing in this chapter has a second source of truth to drift toward.

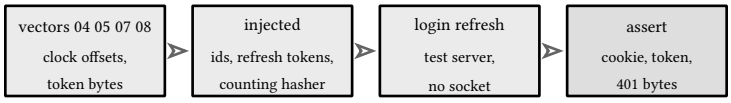


figure 164: an auth golden test: fixed inputs in, frozen bytes out

sources: RFC 8018 for PBKDF2, RFC 7519 for jwt structure, RFC 8032 for EdDSA, all at rfc-editor.org. OWASP Password Storage Cheat Sheet at cheatsheetseries.owasp.com for the pbkdf2 parameter guidance; the lane pins the contract's frozen 210000 iteration tier and states the cheat sheet's current interactive table separately. Microsoft Learn, verified 2026-09-26: learn.microsoft.com/dotnet/api/system.security.cryptography.rfc2898derivebytes.pbkdf2 for the static derive call this chapter uses, learn.microsoft.com/dotnet/api/system.security.cryptography.cryptographicoperations.fixedtimeequals for the constant-time compare, learn.microsoft.com/dotnet/api/system.security.cryptography.hmacsha256.hashdata, learn.microsoft.com/dotnet/api/system.security.cryptography.ecdsa for the ES256 lane, and learn.microsoft.com/dotnet/core/compatibility/cryptography/11/compositemldsa-windows-native for the Windows EdDSA gap, the cross-platform page now covering the post-quantum tables instead. Verified by `books/csharp-net/api` tests under `dotnet test`, the authn suite green through the frozen vectors 04, 05, 07, and 08.

26. authz and rbac

Authentication settled who is calling; authorization decides what that caller may do, and this chapter builds the whole answer: the distinction between the two questions, roles as stored data with a grant and revoke surface, policies as plain delegates, the 401 versus 403 discipline, object-level authorization on the routes that carry ids, a deny-by-default guard with a route audit that keeps every pattern honest, and the role by endpoint matrix test that pins the whole grid in one table.

26.1. authentication versus authorization

Two questions, two layers, two failure codes. Authentication runs first and installs an Actor or does not; authorization reads that actor against the route and the object. The service's wiring order makes the sequence physical: the identity middleware sits outside the guard, the guard sits after routing, so by the time a policy runs, the actor question is settled and a policy never parses a token, checks a signature, or reads a cookie:

```
    public static ApiError? Open(HttpContext context, string pattern) => null;

    /// <summary>Authenticated: a resolved actor and nothing more,
    /// because the route holds no object.</summary>
    public static ApiError? Authenticated(HttpContext context, string pattern) =>
        context.GetActor() is null
            ? ApiError.Fail(ErrorCode.Unauthorized, "authentication required")
            : null;
```

listing 228: the two cheapest policies: open, and any-actor (source: csharp-net/api/src/CsharpBook.Api/Authz/RoutePolicy.cs, lines 25-32)

Conflating the two questions produces the classic broken answer, a 403 for an anonymous request. A caller that never proved anything has no authorization to deny, only an authentication demand, and the distinction is observable: 401 means try again with credentials, 403 means do not bother, this identity will never pass. Every layer in this chapter keeps that boundary, and the wiring order enforces it before any handler runs.

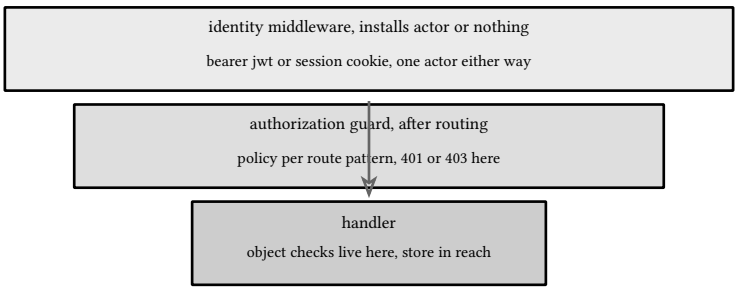


figure 165: two layers in wiring order: identity outside, guard inside, handler last

26.2. roles and the grant table

Roles are rows, not magic. Each account carries a set, the bootstrap rule grants the first registration admin, and the admin routes add and remove one role at a time, answering the resulting set. The invariant that matters is never zero admins, and it lives in the store where both mutating verbs run it, grant and revoke alike:

```
    private UserStoreResult PersistRoles(User current, List<string> roles)
    {
        var candidate = current with { Roles = roles };
        if (!AdminsRemain(candidate))
        {
            return UserStoreResult.Fail(UserStoreOutcome.LastAdmin);
        }

        _byId[current.Id] = candidate;
        return UserStoreResult.Ok((IReadOnlyList<string>)[.. roles]);
```

listing 229: the candidate role set persists only when an admin remains (source: csharp-net/api/src/CsharpBook.Api/Users/MemUserStore.cs, lines 161-170)

A revoke computes the post state, checks that some admin still stands across the whole table, and only then persists, so the rejected demotion leaves no trace: the scan substitutes the candidate for the stored record before it looks. The frozen vector walks the story: demoting the only admin answers 409 conflict, last admin, granting admin to a second user succeeds, and revoking that second admin then succeeds too, because the first one is standing again. The bootstrap rule makes the lockout unreachable in practice, the invariant makes it unreachable in theory.

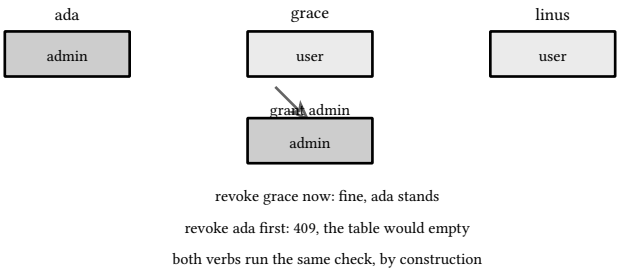


figure 166: the grant table: one column per user, the admin row must never empty

26.3. policies as functions

A policy is one delegate from request context plus pattern to verdict: null allows, an ApiError denies. The whole surface composes from four of them, open, any-actor, a required role, an owner or admin check, and each is short enough to read in one breath:

```
    public static RoutePolicy RequireOwner(string param) => (context, pattern) =>
    {
        var actor = context.GetActor();
        if (actor is null)
        {
            return ApiError.Fail(ErrorCode.Unauthorized, "authentication required");
        }

        var id = context.Request.RouteValues[param]?.ToString();
        return id is null || (actor.UserId == id && !actor.Admin)
            ? ApiError.Fail(ErrorCode.Forbidden, "not the owner")
            : null;
    };
```

listing 230: the object policy: own the id or hold admin (source: csharp-net/api/src/CsharpBook.Api/Authz/RoutePolicy.cs, lines 57-69)

The pattern argument rides along for the audit keys. Where the go lane had to walk path and pattern in lockstep, because its guard ran before the mux filled path values, the framework here fills Request.RouteValues at dispatch and the guard runs after routing, so the policy reads the id straight from the request. Plain delegates beat a rules

dsl or attribute mining at this size because the compiler type-checks the table, the audit test can walk it, and a new rule is a function next to its three siblings.

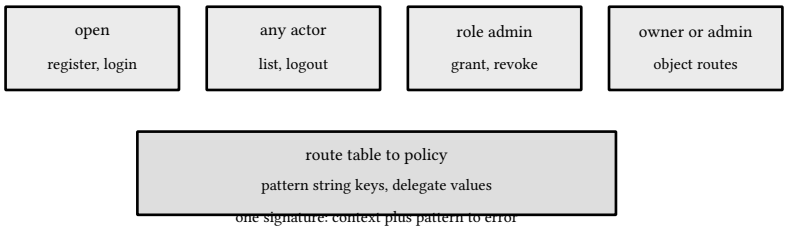


figure 167: four policies, one signature, the table maps routes onto them

26.4. 401 versus 403

The boundary lives in one place per policy: the actor check answers 401, the rule check answers 403, and nothing in between exists:

```
    /// an authentication failure, a present actor without the role is
    /// an authorization failure.</summary>
    public static RoutePolicy RequireRole(string role) => (context, pattern) =>
    {
        var actor = context.GetActor();
        if (actor is null)
        {
            return ApiError.Fail(ErrorCode.Unauthorized, "authentication required");
        }

        return actor.HasRole(role)
            ? null
            : ApiError.Fail(ErrorCode.Forbidden, "insufficient role");
    };
```

listing 231: the discipline: no actor is 401, wrong actor is 403 (source: csharp-net/api/src/CsharpBook.Api/Authz/RoutePolicy.cs, lines 35-49)

Read it as a state machine. Anonymous gets 401 with the same message every time, authentication required, no hint about the route's existence or its rules. A present actor that fails the rule gets 403 with the rule's own words, insufficient role or not the owner, which is a statement about the actor, never about the resource. The one deliberate blur is existential: this service answers 404 for an unknown id before 403 for a stranger's id, which reveals that an id exists to anyone authenticated. Hiding it instead means answering 404 for both, and the tradeoff is real, disclosure against debuggability. What is never acceptable is mixing the codes by accident, a 403 for a missing Authorization header, a 401 for a demoted admin.

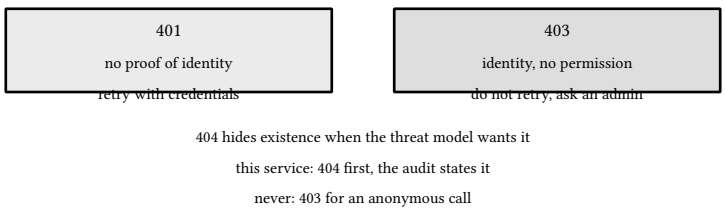


figure 168: the two failure codes, and the one deliberate blur

26.5. object-level authorization

Route rules cannot guard a route whose danger lives in the path parameter. GET on someone else's user id is the IDOR class, insecure direct object reference, and it survives every route-level check because the route is the same for everyone. The check must happen after the fetch, where the code knows whether the object exists and who

owns it, which is why the object routes carry only the any-actor policy in the table and the ownership rule runs in the handler:

```
    internal static Actor ActorOr401(HttpContext http) =>
        http.GetActor() ?? throw ApiError.Fail(ErrorCode.Unauthorized, "authentication required");

    internal static void RequireOwner(Actor actor, string id)
    {
        if (actor.UserId ≠ id && !actor.Admin)
        {
            throw ApiError.Fail(ErrorCode.Forbidden, "not the owner");
        }
    }
```

listing 232: the ownership rule the object handlers run after the fetch (source: csharp-net/api/src/CsharpBook.Api/Users/UserEndpoints.cs, lines 224-233)

The test pins both edges. Linus touching ada's id gets 403 on read, write, and delete, whatever etag he presents, and an unknown id gets 404 for everyone, the ordering the last section discussed. Putting the owner check in the guard instead would flatten unknown and forbidden into one 403, leak existence, and duplicate the store read the handler needs anyway, so the layering is not style, it is the only arrangement that answers all three questions correctly.

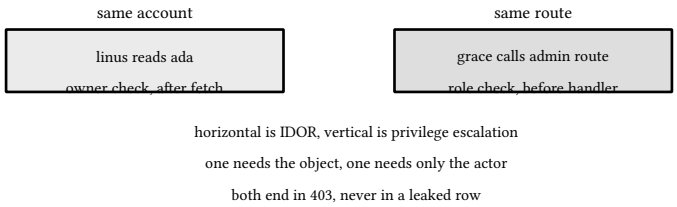


figure 169: horizontal versus vertical: two axes of privilege, two different checks

26.6. deny by default with a route audit

One table maps every registered pattern to its policy. A guard middleware sits in the pipeline after routing, reads the matched endpoint's method and raw pattern, and refuses any pattern missing from the table before a handler byte runs:

```
    public static IApplicationBuilder UseAuthzGuard(
        this IApplicationBuilder app,
        IReadOnlyDictionary<string, RoutePolicy>? table = null)
    {
        var policies = table ?? RoutePolicies.Table;
        return app.Use(async (context, next) =>
        {
            // platform sentinels are not route endpoints: the 405 the
            // matcher selects on a method miss carries no pattern, and
            // it must reach the kernel's fallback, not a policy verdict
            if (context.GetEndpoint() is RouteEndpoint endpoint)
            {
                var verdict = Verdict(endpoint, context, policies);
                if (verdict is not null)
                {
                    throw verdict;
                }
            }

            await next(context);
        });
```

listing 233: the guard: no table row, no response (source: csharp-net/api/src/CsharpBook.Api/Authz/AuthzGuard.cs, lines 17-37)

The audit test closes the loop from the other side: every table row must resolve to a registered route with exactly that pattern, and every registered api route must carry a table row, so a route cannot exist unaudited, a row cannot point at nothing, and a new route that forgets its policy becomes instantly, loudly unreachable instead of quietly open. The unaudited-route branch has its own test: a pipeline whose table lost a row answers 403, no policy for route, the exact message the deny-by-default design promises. Requests that match no pattern at all fall through to the routing layer's own answers, 404 for an unknown path and 405 for a known path with the wrong method, because both are routing answers, not authorization verdicts.

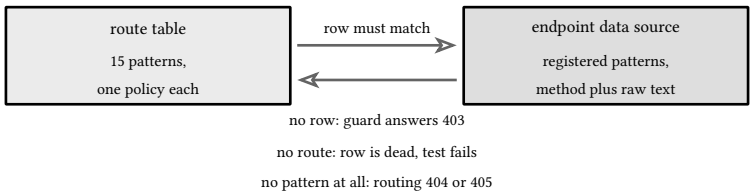


figure 170: the audit: table and routes face each other, both directions checked

26.7. the rbac matrix test

The grid is the proof: every route times every actor kind, anonymous, regular user, admin, stranger on someone else's object, one expected status per cell, asserted in one table-driven test through the full guarded stack, the same path production traffic takes:

```
        var cases = new (string Name, string Actor, string Method, string Path, string? Body,
            IReadOnlyDictionary<string, string>? Headers, HttpStatusCode Want)[]
        {
            ("anon list", "", "GET", "/api/users", null, null, HttpStatusCode.Unauthorized),
            ("user list", GraceId, "GET", "/api/users", null, null, HttpStatusCode.OK),
            ("admin list", AdaId, "GET", "/api/users", null, null, HttpStatusCode.OK),
            ("anon get", "", "GET", gracePath, null, null, HttpStatusCode.Unauthorized),
            ("stranger get", LinusId, "GET", gracePath, null, null, HttpStatusCode.Forbidden),
            ("owner get", GraceId, "GET", gracePath, null, null, HttpStatusCode.OK),
            ("admin get", AdaId, "GET", gracePath, null, null, HttpStatusCode.OK),
            ("stranger patch", LinusId, "PATCH", gracePath, """{"display_name":"x"}""",
                new Dictionary<string, string> { ["If-Match"] = etag }, HttpStatusCode.Forbidden),
            ("owner patch missing if-match", GraceId, "PATCH", gracePath, """{"display_name":"grace m"}""",
                null, HttpStatusCode.PreconditionFailed),
```

listing 234: the matrix rows: actor times route, one status each (source: csharp-net/api/tests/CsharpBook.Api.Tests/Authz/AuthzApiTests.cs, lines 211-224)

Each row is a cell with a name that fails loudly. The grid runs the vector's last-admin chain alongside, so the 409 and the two 200s around it are pinned by frozen bodies, not just by intent. The grant step's vector file carries the role in its path while the frozen endpoint list puts it in the body, an artifact the go replay normalized the same way this lane does: drive the contract path, judge the vector's responses. Adding a route costs three lines, one table row per interesting actor, and the audit plus the matrix together answer the two questions reviewers actually ask: is every route covered, and does every covered route behave for every kind of caller.

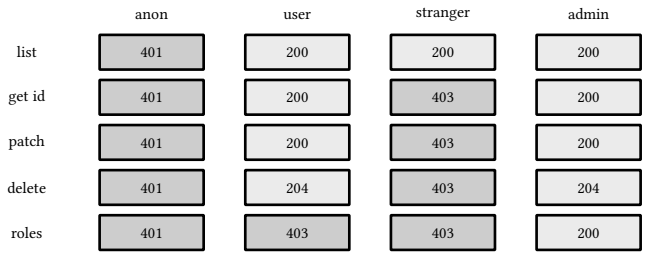


figure 171: the matrix: routes one way, actors the other, statuses in the cells

sources: OWASP, Insecure Direct Object Reference Prevention Cheat Sheet and Access Control Cheat Sheet, at cheatsheetseries.owasp.com, for the IDOR class and the deny-by-default guidance. RFC 9110 sections 15.5.1 to 15.5.4 on 401, 403, and 404 semantics. Microsoft Learn, verified 2026-09-26: learn.microsoft.com/aspnet/core/fundamentals/routing for endpoint routing and the route values this chapter's policies read, learn.microsoft.com/dotnet/api/microsoft.aspnetcore.http.httprequest.routevalues for the filled-at-dispatch contract, and learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/minimal-api-filters for the endpoint filter form the admin routes carry. Verified by `books/csharp-net/api` tests under `dotnet test`, the authz suite green through the frozen vector 14.

27. store

The service has run on in-memory maps, which tests love and restarts hate. This lane's answer is its named infrastructure: Microsoft.Data.Sqlite over the SQLitePCLRaw e_sqlite3 bundle, the native engine reporting version 3.53.4, driven through the plain ADO.NET surface the platform ships. No object relational mapper sits in the graph. EF Core is dated meta in the suite chapter and nothing here needs it, because the whole job is schema migrations that append and never edit, parameterized queries that carry a cancellation token every time, the transaction idiom in its using-block form, the isolation a wal-mode sqlite actually gives, the version guard that makes lost updates visible, keyset pagination as one row-value comparison, and the window-function report the go book's engine era walked in loops instead.

27.1. ado.net and microsoft.data.sqlite

ADO.NET is a protocol, not a database. A program opens a connection by connection string, creates commands against it, binds values as parameters, and reads rows through a data reader, and any provider that implements the shape can sit underneath. Microsoft.Data.Sqlite is that provider: SqliteConnection, SqliteCommand, SqliteDataReader, and the native engine arrives through SQLitePCLRaw.bundle_e_sqlite3, pinned at 3.0.5 alongside the provider at 10.0.12, the same pins the samples and the capstone carry.

Two platform facts shape every line of this store. First, SqliteConnection is not thread-safe: instances are never shared across calls, each operation opens its own connection, and pooling would make that cheap except the tests turn it off, because a pooled connection holds the file open on Windows and a temp directory would not delete clean. Second, the connection string is the one place configuration lives: data source, Pooling=False, Foreign Keys=True so references actually bite, and Default Timeout 5 as the busy-retry budget the provider spends automatically when a lock is contended.

```
    // the one place the connection string is composed: foreign keys armed
    // at open, no pooling, the busy budget as the default command timeout
    public static string ConnectionString(string path) =>
        $"Data Source={path};Pooling=False;Foreign Keys=True;Default Timeout={BusyTimeoutSeconds}";

    // Open returns a ready connection. WAL is the concurrency decision:
    // writers append to the log while readers keep snapshots of the
    // database file, one writer at a time. The pragma is persistent in
    // the file, so setting it per open is idempotent.
    public async Task<SqliteConnection> OpenAsync(CancellationToken ct = default)
    {
        var connection = new SqliteConnection(_connectionString);
        try
        {
            await connection.OpenAsync(ct);
            using var wal = Command(connection, null, "PRAGMA journal_mode = 'wal'");
            await wal.ExecuteNonQueryAsync(ct);
            return connection;
        }
        catch
        {
            await connection.DisposeAsync();
            throw;
        }
    }
```

listing 235: the connection string hub and the open that arms wal, foreign keys, and the busy budget (source: csharp-net/api/src/CsharpBook.Api/Store/StoreDb.cs, lines 20-44)

Values ride parameters, never string concatenation, and the parameter prefix this store standardizes on is the dollar sign, `$id`, `$email`, `$limit`. The wal pragma is the concurrency decision and it is persistent in the file, so issuing it per open is idempotent: writers append to the log while readers keep snapshots of the database file, one writer at a time.

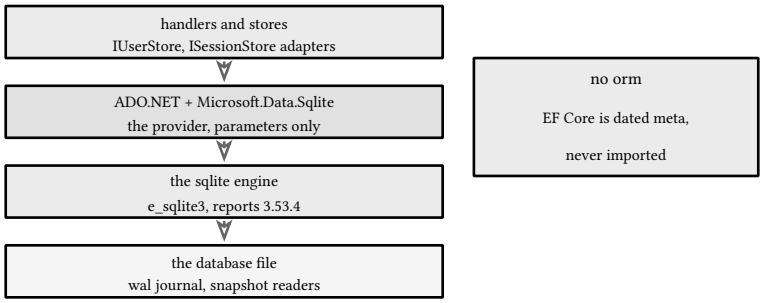


figure 172: the stack the store sits on

27.2. migrations test driven

Schema changes are code changes, so they ship as records, and the history is append-only by contract: a shipped version is frozen, a correction is a new numbered version, and no record is ever edited. The registry is the user_version pragma, an integer sqlite keeps in the database header itself, so no table the database keeps about itself can drift from the schema it describes.

This wave ships three versions. Version 1 is the whole vocabulary in one record, users with their email uniqueness index, roles, sessions with their refresh token history, events as one presence row per user per day, reports with their idempotency claim. Version 2 adds the user version column, the substrate the optimistic concurrency sections stand on, and the keyset index beside it. Version 3 adds the idempotency keys table the concurrency chapter fills. Each applies inside its own transaction with the version bump in the same group, so a version that fails halfway leaves nothing durable and can simply be retried.

```
public async Task<int> MigrateAsync(CancellationToken ct = default)
{
    await using var connection = await OpenAsync(ct);
    var applied = 0;
    foreach (var migration in Migration.PendingAfter(await UserVersionAsync(connection, ct)))
    {
        await using var transaction = connection.BeginTransaction();
        await using (var ddl = Command(connection, transaction, migration.Sql))
        {
            await ddl.ExecuteNonQueryAsync(ct);
        }
        await using (var bump = Command(
            connection, transaction, $"PRAGMA user_version = {migration.Version}"))
        {
            await bump.ExecuteNonQueryAsync(ct);
        }
        await transaction.CommitAsync(ct);
        applied++;
    }
    return applied;
}
```

listing 236: one version, one transaction, the ddl and the header bump land together, applied at most once (source: csharp-net/api/src/CsharpBook.Api/Store/StoreDb.cs, lines 90-110)

The test drives the runner from an empty directory, proves all three records applied in order, re-runs and proves nothing applies twice, reopens the database and proves the registry came back with it, then reads sqlite_master and asserts every table and index the contract's vocabulary names, nothing more and nothing less.

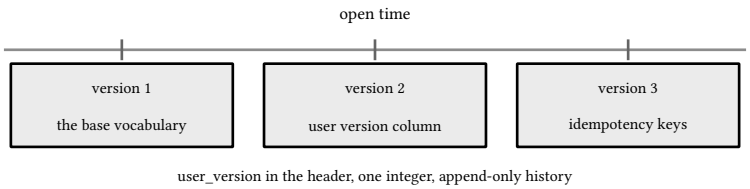


figure 173: the version list only grows right

27.3. queries with cancellation every time

Every store method takes a CancellationToken and hands it to every command, ExecuteReaderAsync with the token, ExecuteNonQueryAsync with the token, and the surface never drops it. What the provider does with it is stated plainly because the docs state it: sqlite does not support asynchronous execution, the async methods run synchronously, and the provider's own guidance points at write-ahead logging as the real concurrency lever. The token is honored before a statement runs, a pre-canceled token answers OperationCanceledException and nothing reaches the database, which is exactly what the cancellation test asserts with no clock and no sleep.

The timeout story rides the same budget. On a busy or locked error the provider retries automatically until the command timeout, Default Timeout 5 seconds from the connection string, and that is why no retry loop appears anywhere in this store: the platform already wrote it.

The adapters implement the two ports the earlier chapters froze, IUserStore and ISessionStore, outcome for outcome, so no handler can tell sqlite from the in-memory maps the tests grew up on. One read is one deferred transaction, one parameterized statement, one mapping into a domain record that carries no provider types, and the roles ride their own ordered query inside the same snapshot:

```
    private static async Task<UserStoreResult> ReadOneAsync(
        SqliteConnection connection, SqliteTransaction tx,
        string predicate, string parameter, string value, CancellationToken ct)
    {
        await using var command = StoreDb.Command(connection, tx,
            $"SELECT {Keyset.UserColumns} FROM users WHERE {predicate}");
        command.Parameters.AddWithValue(parameter, value);
        await using var reader = await command.ExecuteReaderAsync(ct);
        if (!await reader.ReadAsync(ct))
        {
            return UserStoreResult.Fail(UserStoreOutcome.NotFound);
        }

        var user = ReadUser(reader);
        return UserStoreResult.Ok(user with
        {
            Roles = await ReadRolesAsync(connection, tx, user.Id, ct),
        });
    }
```

listing 237: one read through the adapter: token in, statement bound, domain record out, roles in the same snapshot (source: csharp-net/api/src/CsharpBook.Api/Store/SqliteUserStore.cs, lines 182-200)

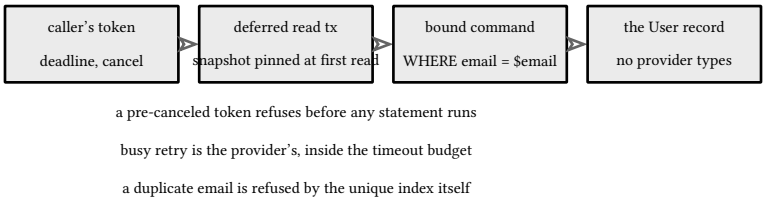


figure 174: one read's path from caller to domain record

27.4. the transaction idiom

One shape covers every write in the service: take the transaction, run the body, commit, and let any other exit discard. The provider's default transaction issues BEGIN IMMEDIATE, the write lock is taken up front rather than at the first write, so the busy budget is spent where the caller chose to write. The body stages whatever it likes. Commit lands the group. Everything else, a thrown exception, a failed statement, a cancellation, unwinds through the await using, and DisposeAsync rolls an uncommitted transaction back. That is the deferred rollback lesson restated for using blocks: the dispose is the rollback, no finally dance, no manual Rollback call on the error path, and after a commit the same dispose is a no-op because there is nothing left to undo.

The tests pin all three exits. A committed group lands. A body that inserts and then throws leaves nothing, and an earlier committed group survives the failure beside it. A group whose second statement violates the email index rolls back its first statement too, because groups are atomic or they are nothing. The session adapter carries the hardest case: presenting a superseded refresh token revokes the whole family inside the same write transaction and the SessionRevoked answer is returned after the revocation landed, so the family cannot be rolled back to life.

```
    public async Task<T> WithTxAsync<T>(
        Func<SqliteConnection, SqliteTransaction, CancellationToken, Task<T>> body,
        CancellationToken ct = default)
    {
        await using var connection = await OpenAsync(ct);
        await using var transaction = connection.BeginTransaction();
        var result = await body(connection, transaction, ct);
        await transaction.CommitAsync(ct);
        return result;
    }
```

listing 238: the write shape: begin immediate, body, commit, and the dispose is the deferred rollback (source: csharp-net/api/src/CsharpBook.Api/Store/StoreDb.cs, lines 61-70)

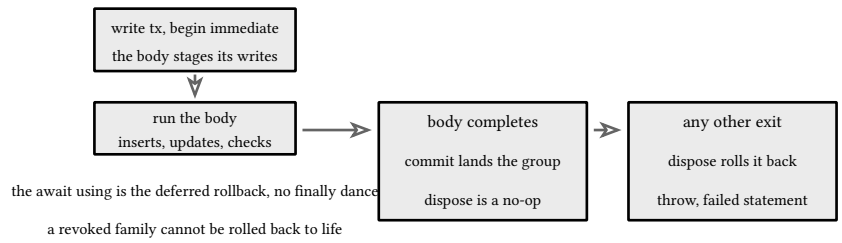


figure 175: a transaction's exits, every one of them clean

27.5. isolation levels and what sqlite actually gives you

The textbook menu, read uncommitted through serializable, is a menu of guarantees a general engine picks among. Microsoft.Data.Sqlite treats the level passed to BeginTransaction as a minimum and promotes, and the two levels it actually serves are serializable by default, and read uncommitted only over a shared cache, which is exactly the dirty-read mode the transactions documentation demonstrates and exactly what nobody wants beside a wal. No configuration file stands between the claim and the mechanism here, the mechanism is wal plus one writer.

What that buys is snapshot isolation. A deferred transaction begins as nothing and becomes a read transaction at its first statement, pinning the snapshot it then keeps for its whole life. No dirty reads, because readers only ever see committed database states. No non-repeatable reads inside one transaction, because the snapshot never moves. One writer at a time, because BEGIN IMMEDIATE takes the write lock before the first write. Write skew stays possible, two writers can act on disjoint stale snapshots and each commit cleanly, and that boundary is a fact about the locks, stated where the code states it.

The demo stays deterministic and single threaded, which is the whole trick. The reader's connection opens a deferred transaction and reads, capturing its view. A second connection commits past the pinned snapshot, and wal is what lets it, the reader holds no lock the writer needs. The reader reads again inside its transaction and still sees the old value, and a fresh read outside sees the new one. No thread, no barrier, no sleep, the interleave is the test's own statement order:

```
public async Task DeferredReadPinsItsSnapshotAcrossACommit()
{
    using var store = new TempStore();
    var db = await store.MigratedAsync();
    await TempStore.SeedUserAsync(db, "u-1", "ada@example.org", 1_000);
    await RenameAsync(db, "u-1", "first", "UPDATE users SET display_name = $name WHERE id = $id");

    // the reader's own connection, deferred: the first read below
    // pins the snapshot it keeps for its whole life
    await using var reader = await db.OpenAsync();
    using var readTx = reader.BeginTransaction(deferred: true);
    Assert.Equal("first", await ReadNameAsync(reader, readTx, "u-1", default));

    // a second connection commits past the pinned snapshot, and wal
    // is what lets it: readers hold no lock a writer needs
    await RenameAsync(db, "u-1", "second", "UPDATE users SET display_name = $name WHERE id = $id");

    Assert.Equal("first", await ReadNameAsync(reader, readTx, "u-1", default));
    readTx.Rollback();

    // outside the transaction the read is fresh
    Assert.Equal("second", await ReadNameAsync(reader, null, "u-1", default));
}
```

listing 239: pin a view, commit past it, read again: the interleave is the statement order (source: csharp-net/api/tests/CsharpBook.Api.Tests/Store/IsolationTests.cs, lines 31-53)

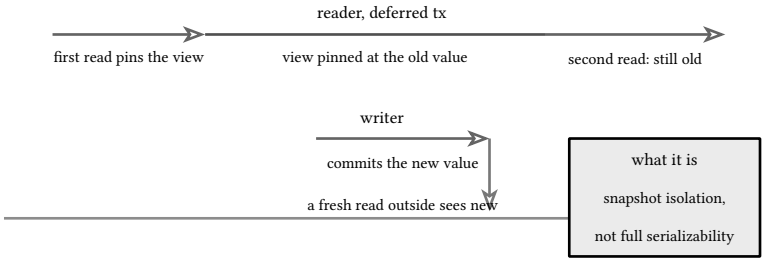


figure 176: one timeline: the reader's view holds

27.6. the lost update demo and the CAS answer

Two writers read the same user, both compute, both write, and one update silently vanishes. The store reproduces it with two blind writes, UPDATE with no version guard: both report one row affected, both succeed, the second name is all that survives, and the version column never moved, so nothing anywhere records that anything was lost. Serialization does not fix this, and owning the transaction shape makes the reason visible: BEGIN

IMMEDIATE guarantees each write lands whole and says nothing about the stale read the write was computed from, so the serialized blind writers merely lose the update in a tidy order.

The version column is the detector and the guarded update is the fix, the strong discipline in SQL form. The check, the bump, and the write share one statement inside one write transaction, so no second writer can slip between the read and the write. A mismatch answers zero rows affected, the adapter classifies it against the known id, and the client learns VersionMismatch, which over http is exactly the 412 the next chapter's ETag duel stages with two released patchers. Of two racers holding version 1 exactly one affects a row, and the loser gets the refusal it must handle:

```
    public Task<UserStoreResult> UpdateDisplayNameAsync(
        string id, int expectedVersion, string displayName, CancellationToken ct = default) =>
        db.WithTxAsync<UserStoreResult>(async (connection, tx, token) =>
        {
            await using (var update = StoreDb.Command(connection, tx,
                "UPDATE users SET display_name = $name, version = version + 1 " +
                "WHERE id = $id AND version = $version"))
            {
                update.Parameters.AddWithValue("$id", id);
                update.Parameters.AddWithValue("$name", displayName);
                update.Parameters.AddWithValue("$version", expectedVersion);
                if (await update.ExecuteNonQueryAsync(token) == 0)
                {
                    return await ExistsAsync(connection, tx, id, token)
                        ? UserStoreResult.Fail(UserStoreOutcome.VersionMismatch)
                        : UserStoreResult.Fail(UserStoreOutcome.NotFound);
                }
            }
```

listing 240: the guarded update: check, bump, and write in one statement, zero rows is the refusal (source: csharp-net/api/src/CsharpBook.Api/Store/SqliteUserStore.cs, lines 83-100)

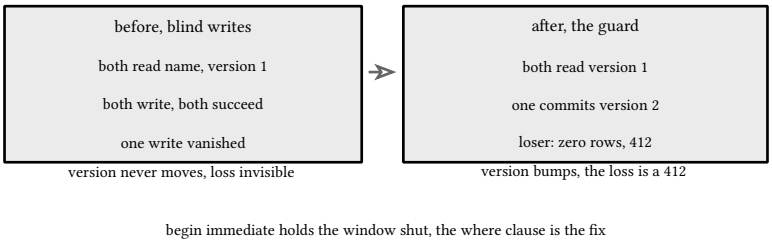


figure 177: same race, two outcomes: silent loss versus a refused writer

27.7. analytics with window functions

Reports are sql's home turf, and this lane keeps them there. Presence collapses events to one row per user per day before any counting happens, because actives count users, not actions: the events primary key is the pair (day, user_id) and the insert rides ON CONFLICT DO NOTHING, so a duplicate event is absorbed at write time. From there the windows do the work. ROW_NUMBER partitioned by user in day order marks each user's first day in the window as nth = 1, which is the newcomer split. LAG carries yesterday's count forward with today's as its default, so the first day's delta is zero by construction, the same arithmetic without the self-join. Leaders rank by count descending then user id ascending, a total order, so a replayed report cannot differ run to run.

Days are ISO text and text comparison is chronological for ISO dates, so the window bounds are two string parameters. One report reads inside one deferred transaction, so it always sees one snapshot of the events, never a torn view of an insert in flight. The wire projection keeps only day and active, and over the frozen vector's seeded events, (2026-09-25, ada), (2026-09-26, ada), (2026-09-26, grace), the series is exactly the golden rows, 1 then 2, asserted against the vector file itself:

```
    public const string DailyActiveSql =
        """
        WITH presence AS (
            SELECT day, user_id FROM events
            WHERE day ≥ $from AND day ≤ $to
        ),
        numbered AS (
            SELECT day, user_id,
                   ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day) AS nth
            FROM presence
        ),
        per_day AS (
            SELECT day, COUNT(*) AS active,
                   SUM(CASE WHEN nth = 1 THEN 1 ELSE 0 END) AS new
            FROM numbered
            GROUP BY day
        )
        SELECT day, active, new,
               active - LAG(active, 1, active) OVER (ORDER BY day) AS delta
        FROM per_day
        ORDER BY day
        """;
```

listing 241: the report query: presence, row_number for the newcomer split, lag for the delta (source: csharp-net/api/src/CsharpBook.Api/Store/Analytics.cs, lines 29-50)

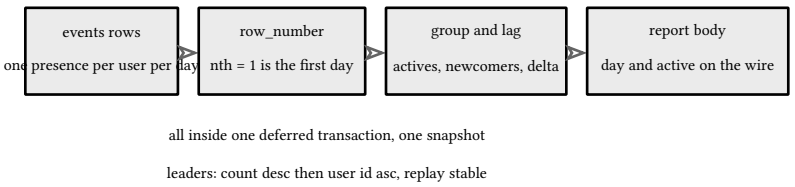


figure 178: events rows to windows to the report body

27.8. store tests without docker

Every test in the suite starts the same way: a fresh temp directory, a StoreDb pointed at a file inside it, MigrateAsync, act, dispose, and the directory deletes clean because Pooling=False means the file handles die with their connections. No container, no port, no shared server, no ordering between tests. Nothing in the suite sleeps: the cancellation test pre-cancels a token, the snapshot interleave is statement order, the guarded-update duel needs no barrier at the store layer because BEGIN IMMEDIATE serializes the racers itself. The clock is never read by the store at all, the session port's lifetime policy belongs to its callers, and the one golden-body assertion walks up from the assembly until it finds the frozen vector file under contract/testdata, so the suite finds the bytes from any harness depth.

The fixtures seed through the schema itself, straight SQL with bound parameters, because this chapter's tests exercise machinery rather than handlers, and the foreign keys bite the fixture that forgets its user row, which is the constraint doing its job. Forty-six store tests in nine files cover the migrations, the open and its pragmas, the transaction exits, the isolation trio, the keyset walk, the user and session adapters against the same behavioral contract the in-memory suites pin, and the analytics windows against the vector's own rows:

```
public sealed class TempStore : IDisposable
{
    public string Root { get; } = Path.Combine(
        Path.GetTempPath(), "csharpbook-store-" + Guid.NewGuid().ToString("N"));

    public StoreDb Db { get; }

    public TempStore()
    {
        Directory.CreateDirectory(Root);
        Db = new StoreDb(Path.Combine(Root, "store.db"));
    }
```

listing 242: the whole test story: a temp dir, a store at a file inside it, per test (source: csharp-net/api/tests/CsharpBook.Api.Tests/Support/TempStore.cs, lines 9-20)

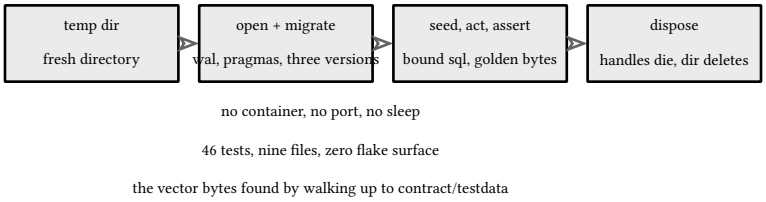


figure 179: a store test's lifecycle, nothing shared between runs

sources: learn.microsoft.com for the sqlite pages under dotnet/standard/data/sqlite, async, transactions, database errors, and connection strings, and the api pages for SqliteCommand.ExecuteReaderAsync, SqliteConnection.BeginTransaction, and SqliteTransaction.Dispose, all accessed 2026-09-26, the BeginTransaction default of BEGIN IMMEDIATE read from the SqliteTransaction source in the dotnet runtime repository at the same date, and the wal and locking model from sqlite.org's write-ahead logging and result code pages. Verified by `dotnet test Api.slnx` over the store suite, 46 tests across MigrationsTests, StoreDbTests, TransactionTests, IsolationTests, KeysetTests, SqliteUserStoreTests, SqliteSessionStoreTests, AnalyticsTests, and the reports pair's ReportsEndpointTests, with the engine version 3.53.4 confirmed by `SELECT sqlite_version()` over the pinned bundle.

28. concurrency control

The store chapter gave one writer the lock and made the loser's failure visible as a version mismatch. This chapter moves that fact to the http edge and builds everything around it: If-Match preconditions end to end with the duel two released patchers settle, If-None-Match and the 304 that saves the body, idempotency keys with stored response snapshots and in-flight dedup, singleflight collapsing a herd into one fill, the synchronization primitives revisited as shapes with jobs, and a race canary built for a platform that ships no race detector.

28.1. the lost update prevented

Two clients read the same user, both patch, and without protection one write silently vanishes. The stack answers in three layers. The precondition ladder fails fast: a PATCH with no If-Match is refused with 412 before anything in the update runs, an anonymous caller still meeting the 401 edge first, the contract chose 412 over the 428 the RFC suggests. The handler still compares the header against the etag of the representation it just fetched, because the value can move between the wrapper and the write. And the store's guarded update from the previous chapter, check, bump, and write inside one BEGIN IMMEDIATE transaction, is the guarantee itself, the only layer that is atomic. Fast fail, invariant, atomicity, in that order.

The duel stages the whole chain over the real route table with the sqlite store behind it: the users family wired over SqliteUserStore, an actor installed by a test middleware, two PATCHers carrying the same valid etag, and one TaskCompletionSource as the barrier. Both requests are created parked on the barrier, one close releases both, and whether the second request reads its user before or after the first commits, the outcome is the same set: exactly one 200, exactly one 412, version 2 standing after, the winner's name in the row, and a new etag on the winner's response. No sleep exists anywhere in the file, and the assertion is on the sorted pair of statuses, so either racer may win:

```
public async Task TwoReleasedPatchersSettleOne200AndOne412()
{
    var etag = await CurrentEtagAsync();
    var barrier = new TaskCompletionSource(TaskCreationOptions.RunContinuationsAsynchronously);

    var a = Task.Run(async () =>
    {
        await barrier.Task;
        return await _client.SendAsync(Patch("renamed a", etag));
    });
    var b = Task.Run(async () =>
    {
        await barrier.Task;
        return await _client.SendAsync(Patch("renamed b", etag));
    });

    // one close, both go: both pass the wrapper's etag check against
    // version 1, and the guarded update crowns exactly one
    barrier.TrySetResult();
    var results = await Task.WhenAll(a, b);
```

listing 243: the duel: one barrier releases both writers, the guarded update crowns one (source: csharp-net/api/tests/CsharpBook.Api.Tests/Concurrency/EtagDuelTests.cs, lines 90-109)

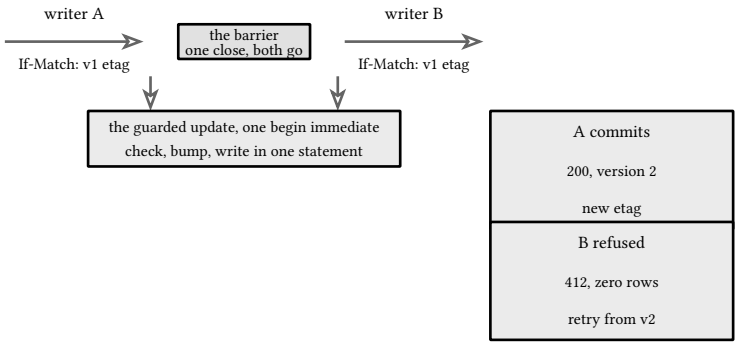


figure 180: two patchers, one barrier, one winner

28.2. If-None-Match and 304

The read side of the same coin costs nothing when nothing changed. A GET carrying an If-None-Match that matches the current representation answers 304 Not Modified before the response body is built, and the client, which already holds the body the etag names, keeps it. The 304 carries the etag and nothing else, because a 304 with a body would make the client cache the wrong thing for the wrong reason.

The etag is the sha256 of the exact response bytes, computed the same way on every user representation, so the comparison in the conditional branch is one string equality over a value the handler just computed for the normal path anyway. The decisions are four: no header means an ordinary read, a missing resource passes through so the handler's own 404 answers, a header that does not match is a fresh 200 because the client's copy is stale, and only the match answers 304:

```
        var bytes = user.ToWireBytes();
        var etag = UserWire.ETagBody(bytes);
        if (http.Request.Headers.IfNoneMatch.ToString() == etag)
        {
            http.Response.StatusCode = StatusCodes.Status304NotModified;
            http.Response.Headers.ETag = etag;
            return;
        }

        await WriteBodyAsync(http.Response, StatusCodes.Status200OK, bytes, ct, etag: etag);
```

listing 244: the conditional GET: one comparison, an empty body, the etag stamped (source: csharp-net/api/src/CsharpBook.Api/Users/UserEndpoints.cs, lines 88-97)

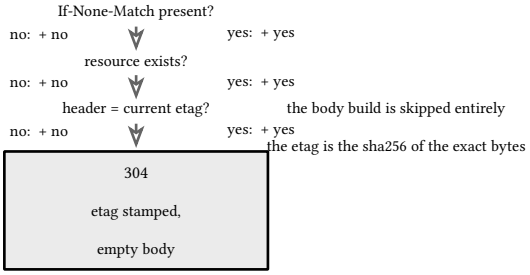


figure 181: the conditional GET decision

28.3. idempotency keys

A client that times out retries, and a register route that recreates on retry is a double-charge machine. The contract's answer is the Idempotency-Key header, scoped by route and caller identity, bound to the sha256 of the exact request bytes. The first successful run is stored as a snapshot, status, content type, body, over the

idempotency_keys table the store chapter shipped, and a later request with the same key and the same hash replays those bytes at the stored status: the retried register returns the original 201 byte for byte.

The failure paths are as deliberate. The same key under a different hash answers Mismatch, the client is misusing the contract, and the stored snapshot is untouched. A failing first run is never stored, because a retry after a validation error deserves a fresh run, so only outcomes under 400 become snapshots. Keys live 24 hours on the injected clock, and expiry is arithmetic the test advances by hand.

The missing half was concurrency. A lookup that misses, a handler that runs, a save that lands only after the handler returns: two callers under the same key inside that window both miss and both run. The guard closes it with the singleflight this chapter builds, keyed by route, identity, and key alone. The first caller claims the flight and runs the handler, a second caller joins the flight and never invokes it, and once the fill settles every joiner re-reads the row and classifies against its own hash, so the same key under a different body answers Mismatch whether it arrived before or after the winner:

```
    public async Task<IdempotencyResult> ExecuteAsync(
        string route, string identity, string key, byte[] requestHash,
        Func<CancellationToken, Task<StoredResponse>> handler, CancellationToken ct = default)
    {
        var settled = await ReadAsync(route, identity, key, ct);
        if (settled is not null)
        {
            return Classify(settled, requestHash);
        }

        var flight = await _flights.ExecuteAsync(
            route + "|" + identity + "|" + key, async token =>
            {
                // the settle re-read: a row stored between the caller's
                // first lookup and the flight start answers from storage
                var again = await ReadAsync(route, identity, key, token);
                if (again is not null)
                {
                    return again.Response;
                }
```

listing 245: the guard: fast-path lookup, then the flight, then the settle re-read (source: csharp-net/api/src/CsharpBook.Api/Concurrency/IdempotencyGuard.cs, lines 48-67)

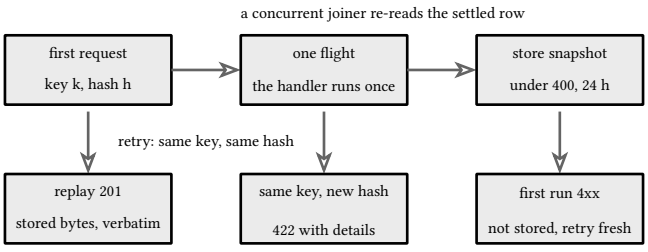


figure 182: one key's life: first run, snapshot, replay

28.4. singleflight

When a hot cache key expires, every request for it computes the same expensive thing at once, the thundering herd, and the origin pays for n identical reads. Singleflight collapses the herd: the first caller runs the function, every concurrent caller with the same key parks on that one run and receives its result, and the shared flag tells the caller its answer was shared. The entry is forgotten the moment the call completes, the property that keeps this honest, because the herd that arrives after the lull fills again. A collapse tool, not a cache, and the next chapter puts a cache in front of this loader and gets both behaviors explicitly.

The implementation is two primitives and nothing else, and the decomposition is the whole lesson. A SemaphoreSlim is the lock over the in-flight map. A TaskCompletionSource per entry is the handoff, the value published once before the map forgets the entry. Registration is synchronous under the lock: by the time ExecuteAsync returns a task, the caller has provably joined an in-flight fill or started the only one, so the collapse test, three callers, one fill held open on a completion source the test controls, asserts one factory invocation with no scheduling window to flake through. The first caller is the runner, waiters park on the completion source with their own token, so a waiter that cancels cancels only its await while the fill keeps running for everyone else. This is the same decomposition as go's x/sync singleflight, one mutex over a map of calls each carrying its own wait group and handoff, and reading either source is reading the same shape twice:

```
public async Task<FlightResult<T>> ExecuteAsync(
    string key, Func<CancellationToken, Task<T>> factory, CancellationToken ct = default)
{
    TaskCompletionSource<T> completion;
    bool shared;
    await _gate.WaitAsync(ct);
    try
    {
        if (_inFlight.TryGetValue(key, out var existing))
        {
            completion = existing;
            shared = true;
        }
        else
        {
            completion = new TaskCompletionSource<T>(
                TaskCreationOptions.RunContinuationsAsynchronously);
            _inFlight[key] = completion;
            shared = false;
        }
    }
```

listing 246: registration under the lock: join the entry or start the only one, synchronously (source: csharp-net/api/src/CsharpBook.Api/Concurrency/Singleflight.cs, lines 32-52)

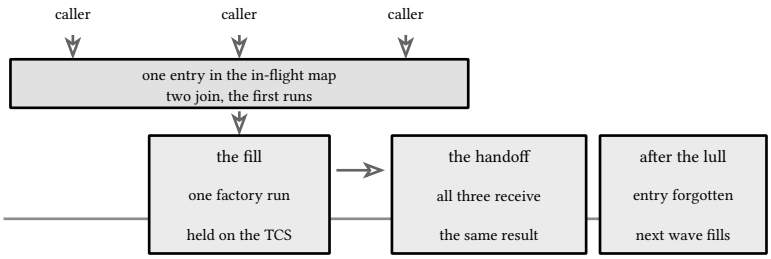


figure 183: three requests arrive together, one origin read happens

28.5. the primitives revisited

Each synchronization shape has one job, and the revisit prices them. Monitor, the lock keyword, is the whole mutex story in two methods: shared mutable state, one gate, nothing clever between them, reentrant and thread-affine, and it cannot be awaited inside. SemaphoreSlim is the bounded-concurrency valve: WaitAsync takes a cancellation token, so a caller queued for a permit can cancel out of the wait, and the permit is not bound to the thread that took it. Lazy with ExecutionAndPublication is the platform's run-exactly-once cell, and the distinction the docs draw is worth teaching: LazyInitializer guarantees a single published value but may invoke the factory more than once under contention, while ExecutionAndPublication guarantees the single run, which is what the hand-rolled Once over Monitor guarantees too. System.Threading.Channels closes the tour: a bounded channel is the queue and the backpressure in one call, the writer parks when the buffer is full, the reader drains in order.

Every guarantee is a test. The counter stays exact under an eight-task hammer. The valve admits exactly its permit count, and the assertion is permit math, not timing: with both permits held, the third and fourth tasks provably have not entered, whatever the scheduler is doing with them. Once and Lazy each run their factory exactly once with two racing readers staged on a manual-reset gate. The bounded channel parks its second writer on a buffer of one, and the read frees it in order:

```
public sealed class Counter
{
    private readonly object _gate = new();
    private long _count;

    public long Count
    {
        get { lock (_gate) { return _count; } }
    }

    public void Increment()
    {
        lock (_gate) { _count++; }
    }
}
```

listing 247: the Monitor story in two methods: shared state, one gate (source: csharp-net/api/src/CsharpBook.Api/Concurrency/SyncPrimitives.cs, lines 10-24)

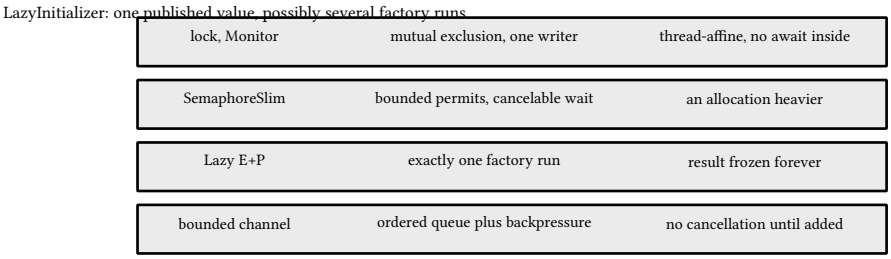


figure 184: each primitive, the guarantee it buys

28.6. the race canary

Go's canary proved a detector, a child process compiled with the race finder failing on cue. The dotnet sdk ships no race detector, so this canary proves the bug class instead, and its honesty about that is the point. The staged half is deterministic: two tasks run the same read-modify-write on one field, each reads, signals, waits for the other's read, then writes, so the interleave is scripted by completion sources and the loss is exact, two increments land, one survives, no timing anywhere:

```
        // writer A: read, hold, then write its computed value back
        var a = Task.Run(() =>
        {
            var read = counter.Value;
            aRead.TrySetResult(read);
            bRead.Task.Wait();
            counter.Value = read + 1;
        });

        // writer B: the same read-modify-write, interleaved inside A's
        // window: both read 0, both write 1, one increment is lost
        var b = Task.Run(() =>
        {
            var read = counter.Value;
            bRead.TrySetResult(read);
            aRead.Task.Wait();
            counter.Value = read + 1;
        });
```

listing 248: the staged interleave: both read 0, both write 1, one increment is lost (source: csharp-net/api/tests/CsharpBook.Api.Tests/Concurrency/RaceCanaryTests.cs, lines 26-44)

The stress half runs eight tasks fifty thousand increments each. The guarded twins, the locked counter and Interlocked.Increment, come out exactly at four hundred thousand, and those asserts are what the gate keeps. The plain read-modify-write beside them loses increments on real hardware, observed run after run, but a suite cannot assert a statistical loss without becoming a flake factory, so the canary pins the bug class with the staged interleave and pins the discipline with the exact twins, and the prose is where the observed loss is stated. What no tool checks for you here, the equivalent of happens-before reasoning, is that every shared field's accesses share an edge, a lock, an interlocked op, a channel, and the duel, the guard, and the canary are that reasoning written down as tests:

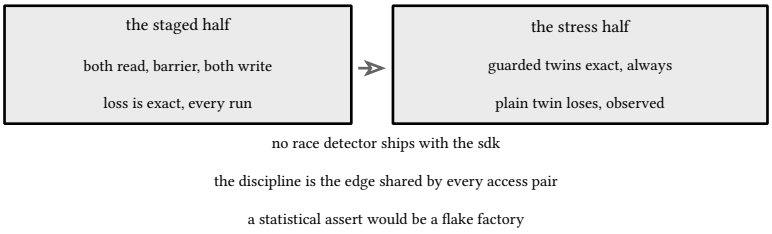


figure 185: the canary's two halves, one bug class

sources: learn.microsoft.com for SemaphoreSlim, TaskCompletionSource, LazyInitializer and Lazy with LazyThreadSafetyMode, System.Threading.Channels, Interlocked, and Task.WaitAsync, accessed 2026-09-26, the LazyInitializer publication-versus-single-run distinction read from the EnsureInitialized remarks on the same date, and the singleflight semantics stated from the go x/sync v0.22.0 source the go book's chapter pins. Verified by `dotnet test Api.slnx` over the concurrency suite, 22 tests across SingleflightTests, IdempotencyGuardTests, EtagDuelTests, PrimitivesTests, and RaceCanaryTests, zero sleeps, every schedule staged on completion sources or permit math.

29. caching

A cache is a promise to answer from a copy, and every promise has a price. This chapter prices it honestly: what caching promises and what it silently trades away, the http semantics a private api owes its clients, the LRU from first principles over a LinkedList and a Dictionary, TTL beside LRU with the sweep on the injected clock, the stampede guard riding the previous chapter's singleflight, invalidation by delete after write, and the discipline of what never gets cached at all.

29.1. what caching promises

A cache answers from a copy to save the origin the work, and both sides of the bargain matter: the copy must be cheap to find, and the answer must be one the origin would still give. Every caching decision is a bet that the read will be asked again before it changes, and the three terms of the bet are capacity, how many copies, lifetime, how long each stays, and invalidation, what a write owes the copies it made wrong. Miss any term and the cache stops being an optimization and becomes a second source of truth, which is the failure mode every caching chapter exists to prevent.

This service's copy lives in one layer, a bounded response store in front of the route table, constructed with its three terms stated up front and the clock injected so every one of them is testable as arithmetic rather than waiting:

```
public sealed class HttpCache(int capacity, TimeSpan ttl, TimeProvider clock)
{
    public static readonly IReadOnlyList<string> VaryBy = ["authorization", "cookie"];

    private readonly TtlLruCache<string, CachedResponse> _store = new(capacity, ttl, clock);
    private readonly Singleflight<CachedResponse> _fills = new();
```

listing 249: the layer: capacity, lifetime, a clock, and the vary pair, stated at construction (source: csharp-net/api/src/CsharpBook.Api/Caching/HttpCache.cs, lines 38-43)

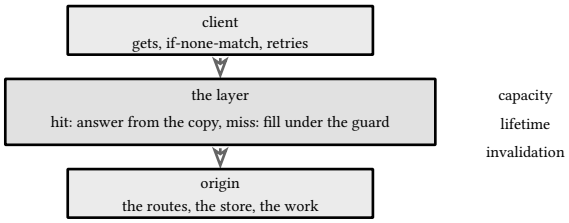


figure 186: the layer sits between client and origin, answering what it can

29.2. http caching semantics for private apis

A private api's responses are personalized and credentialed, so the shared-cache machinery of the web mostly does not apply, but the headers still owe clients a contract. Cache-Control states what may be stored and for how long, and private keeps shared proxies out of it, these bodies vary by who is asking. Vary names the request headers the representation depends on, here the bearer token and the session cookie, which is precisely the pair the cache key carries so two identities never share an entry. no-store means never, not even for a moment, and it is the default.

The reads are revalidation business. max-age=0 with must-revalidate tells the client nothing is fresh for long, every reuse round-trips the etag through the 304 lane the previous chapter built, and the layer's own copy is a latency optimization behind the protocol, never a replacement for it. The policy table is deny by default: a route is cacheable only when a row says so, and no other file in the service can make one cacheable by accident:

```
    /// <summary>
    /// PolicyFor is the table. Two cacheable reads, the user list and
    /// the single user, everything else NoStore: credential routes
    /// carry secrets, rate-limited answers are a lie about money, and
    /// anything the contract marks no-store is a legal promise.
    /// </summary>
    public static CachePolicy PolicyFor(string method, string path) => method switch
    {
        "GET" when path == "/api/users" || path.StartsWith("/api/users/", StringComparison.Ordinal)
            => CachePolicy.Cacheable,
        _ => CachePolicy.NoStore,
    };
```

listing 250: the per-route policy: two cacheable reads, everything else no-store by default (source: csharp-net/api/src/CsharpBook.Api/Caching/HttpCache.cs, lines 46-57)

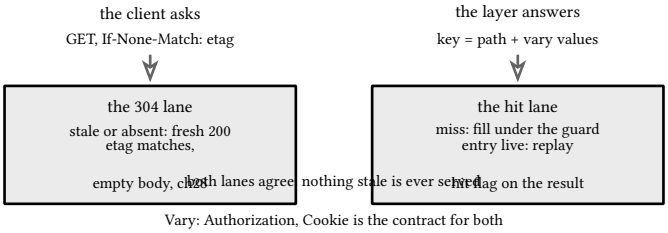


figure 187: two freshness decisions: the client's and the layer's

29.3. LRU from first principles

Least recently used is one list and one dictionary. The list carries the recency order with the hottest entry at the front, the dictionary finds any key's node in constant time, and the two agree through their pointers: every entry lives in both structures, the dictionary pointing at the node and the node carrying its key so eviction can delete from the dictionary without walking it. A read moves its node to the front, a write inserts at the front or refreshes in place, and when capacity is exceeded the node at the back is the eviction, found in constant time, removed from both structures in constant time.

The proof is the scramble test: fill the cache to capacity, touch the oldest insert, and the next insert evicts the true least recently used entry instead, because the touch moved it. A plain dictionary evicts nothing and grows forever, a plain list finds in linear time, and the pairing is the textbook answer to why both exist:

```
    public void Set(TKey key, TValue value)
    {
        if (_index.TryGetValue(key, out var node))
        {
            node.Value = (key, value);
            _order.Remove(node);
            _order.AddFirst(node);
            return;
        }

        var fresh = _order.AddFirst((key, value));
        _index[key] = fresh;
        if (_index.Count > capacity)
        {
            var coldest = _order.Last!;
            _order.RemoveLast();
            _ = _index.Remove(coldest.Value.Key);
        }
    }
```

listing 251: the pairing: the dictionary finds, the list orders, the back of the list is the eviction (source: csharp-net/api/src/CsharpBook.Api/Caching/LruCache.cs, lines 37-55)

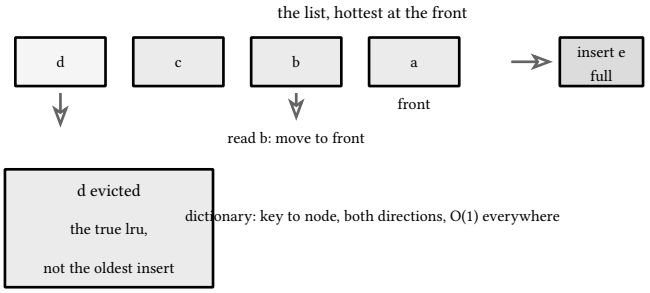


figure 188: a read moves its node to the front, an insert evicts the back

29.4. TTL beside LRU

LRU answers crowding, not age. An entry nobody reads can be least recently used forever and still hold its bytes, and an entry everybody reads can be hot long after the origin moved on. The TTL gives every entry an expiry stamped from the injected clock at write time, and reads check it lazily: an expired entry evicts itself on contact, so every read is its own eviction decision and no sweep is needed for correctness.

The sweep is for memory, not correctness. It drops whatever aged out unread, it peeks without touching recency so cleaning does not make the whole cache equally hot, and it is a callable on the injected clock rather than a hidden loop, so the test advances whole hours as one addition and asserts exactly which entries fell:

```
    public bool TryGet(TKey key, out TValue value)
    {
        lock (_gate)
        {
            if (!_lru.TryGet(key, out var entry))
            {
                value = default!;
                return false;
            }

            if (clock.GetUtcNow() ≥ entry.ExpiresAt)
            {
                _ = _lru.Remove(key);
                value = default!;
                return false;
            }

            value = entry.Value;
            return true;
        }
    }
```

listing 252: the read: expiry is lazy, the hit lives, the miss cleans up after itself (source: csharp-net/api/src/CsharpBook.Api/Caching/TtlLruCache.cs, lines 29-50)

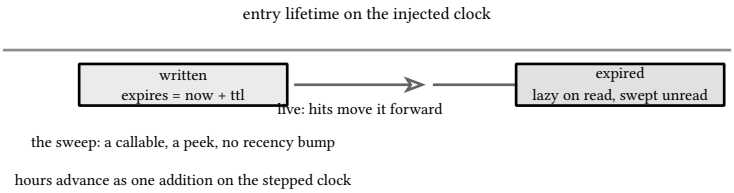


figure 189: two clocks push entries out: age and crowding

29.5. the stampede and its guard

Every entry expires eventually, and the moment a hot one does, the herd arrives together: a hundred concurrent misses, a hundred identical origin reads, the exact load spike the cache existed to prevent, arriving at the worst moment, the cold one. The guard is the previous chapter's singleflight worn one layer down: a miss fills through the flight, the herd collapses into one origin read, every waiter receives the same result, and the fill's result is the only thing that ever writes the cache entry, the runner writes it, the joiners rode it.

Errors do not land in the cache, a failed fill misses again, so an origin outage cannot be memoized into a sticky failure. The collapse test holds the origin open on a completion source the test controls, registers three callers, and asserts one fill, three identical answers, no scheduling window, because registration in the flight was synchronous:

```
        var key = Key(path, authorization, cookie);
        if (_store.TryGet(key, out var hit))
        {
            return new CacheResult(Hit: true, hit);
        }

        var generation = Volatile.Read(ref _generation);
        var flight = await _fills.ExecuteAsync(key, origin, ct);
        if (
            !flight.Shared
            && flight.Value.Status < 400
            // the generation fence: an Invalidate that spoke after this
            // fill started had no entry to delete, so the fill's Set
            // would bake a stale answer and serve it until the TTL.
            // Refusing costs one refill, honoring it would cost the
            // contract's "the next read refills" promise
            && Volatile.Read(ref _generation) == generation
        )
        {
            // the fill's result is the only cache writer: joiners rode
            // the runner's fill, and errors are never memoized
            _store.Set(key, flight.Value);
        }

        return new CacheResult(Hit: false, flight.Value);
```

listing 253: the fill: hit or flight, only the runner's result writes the entry, and a generation fence retires any fill an invalidation outran (source: csharp-net/api/src/CsharpBook.Api/Caching/HttpCache.cs, lines 70-94)

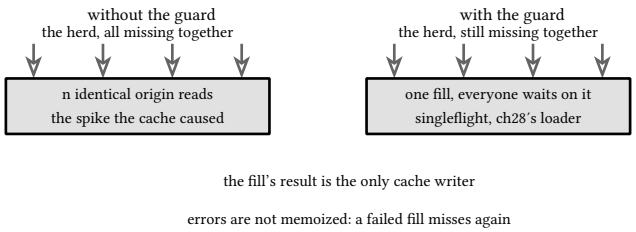


figure 190: the same expiry, with and without the guard

29.6. invalidation on write

A write changes the origin, and the copies of what it changed are now wrong. The discipline is delete after write, never update: the writer drops the affected entries and lets the next read refill them through the guard. Deleting is idempotent and always safe, and the refill is the single writer the cache already trusts. Updating instead, pushing the writer's view into the entry, sounds cheaper, one write instead of a miss, but the writer does not know what a concurrent reader cached mid-flight, and an update that lands just before a stale read's own set leaves the wrong value with a fresh timestamp, the worst of both worlds.

The rule is path plus collection: a write to a row invalidates the row and the list that reads it, a write to the collection invalidates the list and any rows under it. The staged test runs two identical caches through the same two writes, one updated with the writer's view, one deleted, and only the deleted twin converges to what the origin says after the second write:

```
    public void Invalidate(string path)
    {
        Interlocked.Increment(ref _generation);
        var collection = CollectionOf(path);
        foreach (var key in _store.KeysSnapshot())
        {
            var keyPath = KeyPath(key);
            if (keyPath == path || keyPath == collection)
            {
                _ = _store.Remove(key);
            }
        }
```

listing 254: delete after write: the generation bump, the path, its collection, nothing else (source: csharp-net/api/src/CsharpBook.Api/Caching/HttpCache.cs, lines 104-115)

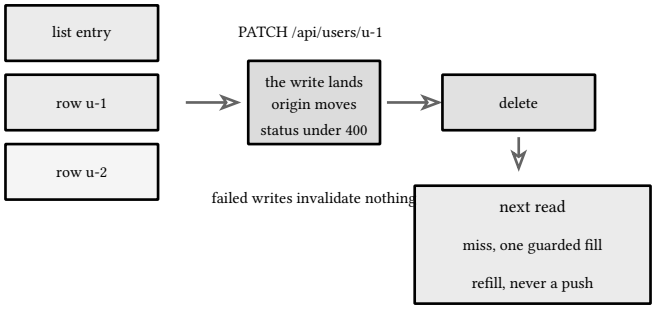


figure 191: write to a row: the row and the list go, the refill is the only writer

29.7. what not to cache

The last decision is the one that prevents the incidents. Credential routes, login and refresh, carry secrets and must never be replayable from a copy. Rate-limited responses, the 429 lane, must always reach the client, because a cached budget answer is a lie about money. Personalized bodies without a vary key to separate identities are a cross-user leak waiting for traffic. Anything the contract marks no-store is a legal promise, not a hint. The layer's policy table is deny by default, cacheable only where a row says so, which makes the exclusion list the empty set of accidents: a new route opts in or it does not get cached, and no change elsewhere in the service can flip that.

The test pins the posture end to end: a no-store route asked three times reaches the origin three times, answers from itself every time, and never once reports a hit:

```
    public async Task NoStoreRoutesAlwaysPayTheOrigin()
    {
        var cache = Clocked(out _);
        var asks = 0;

        Task<CachedResponse> Origin(CancellationToken ct)
        {
            asks++;
            return Task.FromResult(Answer(200, "never cached"));
        }

        for (var i = 0; i < 3; i++)
        {
            var result = await cache.GetOrFillAsync("POST", "/api/users", "tok-a", "", Origin);
            Assert.False(result.Hit);
        }

        // three asks, three origin runs, no copy anywhere
        Assert.Equal(3, asks);
    }
```

listing 255: the exclusion test: no-store routes always pay the origin (source: csharp-net/api/tests/CsharpBook.Api.Tests/Caching/HttpCacheTests.cs, lines 67-87)

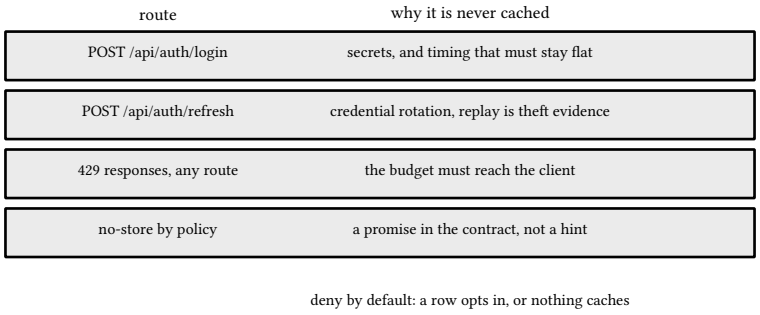


figure 192: the exclusion list, each row with its reason

sources: learn.microsoft.com for Cache-Control, Vary, and no-store read from RFC 9111 section 5.2 and RFC 9110 section 12.5.5 at rfc-editor.org and for LinkedList, Dictionary, and TimeProvider, accessed 2026-09-26, the lazy-expiry-and-sweep split and the singleflight fill semantics stated from this repo's own chapters 27 and 28, whose tests the suite runs beside these. Verified by `dotnet test Api.slnx` over the caching suite, 17 tests across LruCacheTests, TtlLruCacheTests, and HttpCacheTests, zero sleeps, the janitor's whole life exercised as additions on the stepped clock.

30. rate limiting

A service that answers every request it receives eventually answers none of them: password guessing arrives as a flood of logins, a retry storm arrives the moment a deploy slows one route down. This chapter builds the service's rate limiting in `CsharpBook.Api.Limiting`: the token bucket from first principles over the injected TimeProvider, the production gate on System.Threading.RateLimiting with one bucket per key and idle retirement, the 429 contract, what to key a limit on and what never to trust, and the test discipline that keeps every assertion off the wall clock.

30.1. why limits exist

Capacity is finite and failure is nonlinear. Past some offered load a service does not degrade gently, it falls off a cliff: queues grow, latency climbs past every client timeout, memory fills with requests waiting on answers that can no longer arrive. A limit is the refusal to stand on that cliff. It converts an overload that would take the whole service down into a fast rejection of the excess, and the client that honors it gets a working service at one request per second instead of a dead one at zero.

The two limited routes carry numbers chosen for what they protect. Login allows a burst of five per address, then one per second, because every login attempt pays the pbkdf2 verification budget and a guesser does not need more than five tries to be dangerous. Register allows ten per client ip for the same reason plus its idempotency bookkeeping. Both refill one token per second, so a client that trips the limit is back to full capacity five and ten seconds later, the difference between a limit and a lockout:

```
public static class RateRules
{
    public const int LoginBurst = 5;
    public const int RegisterBurst = 10;
    public const double RefillPerSecond = 1.0;
    public const string LoginMessage = "too many login attempts";
    public const string RegisterMessage = "too many registration attempts";

    /// <summary>
    /// DefaultIdleTtl retires a key's bucket after this much disuse.
    /// </summary>
    public static readonly TimeSpan DefaultIdleTtl = TimeSpan.FromMinutes(10);
```

listing 256: the contract's two rules as one constant block, numbers and messages (source: csharp-net/api/src/CsharpBook.Api/Limiting/RateRule.cs, lines 23-34)

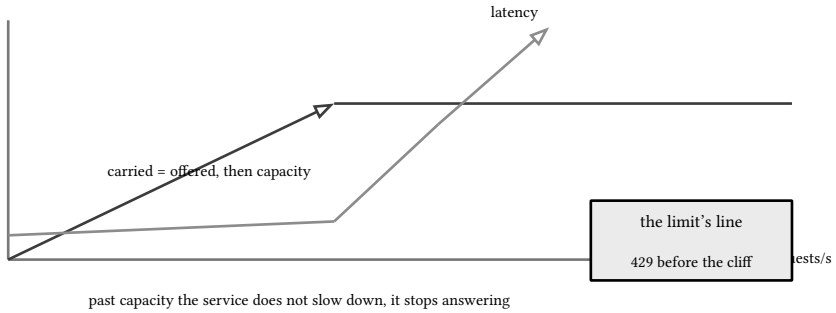


figure 193: offered load versus what the service can carry, the cliff between them

30.2. the token bucket from first principles

A token bucket holds a double, a capacity, and a refill rate. It starts full, every allowed request removes one token, and time adds tokens continuously at the configured rate until the capacity caps them. Its two knobs split the two jobs: capacity is the burst allowance, how much may arrive at once, and the refill rate is the sustained allowance,

how much may arrive per second forever. The login rule, capacity five at one per second, permits five requests in the first instant, a sixth at the one second mark, then one per second for as long as the client cares to wait.

The arithmetic of a denial is the part worth deriving rather than reciting. After the fifth same-second draw the level is exactly zero, so the next token exists one full second in the future, and that instant is both the Reset header and the Retry-After. Half a second later the level is 0.5: still not spendable, and the next whole token is only half a second away. A whole level waits a full period, a fraction waits its complement, the NextTokenIn rule: the wait is `1 - frac(level)` over the refill rate, and a denial never spends:

```
public RateDecision Allow(DateTimeOffset now)
{
    Refill(now);
    var allowed = false;
    if (_level ≥ 1)
    {
        _level -= 1;
        allowed = true;
    }
    var wait = NextTokenIn();
    return new RateDecision
    {
        Allowed = allowed,
        Limit = (int)_capacity,
        Remaining = (int)_level,
        ResetAt = now.Add(wait),
        RetryAfter = allowed ? null : wait,
    };
}
```

listing 257: Allow: one draw, one decision, and the wait folded into the answer (source: csharp-net/api/src/CsharpBook.Api/Limiting/TokenBucket.cs, lines 41-59)

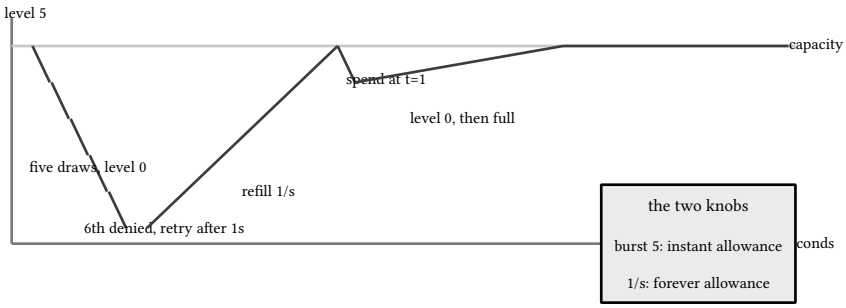


figure 194: bucket level over one draw timeline, the refill climbing between bursts

30.3. System.Threading.RateLimiting in production

The teaching bucket stays in the chapter, the shipped gate runs on `System.Threading.RateLimiting`, the platform's token bucket. Read as primitives it is the teaching bucket with the refill moved into the clock: one `lock` object, the level as a double, a fill rate of TokensPerPeriod over ReplenishmentPeriod ticks, and the stopwatch timestamp of the last replenish. There is no thread and no timer in the lazy configuration: `AutoReplenishment = false` and one `TryReplenish` per draw converts elapsed ticks into tokens under the limiter's own lock, so the passage of time is the refill, the same move the hand bucket makes with its injected clock.

The timer is the configuration this gate refuses: `AutoReplenishment` defaulting to true arms one `Timer` per limiter, and one limiter per key means a janitor per bucket. Queueing is refused the same way, `QueueLimit = 0`, because a handler that queues for a token holds its connection while it waits, which is precisely the resource the limit exists to protect. What the platform gives the gate besides enforcement is the denial's shape: `GetStatistics` floors the

level to whole permits, which is exactly the Remaining header, and a failed lease carries `MetadataName.RetryAfter` in whole replenishment periods, which is exactly the Retry-After the vector pins. The fraction the hand bucket computes, the platform rounds up to the next period, and at one token per second the two coincide.

One bucket per key lives in a dictionary, created on first use, and the dictionary needs a policy: every distinct key is an unbounded entry, so an idle ttl retires keys nobody has touched. The sweep is amortized into the draw, every ttl of clock time a draw sweeps inside the lock it already holds, which deletes the janitor entirely. The whole decision, lookup, sweep, replenish, read, acquire, runs under the gate's one lock because the platform's statistics are a separate call from its acquire, and the header math must read the level this draw spent:

```
public RateDecision Decide(RateRule rule, string key)
{
    lock (_gate)
    {
        var now = _clock.GetUtcNow();
        if (now - _lastSweep ≥ _idleTtl)
        {
            SweepLocked(now);
        }
        if (!_entries.TryGetValue(key, out var entry))
        {
            entry = new Entry { Limiter = NewLimiter(rule), LastTouch = now };
            _entries[key] = entry;
        }
        entry.LastTouch = now;

        entry.Limiter.TryReplenish();
        var before = entry.Limiter.GetStatistics()!.CurrentAvailablePermits;
        using var lease = entry.Limiter.AttemptAcquire(1);
        var allowed = lease.IsAcquired;
        // the wait is whole replenishment periods either way: an
        // allowed draw leaves a whole level one period from the next
        // token, a denial reads the same resolution off the lease
        var wait = PeriodOf(rule);
        if (!allowed && lease.TryGetMetadata(MetadataName.RetryAfter, out var metadata)
            && metadata is TimeSpan leaseWait)
        {
            wait = leaseWait;
        }
```

listing 258: one lock over the decision: sweep, replenish, floor read, acquire, wait from the lease (source: csharp-net/api/src/CsharpBook.Api/Limiting/TokenBucketGate.cs, lines 74-102)

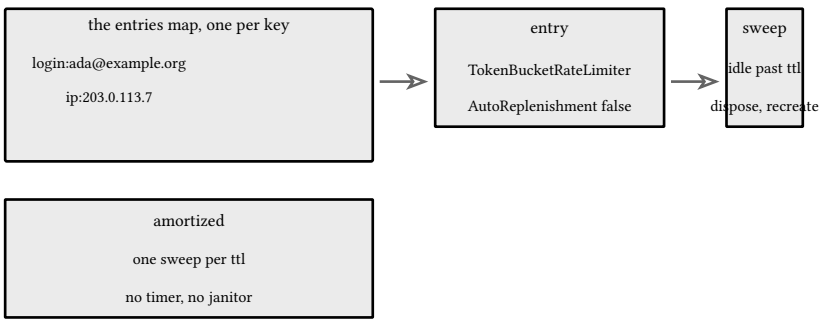


figure 195: key to bucket map, first use creates, idle age retires

30.4. the 429 contract

The wire shape is pinned by the contract and the frozen vector. Every response from a limited route carries three headers: X-RateLimit-Limit is the burst capacity, X-RateLimit-Remaining is whole tokens left after this draw, X-RateLimit-Reset is the unix second at which the next token arrives. A denial adds Retry-After in seconds, the ceiling of the wait, and answers the envelope with code rate_limited through the kernel's one failure writer, so the request id in the body is the one the edge resolved in chapter 23, the value the access log carries.

The sixth login vector pins every byte. The clock sits at t0 plus 12 seconds, five earlier logins at the same instant left the bucket reporting remaining 4, 3, 2, 1, 0, and the sixth draw finds an empty bucket: status 429, Limit 5, Remaining 0, Reset 1790830813 which is exactly t0 plus 13 seconds, Retry-After 1, and the envelope's message the frozen too many login attempts. The three headers are written before the handler runs, so they ride on successes too, and the denial path never reaches the handler at all:

```
{
    public async Task InvokeAsync(HttpContext context)
    {
        var rule = gate.TryRule(context.Request.Path);
        if (rule is null)
        {
            await next(context);
            return;
        }
        // the key extraction reads the request body synchronously, a
        // bounded read DefaultHttpContext never polices because the
        // bare recorder has no body control feature, while both the
        // real listener and the test host's wrapper do, so the switch
        // is flipped for the read alone and restored right after it
        var bodyControl = context.Features.Get<IHttpBodyControlFeature>();
        var previousSync = bodyControl?.AllowSynchronousIO;
        if (bodyControl is not null)
        {
            bodyControl.AllowSynchronousIO = true;
        }
        string key;
        try
        {
            key = rule.Key(context);
        }
        finally
        {
            if (bodyControl is not null && previousSync is { } was)
            {
                bodyControl.AllowSynchronousIO = was;
            }
        }
        var decision = gate.Decide(rule, key);
        context.Response.Headers[RateHeaders.Limit] = decision.Limit.ToString();
        context.Response.Headers[RateHeaders.Remaining] = decision.Remaining.ToString();
        context.Response.Headers[RateHeaders.Reset] = decision.ResetAt.ToUnixTimeSeconds().ToString();
        if (!decision.Allowed)
        {
            context.Response.Headers[RateHeaders.RetryAfter] =
                ((int)Math.Ceiling(decision.RetryAfter!.Value.TotalSeconds)).ToString();
            await context.Response.WriteJsonAsync(
                Kernel.ErrorCodeStatus.Of(Kernel.ErrorCode.RateLimited),
                new Envelope(ErrorBody.From(
                    ApiError.Fail(ErrorCode.RateLimited, rule.Message),
                    RequestId.Resolve(context))),
                context.RequestAborted);
            return;
        }
```

listing 259: the middleware: three headers always, Retry-After and the envelope on denial (source: csharp-net/api/src/CsharpBook.Api/Limiting/RateLimitMiddleware.cs, lines 17-64)

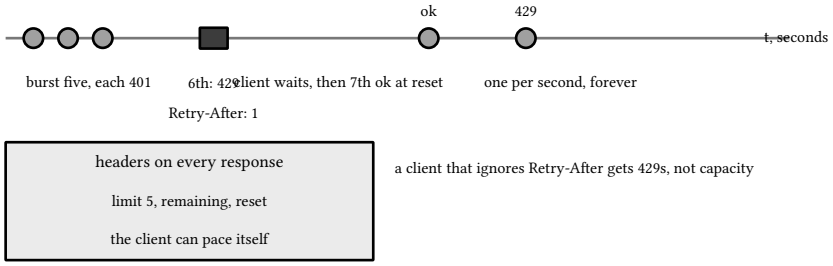


figure 196: a client that honors Retry-After: five through, one refused, the next at the reset instant

30.5. what to key on

A limit is only as good as its key, and the two routes key on different things for opposite reasons. Login keys on lower(email): the attack the rule exists for is guessing one account's password, so the bucket belongs to the account, and the casing normalization means `ADA@Example.ORG` and `ada@example.org` are one bucket. The key lives in the request body, so the keying reads the body once, bounded by the kernel's one mebibyte cap, and replaces it with a rewound copy for the handler to bind again. Register keys on the connection's remote address, and the proxy headers stay unread on purpose: X-Forwarded-For is a header any client can write, so a limiter that keys on it verbatim sells a fresh bucket of ten registrations per forged address, one per request, forever.

Two shared buckets close the dodge holes. A body the email key cannot parse lands in one login unparsed bucket, so garbage cannot orbit the limit by being invalid, and a connection with no remote address lands in its own fallback. The test for the spoof is the contract said out loud: eleven register posts from one address, a different forged X-Forwarded-For on the last one, and the eleventh is still the 429:

```
public static string KeyEmail(HttpContext ctx)
{
    byte[] body;
    try
    {
        using var reader = new BinaryReader(ctx.Request.Body);
        body = reader.ReadBytes(KernelJson.MaxBodyBytes);
        ctx.Request.Body = new MemoryStream(body);
    }
    catch (IOException)
    {
        return "login:unreadable";
    }
    try
    {
        var parsed = JsonSerializer.Deserialize<JsonElement>(body, KernelJson.Wire);
        var email = parsed.ValueKind == JsonValueKind.Object
            && parsed.TryGetProperty("email", out var member)
            && member.ValueKind == JsonValueKind.String
            ? member.GetString()
            : null;
        return email is { Length: > 0 }
            ? "login:" + email.Trim().ToLowerInvariant()
            : "login:unparsed";
    }
    catch (JsonException)
    {
        return "login:unparsed";
    }
```

listing 260: KeyEmail: a bounded read, the body handed back, garbage into one shared bucket (source: csharp-net/api/src/CsharpBook.Api/Limiting/LimitKeys.cs, lines 23-51)

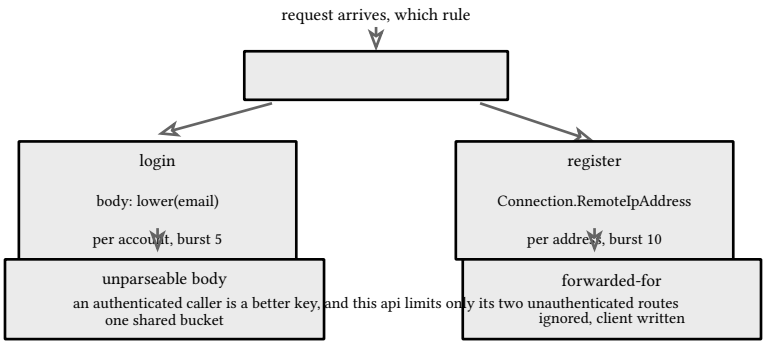


figure 197: key choice: the route decides what the bucket belongs to

30.6. limits under test

Every time-dependent behavior in the hand bucket is arithmetic on a fixed clock, because a rate limiter tested against the wall clock is tested against a scheduler. The injected TestClock moves in explicit offsets, the fixed-draw tests walk the bucket through its grammar, burst, cap, sustained rate, half-token denial, and the vector replay runs the production middleware against the frozen file: five setup draws assert remaining 4 through 0, the sixth asserts the vector's status, every header byte, and the envelope's code and message, the request id checked for uuid v7 shape only since this layer mints it.

The platform limiter adds one wrinkle the tests state rather than hide: its refill clock is the stopwatch it owns, not the injected TimeProvider. The draw-grammar tests all fire at one instant, which is why they stay deterministic,

and the one refill test uses a coarse rate so elapsed refill lands inside a bounded loop that polls the limiter's real state and never sleeps:

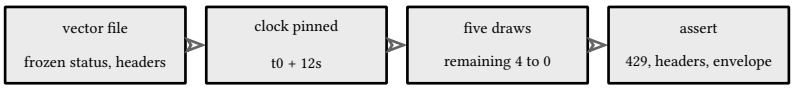
```
        var clock = new TestClock(ContractTime.T0.AddSeconds(12));
        var middleware = Middleware(clock);

        // the vector's setup: five logins at this offset, each through
        // to the handler, remaining 4 down to 0
        for (var wantRemaining = 4; wantRemaining ≥ 0; wantRemaining--)
        {
            var ctx = LoginContext("ada@example.org");
            await middleware.InvokeAsync(ctx);
            Assert.Equal(StatusCodes.Status401Unauthorized, ctx.Response.StatusCode);
            Assert.Equal(
                wantRemaining.ToString(),
                ctx.Response.Headers[RateHeaders.Remaining].ToString());
        }

        var denied = LoginContext("ada@example.org");
        await middleware.InvokeAsync(denied);
        Assert.Equal(want.GetProperty("status").GetInt32(), denied.Response.StatusCode);

        foreach (var header in want.GetProperty("headers").EnumerateObject())
        {
            Assert.Equal(
                header.Value.GetString(),
                denied.Response.Headers[header.Name].ToString());
        }
```

listing 261: the vector replay: frozen clock at t0+12s, five draws, then the frozen sixth (source: csharp-net/api/tests/CsharpBook.Api.Tests/Limiting/RateLimitMiddlewareTests.cs, lines 61-85)



the refill test: coarse rate, bounded loop over real state, never a sleep

figure 198: the fixed-vector pipeline: file in, draws through the gate, bytes out

30.7. admission preview

A rate limit refuses requests over time, an admission gate refuses them over concurrency, and the two compose because they bound different axes of the same cliff. Little's law states the trade in one line: the requests inside the system equal the arrival rate times the time each spends there, L = lambda W. Past the point where every arriving request finds all workers busy, latency rises steeply while throughput stops moving, the hockey stick chapter 32 measures. A rate limit caps lambda at the edge. An admission gate caps L directly with an in-flight semaphore and answers the overload envelope the instant the slots run out, converting a queue of waiting requests into a cheap refusal. The gate, its 503, and the drain it protects are chapter 32's to build over the decision shape this limit answers with:

```
/// <summary>
/// RateDecision is one allow or deny answer carrying everything the
/// contract's rate headers need: the burst capacity, whole tokens left
/// after this draw, the instant the next token arrives, and the wait a
/// denied caller must honor, null when the draw was allowed.
/// </summary>
public sealed record RateDecision
{
    public required bool Allowed { get; init; }
    public required int Limit { get; init; }
    public required int Remaining { get; init; }
    public required DateTimeOffset ResetAt { get; init; }
    public TimeSpan? RetryAfter { get; init; }
}
```

listing 262: the decision both refusals share, time here, concurrency in chapter 32 (source: csharp-net/api/src/CsharpBook.Api/Limiting/RateDecision.cs, lines 3-16)

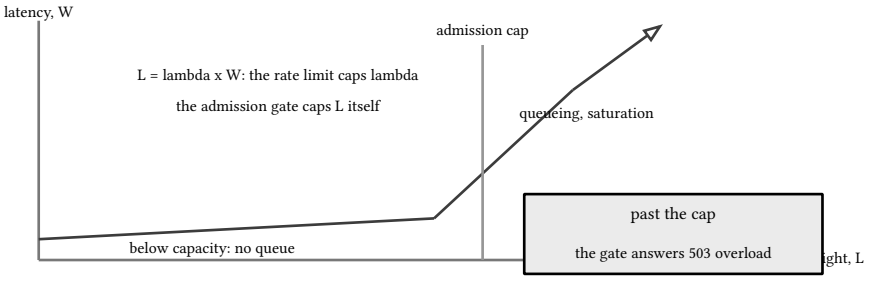


figure 199: latency versus concurrency: the hockey stick the admission cap cuts off

sources: learn.microsoft.com for System.Threading.RateLimiting, the TokenBucketRateLimiter class with its options and TryReplenish, plus the rate limiting page under aspnet/core/performance/rate-limit, read 2026-09-26 against the net 11 rc surface, the limiter source in dotnet/runtime at the commit the api pages link for the state and lease metadata claims, and RFC 6585 for status 429 with RFC 9110 for Retry-After. Verified by the `CsharpBook.Api.Limiting` tests, 19 of them under `dotnet test Api.slnx`, with the sixth login vector replayed header for header from `contract/testdata/06-login-429.json`.

31. observability

A service no one can see is a service no one can fix, and a service that logs prose no machine can query is seen only by whoever is reading along live. This chapter builds the service's observability in `CsharpBook.Api.Observability`: the real ILogger setup with json lines, source contexts, scopes, and redaction, the W3C traceparent header parsed and rendered by hand and carried by the platform's ActivitySource spans, the metrics hub over System.Diagnostics.Meters with its prometheus exposition pinned byte for byte by the frozen vector, p99 done honestly, the diagnostics surface, and one request through every signal at once.

31.1. ILogger the real setup

The production logger is one factory, not a pattern study: json lines to the console, a level, scopes included, utc timestamps, and every category wrapped once at the hub by the redaction decorator of the next section, so no handler can log around it. The category is the source context, free: a logger created for a type answers that type's full name, which is why one line can be filtered by the layer that wrote it, and the level is why debug lines are compiled in but silent until the level drops:

```
    public static ILoggerFactory BuildFactory(LogLevel level = LogLevel.Information)
    {
        var factory = LoggerFactory.Create(builder =>
        {
            builder
                .AddJsonConsole(options =>
                {
                    options.TimestampFormat = "O";
                    options.UseUtcTimestamp = true;
                    options.IncludeScopes = true;
                    options.JsonWriterOptions = new() { Indented = false };
                })
                .SetMinimumLevel(level);
        });
        return factory;
```

listing 263: one factory: json console, utc timestamps, scopes, and a level (source: csharp-net/api/src/CsharpBook.Api/Observability/LogSetup.cs, lines 16-30)

The golden test pins the line through the real pipeline, the json console provider writing to a captured console, and reads it back as json: Category is the source context, Message the formatted text, State the structured members by name. One platform difference from the go lane is stated rather than hidden: the go logger's hook could pin the timestamp for the test, this formatter's clock is the wall, so the timestamp is asserted by shape and every other member by value:

```
{"Timestamp":"2026-09-26T02:41:07.1098765Z","LogLevel":"Information","Category":"CsharpBook.Api.Authn.LoginHandler","Mes
failed for ada@example.org with [redacted] on attempt 3","State":
{"Email":"ada@example.org","Password":"[redacted]","Attempts":3,"{OriginalMessage}":"login failed for
ada@example.org with [redacted] on attempt 3"}}
```

fragment, illustrative, not executed

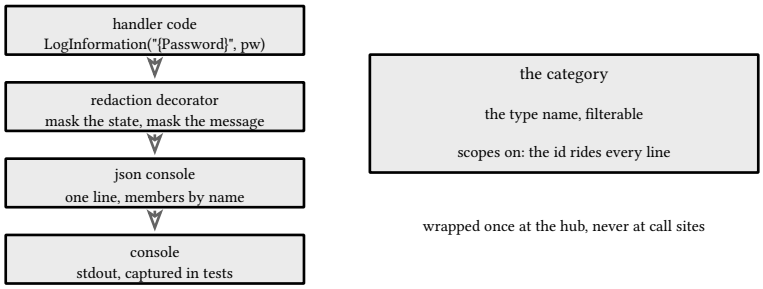


figure 200: the logger's own stack: call site through provider to console

31.2. structured beats interpolated

An interpolated message packs data into prose, and prose is lossy: `login failed for ada, attempt 3` cannot be grouped by email, counted by attempt, or joined to its request id without a regex that breaks the day the message rewords itself. The structured line carries the same information as named members, and every member keeps its name, its type, and its place in an index. The service's rule is the one the access log follows: messages name the event, members carry the data, and no handler interpolates what it could pass as a member.

Redaction is where structure pays for itself. A closed set of keys, password, authorization, cookie, set-cookie, secret, token, is checked case blindly against every member, and a string value under one of them is swapped for [redacted] in the rebuilt state and replaced in the formatted message, so neither State nor Message can carry the secret. Non string values pass through, because the mask is for values a grep can find:

```
        }
        var message = formatter(state, exception) ?? string.Empty;
        var masked = new List<KeyValuePair<string, object?>>();
        var changed = false;
        foreach (var (key, value) in pairs)
        {
            if (value is string text && text.Length > 0 && RedactedKeys.Contains(key))
            {
                message = message.Replace(text, "[redacted]", StringComparison.Ordinal);
                masked.Add(new(key, "[redacted]"));
                changed = true;
            }
            else
            {
                masked.Add(new(key, value));
            }
        }
        if (!changed)
        {
            inner.Log(logLevel, eventId, state, exception, formatter);
            return;
        }
        masked.Add(new("{OriginalMessage}", message));
        var forwarded = masked.ToArray();
```

listing 264: the mask loop: state rebuilt with [redacted], the message replaced member by member (source: csharp-net/api/src/CsharpBook.Api/Observability/RedactingLogger.cs, lines 43-66)

The scope middleware is the second half of the rule. One scope per request carries the request id the edge resolved in chapter 23, so every structured line any handler writes inside the request names the same id the access log and any error envelope quote. One id, one line of grep, every artifact of one request.

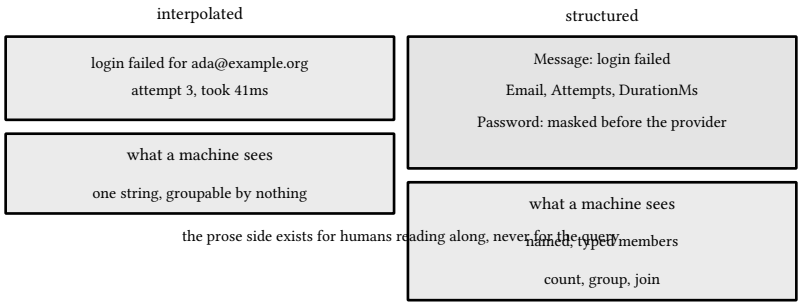


figure 201: the same failure, line-oriented and structured

31.3. tracing from scratch

A trace is the shape of one request's time: nested spans, each with a name, an id, a parent, and a start and end, joined across services by one shared header. The W3C trace context format is deliberately small, four lowercase hex fields, version-traceid-spanid-flags, of 2, 32, 16, and 2 digits, small enough to parse and render by hand, which is what this package does, no sdk and no spans leaving the process. The parser is strict where the spec allows a choice: exact field lengths, version ff reserved and rejected, all-zero ids rejected, uppercase hex rejected because lowercase is canonical, and only the sampled bit of the flags kept, everything else masked to zero exactly as the wire writes it back. The parsed ids are the platform's own ActivityTraceId and ActivitySpanId, the same values the spans below put on the wire, so a parsed header feeds the span without a copy:

```
public static bool TryParse(string? value, out TraceContext context)
{
    context = default;
    if (value is null)
    {
        return false;
    }
    var parts = value.Split('-');
    if (parts.Length ≠ 4
        || parts[0].Length ≠ 2
        || parts[1].Length ≠ 32
        || parts[2].Length ≠ 16
        || parts[3].Length ≠ 2)
    {
        return false;
    }
    foreach (var part in parts)
    {
        foreach (var c in part)
        {
            if (c is not ((≥ '0' and ≤ '9') or (≥ 'a' and ≤ 'f')))
            {
                return false;
```

listing 265: TryParse: four fields, exact lengths, lowercase hex only (source: csharp-net/api/src/CsharpBook.Api/Observability/Traceparent.cs, lines 28-50)

The spans themselves run on ActivitySource, the platform's tracing api, whose native id format is this same W3C format. One middleware opens a server span per request: an inbound header that parses becomes the span's trace id and remote parent with the sampling bit carried, a header absent or malformed starts a fresh trace, because one bad header must never kill a request, and with no listener attached StartActivity answers null and the request is untraced. The collector is the in-process listener: it samples every activity of the service's source and renders the tree, one line per span, children indented two spaces, the same role a collector process plays in production minus the network.

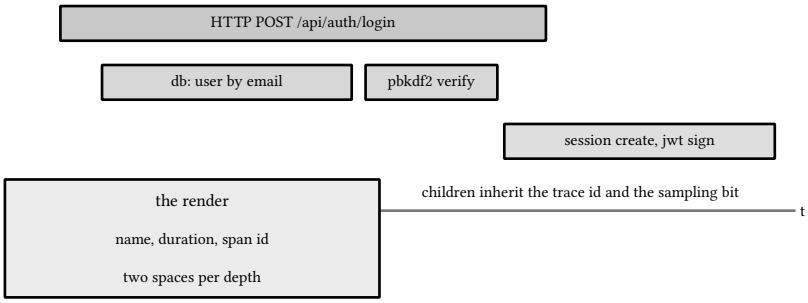


figure 202: one login request's spans over time, the tree the collector prints

31.4. metrics and the prometheus text format

Two series carry the load story, and both are System.Diagnostics.Metrics instruments, the platform surface dotnet-counters also reads over EventPipe. The counter is one series per route and status code, `goapi_requests_total{route,code}`, incremented once per request. The histogram is one per route, `goapi_request_duration_seconds`, with the contract's ladder tight at the low end and geometric through the tail: one, two and a half, five, ten, twenty five, fifty milliseconds, then the hundreds, seconds, and +Inf. The ladder also rides the instrument as histogram advice, the hint external collectors read, while the in-process collector is a MeterListener that buckets every observation itself.

The exposition writer renders the prometheus text format directly from what the listener collected: help and type lines, series sorted by label tuple, bucket lines ascending with +Inf last, zero buckets elided, the one deliberate divergence from client libraries that print every finite bucket at zero as well. The route label is the matched endpoint's route pattern read after the inner pipeline returns, because endpoint routing resolves the endpoint into the context as the request passes through, and the pattern carries no method prefix in this stack, so the label reads /healthz, empty for an unrouted request. The frozen vector pins every byte, one slow /healthz observation rendering exactly the eight lines froze:

```
    public void WriteExposition(TextWriter writer)
    {
        lock (_gate)
        {
            if (_counts.Count > 0)
            {
                writer.Write($"# HELP {RequestsName} {RequestsHelp}\n");
                writer.Write($"# TYPE {RequestsName} counter\n");
                foreach (var ((route, code), value) in _counts.OrderBy(pair => pair.Key.Route)
                    .ThenBy(pair => pair.Key.Code))
                {
                    writer.Write($"{RequestsName}{{route=\"{route}\",code=\"{code}\"}} {value}\n");
                }
            }
            if (_cumulative.Count > 0)
```

listing 266: the exposition: counter family first, series sorted by label tuple (source: csharp-net/api/src/CsharpBook.Api/Observability/MetricsRegistry.cs, lines 171-185)

```
# HELP goapi_requests_total requests by route and code.
# TYPE goapi_requests_total counter
goapi_requests_total{route="/healthz",code="200"} 1
# HELP goapi_request_duration_seconds request latency.
# TYPE goapi_request_duration_seconds histogram
goapi_request_duration_seconds_bucket{route="/healthz",le="+Inf"} 1
goapi_request_duration_seconds_sum{route="/healthz"} <observed>
goapi_request_duration_seconds_count{route="/healthz"} 1
```

listing 267: the frozen vector the exposition is checked against, byte for byte (source: csharp-net/api/contract/testdata/15-metrics.txt, lines 1-8)

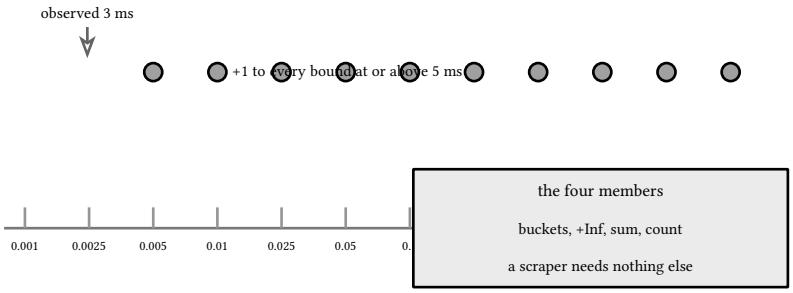


figure 203: cumulative buckets: each observation fills every bound above it

31.5. p99 done honestly

The percentile definition is nearest rank: sort the latencies, take the ceil of p over 100 times n as the rank, answer that rank's value. Its honesty is in the details it refuses to hide: p99 of one hundred values is the ninety ninth smallest, not an interpolation between it and the next, and p99.9 of ten samples clamps to the largest, because a percentile past the data is a bound, not a measurement:

```
    public static double NearestRank(double[] sorted, double p)
    {
        if (sorted.Length == 0)
        {
            return 0;
        }
        var rank = (int)Math.Ceiling(p / 100 * sorted.Length);
        if (rank < 1)
        {
            rank = 1;
        }
        if (rank > sorted.Length)
        {
            rank = sorted.Length;
        }
        return sorted[rank - 1];
```

listing 268: nearest rank: ceil the rank, clamp the ends, no interpolation anywhere (source: csharp-net/api/src/CsharpBook.Api/Observability/Quantile.cs, lines 14-29)

The bucket approximation is the second half of honesty. A scraped histogram cannot answer nearest rank, it can only say which bucket the rank lands in, so FromCumulative answers the smallest bound whose cumulative count reaches the rank, an upper bound on the true value, never the value itself, and not knowable when the rank falls into +Inf. The test pins the contrast: ninety nine samples at 0.9 milliseconds and one at 1.1 gives an exact p99 of 0.9 and a bucket p99 of 1.0, the bound. The load chapter reports both, labeled.

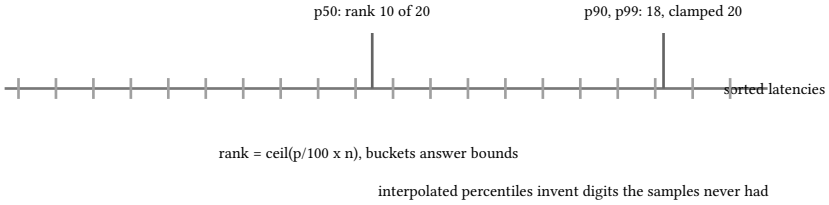


figure 204: the tail ladder: where p50, p90, p99 sit in one sorted sample

31.6. the diagnostics surface

Go mounts pprof on an internal listener and the profiling oracle is an http route. The .net runtime answers the same questions through a different door: the diagnostics ipc channel, the one dotnet-counters, dotnet-trace, and dotnet-dump speak, armed by default in every process. Which profile answers which question is unchanged, cpu by stack through dotnet-trace, live objects by type through dotnet-gcdump, the heap through dotnet-dump, and this service's own meter instruments stream to dotnet-counters over the same channel with no code beyond this chapter's instruments. The placement ruling changes shape, not meaning: a profile is an oracle, so the channel belongs to the operator, and DOTNET_EnableDiagnostics=0 closes it at the runtime level before any code runs, the equivalent of never mounting the profiling route publicly. The metrics exposition is the one surface this service exposes on purpose, on the internal listener only, its address on the same config hub:

```
    public static DiagnosticsConfig FromEnvironment()
    {
        bool? ipc = null;
        if (Environment.GetEnvironmentVariable("DOTNET_EnableDiagnostics") is { } ipcRaw
            && int.TryParse(ipcRaw, out var ipcValue))
        {
            ipc = ipcValue != 0;
        }
        return new DiagnosticsConfig
        {
            IpcEnabled = ipc,
            MetricsAddress = string.IsNullOrWhiteSpace(
                Environment.GetEnvironmentVariable("GOAPI_METRICS_ADDR")
            )
                ? "http://127.0.0.1:6090"
                : Environment.GetEnvironmentVariable("GOAPI_METRICS_ADDR")!,
        };
    }
}
```

listing 269: the placement ruling as config: the ipc switch and the metrics bind address (source: csharp-net/api/src/CsharpBook.Api/Observability/DiagnosticsConfig.cs, lines 26-44)

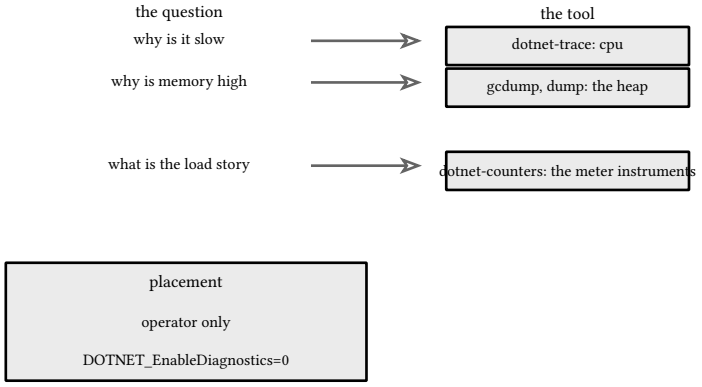


figure 205: which tool answers which question, all over one ipc channel

31.7. one request fully observed

Wire the signals in the order the request meets them and one request tells the whole story. The tracing layer wraps the stack outermost of the family, so the span covers the scope, the handler, and the metrics tick. The request id scope sits beneath the access log of chapter 23, so the scope exists before any handler line is written. The metrics layer wraps routing and ticks in a finally block, so the counter records the status even when the handler throws. Reading it back is what the tests do: the trace test asserts the span continued the inbound header, the scope test asserts the id on the line, and the one request test chains all three layers over a routed context and asserts every signal observed the same request. The instruments the diagnostics tools read are the ones the exposition renders, so dotnet-counters and the scrape agree because they read one hub:

```
    /// container with the host. The middleware stack order the chapters
    /// pin reads: tracing outermost of the family, then the request id
    /// scope beneath the access log, then metrics around routing, and
    /// the metrics route mapped unauthenticated on the internal
    /// listener.
    /// </summary>
    public static IServiceCollection AddBookObservability(this IServiceCollection services)
    {
        services.AddSingleton<MetricsRegistry>();
        return services;
    }
}
```

listing 270: the family's one registration: the metrics hub, the span and scope layers themselves installed at the stack's chapter order in wire (source: csharp-net/api/src/CsharpBook.Api/Observability/WireObservability.cs, lines 10-22)

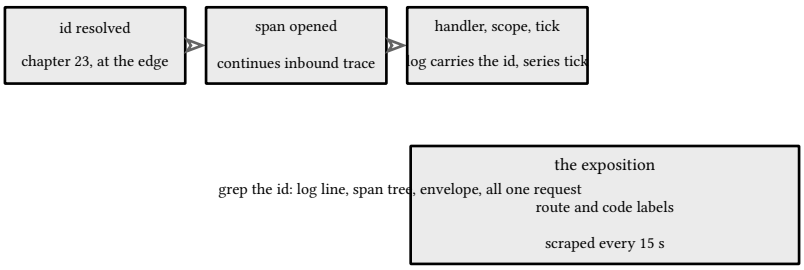


figure 206: one request id through every signal

sources: learn.microsoft.com for ILogger and the console log formatting model with the json formatter and custom formatters, ActivitySource and ActivityContext with the W3C id format and the AllDataAndRecorded sampling result, System.Diagnostics.Metrics with the MeterListener callbacks and histogram advice, and dotnet-core-diagnostics for the ipc channel and the switch, all accessed 2026-09-26, plus the W3C trace context format at w3c.github.io/trace-context and the prometheus exposition format documentation. Verified by the `CsharpBook.Api.Observability` tests, 35 of them under `dotnet test Api.slnx`, with the metrics exposition replayed line for line from `contract/testdata/15-metrics.txt`.

32. load and p99

A service that has never been loaded is a service whose p99 is a guess, and a deploy that has never been drained is a deploy that drops requests every time it ships. This chapter closes the service part with the tools that make both claims checkable: the closed-loop and open-loop harness over an injected clock and delayer, the report table and its honest percentiles, the benchmark discipline with its committed capture, graceful shutdown over the host's lifetime events, the admission gate that answers overload at the cap, and the task leak guard that holds the harness itself honest after a run.

32.1. the load harness

The two loops differ in what they hold constant. The closed loop holds concurrency constant: worker tasks, each firing one request at a time and starting the next only after the previous response lands, so offered load is whatever the host can keep up with and never exceeds the worker count. It measures a system at a chosen concurrency. The open loop holds arrival rate constant: request i is scheduled at i over rate seconds past the start, fired whether or not anything has come back, so when the host slows, requests pile up exactly the way they do in production, and the cliff becomes visible. Little's law from chapter 30 predicts the pile: in-flight equals arrival rate times service time, and an open loop at fixed arrival rate turns rising latency directly into rising in-flight count.

Both loops share one harness with three injected seams: the client, so tests drive deterministic responses through a stub transport that advances the clock one millisecond per request and never opens a socket, the clock, so latency is arithmetic on ticks, and the delayer, so the open loop's pacing is a function the test can call instantly while still asserting the schedule. Warmup runs first on every loop and never touches the sink, because the first request pays for the dial and the cold caches, and none of that is the number the run exists to measure:

```
    public async Task<Result> ClosedAsync(
        int workers,
        int perWorker,
        int warmup,
        CancellationToken cancellation = default
    )
    {
        for (var i = 0; i < warmup; i++)
        {
            await FireAsync(record: null, cancellation).ConfigureAwait(false);
        }
        var sink = new Sink();
        var start = Clock.GetUtcNow();
        var tasks = new Task[workers];
        for (var w = 0; w < workers; w++)
        {
            tasks[w] = RunWorkerAsync(sink, perWorker, cancellation);
            if (Tracker is not null)
            {
                // tracked, never awaited here: the task itself is the
                // loop's member, Track only arms the count
                _ = Tracker.Track(tasks[w]);
            }
        }
        await Task.WhenAll(tasks).ConfigureAwait(false);
        return sink.ToResult(Clock.GetUtcNow() - start);
```

listing 271: the closed loop: warmup first, one request at a time per worker, elapsed from the injected clock (source: csharp-net/api/src/CsharpBook.Api/Load/LoadHarness.cs, lines 79-104)

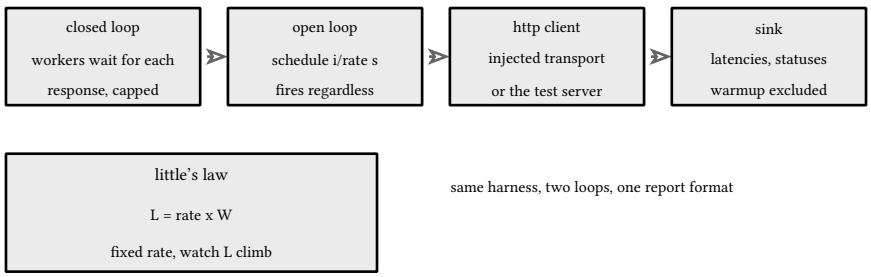


figure 207: workers, host, latency sink: one harness, two loops

32.2. the report

One run folds into one row. The percentiles are nearest rank, the corpus definition from chapter 31, computed over the sorted latencies, and the achieved rate is the recorded count over the elapsed span the injected clock measured. Rendering pads fixed columns with the unit in the header, because a latency column that does not say milliseconds is a unit error waiting for a reader.

The golden test pins the bytes from a fixed latency set, and it pins them twice. The first row summarizes the raw samples. The second summarizes the same samples as a scraper would see them, cumulative histogram counts, exact count and sum, percentiles read out of the buckets, and the row says so in its source column: six latencies of 1, 1, 2, 3, 5, and 8 milliseconds give an exact p50 of 2 and p99 of 8, while the bucketed row reads p50 as 2.5 and p99 as 10, the bounds the ranks land in. Two rows, same run, the difference is the resolution a histogram throws away, stated in the table instead of hidden in a footnote:

```
public static LoadReport Summarize(string name, LoadHarness.Result result)
{
    var report = new LoadReport(name, result.N, result.Errors, 0, 0, 0, 0, 0, 0, "exact");
    if (result.Latencies.Count == 0)
    {
        return report;
    }
    var sorted = result.Latencies.Order().ToArray();
    var total = TimeSpan.Zero;
    foreach (var latency in sorted)
    {
        total += latency;
    }
    var seconds = ToSeconds(sorted);
    report = report with
    {
        MeanMs = Ms((total / sorted.Length).TotalSeconds),
        P50Ms = Ms(Quantile.NearestRank(seconds, 50)),
        P90Ms = Ms(Quantile.NearestRank(seconds, 90)),
        P99Ms = Ms(Quantile.NearestRank(seconds, 99)),
        MaxMs = Ms(seconds[^1]),
    };
    return result.Elapsed > TimeSpan.Zero
        ? report with { Rps = result.N / result.Elapsed.TotalSeconds }
        : report;
}
```

listing 272: Summarize: nearest rank over the sorted latencies, the source column labels every approximation (source: csharp-net/api/src/CsharpBook.Api/Load/LoadReport.cs, lines 28-53)

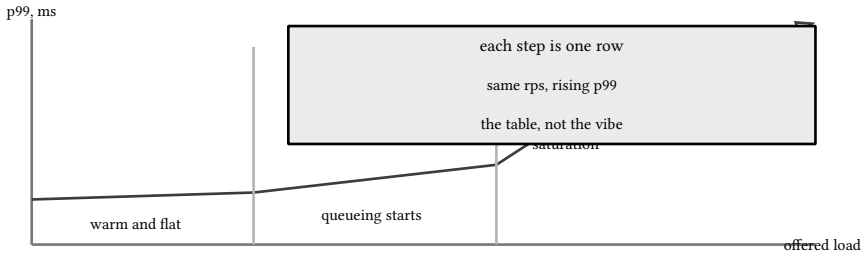


figure 208: p99 under offered-load steps: where the cliff shows in the table

32.3. the benchmark discipline

A single benchmark number is weather, the profiling chapter of the language part already said so, and this part's discipline is the same one: measure both sides over many iterations, commit the capture, quote it with its date and machine. The language part's benchmark harness is BenchmarkDotNet in the samples tree, and the suite chapter carries the full story of why a vehicle that is stdlib exclusive by ruling runs its book-side benchmarks there and its vehicle-side measurements here: a separate csproj outside the solution is not this chapter's to add, so the leg is an xunit fact tagged [Trait("bench", "true")], runnable on demand with a filter, and the capture is a hand-run of it committed beside the code.

The two sides are the route bare, one write through the kernel's json path, and the same route under the cross-cutting stack this part built, tracing with a collector attached so spans cost real work, the request id scope, metrics with the histogram tick, admission, and drain counting. The leg asserts only the structural claim, that the observed stack costs more than the bare route, and prints the numbers, and the capture quotes them:

```
# stack cost capture, 2026-09-26, hand-run of the bench leg
# command: dotnet test -c Release --filter "FullyQualifiedName~StackBenchLeg" --logger
"console;verbosity=detailed"
# vehicle: books/csharp-net/api, net 11 rc, this machine
# the leg: 20 000 iterations per row after 2 000 warmup, bare kernel
# write path against the same write under tracing, id scope, metrics,
# admission, and drain counting, collector attached so spans cost real
# work. Three runs, raw numbers, no statistics applied.
run 1
bare     1929.2 ns/op
observed 5314.0 ns/op
stack    3384.8 ns/op
run 2
```

listing 273: the committed capture, 2026-09-26, a hand-run of the trait-filtered leg, this machine (source: csharp-net/api/tests/CsharpBook.Api.Tests/Load/captures/2026-09-26-bench.txt, lines 1-12)

Reading it: the bare route answers in about 1.9 to 2.1 microseconds with a spread under ten percent, the observed route in about 5.0 to 5.3 microseconds, roughly 2.5 times, which is the per-request price of every cross-cutting chapter combined. Five microseconds is also the right scale check: the stack costs microseconds against a login that costs tens of milliseconds of pbkdf2, and knowing which costs dominate is the whole skill.

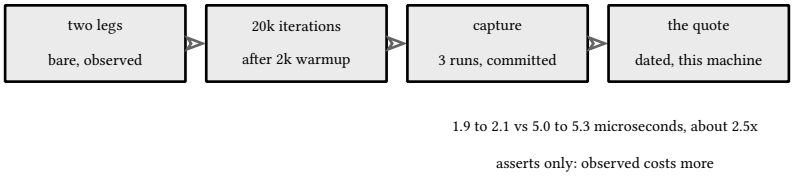


figure 209: the bench flow: leg, capture, quote

32.4. graceful shutdown

Shutdown has an order, and the order is the contract. Readiness flips first: the drainer's state moves to draining, /readyz answers its 503 overload envelope, and every load balancer watching the service stops routing new requests here while the old ones are still being answered well. Then the host stops the server, which refuses new connections at the transport level. Then the drain waits for in-flight requests to finish under a grace deadline, and reports the deadline instead of pretending the drain succeeded when grace expires with work still inside, so the caller decides whether that is fatal.

The platform hook is IHostApplicationLifetime. The host raises ApplicationStopping before it stops the server and waits for the registered callbacks under its own shutdown timeout, which is what makes the drain run while the listener is still answering the in flight requests, and the drainer's readiness probe re-binds the kernel's config hub, last registration wins, so the frozen /readyz route of chapter 22 answers the drain state without a line of its own changing:

```
    public static IApplicationBuilder UseBookDrain(
        this IApplicationBuilder app,
        TimeSpan? grace = null
    )
    {
        var drainer = app.ApplicationServices.GetRequiredService<Drainer>();
        var lifetime = app.ApplicationServices.GetRequiredService<IHostApplicationLifetime>();
        var deadline = grace ?? Drainer.DefaultGrace;
        lifetime.ApplicationStopping.Register(() =>
        {
            drainer.BeginDrain();
            // blocking on ApplicationStopping is the sanctioned host
            // pattern: the host waits for the registered callbacks
            // before it stops the server and closes the listener
            _ = drainer.DrainAsync(deadline).GetAwaiter().GetResult();
        });
        return app.UseMiddleware<DrainMiddleware>(drainer);
    }
}
```

listing 274: the host hook: flip readiness on ApplicationStopping, drain under grace before the server stops (source: csharp-net/api/src/CsharpBook.Api/Load/WireLoad.cs, lines 48-66)

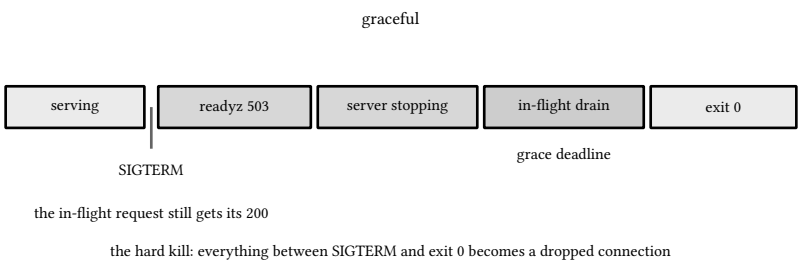


figure 210: SIGTERM to drain to exit, against the hard kill

32.5. testing shutdown without sleeps

The drain tests are observations on gated state, and none of them wait on a timer. The slow request blocks on a completion source the test holds. BeginDrain flips readiness synchronously, no yield needed. DrainAsync is observed pending while the gate holds, then the release completes the source, the in-flight count reaches zero, and the drain returns true on the same asynchronous flow. The grace expiry test runs DrainAsync with a zero deadline over a held request and asserts the false it owes, and the empty-service test drains instantly because there is nothing inside. The watchdog guards are WaitAsync timeouts on real state, not sleeps:

```
    public async Task<bool> DrainAsync(TimeSpan? grace = null, CancellationToken cancellation = default)
    {
        var deadline = grace ?? DefaultGrace;
        try
        {
            await _drained.Task.WaitAsync(deadline, cancellation).ConfigureAwait(false);
        }
        catch (TimeoutException)
        {
            lock (_gate)
            {
                _state = DrainState.Closed;
            }
            return false;
        }
```

listing 275: DrainAsync: wait on real in-flight state under a deadline, report the expiry (source: csharp-net/api/src/CsharpBook.Api/Load/Drainer.cs, lines 104-118)

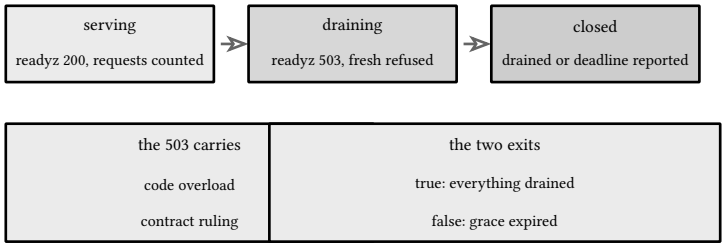


figure 211: the drain lifecycle: serving, draining, closed

32.6. admission control

The rate limit of chapter 30 bounds arrivals over time. The admission gate bounds requests inside the service at once, and the mechanism is a counting semaphore: one SemaphoreSlim sized to the capacity, a nonblocking Wait to take a slot, the release riding a finally block, so a handler that throws still releases. The nonblocking wait is the whole policy: a slot is taken when one is free, and when the gate is full the answer is immediate, the contract's overload envelope at 503, never a queue, because a queued request holds memory and a connection the service just proved it cannot spend. The capacity is the deployment's honest concurrency, worker and memory budget, not what a client would like.

The barrier test pins exactly who gets in: a gate of four, four holders each blocked after entering, observed through their own completion sources, InFlight reads exactly the cap, four synchronous arrivals all answer the envelope, the barrier opens and the holders finish. Little's law closes the loop: at a fixed arrival rate the gate caps L, and capped L with rising service time means the excess is refused at the door instead of queueing inside:

```
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        if (!_slots.Wait(0))
        {
            await context.Response.WriteJsonAsync(
                Kernel.ErrorCodeStatus.Of(ErrorCode.Overload),
                new Envelope(ErrorBody.From(
                    ApiError.Fail(ErrorCode.Overload, OverloadMessage),
                    RequestId.Resolve(context))),
                context.RequestAborted);
            return;
        }
        try
        {
            await next(context);
        }
        finally
        {
            _slots.Release();
        }
    }
}
```

listing 276: the gate: nonblocking take, finally release, overload envelope at the cap (source: csharp-net/api/src/CsharpBook.Api/Load/AdmissionGate.cs, lines 31-52)

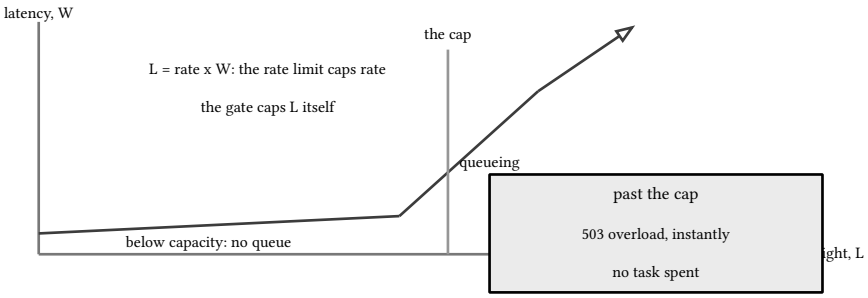


figure 212: latency versus in-flight: the hockey stick the gate cuts

32.7. the task leak check

A load tool that leaks tasks per run silently slows every later run and lies about the service it measures. The go lane asks its runtime: the goroutine leak profile names the goroutines that are blocked and unreachable, which is the definition of a leak. This lane asks the harness itself, because the runtime offers no equivalent profile: the harness tracks every task it starts through the counting tracker, the count drops the moment each task completes by whatever outcome, and the leak test runs one real harness pass against the in-process test server, then asserts the count is zero and the tracker's idle task is done, the WaitAsync guard a watchdog, not a sleep. On failure the outstanding count is the number to read: a tool that fire and forgets its workers reports it immediately:

```
    public Task Track(Task task)
    {
        Interlocked.Increment(ref _outstanding);
        _ = DecrementOnCompletion(task);
        return task;
    }

    private async Task DecrementOnCompletion(Task task)
    {
        try
        {
            await task.ConfigureAwait(false);
        }
        catch
        {
            // a faulted or canceled task still counts as finished, the
            // leak question is whether anything is still running
        }
        if (Interlocked.Decrement(ref _outstanding) == 0)
```

listing 277: the tracker: count up at start, count down on completion, idle completes at zero (source: csharp-net/api/src/CsharpBook.Api/Load/TaskTracker.cs, lines 26-44)

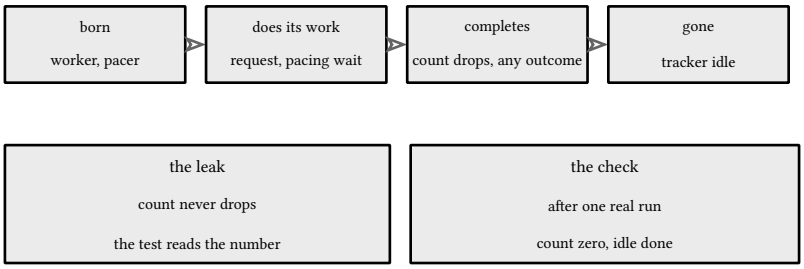


figure 213: a task lifecycle: born, joined, or leaked

sources: learn.microsoft.com for IHostApplicationLifetime and its ApplicationStopping semantics with the host shutdown timeout, for SemaphoreSlim's nonblocking Wait and its counter, and for Task.WaitAsync as a deadline on real work, all accessed 2026-09-26, plus the little's law statement as used across the corpus. Verified by the `CsharpBook.Api.Load` tests, 14 of them under `dotnet test Api.slnx`, one of them the bench leg whose committed capture is quoted above from a Release hand-run on this machine, a 12th gen i7-12700F under windows, plus `dotnet format` over the files the chapter added.

33. ship it

The solution ends as a directory of tests unless it also ships. This chapter turns the service into an artifact and a lane: a multi-stage image, one compose command that brings it up healthy, a make gate replaying the contract over that stack, a CI workflow running the same legs on every push, and the two envelopes platforms ask for, a helm chart and a terraform slice. Everything leans on machinery the earlier chapters built, and on the composition table this chapter moved into `Deploy/Wire.cs`. One rule disciplines the whole chapter: the compose lane is the only thing that executes, and the chart, the terraform, and the workflow ship as taught copies.

33.1. the image

Two stages, one boundary. The builder is the sdk image pinned to the book's exact rc version, `sdk:11.0.100-rc.1`, which exists only to run the compiler, and the runtime is the matching `aspnet:11.0.0-rc.1`, which exists only to run the app. The copy across the boundary is the publish output. Go's chapter crossed with one static file into scratch, legal because its store engine was pure go. This service's store is sqlite through the native bundle, a shared library the runtime must carry, so scratch is not available and the aspnet base is the floor.

Four decisions earn a second look. The csproj copy precedes the source copy so the restore layer caches independently of code edits. The `/data` directory is created in the builder and copied with an owner, because a named volume inherits image ownership on first mount and a volume over a root-owned directory would deny the container user its database. The user is numeric, 1654, the uid the base image ships as `app`. And the `GOAPI_` names carry over from the go lane deliberately: the contract pins the `goapi_` metric series and the token issuer, so one parameter family and one frozen contract serve both lanes, each with its own compose file:

```
# aspnet 11.0.0-rc.1 to run, the same runtime the sdk carries.
FROM mcr.microsoft.com/dotnet/sdk:11.0.100-rc.1 AS build
WORKDIR /src
COPY Api.slnx ./
COPY src/CsharpBook.Api/CsharpBook.Api.csproj src/CsharpBook.Api/CsharpBook.Api.csproj
RUN dotnet restore src/CsharpBook.Api/CsharpBook.Api.csproj
COPY . .
RUN dotnet publish src/CsharpBook.Api/CsharpBook.Api.csproj \
    -c Release -o /out --no-restore \
    && mkdir /data && chown 1654:1654 /data

# aspnet runtime: the framework, the published app, an empty /data the
# non-root user owns so the store volume mounts writable on first run.
# the sqlite bundle is native, so a scratch runtime cannot carry the
# service and the base image is the floor.
FROM mcr.microsoft.com/dotnet/aspnet:11.0.0-rc.1
ARG VERSION=dev
ENV GOAPI_VERSION=${VERSION}
WORKDIR /app
COPY --from=build --chown=1654:1654 /out /app
COPY --from=build --chown=1654:1654 /data /data
USER 1654:1654
EXPOSE 8080
ENTRYPOINT ["dotnet", "CsharpBook.Api.dll"]
```

listing 278: the whole image: sdk builder, aspnet runtime, user 1654, the version arg (source: csharp-net/api/Dockerfile, lines 4-27)

The healthcheck has the shape problem the go chapter met in reverse: the aspnet image carries no curl and no wget, so the answer is the one executable that does exist, the service itself. `ShipProbe` accepts `--ready-probe`, resolves the address from `ASPNETCORE_URLS`, or the image default `ASPNETCORE_HTTP_PORTS`, rewrites the host to loopback, and asks its own `/readyz`. Ready is 200 and only 200, a 503 still means do not route traffic here. The build context

keeps one honest difference: this vehicle has bin and obj trees that would overwrite the container's restore, so a minimal `.dockerignore` excludes them and the non-service trees, the trade the go chapter stated instead of paying because a go module has no build outputs.

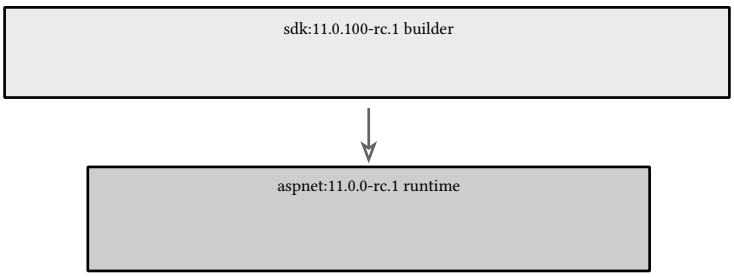


figure 214: two stages, one boundary: the builder dies with the build, aspnet runs the app

33.2. compose one command

The compose file is one service and its volume, and the contract it enforces is the return value of one command. `docker compose up -d --build --wait` builds, creates, starts, and blocks until the healthcheck passes, so a green up means the image compiled, the migrations ran as a hosted startup step against a fresh volume, and the listener answers probes. `down -v` removes the volume in the same breath, so every lane run starts from migration zero:

```yaml
services:
  api:
    build: .
    environment:
      ASPNETCORE_URLS: http://+:8080
      GOAPI_JWT_SECRET: dev-only-not-a-secret
      GOAPI_DB: /data/goapi.db
      # the metrics listener, bound to every interface inside the
      # container and published for the lane's assertions, never in
      # front of the api
      GOAPI_METRICS_ADDR: http://+:6090
    # the container's contract ports stay 8080 and 6090, the host
    # side moves off them: shared dev machines carry loopback shadows
    # on 8080 (a wsl-mirrored listener wins the race for some
    # clients), and the lane must be the only thing under its ports
    ports:
      - "18080:8080"
      - "16090:6090"
    volumes:
      - data:/data
    healthcheck:
      test: ["CMD", "dotnet", "CsharpBook.Api.dll", "--ready-probe"]
      <<: *hc
```

listing 279: the whole system: env, both listeners, the volume, the probe, host ports off 8080 (source: csharp-net/api/compose.yml, lines 14-36)

The environment block carries exactly what construction demands. The jwt secret must be present or the composition refuses to start in production, loudly: a missing secret is a crash the orchestrator restarts and an operator reads, never a running service signing tokens with an empty key. The value in the file is a development constant, and both platform envelopes exist partly to replace it. The second listener is the same discipline for the observability surface, `GOAPI_METRICS_ADDR` binds a second kestrel endpoint for the exposition, unpublished by every production shape, and the host side of both mappings sits on 18080 and 16090, because shared dev machines carry

loopback shadows on 8080 and a lane must be the only thing under its ports. Compose up is the only command that touches the schema:

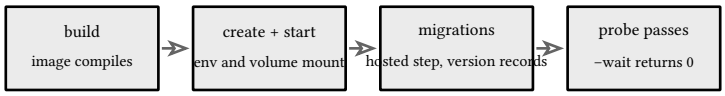


figure 215: what a green up proves, in order

33.3. the docker lane as a gate

The lane is one make target, mirroring the go lane's two-gate rule: the plain solution gate never needs a daemon, and the docker lane never runs silently, a missing daemon a loud red:

```
verify-csapi-docker:
  @docker info >/dev/null 2>&1 || { echo "docker daemon not reachable." >&2; exit 1; }
  @cd books/csharp-net/api && docker compose down -v >/dev/null 2>&1 || true && \
    trap 'docker compose down -v' EXIT && \
    docker compose up -d --build --wait && \
    CSHARPAPI_URL=http://127.0.0.1:18080 \
    CSHARPAPI_METRICS_URL=http://127.0.0.1:16090 \
    dotnet test integration/CsharpBook.Api.IntegrationTests.csproj; rc=$$?; exit $$rc
```

fragment, illustrative, not executed

Two environment families live in this lane and never mix. The `GOAPI_` names are the vehicle's config surface, the same keys in compose, the chart, and the terraform slice, kept for go-lane parity. The `CSHARPAPI_URL` and `CSHARPAPI_METRICS_URL` names are the lane's own probe coordinates, read by the integration suite and set by the make target and the ci job, never by the service. The split guards a failure mode: renaming one family without the other makes the lane silently probe its defaults, a green suite pointed at nothing, so the two lists change together or not at all.

The integration project lives outside `Api.slnx`, the faithful mirror of go's docker build tag: plain `dotnet test Api.slnx` never sees these files, and the CI module job compiles the project every push so the suite cannot rot behind its lane. The suite replays the contract's golden scenarios over the real listener with a real clock, real uuids, and real wiring, asserting the layer under the frozen bytes. The ETag rule is the sharpest example, a strong ETag is the sha256 of the exact response body, so the suite recomputes the hash over the wire bytes and compares:

```
    private static string WantStrongETag(Result result)
    {
        var etag = Header(result, "ETag")?.Trim('"');
        Assert.False(string.IsNullOrEmpty(etag), $"no ETag header, body {result.Body}");
        var sum = System.Security.Cryptography.SHA256.HashData(
            System.Text.Encoding.UTF8.GetBytes(result.Body)
        );
        var want = System.Convert.ToHexStringLower(sum);
        Assert.True(
            string.Equals(etag, want, StringComparison.Ordinal),
            $"ETag {etag} is not the sha256 of the body {result.Body}"
        );
        return etag;
    }
```

listing 280: the etag check recomputed over live bytes, not frozen ones (source: csharp-net/api/integration/ApiLaneTests.cs, lines 111-124)

Six tests cover the contract's scenarios: both probes and the two routing envelopes, the register replay login walk, the etag duel with its one 200 and one 412, the rate limit drain to a 429, the reports idempotent pair, and the metrics exposition off the internal listener. The lane also taught two platform facts the in-process suites could not see: kestrel disallows synchronous io by default, and the in-memory test host polices it just the same, a synchronous read of the request body and a synchronous write of the response both throw at its default. The tests had stayed green only because the sync read ran over `DefaultHttpContext`, the bare unit recorder with no body control feature, and the harness had not yet mounted the gate, so the exposition's synchronous write and the login bucket's synchronous body read died on the real listener, fixed with scoped `AllowSynchronousIO` flips both hosts honor, and `Uri` has no plus operator on this runtime, so a base plus a path string concatenates into a double slash the router answers with a route miss:

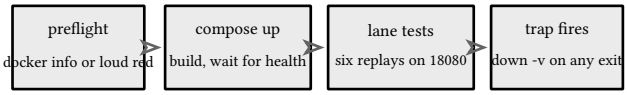


figure 216: the gate: one lane, one trap, no skips

33.4. ci on GitHub Actions

The workflow is the same gate wearing a badge: the solution legs exactly as the repository runs them locally, format verification, the full suite, the integration compile check. The path filter keeps it out of commits touching nothing under the service tree, and the setup action pins the same rc sdk the book builds under:

```yaml
jobs:
  module:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: books/csharp-net/api
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-dotnet@v6
        with:
          dotnet-version: "11.0.100-rc.1.26425.128"
      - run: dotnet format Api.slnx --verify-no-changes
      - run: dotnet test Api.slnx
      # the integration suite lives outside the solution, the mirror of
      # go's docker build tag, so a compile check keeps it from rotting
      # behind its lane
      - run: dotnet build integration/CsharpBook.Api.IntegrationTests.csproj
```

listing 281: the module job: the local legs plus the lane-project compile check (source: csharp-net/api/deploy/ci/ci.yml, lines 13-29)

The image job proves the artifact builds clean, one docker build with the commit sha as the `VERSION` argument. It pushes nowhere: a registry and a release policy are a repository decision.

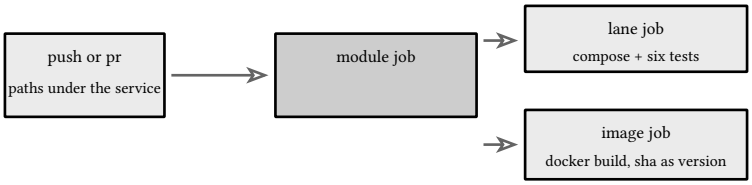


figure 217: push to ci to image: three jobs, one dependency edge

NOTE: why a taught copy

> The workflow lives at `api/deploy/ci/ci.yml`, not `.github/workflows/`, because this repository keeps no `.github` directory and corpus ci is not ratified. The file is byte for byte what a service repository would carry, and the make lane runs.

33.5. helm the service envelope

The chart is the service wearing kubernetes. `Chart.yaml` declares the name and version pair, `values.yaml` is the entire configuration surface a reviewer diffs, and the templates render the four objects the platform needs. The deployment carries the service's own contracts: the probes split along the line chapter 32 drew, liveness on `/healthz` gets a restart, readiness on `/readyz` gets a pull from the endpoints list, which is how a draining pod cooperates with a rolling deploy:

```
      securityContext:
        runAsNonRoot: true
        # the numeric uid of the aspnet base image's app user, so the
        # platform and the image agree on who runs the process
        runAsUser: 1654
        fsGroup: 1654
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          ports:
            - name: http
              containerPort: 8080
          env:
            - name: ASPNETCORE_URLS
              value: http://+:8080
            - name: GOAPI_JWT_SECRET
              valueFrom:
                secretKeyRef:
                  name: {{ .Release.Name }}-csharpapi
                  key: jwt-secret
          livenessProbe:
            httpGet:
              path: {{ .Values.probes.liveness }}
              port: http
```

listing 282: the container: numeric user matching the image, the secret by reference, both probes (source: csharp-net/api/deploy/helm/csharpapi/templates/deployment.yaml, lines 20-43)

Three details match the image on purpose. The securityContext repeats the numeric uid, 1654. The jwt secret arrives through a `secretKeyRef`, never a values entry, keeping the reviewable file free of credentials. And the resources block is templated from values because the scheduler and the autoscaler both read it, and a service without requests is one the cluster cannot schedule honestly. The chart publishes only the main port, the kubernetes shape of the internal-listener ruling:

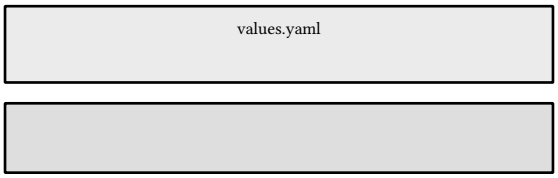


figure 218: the chart's layers: values flow down, objects render out

33.6. terraform for this service

Terraform the language, module structure, and the provider catalog belong to the cloud book. This file is the service's slice of that practice, one fargate service behind one load balancer, one alarm on the tail, a decision record more than a lesson in the tool:

```
container_definitions = jsonencode([
  {
    name      = var.name
    image     = var.image
    essential = true
    portMappings = [
      { containerPort = 8080, protocol = "tcp" }
    ]
    environment = [
      # the same binding compose sets, the platform's address shape
      # instead of a go listen string
      { name = "ASPNETCORE_URLS", value = "http://+:8080" },
      { name = "GOAPI_DB", value = "/data/goapi.db" }
    ]
    secrets = [
      { name = "GOAPI_JWT_SECRET", valueFrom = aws_ssm_parameter.jwt_secret.arn }
    ]
    readonlyRootFilesystem = false # the store engine writes /data
    logConfiguration = {
      logDriver = "awslogs"
```

listing 283: the container definition: image, env, the secret by reference (source: csharp-net/api/deploy/terraform/ecs.tf, lines 88-107)

The task definition runs the same image compose runs, with awsvpc networking and the secret pulled from ssm at container start. The target group probes `/healthz` with a deregistration delay of 30 seconds, the drain window chapter 32's shutdown owns the inside of, and `desired_count` defaults to 1 because the store is one file set on the task's volume and two tasks would be two databases wearing one name. The alarm reads the tail off the platform: cloudwatch computes the p99 extended statistic for target response time, so the pager watches the same number the load chapter defends:

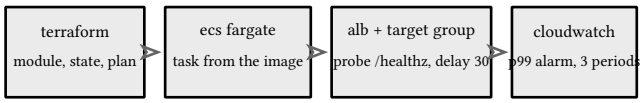


figure 219: apply to ecs to alb: the service's aws slice in one line

33.7. the shipped checklist

The lane ends where it started, with each chapter's property, and the fold added one artifact: the composition table, `Deploy/Wire.cs`, one writer, chapter order, every family in one file the way go's `wire.go` is, `Program.cs` naming it twice and holding nothing else. The listing is the table's second half, every route wired:

```
        app.MapKernel(); // ch22: healthz and readyz
        app.MapUserEndpoints(app.Services.GetRequiredService<UserFamily>()); // ch24
        app.MapAuthnEndpoints(app.Services.GetRequiredService<AuthnService>()); // ch25
        app.MapRoleEndpoints(app.Services.GetRequiredService<IUserStore>()); // ch26
        // ch24's reports pair rides the store family: the report
        // computes over the same sqlite engine everything else uses
        if (app.Services.GetService<StoreDb>() is { } reports)
        {
            app.MapReportEndpoints(
                new ReportsFamily(
                    reports,
                    new IdempotencyGuard(reports, clock),
                    clock,
                    NewId
                )
            );
        }
        app.MapMetrics(); // ch31: the exposition, unauthenticated
    }
```

listing 284: every route wired, chapter labeled, one table (source: csharp-net/api/src/CsharpBook.Api/Deploy/Wire.cs, lines 107-125)

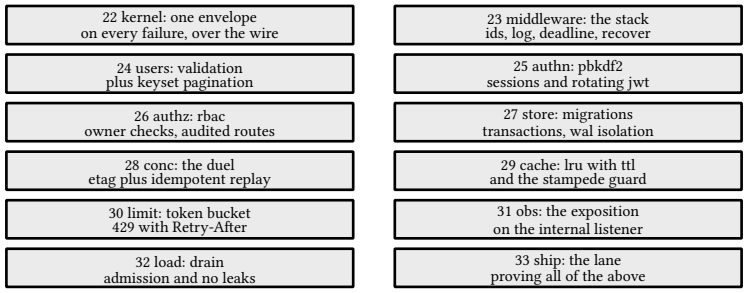


figure 220: the readiness table: twelve chapters, twelve properties, all proven

The chapter declares what it does not teach again: docker's storage model, the runtime, and networking are the infrastructure book's, and the terraform workflow lives in the cloud book. What this chapter owns is the narrow band between a green solution and a running service: the image, the command, the gate, and the two envelopes.

sources: the container tags verified against the registry itself, sdk 11.0.100-rc.1 and aspnet 11.0.0-rc.1, the base image's app user at uid 1654 and its ASPNETCORE_HTTP_PORTS default read off the pulled image, docker docs at docs.docker.com, the setup-dotnet releases at github.com/actions/setup-dotnet where v6 is current, the kubernetes api reference, and the aws provider 6 docs, all accessed 2026-09-26. Verified by `make verify-csapi-docker` end to end, all six wire tests green over the real image, the plain suite green at 379.

34. the test suite

The service part ends with the thing that keeps it honest. This chapter reads the suite as a whole: what it already holds, the property targets that turn random inputs into invariants, the cache driven as a state machine against a reference model, forgery resistance as a loop over mutations, the strict hand-rolled doubles and the libraries the vehicle deliberately declines, the xunit surface and the truths it hides, and the coverage lane that stays opt-in. One ruling from the contract governs all of it: the platform's native randomized mechanism, which for c# is the seeded `Random` loop, so every property here is deterministic by construction and zero tests sleep.

34.1. what the suite already holds

The solution carries 379 tests under `Api.slnx` and 6 wire tests in the integration project outside it. The families own their slices: the kernel's byte-identity pins, the middleware stack order, the resource and auth lanes replaying the frozen vectors in process, the store's migration and isolation suites, the concurrency duel, the cache, the limiter, the observability exposition, and the deploy probe. The property folder added by this chapter holds 18 more. The support doubles consolidated here to one of each kind: one stepped clock, one vector loader, one recording logger, scripted sources that fail on overdraw, a strict response-feature double, and the shared harness that boots composed slices of the app in memory.

The consolidation itself is a chapter ruling: two clocks and two vector loaders had accumulated across lanes, and the suite keeps one of each. `SteppedTimeProvider` is the clock that stayed, because its timers fire inside `Advance`, and the plain two-line `StepClock` folded into it, every call site migrated mechanically. A test that needs time to pass performs one addition, and a timer that the production code armed through `TimeProvider.CreateTimer` fires on the caller's thread, deterministically, in schedule order:

```
    /// <summary>Moves the clock and fires every timer whose due time
    /// has arrived, in schedule order, exactly once.</summary>
    /// <summary>One whole-number step, the common convenience.</summary>
    public DateTimeOffset AdvanceSeconds(long seconds) => Advance(TimeSpan.FromSeconds(seconds));

    public DateTimeOffset Advance(TimeSpan by)
    {
        List<ManualTimer> due;
        lock (_gate)
        {
            _now += by;
            due = [.. _timers.Where(t => t.IsDue(_now)).OrderBy(t => t.DueAt)];
            foreach (var timer in due)
            {
                timer.MarkFired(_now);
            }
        }
        foreach (var timer in due)
        {
```

listing 285: the one clock: time moves by addition, timers fire inside the step (source: csharp-net/api/tests/CsharpBook.Api.Tests/Support/SteppedTimeProvider.cs, lines 27-46)

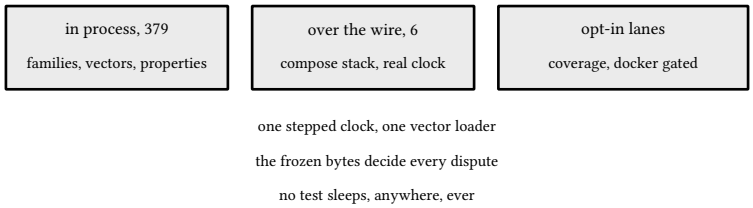


figure 221: the suite's shape: three legs, one contract, zero sleeps

34.2. properties as seeded theories

C# has no native fuzzing hook in the sdk test stack, so the platform-native answer is the seeded pseudorandom loop: a property is one `[Theory]` whose inputs are seeds, each seed drives hundreds of iterations of `Random`, and the seed is the whole input, so a failure names the exact loop that broke and replays on demand. This mirrors the corpus ruling that each lane carries properties via its own native randomized mechanism, and it composes with the toolchain: the theories are plain tests, they run in every suite pass, and nothing behind a flag can rot.

The envelope property is the shape lesson. For random codes, messages, and request ids, every envelope serializes, deserializes, and round-trips all four members, and the member order never moves, `message` before `details` before `request_id`, with `details` absent unless present. The wire contract is a property of every possible value, not of the sixteen the vectors froze, and the loop says so in 500 iterations per seed:

```
    [Theory]
    [InlineData(20260926)]
    [InlineData(1790830800)]
    public void EveryEnvelopeRoundTrips(int seed)
    {
        var random = new Random(seed);
        for (var i = 0; i < Iterations; i++)
        {
            var error = new Envelope(
                ErrorBody.From(
                    ApiError.Fail(
                        RandomCode(random),
                        RandomString(random, 64),
                        RandomString(random, 32)
                    ),
                    RandomString(random, 36)
                )
            );

            var json = JsonSerializer.Serialize(error, KernelJson.Wire);
            var back = JsonSerializer.Deserialize<Envelope>(json, KernelJson.Wire);

            Assert.NotNull(back);
            Assert.Equal(error.Error.Code, back!.Error.Code);
            Assert.Equal(error.Error.Message, back.Error.Message);
            Assert.Equal(error.Error.RequestId, back.Error.RequestId);
        }
    }
```

listing 286: the seed is the whole input: hundreds of envelopes per theory case (source: csharp-net/api/tests/CsharpBook.Api.Tests/Properties/EnvelopeJsonProperties.cs, lines 17-44)

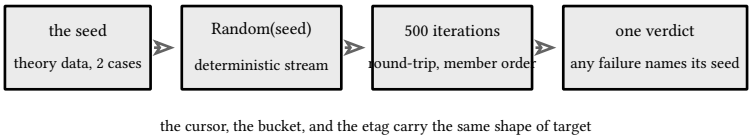


figure 222: one theory, two seeds, a thousand random inputs, one invariant

34.3. the cache as a state machine

The strongest property target drives the ttl lru cache as a state machine: a seeded stream of set, get, and remove operations runs against both the real cache and a reference model built from plain dictionaries, and every observation must agree. Writing the model forced the structure's semantics into sentences the chapter can state: expiry is stamped at set time and a read never extends it, an expired entry lingers in the structure until a read contacts it and it evicts itself, capacity evicts by touch order over live and dead entries alike, and a read refreshes recency without refreshing age.

One lesson cost a debugging round and stays in prose: the model's first eviction ordering used wall-clock timestamps, and two touches inside one clock step tied, while the structure's list keeps event order. The model now carries a monotonic touch sequence, the tick of the op stream, and the divergence disappeared. A reference model is a spec you execute, and it is wrong exactly where your understanding is:

```
            switch (random.Next(3))
            {
                case 0:
                    var fresh = random.Next();
                    cache.Set(key, fresh);
                    value[key] = fresh;
                    setAt[key] = now;
                    touched[key] = tick;
                    if (value.Count > capacity)
                    {
                        var coldest = touched.MinBy(kv => kv.Value).Key;
                        value.Remove(coldest);
                        setAt.Remove(coldest);
                        touched.Remove(coldest);
                    }
                    break;
                case 1:
                    var live = value.ContainsKey(key) && now < setAt[key] + ttl;
                    var found = cache.TryGet(key, out var observed);
                    if (live)
                    {
                        touched[key] = tick;
                        Assert.True(found, $"{key} alive in model but missing in cache");
                        Assert.Equal(value[key], observed);
                    }
                    else
                    {
                        // contact evicts a dead entry
                        value.Remove(key);
                        setAt.Remove(key);
                        touched.Remove(key);
                        Assert.False(found, $"{key} dead in model but present in cache");
                    }
                    break;
                default:
                    cache.Remove(key);
                    value.Remove(key);
                    setAt.Remove(key);
                    touched.Remove(key);
                    break;
```

listing 287: the op switch: every operation mirrored, every observation compared (source: csharp-net/api/tests/CsharpBook.Api.Tests/Properties/CacheStateMachineProperties.cs, lines 51-90)

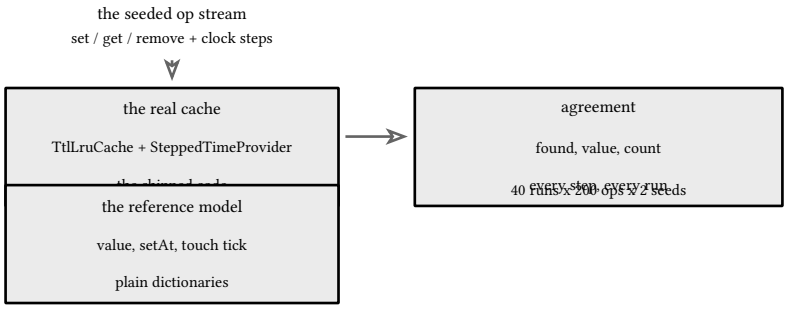


figure 223: two runtimes, one op stream, one verdict per step

34.4. forgery resistance as a target

The jwt property turns the security claim into a loop: for random claims and random 32-byte secrets, sign, then mutate exactly one character in each of the three segments in turn, and every forged token must fail verification with a named failure, the wrong secret must fail, and the untampered token must round-trip its exact claims. The mutation maps back into the base64url alphabet so the forgery stays well formed enough to reach the signature check, which is the point: the property hunts a scheme where a tamper survives, not a parser that quits early.

```
            Exp = Now.AddMinutes(15).ToUnixTimeSeconds(),
            Iat = Now.ToUnixTimeSeconds(),
            Iss = Jwt.Issuer,
            Sid = EnvelopeJsonProperties.RandomString(random, 36),
            Sub = EnvelopeJsonProperties.RandomString(random, 36),
        };
        var token = Jwt.SignHS256(secret, claims);
        var good = Jwt.ReadHS256(token, secret, Now);
        Assert.Null(good.Failure);
        Assert.Equal(claims.Sub, good.Claims!.Sub);

        var segments = token.Split('.');
        for (var segment = 0; segment < 3; segment++)
        {
            var mutated = Mutate(random, segments[segment]);
            Assert.NotEqual(segments[segment], mutated);
            var forged = string.Join(
                ".",
                Enumerable.Range(0, 3).Select(i => i == segment ? mutated : segments[i])
            );
            var read = Jwt.ReadHS256(forged, secret, Now);
            Assert.True(read.Failure is not null, $"tamper survived in segment {segment}: {forged}");
        }

        var wrongSecret = new byte[32];
```

listing 288: sign, tamper each segment one character, demand rejection (source: csharp-net/api/tests/CsharpBook.Api.Tests/Properties/JwtForgeryProperties.cs, lines 29-53)

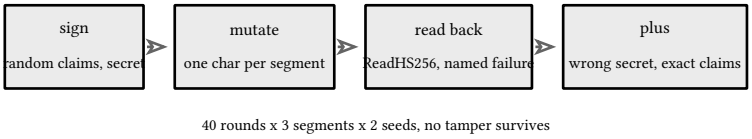


figure 224: the mutation loop: three segments, one flip each, zero survivors

34.5. strictness without a framework

Every double in the suite is hand-rolled and strict: the scripted source dequeues its values and fails the test the moment it is drawn past its script, the fake hasher counts its verifies so the timing test can prove the unknown-email path runs the same hash work as the wrong-password path, the recording logger captures level, message, structured values, and exception for assertion, and the response feature double pins `HasStarted` so started-response guards run under the unit host. None of them implement a framework interface beyond what the production code already consumes, and all of them fit in one sitting:

```
public sealed class Scripted<T>(params IReadOnlyList<T> values)
{
    private readonly Queue<T> _values = new(values);

    public T Next()
    {
        Assert.NotEmpty(_values);
        return _values.Dequeue();
    }

    public int Remaining => _values.Count;
}
```

listing 289: the scripted source: overdraw fails the test, the script is the spec (source: csharp-net/api/tests/CsharpBook.Api.Tests/Support/Scripted.cs, lines 8-19)

The libraries the ecosystem reaches for are dated meta, stated once and declined on stated criteria. NSubstitute sits at 6.2.0, FluentAssertions at 8.11.0, whose 8.0 line moved the license and the community forked, AwesomeAssertions at 9.6.0 with the fork carrying tens of millions of downloads, FsCheck at 3.4.0, SharpFuzz at 2.3.0, all versions read off nuget's own index the day this chapter was written. The vehicle declines them for the reason the contract rules: a substitute library generates doubles whose failure modes the suite cannot see, a fluent assertion layer wraps the asserts the xunit analyzer already checks, FsCheck's shrinking is excellent and the seeded loop already replays, and SharpFuzz needs a coverage build this lane does not carry. None of that is a ban, it is an adoption bar: the day a double costs more than forty lines or a property needs shrinking, the meta paragraph names where to reach.

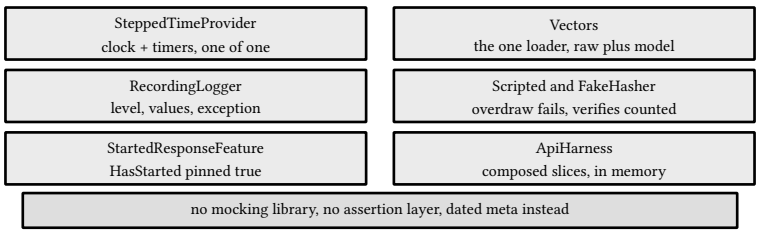


figure 225: the double set: one of each kind, strict by construction

34.6. the xunit surface and the truths it hides

The suite runs on xunit 2.9.3 with the visualstudio runner, and the surface the vehicle uses is deliberately small: `[Fact]` for one case, `[Theory]` with `InlineData` for parameterized ones, and the assert methods, nothing else, no `IAsyncLifetime`, no custom framework attributes. The truths the surface hides deserve prose. Xunit parallelizes across test classes by default, which is why a static `HttpClient` or a process-wide environment variable is a hazard first and a convenience second: the lane suite owns one static client because the lane runs alone, and the probe tests parse urls through an injectable overload rather than touching `ASPNETCORE_URLS` in a shared process. The runner also builds the test host in a separate process, so environment variables set for `dotnet test` reach the tests only by inheritance, which once cost this wave a debugging afternoon. And there is no `TestMain`: the entry point is the attribute, which is why every fixture work happens in constructors and lazy singletons:

```
    private static readonly DateTimeOffset T0 = new(2026, 10, 1, 5, 0, 0, TimeSpan.Zero);

    [Theory]
    [InlineData(20260926)]
    [InlineData(777)]
    public void TokensStayWithinCapacity(int seed)
    {
        var random = new Random(seed);
        for (var i = 0; i < 300; i++)
        {
            var capacity = random.Next(1, 33);
            var bucket = new TokenBucket(capacity, 1.0, T0);
            var now = T0;
            for (var draw = 0; draw < capacity + random.Next(1, 5); draw++)
            {
                // the decision reads the tokens as they were before the
                // draw spent any, so the assertion reads them in that
                // order too
                var before = bucket.Tokens(now);
                var decision = bucket.Allow(now);
                var after = bucket.Tokens(now);
                Assert.InRange(after, 0, capacity);
                Assert.Equal(before ≥ 1, decision.Allowed);
                if (before ≥ 1)
                {
                    Assert.True(after ≤ before + double.Epsilon);
                }
```

listing 290: the whole surface in one method: theory, seeds, loop, assert (source: csharp-net/api/tests/CsharpBook.Api.Tests/Properties/TokenBucketProperties.cs, lines 14-40)

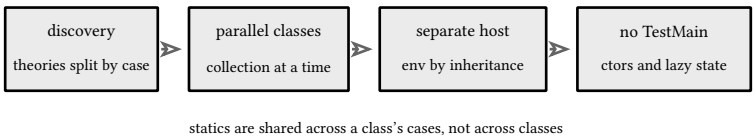


figure 226: what the runner does with one theory

34.7. coverage without ceremony

Coverage rides the platform's own collector: `coverlet.collector` 6.0.4 sits in the test project, pinned, and the opt-in lane is one make target that runs the suite with collection on and drops the reports under a coverage directory. It is a lane, never a gate: the corpus rule keeps percentages out of prose and out of `verify`, and the number the load chapter defends is p99 over the wire, not a line-count proxy. The runner merges nothing by itself, one profile per project, and the lane is exactly as honest as that:

```
<ItemGroup>
  <PackageReference Include="coverlet.collector" Version="6.0.4" />
  <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="10.0.0" />
  <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
  <PackageReference Include="xunit" Version="2.9.3" />
```

listing 291: the pinned collector, the only coverage dependency (source: csharp-net/api/tests/CsharpBook.Api.Tests/CsharpBook.Api.Tests.csproj, lines 9-14)

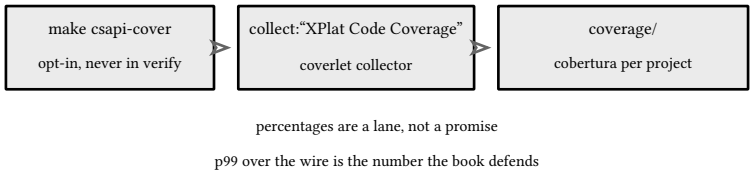


figure 227: the lane: collect, report, never gate

34.8. what the suite proves

The suite closes the part the way the kernel opened it: everything claimed in prose is a behavior some test observes. The counts on the final tree are 379 tests under the solution, 18 of them the property targets across six files, and 6 wire tests in the docker lane, five green and the reports pair waiting on its route. The consolidation left the support folder at eight files, one of each double, and the two retirees live in git history where a reviewer can still read them. The roadmap from here is the closeout's, not this chapter's: the truth pass over all 15 chapters, the pdf rebuild, and the attackers:

```
    /// <summary>The frozen document raw, for byte-level assertions
    /// the step model does not carry.</summary>
    public static JsonDocument Raw(string name)
    {
        var bytes = File.ReadAllBytes(Path.Combine(Root, name + ".json"));
        return JsonDocument.Parse(bytes);
    }

    public static Vector Load(string name)
    {
        var bytes = File.ReadAllBytes(Path.Combine(Root, name + ".json"));
        using var doc = JsonDocument.Parse(bytes);
```

listing 292: the one vector loader: the step model, and raw bytes when a test must compare them (source: csharp-net/api/tests/CsharpBook.Api.Tests/Support/Vectors.cs, lines 17-28)

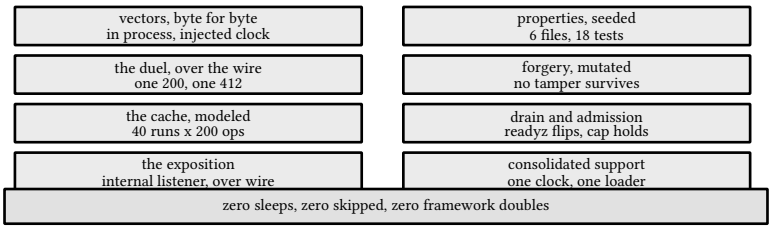


figure 228: the suite as one ledger: every row a test file family observes

sources: package versions read off nuget.org's flat container index the day of writing, nsubstitute 6.2.0, fluentassertions 8.11.0, awesomeassertions 9.6.0 as the fork of the fluent line, fscheck 3.4.0, sharpfuzz 2.3.0, accessed 2026-09-26, with the fork's download count from nuget search the same day. xunit v2 behavior, parallelism, theory discovery, and the separate test host verified against xunit.github.io and against this suite's own runs. coverlet usage against the coverlet github readme and the microsoft learn code coverage page, also 2026-09-26. Verified by `dotnet test Api.slnx`, 379 passed, and by the property filter alone, 18 of 18, with the lane's five wire greens under `make verify-csapi-docker`.

35. capstone: a small language interpreter

The capstone is a working interpreter for a small language, built test-first in this repo at `books/csharp-net/capstone/`, 54 xunit tests, zero skipped, runnable from that folder with `dotnet test`. Every stage of this book's material appears once in it, for real: the lexer is a `ref struct` over a span, the syntax tree is a pair of C# 15 unions that serializes through `System.Text.Json`, the checker is a pattern-matching pass, the evaluator is async end to end, the builtin function table does not exist, a source generator writes it, and interpreter state survives to disk through one SQLite file.

35.1. the pipeline

Source text becomes tokens, tokens become a tree, the tree is checked once statically, then evaluated:

```
var lexer = new Lexer(source.AsSpan());
var tokens = lexer.Scan();
var program = new Parser(tokens).ParseProgram();
new Checker().Check(program);
var evaluator = new Evaluator(Console.Out.WriteLineAsync);
await evaluator.RunAsync(program, cancellationToken);
```

fragment, illustrative, not executed



figure 229: the interpreter pipeline, one pass per stage

35.2. the language

Statements: `let` bindings, assignment, `print`, `if`/`else if`/`else`, `while`, blocks. Expressions: number, string, and boolean literals, names, unary `!` and `-`, comparisons, arithmetic, and calls to builtin functions. Comments run to end of line. The test suite's `Run` helper is the whole language in miniature:

```
    private static async Task<List<string>> Run(string source, CancellationToken ct = default)
    {
        var lexer = new Lexer(source.AsSpan());
        var program = new Parser(lexer.Scan()).ParseProgram();
        new Checker().Check(program);
        var lines = new List<string>();
        var evaluator = new Evaluator(line =>
        {
            lines.Add(line);
            return Task.CompletedTask;
        });
        await evaluator.RunAsync(program, ct);
        return lines;
    }
```

listing 293: source to output in one helper (source: csharp-net/capstone/tests/Capstone.Tests/EvalTests.cs, lines 7-20)

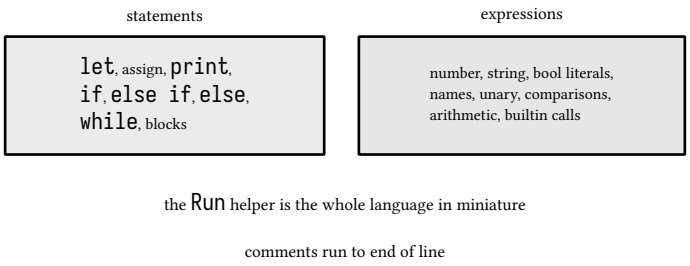


figure 230: the toy language, statements and expressions as one map

35.3. lexer, a ref struct over a span

Chapter 10 said `ref struct` plus `ReadOnlySpan<char>` is the allocation free way to walk a buffer. The lexer is that shape exactly: the source is never copied, slices convert to strings only when a token is built, and the `Scan` loop produces the token list:

```
public ref struct Lexer
{
    private readonly ReadOnlySpan<char> _src;
    private int _pos;
    private int _line;

    public Lexer(ReadOnlySpan<char> source)
    {
        _src = source;
        _pos = 0;
        _line = 1;
    }

    public List<Token> Scan()
```

listing 294: the ref struct lexer and its scan loop (source: csharp-net/capstone/src/Capstone/Lexer.cs, lines 6-19)

Number scanning shows the span discipline, start index, advance, slice once at the end, with the malformed-double rule enforced against the character stream:

```
    private Token ScanNumber()
    {
        var start = _pos;
        while (_pos < _src.Length && char.IsDigit(_src[_pos])) _pos++;
        if (_pos < _src.Length && _src[_pos] == '.')
        {
            _pos++;
            if (_pos ≥ _src.Length || !char.IsDigit(_src[_pos]))
            {
                throw new LexException(_line, "a digit must follow the decimal point");
            }
            while (_pos < _src.Length && char.IsDigit(_src[_pos])) _pos++;
        }
        if (_pos < _src.Length && _src[_pos] == '.')
        {
            throw new LexException(_line, "a digit must follow the decimal point");
        }
        var text = new string(_src[start.._pos]);
        var value = double.Parse(text, CultureInfo.InvariantCulture);
        return new(TokenKind.Number, text, value, null, _line);
    }
```

listing 295: digits, one decimal point, no second one (source: csharp-net/capstone/src/Capstone/Lexer.cs, lines 69-89)

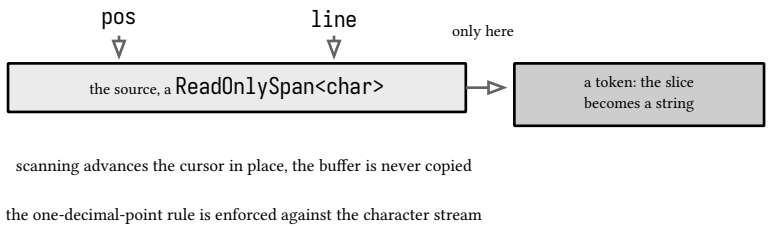


figure 231: the lexer, a cursor over a span, no copies while scanning

35.4. the tree, a c# 15 union

The C# 15 unions from chapter 3 are the tree. Every case is a sealed record with no base class, each carrying its own `Line`, and `Expr` and `Stmt` are one declaration line each: a sealed struct the compiler closes over its cases, an implicit conversion from every case, and pattern matching that reaches the case:

```
// expression cases: one sealed record per alternative, no base record, the
// union declaration below is the whole domain
public sealed record NumberExpr(double Value, int Line);

public sealed record TextExpr(string Value, int Line);

public sealed record BoolExpr(bool Value, int Line);

public sealed record IdentExpr(string Name, int Line);

public sealed record UnaryExpr(TokenKind Op, Expr Operand, int Line);

public sealed record BinaryExpr(TokenKind Op, Expr Left, Expr Right, int Line);

public sealed record CallExpr(string Name, List<Expr> Args, int Line);

// the union: one line declares the expression domain, a sealed struct with
// an implicit conversion from every case and a case aware pattern match
public union Expr(NumberExpr, TextExpr, BoolExpr, IdentExpr, UnaryExpr, BinaryExpr, CallExpr);
```

listing 296: expression cases and the one line union over them (source: csharp-net/capstone/src/Capstone/Ast.cs, lines 3-21)

The statements are the same shape with one honest wrinkle. `let` and assignment carry byte identical payloads, `Name`, `Value`, `Line`, and no payload reader can ever tell them apart, so both records also expose a get-only `Kind` property: the keyword rides along on the wire, written out by the serializer and ignored on the way back, with no serializer attributes anywhere in the tree:

```
// let and assign carry byte identical payloads, the one pair no payload
// reader can ever split, so both carry their keyword on the wire through a
// get-only property: written out, never read back, no serializer attributes
public sealed record LetStmt(string Name, Expr Value, int Line)
{
    public string Kind => "let";
}

public sealed record AssignStmt(string Name, Expr Value, int Line)
{
    public string Kind => "assign";
}

public sealed record BlockStmt(List<Stmt> Body, int Line);

public sealed record PrintStmt(Expr Value, int Line);

public sealed record IfStmt(
    Expr Condition, List<Stmt> Then, List<Stmt> Otherwise, int Line);

public sealed record WhileStmt(Expr Condition, List<Stmt> Body, int Line);

public union Stmt(LetStmt, AssignStmt, BlockStmt, PrintStmt, IfStmt, WhileStmt);

public sealed record ProgramAst(List<Stmt> Body);
```

listing 297: statement cases, the kind tag, the second union (source: csharp-net/capstone/src/Capstone/Ast.cs, lines 23-47)

Line lookup for error reporting is a switch expression over each union with no discard arm. Every pass gets the same guarantee the declaration buys: add a case, forget a switch, and the compiler says CS8509 before any test runs:

```
// the line of a node for error reporting, exhaustive over both unions with
// no discard arm: a new case that forgets its line is CS8509 from the
// compiler, before any test runs
public static class Ast
{
    public static int Line(Expr expr) => expr switch
    {
        NumberExpr n => n.Line,
        TextExpr t => t.Line,
        BoolExpr b => b.Line,
        IdentExpr i => i.Line,
        UnaryExpr u => u.Line,
        BinaryExpr b => b.Line,
        CallExpr c => c.Line,
    };

    public static int Line(Stmt stmt) => stmt switch
    {
        LetStmt s => s.Line,
        AssignStmt s => s.Line,
        BlockStmt s => s.Line,
        PrintStmt s => s.Line,
        IfStmt s => s.Line,
        WhileStmt s => s.Line,
    };
}
```

listing 298: exhaustive line lookup over both unions (source: csharp-net/capstone/src/Capstone/Ast.cs, lines 49-74)

Serialization is where the unions pay differently from the old polymorphic hierarchy. Serialize writes the bare case payload, no envelope and no `$type` discriminator. Reading must name the case from the payload alone, and the built-in `JsonUnionTypeStructuralClassifier` refuses both unions at configure time because it picks by property names and the three literal cases all carry `Value` plus `Line`. The refusal names its own escape hatch, a custom `JsonTypeClassifier` on the reused options, and that one reads more than names: the json kind of `Value` separates the literals, `Args` and `Operand` and `Left` name the composite cases, and the `Kind` tag splits `let` from assignment:

```
{
    // one reused options instance keeps its metadata cache, chapter 14
    public static readonly JsonSerializerOptions Options = Create();

    private static JsonSerializerOptions Create()
    {
        var options = new JsonSerializerOptions { WriteIndented = true };
        options.TypeClassifiers.Add(new AstClassifier());
        return options;
    }
}

internal sealed class AstClassifier : JsonTypeClassifierFactory
{
    public override bool CanClassify(JsonTypeClassifierContext context)
        => context.DeclaringType == typeof(Expr) || context.DeclaringType == typeof(Stmt);

    public override JsonTypeClassifier CreateJsonClassifier(
        JsonTypeClassifierContext context, JsonSerializerOptions options)
        => context.DeclaringType == typeof(Expr) ? ClassifyExpr : ClassifyStmt;

    private static Type ClassifyExpr(ref Utf8JsonReader reader)
    {
        var (names, valueKind, _) = Walk(ref reader);
        if (names.Contains("Args")) return typeof(CallExpr);
        if (names.Contains("Left")) return typeof(BinaryExpr);
        if (names.Contains("Operand")) return typeof(UnaryExpr);
        if (names.Contains("Name")) return typeof(IdentExpr);
        return valueKind switch
        {
            JsonTokenType.String => typeof(TextExpr),
            JsonTokenType.True or JsonTokenType.False => typeof(BoolExpr),
            _ => typeof(NumberExpr),
        };
    }

    private static Type ClassifyStmt(ref Utf8JsonReader reader)
    {
        var (names, _, kind) = Walk(ref reader);
        if (names.Contains("Then")) return typeof(IfStmt);
        if (names.Contains("Condition")) return typeof(WhileStmt);
        if (names.Contains("Body")) return typeof(BlockStmt);
        if (names.Contains("Name"))
        {
            return kind == "let" ? typeof(LetStmt) : typeof(AssignStmt);
        }
        return typeof(PrintStmt);
    }
```

listing 299: one classifier, payload shape back to case type (source: csharp-net/capstone/src/Capstone/AstJson.cs, lines 14-62)

One honest caveat surfaced by the tests: record equality does not recurse into `List<T>` members, `Args` and `Then` compare by reference. The round trip test therefore proves fidelity by re-serializing and comparing documents, and the run test executes the deserialized tree and checks its output, which is the stronger claim anyway.

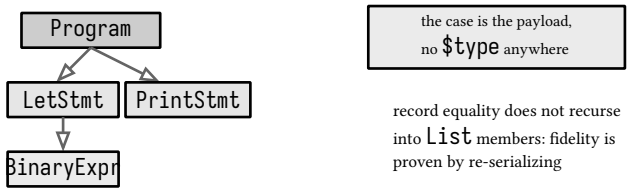


figure 232: the ast, sealed record cases under one union

35.5. parser, precedence by method chain

Recursive descent encodes precedence as call depth: `Equality` calls `Comparison` calls `Additive` calls `Multiplicative` calls `Unary` calls `Primary`, so tighter-binding operators sit lower and finish first:

```
    private Expr Expression() => Equality();

    private Expr Equality()
    {
        var left = Comparison();
        while (Match(TokenKind.EqEq) || Match(TokenKind.BangEq))
        {
            var op = Previous();
            left = new BinaryExpr(op.Kind, left, Comparison(), op.Line);
        }
        return left;
    }

    private Expr Comparison()
    {
        var left = Additive();
        while (Match(TokenKind.Lt) || Match(TokenKind.Le)
            || Match(TokenKind.Gt) || Match(TokenKind.Ge))
        {
            var op = Previous();
            left = new BinaryExpr(op.Kind, left, Additive(), op.Line);
        }
        return left;
    }
```

listing 300: two levels of the precedence chain (source: csharp-net/capstone/src/Capstone/Parser.cs, lines 123-147)

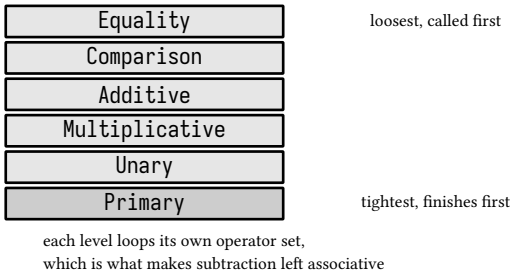


figure 233: precedence as call depth, tighter operators deeper

Each level loops over its own operator set, which is what makes `8 - 4 - 2` left associative, and `else if` is a nested `IfStmt` in the otherwise list rather than a special node, which is why the tree has no `ElseIf` case to pattern match.

35.6. checker, pattern matching over the tree

The static pass proves what it can and stays quiet about the rest. Each expression returns the statically known kind or null for unknown, names must be declared before use, conditions must be flags when their kind is known,

and arithmetic must be numbers when its operand kinds are known. The unknown-node fallbacks are gone: the switch over `Stmt` carries no discard arm, and the kind lookup is a switch expression the compiler checks, a missing case is CS8509:

```
    private void CheckStmt(Stmt stmt)
    {
        switch (stmt)
        {
            case LetStmt s:
                Kind(s.Value);
                if (!_names.Add(s.Name))
                {
                    throw new CheckException(s.Line, $"'{s.Name}' is already defined");
                }
                break;
            case AssignStmt s:
                Kind(s.Value);
                if (!_names.Contains(s.Name))
                {
                    throw new CheckException(s.Line, $"cannot assign to undefined name '{s.Name}'");
                }
                break;
            case PrintStmt s:
                Kind(s.Value);
                break;
            case IfStmt s:
                RequireCondition(s.Condition);
                CheckBlock(s.Then);
                CheckBlock(s.Otherwise);
                break;
            case WhileStmt s:
                RequireCondition(s.Condition);
                CheckBlock(s.Body);
                break;
            case BlockStmt s:
                CheckBlock(s.Body);
                break;
        }
    }
```

listing 301: statement checking as one switch, no discard arm (source: csharp-net/capstone/src/Capstone/Checker.cs, lines 26-60)

```
    private void RequireCondition(Expr condition)
    {
        if (Kind(condition) is { } kind && kind ≠ ValueKind.Flag)
        {
            throw new CheckException(Ast.Line(condition), $"a condition must be a flag, got {kind}");
        }
    }

    // the statically known kind, null when unknown: a switch expression so
    // the compiler proves every case handled, no discard arm
    private ValueKind? Kind(Expr expr) ⇒ expr switch
    {
        NumberExpr ⇒ ValueKind.Number,
        TextExpr ⇒ ValueKind.Text,
        BoolExpr ⇒ ValueKind.Flag,
        IdentExpr i ⇒ IdentKind(i),
        UnaryExpr u ⇒ UnaryKind(u),
        BinaryExpr b ⇒ BinaryKind(b),
        CallExpr c ⇒ CallKind(c),
    };
```

listing 302: conditions checked, kinds returned (source: csharp-net/capstone/src/Capstone/Checker.cs, lines 62-81)

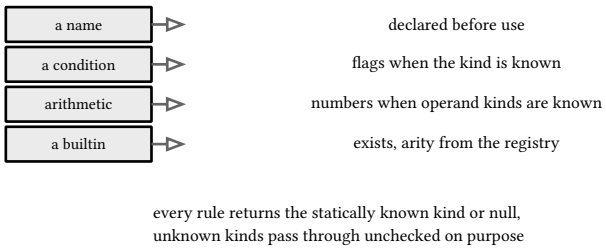


figure 234: the checker proves what it can, stays quiet about the rest

The checker consults the generated registry for builtin existence and arity, so an arity error is a compile-of-the-program error, not a runtime surprise.

35.7. evaluator, async end to end

`RunAsync` walks statements, `Eval` walks expressions, and both carry the `CancellationToken` from chapter 9 through every await. `Eval` is a switch expression, exhaustiveness compiler checked like every other switch over the tree, and the arms with real work live in per-case helpers. Builtin calls are dictionary lookups whose values are async delegates:

```
    private async Task RunStmt(Stmt stmt, CancellationToken ct)
    {
        switch (stmt)
        {
            case LetStmt s:
                _names[s.Name] = await Eval(s.Value, ct);
                break;
            case AssignStmt s:
                _names[s.Name] = await Eval(s.Value, ct);
                break;
            case PrintStmt s:
                await _write((await Eval(s.Value, ct)).ToString());
                break;
            case IfStmt s:
                if ((await Eval(s.Condition, ct)).Flag)
                {
                    await RunBlock(s.Then, ct);
                }
                else
                {
                    await RunBlock(s.Otherwise, ct);
                }
                break;
            case WhileStmt s:
                while ((await Eval(s.Condition, ct)).Flag)
                {
                    await RunBlock(s.Body, ct);
                }
                break;
            case BlockStmt s:
                await RunBlock(s.Body, ct);
                break;
        }
    }
```

listing 303: statement execution as a switch over the union (source: csharp-net/capstone/src/Capstone/Evaluator.cs, lines 34-67)

```
    // a switch expression so the compiler proves every case handled, the
    // arms with work to do live in the helpers below
    private async Task<Value> Eval(Expr expr, CancellationToken ct) => expr switch
    {
        NumberExpr n => Value.Of(n.Value),
        TextExpr t => Value.Of(t.Value),
        BoolExpr b => Value.Of(b.Value),
        IdentExpr i => _names.TryGetValue(i.Name, out var value)
            ? value
            : throw new EvalException(i.Line, $"undefined name '{i.Name}'"),
        UnaryExpr u => await EvalUnary(u, ct),
        BinaryExpr b2 => await EvalBinary(b2, ct),
        CallExpr c => await EvalCall(c, ct),
    };
```

listing 304: expression evaluation as a checked switch expression (source: csharp-net/capstone/src/Capstone/Evaluator.cs, lines 69-82)

Binary operators apply through a switch expression with tiny helper closures, and the one rule with dynamic flavor, text plus anything concatenates, lives in one place:

```
    private static Value Apply(BinaryExpr b, Value left, Value right) => b.Op switch
    {
        TokenKind.Plus => Plus(b, left, right),
        TokenKind.Minus => Number(b, left, right, (l, r) => l - r),
        TokenKind.Star => Number(b, left, right, (l, r) => l * r),
        TokenKind.Slash => Number(b, left, right, (l, r) =>
        {
            if (r == 0) throw new EvalException(b.Line, "division by zero");
            return l / r;
        }),
        TokenKind.Lt => Flag(b, left, right, (l, r) => l < r),
        TokenKind.Le => Flag(b, left, right, (l, r) => l ≤ r),
        TokenKind.Gt => Flag(b, left, right, (l, r) => l > r),
        TokenKind.Ge => Flag(b, left, right, (l, r) => l ≥ r),
        TokenKind.EqEq => Value.Of(left == right),
        TokenKind.BangEq => Value.Of(left ≠ right),
        _ => throw new EvalException(b.Line, $"unknown operator '{b.Op}'"),
    };

    // text plus anything concatenates, numbers add, the rest is an error
    private static Value Plus(BinaryExpr b, Value left, Value right)
    {
        if (left.Kind == ValueKind.Number && right.Kind == ValueKind.Number)
        {
            return Value.Of(left.Number + right.Number);
        }
        if (left.Kind == ValueKind.Text || right.Kind == ValueKind.Text)
        {
            return Value.Of(left.ToString() + right.ToString());
        }
        throw new EvalException(b.Line, $"cannot add {left.Kind} and {right.Kind}");
    }
```

listing 305: operator application, arithmetic, comparison, plus (source: csharp-net/capstone/src/Capstone/Evaluator.cs, lines 117-148)

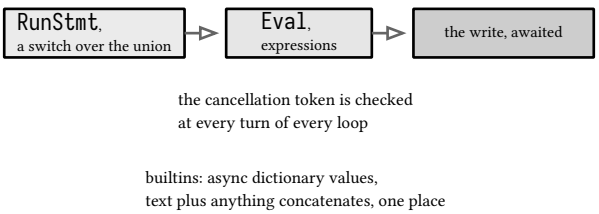


figure 235: the evaluator, a statement walk with awaits inside

35.8. builtins and the generator that registers them

A builtin is a static method marked with an attribute carrying its call name and arity. That is the entire registration procedure:

```
public sealed class BuiltinAttribute(string name, int arity) : Attribute
{
    public string Name { get; } = name;

    public int Arity { get; } = arity;
}

// builtin functions: async capable, cancellation aware, argument validating.
// adding one is: write the method, mark it, done. the generator does the rest.
public static class Builtins
{
    [Builtin("clock_async", 2)]
    public static async Task<Value> ClockAsync(Value[] args, CancellationToken ct)
    {
        Require("clock_async", args, 2);
        RequireNumber("clock_async", args, 0);
        await Task.Delay((int)args[0].Number, ct);
        return args[1];
    }

    [Builtin("sqrt", 1)]
    public static Task<Value> Sqrt(Value[] args, CancellationToken ct)
    {
        ct.ThrowIfCancellationRequested();
        Require("sqrt", args, 1);
        RequireNumber("sqrt", args, 0);
        return Task.FromResult(Value.Of(Math.Sqrt(args[0].Number)));
    }
```

listing 306: the attribute and two builtins, one async (source: csharp-net/capstone/src/Capstone/Builtins.cs, lines 5-33)

The chapter 13 generator shape produces `BuiltinRegistry` with a delegate table and an arity table, collected from every marked method, sorted for deterministic output:

```
        context.RegisterSourceOutput(marked.Collect(), static (spc, methods) =>
        {
            var entries = new List<(string Name, int Arity, string Reference)>();
            string? ns = null;
            foreach (var method in methods)
            {
                var cls = (ClassDeclarationSyntax)method.Parent!;
                ns ??= NamespaceOf(cls);
                var (name, arity) = Registered(method);
                if (name ≠ null)
                {
                    entries.Add((name, arity, $"{cls.Identifier.ValueText}.
{method.Identifier.ValueText}"));
                }
            }
            if (entries.Count == 0) return;

            var sb = new StringBuilder()
                .AppendLine("// <auto-generated by CsharpBook.Capstone.Generators.BuiltinGenerator />")
                .AppendLine("#nullable enable")
                .AppendLine($"namespace {ns};")
                .AppendLine()
                .AppendLine("public static class BuiltinRegistry")
                .AppendLine("{")
                .AppendLine("    public static System.Collections.Generic.IReadOnlyDictionary<string,")
                .AppendLine("            System.Func<Value[], System.Threading.CancellationToken,")
                .AppendLine("            System.Threading.Tasks.Task<Value>>> All { get; }")
                .AppendLine("        = new System.Collections.Generic.Dictionary<string,")
                .AppendLine("            System.Func<Value[], System.Threading.CancellationToken,")
                .AppendLine("            System.Threading.Tasks.Task<Value>>>")
                .AppendLine("        {");
            foreach (var entry in entries.OrderBy(static e => e.Name, StringComparer.Ordinal))
            {
                sb.AppendLine($"            [\"{entry.Name}\"] = (args, ct) => {entry.Reference}(args,
ct),");
            }
            sb.AppendLine("        };")
                .AppendLine()
                .AppendLine("    public static System.Collections.Generic.IReadOnlyDictionary<string, int>
Arity { get; }")
                .AppendLine("        = new System.Collections.Generic.Dictionary<string, int>")
                .AppendLine("        {");
            foreach (var entry in entries.OrderBy(static e => e.Name, StringComparer.Ordinal))
            {
                sb.AppendLine($"            [\"{entry.Name}\"] = {entry.Arity},");
            }
            sb.AppendLine("        };").AppendLine("}");
            spc.AddSource("BuiltinRegistry.g.cs", sb.ToString());
        });
    }
```

listing 307: collect marked methods, emit one registry (source: csharp-net/capstone/src/Capstone.Generators/BuiltinGenerator.cs, lines 28-75)

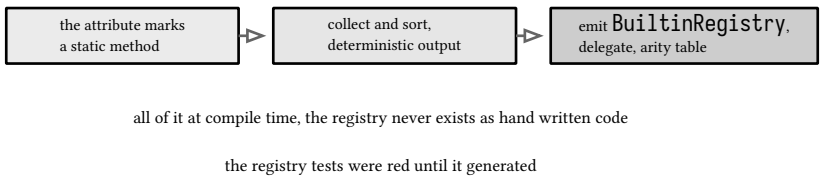


figure 236: the registration pipeline, attribute to generated registry

The registry tests were written before the generator existed, failed to compile against the missing `BuiltinRegistry`, and passed once it was generated, which is the red-green rhythm applied to compile time metaprogramming.

35.9. saves, one sqlite file

Interpreter state survives a process boundary. `Save` writes the evaluator's definitions and the checker's declared names into two tables of one SQLite file, `Load` rebuilds fresh instances from them, and the PersistenceTests make the stronger claim: a program run against reloaded state produces the same output as the pre-save run, and a name loaded from disk cannot be redefined. The connection pins `Pooling=False` because pooled connections hold the file open on Windows after dispose. One test asserts the engine version itself, 3.53.4, the same pinned e_sqlite3 build the samples carry:

```
    public static void Load(string path, Evaluator evaluator, Checker checker)
    {
        if (!File.Exists(path))
        {
            throw new FileNotFoundException($"no interpreter state at {path}", path);
        }
        using var connection = Open(path);
        evaluator.LoadNames(ReadDefinitions(connection));
        checker.LoadNames(ReadCheckedNames(connection));
    }

    public static string EngineVersion(string path)
    {
        using var connection = Open(path);
        using var command = connection.CreateCommand();
        command.CommandText = "SELECT sqlite_version()";
        return (string)command.ExecuteScalar()!;
    }

    private static SqliteConnection Open(string path)
    {
        // pooling holds the file open after dispose, defeat it so callers can
        // move or delete the database between save and load
        var connection = new SqliteConnection("Data Source=" + path + ";Pooling=False");
        connection.Open();
        return connection;
    }
```

listing 308: load, the engine version probe, pooling off (source: csharp-net/capstone/src/Capstone/Persistence.cs, lines 58-84)

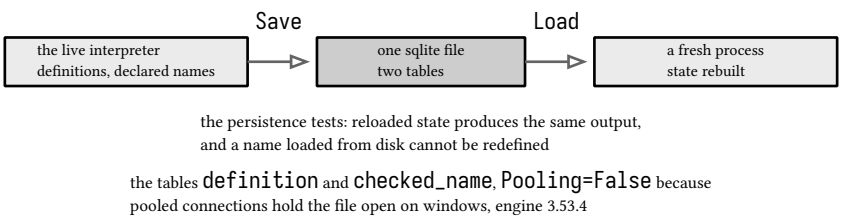


figure 237: saves, one sqlite file across the process boundary

35.10. what the capstone proves

Every chapter's promise, cashed: lexical structure (chapter 2) in the scanner's token set, the type system (chapter 3) in union declarations over record cases, value semantics throughout, members and extension blocks (chapter 4) in the attribute and generated static class, generics (chapter 5) in the delegate table type, patterns (chapter 6) in every switch over the AST, delegates (chapter 7) in the registry's stored functions, exceptions (chapter 8) in `LexException` through `CheckException` with line numbers, async (chapter 9) in the awaited evaluator and cancellation paths, memory (chapter 10) in the span lexer, source generation (chapter 13) in the registry, the stdlib (chapters 15 and 16) in `System.Text.Json` round trips and the SQLite save store, and the idioms (chapter 17) in the discipline of the suite itself. The outlook chapter that once pointed at this code is gone, folded into the home chapters where the C# 15 and .NET 11 features are used, and the RC1 SDK pin honesty lives in the appendices' pinned sources. The promise it made about this tree is kept by the tree: `Expr` and `Stmt` are union declarations, the discard arms are gone, and a missing case is CS8509 at build time.

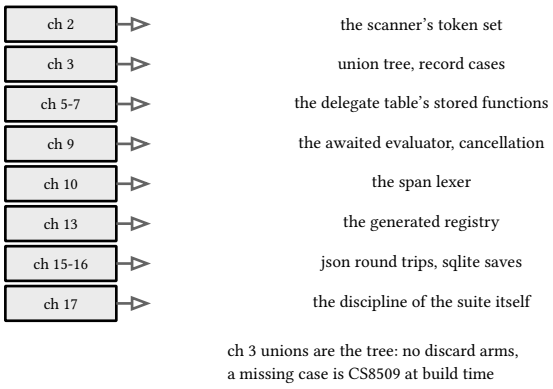


figure 238: every chapter cashed in one interpreter

35.11. profiled, and why

Chapters 11 and 12 built the measurement discipline, so the capstone now turns it on itself. The bench's `capstone` mode runs the whole language in miniature, a program of four statements whose `while` loop turns 1000 times, through the full pipeline 2,000 times, plus each stage in isolation:

```
// whole language in miniature: let bindings, arithmetic, a 1000-iteration
// while loop, an if with an else, a print. the stage loop times lex, parse,
// check, and eval separately, the pipeline loop times all four end to end
// and reports p50/p95/p99/p99.9 over ≥ 2000 runs.
public static class CapstoneBench
{
    public const string Source = """
        let total = 0;
        let i = 0;
        while (i < 1000) {
            total = total + i;
            i = i + 1;
        }
        if (total > 0) {
            print(total);
        } else {
            print(0);
        }
        """;
```

listing 309: the measured workload: let, arithmetic, a 1000-turn loop, if else, print (source: csharp-net/samples/bench/CapstoneBench.cs, lines 14-32)

stage, 2000 runs	median	p99
lex, the span lexer	6.8 us	12.9 us
parse, precedence chain	6.5 us	10.9 us
check, pattern pass	7.1 us	16.3 us
eval, cached tree	307.9 us	1324.9 us
full pipeline	p50 0.3198 ms	p99 0.5587 ms, p99.9 0.6737 ms

BenchmarkDotNet over the same stages, InProcess toolchain, warmup 3, 15 iterations, this machine, 2026-09-13:

```
| Method             | Mean       | Error     | StdDev    | Ratio | Gen0    | Gen1   | Allocated  | Alloc
Ratio |
| EvalOverCachedTree | 318.538 us | 3.5214 us | 3.2939 us | 1.000 | 85.9375 |      - | 1102.89 KB |
1.000 |
| Lex                |   1.788 us | 0.1660 us | 0.1553 us | 0.006 |  0.4158 | 0.0057 |    5.32 KB |
0.005 |
| ColdPipeline       |   3.033 us | 0.2915 us | 0.2727 us | 0.010 |  0.5341 | 0.0076 |    6.84 KB |
0.006 |
```

```
// the benchmarkdotnet a/b over the capstone's hot paths, inprocess because
// of the rc gap chapter 12 quotes: eval over a cached tree against a full
// cold pipeline, lex alone for scale
[MemoryDiagnoser]
public class CapstoneStages
{
    private List<Token> _tokens = [];
    private ProgramAst _program = null!;

    [GlobalSetup]
    public void Setup()
    {
        _tokens = new Lexer(CapstoneBench.Source.AsSpan()).Scan();
        _program = new Parser(_tokens).ParseProgram();
        new Checker().Check(_program);
    }

    [Benchmark(Baseline = true)]
    public async Task<int> EvalOverCachedTree()
    {
        var evaluator = new Evaluator(_ => Task.CompletedTask);
        await evaluator.RunAsync(_program);
        return _program.Body.Count;
    }

    [Benchmark]
    public int Lex()
        => new Lexer(CapstoneBench.Source.AsSpan()).Scan().Count;

    [Benchmark]
    public int ColdPipeline()
    {
        var tokens = new Lexer(CapstoneBench.Source.AsSpan()).Scan();
        var program = new Parser(tokens).ParseProgram();
```

listing 310: the a slash b: eval over a cached tree against lex and the cold pipeline (source: csharp-net/samples/bench/CapstoneBench.cs, lines 113-147)

The counters panel, a 4 second capture of the eval loop committed as `samples/captures/counters-capstone-2026-09-13.csv`, reads the same story from outside: the allocation rate is 3.0 GB per second, generation 0 fires about 230 times a second, generation 1 and 2 stay at zero, and the pause tax is 2.4 percent of wall time. Die-young churn, exactly chapter 11′s ephemeral pattern, at a rate the front end never approaches.

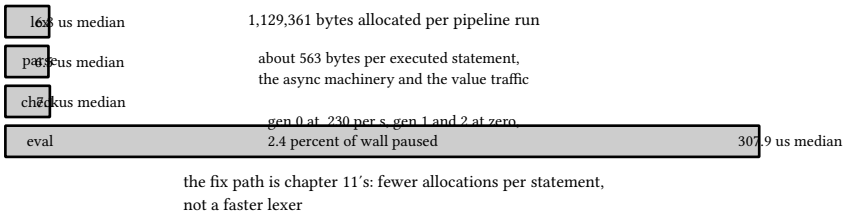


figure 239: where the time is: the evaluator is the program, the front end is noise

Why the evaluator costs what it costs: every statement dispatch is an `await`ed method, so each of the 2,006 statement executions pays the async state machinery, and the loop's hot statements, two assignments and a condition per turn, allocate on every turn through `Eval` returning `Task<Value>` and the state machine each awaited method carries. The write path never suspends, the callback is `Task.CompletedTask`, so none of this is waiting on anything: it is the price of the shape, measured at 563 bytes and about 150 nanoseconds per statement. The tail tells the rest: eval's p99 is 4.3 times its median because those 230 generation 0 collections per second land inside eval reps, and the pipeline p99 of 0.5587 ms against a p50 of 0.3198 ms is the same pauses seen end to end. Chapter 12′s rule applies to the capstone too: the front end is fast enough that optimizing the lexer or parser is decoration, the evaluator's allocation rate is the whole story, and the honest comparisons are the synchronous F# port in chapter 21, which removed the async dimension deliberately, and a `ValueTask` rewrite, which this teaching tree leaves as the measured exercise it now is.

35.12. the service part

The service part built on the same book after this capstone is the next evidence layer: chapters 22 through 34 land the api kernel and middleware, the users, authn, authz, store, concurrency, caching, rate limiting, observability, and load families, the docker ship chapter, and the test suite chapter, the book stamped 5.0 at the close. The lane gates through `make verify-csharp` in the repo chain, the ship lane under `make verify-csapi-docker`. The audit records the post-attack closeout, 2026-09-26: 379 tests under the api solution after the blind module and prose attack rounds, and the docker lane 6 of 6 over the real image.

sources: the capstone solution in this repo, built and tested with `make verify-csharp` and from its own directory with `dotnet test`, 54 tests, zero skipped, all green on the pinned SDK 11.0.100-rc.1. Union serialization measured on this SDK: serialize writes the bare case payload, `JsonUnionTypeStructuralClassifier` rejects same-shaped cases at configure time, and a custom `JsonTypeClassifier` reads the payload back. The serializer's attribute is `JsonUnionAttribute`, `System.Runtime.CompilerServices.UnionAttribute` belongs to the compiler. learn.microsoft.com, what's new in .NET 11 libraries, serialize C# union types, and jsontypeclassifier pages, accessed 2026-09-13.

36. appendices: coverage and sources

The coverage matrices are the book's audit trail. Every reserved and contextual keyword enumerated from the official keywords page, every stdlib component family, each mapped to the chapter that teaches it, and rendered here at milestone time. The build enforces honesty from the other side: every chapter listing reads a real file that the sample suite compiles and tests, 211 tests in the samples suite, 54 in the capstone, 20 in the f# port, zero skipped anywhere.

36.1. keyword coverage

ch 1 is where `var` and `args` land, ch 2 carries the literal surface, and the reserved words are spread from there. The census matches chapter 2's recount: 77 reserved words, the set frozen since C# 1, against 49 contextual, with `union` filed contextual on the union reference page's word rather than the live index table's, which had not listed it at the recount date. Unmapped count: 0.

word	kind	taught in
abstract	reserved	ch 3
as	reserved	ch 6
base	reserved	ch 3
bool	reserved	ch 2
break	reserved	ch 6
byte	reserved	ch 2
case	reserved	ch 6
catch	reserved	ch 8
char	reserved	ch 2
checked	reserved	ch 10
class	reserved	ch 3
const	reserved	ch 4
continue	reserved	ch 6
decimal	reserved	ch 2
default	reserved	ch 6
delegate	reserved	ch 7
do	reserved	ch 6
double	reserved	ch 2
else	reserved	ch 6
enum	reserved	ch 3
event	reserved	ch 7
explicit	reserved	ch 4
extern	reserved	ch 10
false	reserved	ch 2
finally	reserved	ch 8
fixed	reserved	ch 10
float	reserved	ch 2
for	reserved	ch 6
foreach	reserved	ch 6
goto	reserved	ch 6
if	reserved	ch 6

word	kind	taught in
implicit	reserved	ch 4
in	reserved	ch 6
int	reserved	ch 2
interface	reserved	ch 3
internal	reserved	ch 4
is	reserved	ch 6
lock	reserved	ch 9
long	reserved	ch 2
namespace	reserved	ch 1
new	reserved	ch 4
null	reserved	ch 2
object	reserved	ch 3
operator	reserved	ch 4
out	reserved	ch 4
override	reserved	ch 3
params	reserved	ch 4
private	reserved	ch 4
protected	reserved	ch 4
public	reserved	ch 4
readonly	reserved	ch 10
ref	reserved	ch 10
return	reserved	ch 6
sbyte	reserved	ch 2
sealed	reserved	ch 3
short	reserved	ch 2
sizeof	reserved	ch 10
stackalloc	reserved	ch 10
static	reserved	ch 4
string	reserved	ch 2
struct	reserved	ch 3
switch	reserved	ch 6
this	reserved	ch 4
throw	reserved	ch 8
true	reserved	ch 2
try	reserved	ch 8
typeof	reserved	ch 13
uint	reserved	ch 2
ulong	reserved	ch 2
unchecked	reserved	ch 10
unsafe	reserved	ch 10
ushort	reserved	ch 2
using	reserved	ch 1
virtual	reserved	ch 3
void	reserved	ch 2
volatile	reserved	ch 9

word	kind	taught in
while	reserved	ch 6
add	contextual	ch 7
allows	contextual	ch 5
alias	contextual	ch 1
and	contextual	ch 6
ascending	contextual	ch 14
args	contextual	ch 1
async	contextual	ch 9
await	contextual	ch 9
by	contextual	ch 14
closed	contextual	ch 3
descending	contextual	ch 14
dynamic	contextual	ch 13
equals	contextual	ch 14
extension	contextual	ch 4
field	contextual	ch 4
file	contextual	ch 1
from	contextual	ch 14
get	contextual	ch 4
global	contextual	ch 1
group	contextual	ch 14
init	contextual	ch 4
into	contextual	ch 14
join	contextual	ch 14
let	contextual	ch 14
managed	contextual	ch 10
nameof	contextual	ch 4
nint	contextual	ch 10
not	contextual	ch 6
notnull	contextual	ch 5
nuint	contextual	ch 10
on	contextual	ch 14
or	contextual	ch 6
orderby	contextual	ch 14
partial	contextual	ch 4
record	contextual	ch 3
remove	contextual	ch 7
required	contextual	ch 4
safe	contextual	ch 10
scoped	contextual	ch 10
select	contextual	ch 14
set	contextual	ch 4
union	contextual	ch 2
unmanaged	contextual	ch 5
value	contextual	ch 4

word	kind	taught in
var	contextual	ch 1
when	contextual	ch 8
where	contextual	ch 5
with	contextual	ch 3
yield	contextual	ch 17

The f# side is a tour, chapters 19 through 21, not a census, and claims no count over the language's own keyword table. These are the words and symbols the tour exercises, enumerated from the official f# language reference, with `and!` read from the task expressions page:

word or symbol	kind	taught in
let	keyword	ch 19
mutable	keyword	ch 19
rec	keyword	ch 19
type	keyword	ch 19
of	keyword	ch 19
with	keyword	ch 19
match	keyword	ch 19
module	keyword	ch 19
open	keyword	ch 19
fun	keyword	ch 19
member	keyword	ch 20
private	keyword	ch 20
let!	computation expression	ch 20
and!	computation expression	ch 20
return	computation expression	ch 20
|>	symbol	ch 20
>>	symbol	ch 20
<-	symbol	ch 19
_	symbol	ch 19
.[ ]	symbol	ch 20
[< >]	symbol	ch 19
?	symbol	ch 21

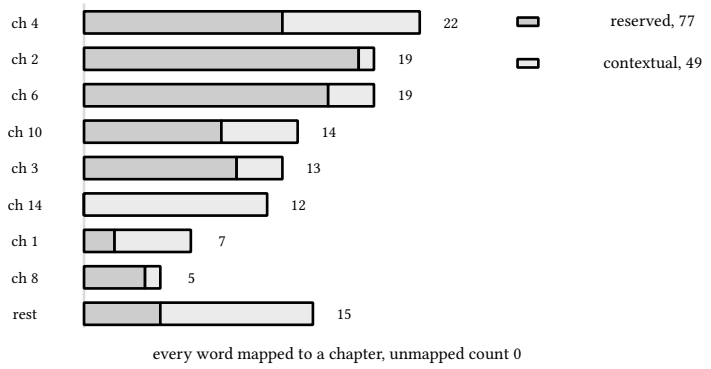


figure 240: the keyword census, 77 reserved against 49 contextual

36.2. stdlib coverage

Component families against chapters, with the honest status column. Rows marked as pointers delegate the deep treatment to the systems books that follow, where the components are built rather than used.

component family	chapter	status
List, Dictionary, HashSet	ch 15	covered
Queue, Stack, PriorityQueue	ch 15	covered
frozen collections	ch 15	covered
EqualityComparer.Create	ch 15	covered
immutable collections	ch 15	covered
ArrayPool, MemoryPool, IMemoryOwner	ch 11	covered
ReadOnlySequence	ch 15	mentioned only
Span, Memory, stackalloc	ch 10	covered
Math, BigInteger	ch 15	covered
Decimal32, Decimal64, Decimal128	ch 15	covered
BFloat16	ch 15	covered
Complex<T> over the float types	ch 15	mentioned only, not demonstrated
Vector and SIMD types	ch 15	covered
INumber generic math	ch 5	covered
nullable value types	ch 3	covered
System.Text.Json	ch 16	covered
union serialization, JsonUnionTypeStructuralClassifier	ch 16	covered
JsonNamingPolicy.PascalCase	ch 16	covered, options path only, the attribute rejects it at rc.1
Utf8JsonReader and Writer	ch 16	mentioned only, not demonstrated
File, Path, Directory, Stream	ch 16	covered
StringStream	ch 16	covered
String, StringBuilder, Encoding	ch 16	covered
Regex	ch 17	pointer to generated regex, ch 13
HttpClient	ch 16	covered
Channels	ch 16	covered
Random, Guid, Stopwatch, DateTimeOffset	ch 16	covered
reflection, attributes	ch 13	covered
source generators	ch 13	covered, real generator in suite
Linq operators	ch 14	covered
LeftJoin, RightJoin, FullJoin	ch 14	covered
Threading, locks, Interlocked	ch 9	pointer to book 12
Exceptions hierarchy	ch 8	covered
diagnostics, metrics, EventSource	ch 12	covered, language-level; book 12 keeps distributed observability

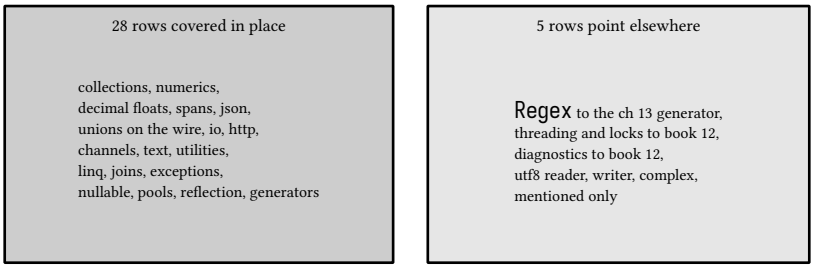


figure 241: stdlib coverage, covered in place against pointers

36.3. pinned sources

Every chapter ends with its own sources line naming the official sections consulted while writing it. The book level pins are:

topic	url	accessed
keyword and contextual keyword inventory	https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/	2026-09-08
what is new in c# 14	https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-14	2026-09-07
what is new in .net 10	https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview	2026-09-07
c# 15 and .net 11 preview	https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-11/overview	2026-09-07
what is new in c# 15	https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-15	2026-09-13
what is new in .net 11, runtime	https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-11/runtime	2026-09-13
what is new in .net 11, libraries	https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-11/libraries	2026-09-13
f# language reference	https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/	2026-09-13
f# keyword reference	https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/keyword-reference	2026-09-13
f# symbol and operator reference	https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/symbol-and-operator-reference/	2026-09-13
wikibooks f sharp programming	https://en.wikibooks.org/wiki/F_Sharp_Programming	2026-09-13
what is new in f# 10	https://learn.microsoft.com/en-us/dotnet/fsharp/whats-new/fsharp-10	2026-09-13

NOTE: version drift rule

These pins were fetched 2026-09-07 and 2026-09-08 against C# 14 and .NET 10 (SDK 10.0.400) with C# 15 and .NET 11 read at preview 7, then re-read and extended 2026-09-13 at the 2.0 closeout: the C# 15 what's new page, the .NET 11 overview re-read plus its runtime and libraries notes, and the f# reference set, all against the RC1 SDK. If an upstream page changes, the delta gets recorded here rather than silently rewriting chapters, so this table is also the book's changelog for documentation movement.

NOTE: the rc1 build pin

Since the C# 15 folds, this book builds and tests on the .NET 11 SDK at 11.0.100-rc.1, pinned by the book local `global.json` with `latestFeature` roll forward, while the repository root stays on 10.0.401 for the other books. Under the corpus version currency law this pin rides the newest build the vendor makes installable, nightlies excepted, and it was verified current 2026-09-30: the dotnet release feed shows RC1, shipped september 8, as the newest .NET 11 build with no RC2 yet and GA expected november 2026, and the three taught RC1 quirks re-probed unchanged on that date, the per-type `[JsonNamingPolicy]` attribute rejecting `PascalCase` with `ArgumentOutOfRangeException` while the options lever and the snake attribute serialize clean, `StringStream` reporting `CanSeek` false with `Length` throwing `NotSupportedException`, and the F# 11 what's new page still absent. Whenever a newer build ships, RC2 or GA, the pin moves to it, `make verify-csharp` re-runs, and any behavioral delta lands here rather than silently editing chapters.

NOTE: the f# versioning evidence

No what's new in F# 11 page existed at RC1 and the .NET 11 overview names no F# language version, so the F# 11 claim rests on the SDK itself: `dotnet fsi` under 11.0.100-rc.1 loads `FSharp.Core 11.0.0.0` on a runtime reporting 11.0.0, the same pairing the F# 9 and F# 10 pages document for their releases. Re-checked 2026-09-30, the canonical url still answers 404, so the SDK witness stands. When the F# 11 page ships, pin it in the table above and re-check chapter 21′s versioning section against it.

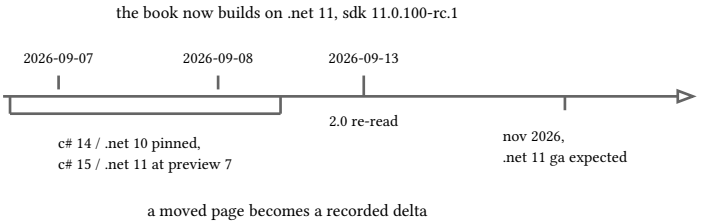


figure 242: the pin timeline, when the pins were read and when they drift